

Complete Documentation

Multi-tenant SaaS for travel agencies

TECHNICAL REFERENCE

GENERATED	2026-07-02
DOCUMENTS	54 documents across 5 parts
LIVE	https://agencetool.vercel.app
REPOSITORY	github.com/ouks1993/agence_tool

Contents

PART I CORE DOCUMENTATION

Atlas · Documentation Index	1
Vision	3
Tech Stack	6
Architecture	15
Domain Model	20
Database	27
Schema Reference	32
API Integrations	51
Booking Architecture (Sprint 1)	55
Server Actions Reference	67
Development Guide	76
Coding Standards	83
UI / UX	85
Design Recommendations (Redesign Target)	88
Business Rules	103
Deployment	109
Roadmap & Changelog	116
Security	125
Analytics & BI	133
AI Features	140

PART II PROJECT ROOT DOCS

Project README	147
Project Overview	157
Agent & Coding Conventions	161
Design System	164
Claude Code Instructions	183

PART III AUDITS & REVIEWS

Atlas · Production Readiness Audit Report	184
Production-Readiness Audit	198
Owner's-Eye UX Audit	202
CTO Review · Atlas	206

PART IV ARCHITECTURE DECISION RECORDS

Decision 0001 · Navigation IA restructure	217
Decision 0002 · Agency Network is a Stage 3 Direction	219
Decision 0003 · Sprint 1 Booking Architecture	221
Decision 0004 · Travel Platform facade over direct supplier access	224
Decision 0005 · Marketing-grade UI redesign of the live app	226
Decision 0006 · Auto-create a booking when a client accepts a proposal	228
Decision 0007 · Commission ledger survives booking/item deletion	230
Decision 0008 · One set of booking lifecycle guards, shared by both status-change entry poi- ...	232
Decision 0009 · Deposit-threshold booking confirmation	235

PART V SPECIFICATIONS

Phase 1 · "Sellable" Implementation Plan	237
UI Polish & Responsive Improvements	241
Action Required: UI Polish & Responsive Improvements	243
Requirements: UI Polish & Responsive Improvements	244
Task 01: globals.css Foundation	246
Task 02: Layout Sticky Footer	250
Task 03: Site Header Responsive	252
Task 04: Site Footer Responsive	254
Task 05: Home Page Overhaul	256
Task 06: Dashboard Page	261
Task 07: Chat Page	266
Task 08: Profile Page	271
Task 09: Auth Pages	275
Task 10: Setup Checklist Card Adoption	278
Task 11: Starter Prompt Modal	281
UI Redesign · Phased Plan	283

PART I

Core Documentation

20 documents

Atlas — Documentation Index

Atlas is the operating system for modern travel agencies. Every decision should support that sentence.

Atlas is a **multi-tenant SaaS for travel agencies**. Each agency runs its clients, sales pipeline, proposals, bookings and finance in a fully isolated workspace; the vendor manages every agency from a platform console. Multilingual (EN/FR/AR with RTL), deployed on Vercel with GitHub auto-deploy.

- **Live:** <https://agencetool.vercel.app>
- **Repo:** github.com/ouks1993/agence_tool
- **Vendor console:** <https://agencetool.vercel.app/platform>

This is the reference index. The detailed reference is split into focused docs under `docs/`. See also `DESIGN.md` (design system) and `AGENTS.md/CLAUDE.md` (working conventions) — those remain the source of truth for UI and coding conventions respectively.

Documentation map

Doc	Covers
Vision	What Atlas is, who it's for, operating model, features at a glance
Architecture	Multi-tenancy · auth · platform admin · impersonation · roles · route map · module layout · i18n
Tech stack	Framework, libraries, services
Domain model	Core entities, the Lead→...→Feedback chain, aggregates & ownership
Database	Schema tables, tenancy columns, migrations, ID convention
API integrations	Duffel · Hotelbeds · Stripe (billing + Connect) · Resend · Gemini / OpenRouter · Blob
Development guide	Setup, env vars, commands, scripts, DB workflow
Coding standards	→ points to <code>AGENTS.md</code> ; conventions summary
UI / UX	→ points to <code>DESIGN.md</code> ; product UX patterns
Business rules	RBAC, currency, booking lifecycle, commissions, vocabularies
Deployment	Vercel, env, prod migrations, demo accounts

Roadmap & changelog	Phase status, open items, commit changelog
Security	Tenant isolation, auth, guards, webhooks, DB safety, known risks
Analytics & BI	Dashboards, currency-safe metrics, CSV/Excel export
AI	Assistant + inline AI features

At a glance

- **Stack:** Next.js 16 (App Router) · React 19 · TypeScript · Tailwind v4 + shadcn/ui · PostgreSQL (Neon) + Drizzle · Better Auth · next-intl · Stripe + Stripe Connect · Resend · Duffel (flights) · Hotelbeds (hotels) · Vercel AI SDK + Google Gemini (primary) / OpenRouter (fallback) · Vercel.
- **Tenancy:** every business row is scoped to an `agencyId`; enforced via `requireAgencyUser()`. Vendor platform admin sits above all tenants.
- **Roles:** admin · manager · finance · support · agent — each with a tailored landing and nav.
- **Deploy:** push to `main` → Vercel auto-deploys production.
- **DBs:** dev `ep-wandering-sunset-aitlty78` · prod `ep-misty-thunder-aixz34vy`. Run `POSTGRES_URL=<prod-url> npx drizzle-kit migrate` after each schema change.

Phases 1–3 + UX + Data-quality/BI + Sprint 1 (booking) + Sprint 2 (Travel Platform) complete, plus a full **marketing-grade UI redesign** of every screen (Phases 0–5). Multi-tenant SaaS with live travel sourcing, billing, client portal, supplier/commission management, and AI throughout. Real supplier search and booking (Duffel + Hotelbeds) fully wired behind a provider-agnostic Travel Platform facade — activate with production credentials. Adding a new provider requires zero consumer changes. Migrations: 26 (latest `0025` — booking-item net cost).

Shipped since:

- **UI redesign — complete.** Every one of the 55 routes brought to the sales-deck standard: shared design tokens + primitives, ⌘K command palette + mobile tab bar, a unified `StatStrip` KPI band app-wide, and per-screen rebuilds. See `specs/ui-redesign/PLAN.md` and decision `0005`. `marketing/` holds the deck, 11 mockups, the demo dataset, and the design guide.
- **AI — Google Gemini** is the primary provider (`@ai-sdk/google`, OpenRouter fallback). The `/assistant` answers across **every data domain** (bookings, clients, proposals, pipeline, finance, commissions) via tenant-safe read tools, keeps a live client/booking context rail, and respects platform-admin "view as". See `docs/ai.md`.
- **Auto-booking** — accepting + signing a proposal now auto-creates a booking in `awaiting_payment` (idempotent, tenant-safe). See `docs/business-rules.md` and decision `0006`.

Vision

Atlas is the operating system for modern travel agencies.

Every decision should support that sentence.

It is the north star: a feature, a design choice, or a piece of scope earns its place only if it makes Atlas more of an agency's single operating system. If it doesn't support the sentence, it doesn't ship.

No OTA knows agency behaviour. No agency knows market behaviour. Only Atlas sees both.

That information asymmetry — across thousands of agencies, in real time — is the business. Not the software. Not the AI features. The information.

Today an agency runs on a patchwork of tools:

- Excel
- WhatsApp
- Email
- Booking portals
- Accounting software

Everything should happen inside Atlas instead — **one workspace**.

Long-term goals

- CRM
- Sales
- Quotations
- Flight booking
- Hotel booking
- Supplier management
- Accounting
- Customer portal
- AI assistant
- BI
- Automation

Operating model

- **Per-agency isolation.** Every business record is scoped to an `agencyId`; agencies never see each other's data. Reference numbers (`BKG-...`, `PRD-...`) are unique per agency. See [architecture.md](#) and [security.md](#).
- **Vendor + tenants.** The vendor (platform operator) sells Atlas to many agencies and bills them by subscription, managing all of them from a platform console. Each agency is a fully isolated tenant.
- **DZD-first, no FX.** The agency operates in Algerian Dinar. EUR/USD are supported for source data but Atlas never converts between currencies — analytics group by currency rather than summing across. See [business-rules.md](#).
- **Graceful degradation.** Every external integration (flights, hotels, email, payments, AI) falls back to sample/logged behaviour when its keys are unset, so the app runs end-to-end with only a database and an auth secret. See [api-integrations.md](#).

Where it is today

Atlas is a deployed, multi-tenant SaaS — multilingual (EN/FR/AR with RTL), live on Vercel with GitHub auto-deploy.

- **Live:** <https://agencetool.vercel.app>
- **Repo:** github.com/ouks1993/agence_tool
- **Vendor console:** <https://agencetool.vercel.app/platform>

Shipped against the long-term goals:

- **CRM** — clients with contacts and a unified funnel timeline.
- **Sales** — opportunities, pipeline stages, conversion analytics.
- **Quotations** — proposals with PDF + e-signature, one-click convert→booking, AI quote builder.
- **Flight booking** — live Duffel search (real order placement is pending).
- **Hotel booking** — live Hotelbeds search + content cache (real booking pending).
- **Supplier management** — directory, contracts & rates, supplier picker.
- **Customer portal** — magic-link login, trip view, online payments, proposal signing.
- **AI assistant** — agency-scoped chat + inline itinerary/quote/email/visa features. See [ai.md](#).
- **BI** — decision dashboards + standardized CSV/Excel export. See [analytics.md](#).
- **Accounting** — partial: payments, two-ledger commissions, AR aging. Full accounting is a long-term goal.
- **Automation** — early: auto-generated commissions, lifecycle guards. Broader automation (e.g. quote-on-stage-change, WhatsApp) is on the roadmap.

Atlas principles

The ten founding principles. Everything else — the design system, the never-rules, the golden workflow — is an expression of these.

1. **Client first.** The client record is the spine; every workflow starts there.
2. **Every entity belongs to an agency.** Multi-tenancy is non-negotiable — `agencyId` on every business row, enforced on every read/write.
3. **No duplicated data.** One source of truth; canonical values over free text.
4. **Automation over manual work.** Generate, don't ask — quotes, itineraries, commissions, documents.
5. **Every workflow has one happy path.** A single, obvious golden route through each flow (business-rules.md).
6. **Managers need insight.** Decision-oriented analytics where managers land.
7. **Agents need speed.** Fast, scoped surfaces — minimal clicks, minimal waiting.
8. **Data quality > flexibility.** Constrain inputs to keep reporting clean.
9. **Everything is reversible.** Prefer soft state and undo over destructive operations.
10. **Every page answers "What do I do next?"** Always surface the next action; never a dead end.

These are operationalized as UI guardrails in ui-ux.md, hard constraints in business-rules.md, and tracked against the code in the gap tracker.

See roadmap.md for phase status and open items, and the index for the full documentation map.

Tech Stack

Atlas is a multi-tenant SaaS for travel agencies, built as a single Next.js App-Router application. This document is the authoritative reference for the exact frameworks, libraries, and external services in use — with the real version numbers from `package.json` and the reason each one is here.

At a glance

Layer	Choice
Framework	Next.js 16 (App Router), React 19, TypeScript 5
Styling	Tailwind CSS v4 (<code>@theme inline</code>), shadcn/ui (new-york), Lucide icons
Fonts	Geist (sans/mono), IBM Plex Sans Arabic (RTL)
Auth	Better Auth (email/password, invitation-gated)
Database	PostgreSQL (Neon) + Drizzle ORM (<code>postgres-js</code> driver)
i18n	next-intl 4 (cookie-based, no URL routing) — <code>en · fr · ar</code> (RTL)
Charts	recharts 3 (tokenized)
AI	Vercel AI SDK + Google Gemini (primary) / OpenRouter (fallback)
Email	Resend (transactional: invites, verification, password reset, proposals)
Billing	Stripe subscriptions (vendor → agency) + webhook
Payments	Stripe Connect (traveler → agency, destination charges)
Flights	Duffel (Amadeus self-service kept only as legacy fallback)
Hotels	Hotelbeds (APITUDE: availability + content)
PDF	<code>@react-pdf/renderer</code> (server-rendered proposals)
Data export	<code>exceljs</code> (XLSX) + built-in CSV (UTF-8 BOM) for BI/Power BI
Storage	Vercel Blob (optional)
Validation	Zod 4 (env schema, action inputs, AI structured output)
Hosting	Vercel + GitHub auto-deploy

Every external integration **degrades gracefully when unconfigured** — see `api-integrations.md`. Flights fall back to sample data, hotels to placeholder photos, email to console logging, billing/AI to disabled.

Core framework & language

Package	Version	Role
next	16.1.6	App-Router framework; RSC, Server Actions, route handlers, image optimization, rewrites/redirects/headers
react / react-dom	19.2.4	UI runtime (React 19 — Server Components, Actions)
typescript	^5.9.3	Language; strict compiler settings (see below)
@types/node	^20.19.41	Node type definitions
@types/react / @types/react-dom	19.2.5 / 19.2.3	React type definitions

Dev server runs with Turbopack (`next dev --turbo`). There is **no** `middleware.ts` in the project — request-time locale resolution and auth gating are handled inside `src/i18n/request.ts` and server components / layouts respectively, not in Edge middleware.

TypeScript configuration

`tsconfig.json` runs in **maximum-strict mode**. Beyond `strict: true`, it additionally enables:

- `noUncheckedIndexedAccess` — indexed access yields `T | undefined`
- `exactOptionalPropertyTypes` — `?` and `| undefined` are distinct
- `noUnusedLocals` / `noUnusedParameters`
- `noImplicitReturns`, `noFallthroughCasesInSwitch`, `noImplicitOverride`
- `forceConsistentCasingInFileNames`

Module resolution is `bundler`, `target` `ES2017`, `jsx`: `react-jsx`, and the `@/*` path alias maps to `./src/*`. The `create-agentic-app` scaffold folder is excluded from the program. Type-checking runs via `pnpm typecheck` (`tsc --noEmit`).

Next.js configuration (`next.config.ts`)

Wrapped with `createNextIntlPlugin()` from `next-intl`. Notable settings:

- `images.remotePatterns` — allow-listed image hosts: Google/GitHub avatars, `*.public.blob.vercel-storage.com` (Vercel Blob), `photos.hotelbeds.com` (GIATA hotel/room CDN), and `picsum.photos` / `fastly.picsum.photos` (placeholder photos when live supplier data is unavailable).
- `compress: true` — gzip response compression.
- **URL canonicalization** — `rewrites()` serve `/proposals/*` from the `/products/*` page files and `/sourcing/hotels/*` from `/hotels/*`; `redirects()` send `/products/*` → `/proposals/*` (permanent) and legacy `/search`, `/hotels/*`, `/operations` to their new canonical paths (temporary).
- **Security headers** applied to all routes: `X-Frame-Options: DENY`, `X-Content-Type-Options:`

`nosniff, Referrer-Policy: strict-origin-when-cross-origin, X-XSS-Protection: 1; mode=block`, and a `Permissions-Policy` disabling camera/microphone/geolocation. See [security.md](#).

Styling & UI

Package	Version	Role
<code>tailwindcss</code>	<code>^4.3.0</code>	Utility-first CSS (v4, CSS-first config)
<code>@tailwindcss/postcss</code>	<code>latest</code>	Tailwind's PostCSS plugin (the only PostCSS plugin configured)
<code>tw-animate-css</code>	<code>^1.4.0</code>	Animation utilities (dialogs/dropdowns), imported in <code>globals.css</code>
<code>shadcn</code>	<code>^3.8.5</code>	Component generator CLI (new-york style, neutral base)
<code>radix-ui</code>	<code>^1.6.0</code>	Unstyled primitives (meta-package) — used by newer components (tabs, tooltip, breadcrumb)
<code>@radix-ui/react-*</code>	see below	Individual Radix primitives underpinning shadcn components
<code>lucide-react</code>	<code>^0.539.0</code>	Icon set
<code>class-variance-authority</code>	<code>^0.7.1</code>	Variant-driven class composition (buttons, badges)
<code>clsx</code>	<code>^2.1.1</code>	Conditional class strings
<code>tailwind-merge</code>	<code>^3.6.0</code>	Dedupe/merge Tailwind classes (<code>cn()</code> helper)
<code>next-themes</code>	<code>^0.4.6</code>	Class-based dark mode (system default, 3-way toggle)
<code>sonner</code>	<code>^2.0.7</code>	Toast notifications
<code>prettier-plugin-tailwindcss</code>	<code>^0.6.14</code>	Sorts Tailwind classes on format

Individually installed Radix primitives: `@radix-ui/react-avatar ^1.1.11`, `@radix-ui/react-dialog ^1.1.15`, `@radix-ui/react-dropdown-menu ^2.1.16`, `@radix-ui/react-label ^2.1.8`, `@radix-ui/react-popover ^1.1.17`, `@radix-ui/react-slot ^1.2.4`.

Tailwind v4 is configured CSS-first — there is **no** `tailwind.config.ts`. `src/app/globals.css` starts with `@import "tailwindcss";` and `@import "tw-animate-css";`, then defines the theme in an `@theme inline { ... }` block that bridges CSS custom properties (colors, radius, shadows, sidebar tokens, chart palette, keyframe animations) to Tailwind utilities. Light-theme tokens are exact marketing-deck hex values; the `.dark` selector swaps them. `postcss.config.mjs` lists a single plugin, `@tailwindcss/postcss`. `shadcn` wiring lives in `components.json` (`style: new-york`, `rsc: true`, `baseColor: neutral`, `cssVariables: true`, `iconLibrary: lucide`, **CSS** → `src/app/`

`globals.css`).

The full visual system — tokens, type scale, spacing, component variants — is defined in `DESIGN.md`; see also `ui-ux.md`.

Fonts

Loaded via `next/font/google` in `src/app/layout.tsx`:

Font	CSS variable	Subset	Usage
Geist	<code>--font-geist-sans</code>	latin	All UI text
Geist Mono	<code>--font-geist-mono</code>	latin	Monospace/code
IBM Plex Sans Arabic	<code>--font-arabic</code>	arabic (weights 400/500/600/700)	Applied via <code>[dir="rtl"]</code> for the Arabic locale

Authentication

Package	Version	Role
<code>better-auth</code>	<code>^1.6.11</code>	Session/auth engine (email + password)
<code>@better-auth/api-key</code>	<code>^1.6.11</code>	API-key plugin (installed; not currently wired into <code>auth.ts</code>)

`src/lib/auth.ts` configures Better Auth with the **Drizzle adapter** (`drizzleAdapter(db, { provider: "pg" })`). Email/password sign-in is enabled; email verification is sent on sign-up. The base URL / trusted origin resolves from `BETTER_AUTH_URL` → `NEXT_PUBLIC_APP_URL` → `http://localhost:3000`.

Registration is invitation-only. A `databaseHooks.user.create.before` hook looks up a pending invite by email (`findPendingInviteByEmail`) and rejects sign-up with a `FORBIDDEN APIError` if none exists — this is also the enforcement point that blocks direct calls to the public sign-up endpoint. The matching invite determines the new user's `agencyId` and `role`; the `after` hook consumes the invite (`markInviteAccepted`). Application columns (`agencyId`, `isPlatformAdmin`, `role`, `active`) are exposed as `user.additionalFields` with `input: false`, so they ride along on the session but cannot be set by clients. Password-reset and verification emails are sent through the Resend-backed `sendEmail` helper (console fallback when Resend is unconfigured). Tenant/role authorization details are in `security.md` and `business-rules.md`.

Database & ORM

Package	Version	Role
---------	---------	------

drizzle-orm	^0.44.7	Type-safe query builder / schema
drizzle-kit	^0.31.10	Migration generation & runner (dev dependency)
postgres	^3.4.9	postgres-js driver used at runtime
pg	^8.20.0	node-postgres (installed alongside @types/pg; not the runtime driver)
@types/pg	^8.20.0	Types for pg

The runtime connection (`src/lib/db.ts`) uses the **postgres-js** driver:

`drizzle(postgres(PG_URL), { schema })`. The connection string is read through the validated env accessor (`getServerEnv()`), so a missing/malformed `PG_URL` fails fast with a clear message. The database is PostgreSQL, hosted on **Neon** in production.

`drizzle.config.ts` sets `dialect: "postgresql"`, `schema: "src/lib/schema.ts"`, `out: "drizzle"`, and reads `PG_URL` for credentials. Schema-change workflow (per `AGENTS.md`):

```
pnpm db:generate # drizzle-kit generate - write a new migration
pnpm db:migrate  # drizzle-kit migrate - apply migrations
```

Never run `drizzle-kit push` for schema changes. All non-BetterAuth ID columns are randomly generated UUIDs. The full schema and table catalog live in `database.md`.

Internationalization

Package	Version	Role
next-intl	^4.13.0	Message catalogs, formatting, request-scoped locale

`i18n` is **cookie-based with no URL-locale routing** — existing routes are unchanged. `src/i18n/config.ts` declares three locales: `en` (default), `fr`, and `ar` (right-to-left). The active locale is read from the `locale` cookie in `src/i18n/request.ts` (`getRequestConfig`), which loads `src/messages/{locale}.json`. Direction (`ltr/rtl`) is derived per-locale; the Arabic locale triggers `[dir="rtl"]` and the IBM Plex Sans Arabic font. The `next-intl` plugin is enabled in `next.config.ts`.

AI

Package	Version	Role
ai	^5.0.188	Vercel AI SDK core (<code>streamText</code> , <code>generateText</code> , <code>generateObject</code>)
@ai-sdk/react	^2.0.190	React hooks (<code>useChat</code>) for the assistant UI

@ai-sdk/google	^2.0.77	Google Gemini provider (primary). Must stay on the 2.x line to match ai@5's @ai-sdk/provider@2 — @ai-sdk/google@4 pulls provider@4 and is incompatible
@openrouter/ai-sdk-provider	^1.5.4	OpenRouter provider (fallback)
react-markdown	^10.1.0	Renders assistant markdown responses

The primary model provider is **Google Gemini** (`createGoogleGenerativeAI` from `@ai-sdk/google`, `GEMINI_API_KEY`, `model` from `GEMINI_MODEL`, default `gemini-2.5-flash`), with **OpenRouter** (`OPENROUTER_API_KEY` / `OPENROUTER_MODEL`) as the automatic fallback. The chat route (`src/app/api/chat/route.ts`) streams responses with `streamText` (Gemini if configured, else OpenRouter); the inline server actions in `src/lib/actions/ai.ts` use `generateObject` (Zod-typed structured output) + `generateText` behind `withAiFallback()` — a runtime chain of Gemini models then OpenRouter, for free-tier-quota resilience. AI features are disabled (with a startup warning) only when **neither** provider key is set. See `ai.md`.

External services & suppliers

All third-party APIs below are called over **raw fetch** — no vendor SDKs are installed. Each reads its credentials from `env` and `no-ops` (or returns sample data) when unconfigured.

Service	Package / transport	Purpose
Resend	<code>resend</code> ^6.14.0	Transactional email (invites, verification, password reset, proposal delivery)
Stripe (billing)	REST over <code>fetch</code> (no SDK)	Vendor → agency SaaS subscriptions
Stripe (payments)	REST over <code>fetch</code> (no SDK)	Traveler → agency booking payments via Connect (destination charges)
Duffel	REST over <code>fetch</code> (no SDK)	Flight search & booking (primary)
Amadeus	REST over <code>fetch</code> (no SDK)	Legacy flight fallback only
Hotelbeds	REST over <code>fetch</code> (no SDK)	Hotel availability & content (APITUDE)
Vercel Blob	<code>@vercel/blob</code> ^2.3.3	Optional file storage

Stripe

Two intentionally separate planes, both hitting `https://api.stripe.com/v1` directly via `fetch`:

- **Billing** (`src/lib/billing/stripe.ts`) — the platform charges each agency a recurring subscription. Active only when `STRIPE_SECRET_KEY` is set. Webhook signatures are verified manually with Node's `crypto` (`createHmac`, `timingSafeEqual`). Blocking subscription statuses (`canceled`, `unpaid`, `incomplete_expired`) lock an agency out.
- **Payments** (`src/lib/payments/stripe.ts`) — traveler → agency booking payments through Stripe Connect (destination charges), with a configurable platform fee (`STRIPE_PLATFORM_FEE_PERCENT`, default 5).

Duffel & Hotelbeds

- **Duffel** (`src/lib/suppliers/duffel.ts`, `providers/duffel-provider.ts`) — base URL `https://api.duffel.com`; every request carries a `Duffel-Version` header (from `DUFFEL_VERSION`, default v2).
- **Hotelbeds** (`src/lib/suppliers/hotelbeds.ts`, `providers/hotelbeds-provider.ts`) — base URL `https://api.hotelbeds.com`; every request carries an `Api-key` header plus an `x-Signature` (SHA-256 of key + secret + timestamp).

Full endpoint/env details are in `api-integrations.md`.

Documents, export & utilities

Package	Version	Role
<code>@react-pdf/renderer</code>	<code>^4.5.1</code>	Server-rendered proposal PDFs (<code>/proposal/[id]/pdf</code> , <code>/p/[token]/pdf</code> route handlers)
<code>exceljs</code>	<code>^4.4.0</code>	XLSX export (<code>src/lib/export/xlsx.ts</code>); CSV export is hand-rolled with a UTF-8 BOM for Excel/Power BI
<code>zod</code>	<code>^4.4.3</code>	Runtime validation — env schema, server-action inputs, AI structured output
<code>date-fns</code>	<code>^4.4.0</code>	Date math/formatting (date-range picker, etc.)
<code>react-day-picker</code>	<code>^10.0.1</code>	Calendar primitive behind the date-range picker

Tooling, quality & scripts

Package	Version	Role
<code>eslint</code>	<code>^9.39.4</code>	Linting (flat config)
<code>eslint-config-next</code>	<code>16.0.7</code>	Next.js core-web-vitals ruleset
<code>prettier</code>	<code>^3.8.3</code>	Formatting

tsx	^4.21.0	Run TS scripts (<code>scripts/*.ts</code> , <code>setup</code>)
vitest	^3.2.6	Unit-test runner (<code>vitest.config.ts</code> : <code>node env</code> , <code>@ alias</code> , <code>colocated src/**/*.test.ts</code>). Pinned to 3.x — vitest 4 pulls vite 8 whose transitive deps currently fail to install
@vitest/ coverage-v8	^3.2.6	Coverage provider

`eslint.config.mjs` extends `eslint-config-next/core-web-vitals` and adds project rules: **enforced** `import/order` grouping (`react/next/@/ ordered`), `no-console` (`warn`, `warn/error allowed`), `prefer-const`, `no-var`, and `eceqeq` (`always`, `null-ignored`). `.next`, `drizzle`, `scripts`, and `create-agentic-app` are ignored.

Key scripts (`package.json`):

```
pnpm dev          # next dev --turbo
pnpm build        # db:migrate, then next build
pnpm build:ci     # next build only (used by CI/Vercel)
pnpm lint         # eslint .
pnpm typecheck    # tsc --noEmit
pnpm check        # lint + typecheck
pnpm test         # vitest run (unit tests)
pnpm test:watch   # vitest watch mode
pnpm format       # prettier --write .
pnpm db:generate  # drizzle-kit generate
pnpm db:migrate   # drizzle-kit migrate
pnpm db:studio    # drizzle-kit studio
```

Environment variables are validated centrally in `src/lib/env.ts` (Zod). Only `POSTGRES_URL` is strictly required; everything else is optional and produces a startup warning when missing (see `checkEnv()`). Env-var reference: `api-integrations.md`.

Hosting & deployment

Deployed on **Vercel** with GitHub auto-deploy. `vercel.json` pins `framework: nextjs`, `buildCommand: npm run db:migrate && npm run build:ci` (migrations run before the build), and `installCommand: npm install --legacy-peer-deps`. Deployment specifics are in `deployment.md`.

Conventions tied to the stack

- **Styling** follows the design system in `DESIGN.md`; see `ui-ux.md`.
- **Working conventions** (sub-agents, UUID ids, Drizzle workflow, testing) live in `AGENTS.md`; see `coding-standards.md`.
- **External integrations** and their env vars are documented in `api-integrations.md`. Every one degrades gracefully when unconfigured.

- **Tailwind v4** is configured CSS-first via `@theme inline` in `globals.css` — there is no `tailwind.config.ts`.

Architecture

Tech choices live in tech-stack.md. This doc covers structure: multi-tenancy, auth, roles, the route map, the module layout, and i18n plumbing.

Multi-tenancy

Every business table carries `agencyId` (tenant roots) or inherits it through a parent (children). **All** reads/writes are scoped by agency, enforced in actions and pages via `requireAgencyUser() → user.agencyId`. References (`BKG-...`, `PRD-...`) are unique **per agency**. Re-verified by `scripts/test-tenant-isolation.ts`. See security.md for the isolation guarantees and database.md for tenancy columns.

Auth & onboarding

- Better Auth (`src/lib/auth.ts`) — email/password. The `user.create.before` hook makes signup **invitation-only**: it requires a pending `agency_invite` matching the email, stamps `agencyId` + role, and rejects everyone else (blocks the raw signup endpoint too). `BETTER_AUTH_URL/baseURL` + `trustedOrigins` set so the deployed domain is trusted.
- Guards (`src/lib/permissions.ts`): `requireUser`, `requireAgencyUser` (tenant + agency-suspension lockout), `requireManager`, `requireCapability`, `requirePlatformAdmin`.

Platform admin (vendor)

A user with `isPlatformAdmin = true`, `agencyId = null` — above all tenants, routed to `/platform`. Cannot enter a tenant app except via impersonation.

Impersonation (cookie-driven, platform-admin only)

- `viewAsAgency(agencyId) → cookie platform_view_agency → acts as agency admin`.
- `viewAsUser(userId) → cookie platform_view_user` (takes precedence) → adopts that user's identity, agency and **role** (full fidelity — an agent sees only their work).
- Resolved in `requireUser`; `user.impersonating = "agency" | "user" | null` drives the exit banner. `exitAgencyView()` clears both cookies.

Per-role landing

`roleHome(role)` routes `finance → /finance`, `support → /support`, `else /dashboard` (which itself adapts: agency-wide for admin/manager, scoped "Your work" for agents). App-shell nav is role-aware via a `show(role)` predicate per item.

Roles & permissions

Defined in `src/lib/domain.ts`. Full capability semantics are in `business-rules.md`.

Role	Sees all data	Team mgmt	Payments	Finance view	Support view	Delete	Home
admin	✓	✓	✓	✓	✓	✓	/dashboard
manager	✓	✓	✓	✓	✓	✓	/dashboard
finance	✓	—	✓	✓	—	—	/finance
support	✓	—	—	—	✓	—	/support
agent	own only	—	—	—	—	—	/dashboard (scoped)

Capability helpers: `seesAllData`, `canManageTeam`, `canAssignAdmin`, `canManagePayments`, `canViewFinance`, `canViewSupport`, `canDeleteRecords`, `roleHome`. Only an **admin** can assign/change the admin role.

Scale targets

The capacity Atlas is designed toward, with implications for the current architecture (single Neon Postgres + Vercel serverless).

Target	Implication / current state
1,000 agencies	Multi-tenant single DB; fine with <code>agencyId</code> indexes (present).
50,000 users	Better Auth + Postgres; fine at this scale.
5,000,000 bookings	Needs solid indexing + pagination (currently hardcoded <code>limits</code> — see tracker).
500,000,000 activities	<code>activity_log</code> at this size needs partitioning + archival/retention — not in place today (single unpartitioned table).
99.9% uptime	Relies on Vercel + Neon SLAs; no DR runbook yet (see <code>security.md</code>).
100 concurrent hotel searches	Bound by Hotelbeds quota; content cache serves photos quota-free. No concurrency/queue control or rate limiting yet.
100 concurrent flight searches	Bound by Duffel quota; <code>safeSearch</code> degrades to sample data on error. No queue/backpressure yet.

These are design targets. The gating work to reach them — table partitioning for `activity_log`, real pagination, rate limiting/backpressure on search, and a DR plan — is in the gap tracker.

Route map

Authenticated app (`(app) /`, gated by `requireAgencyUser`): `dashboard`, `finance`, `commissions` (`canViewFinance`), `support`, `bookings` (+ `new`, `[id]`, `[id]/edit`, `[id]/itinerary`), `clients` (+ `new`, `[id]`, `[id]/edit`), `opportunities` (**sales Pipeline board**) (+ `new`, `[id]`, `[id]/edit`), `products` (+ `new`, `[id]`, `[id]/edit`), `suppliers` (+ `new`, `[id]`, `[id]/edit`) (`canManageTeam`), `operations` (**bookings-by-status board**), `reports` (`canViewFinance` — BI export hub), `sourcing/flights` (live flight search), `hotels` (+ `[code]` **details**), `assistant`, `team` (`canManageTeam`), `billing` (admin-only), `settings`, `profile`.

Canonical URLs moved with the UI redesign; `next.config.ts` keeps old bookmarks working via `rewrites()/redirects()`:

- `/proposals` (+ `/proposals/*`) is the canonical URL, **rewritten** (not redirected) from the `products` route files — `/products*` **redirects** (permanent) to `/proposals*`.
- `/sourcing/hotels` (+ `/sourcing/hotels/*`) is the canonical URL, **rewritten** from the `hotels` route files.
- `/search` → `/sourcing/flights` (temporary redirect, during transition).
- `/hotels` (+ `/hotels/*`) → `/sourcing/hotels*` (temporary redirect).
- `/operations` → `/bookings` (temporary redirect; the board-view toggle lives there).

Client portal (`portal/`, gated by `requirePortalSession`): `portal` (trip list), `portal/bookings/[id]` (detail + pay), `portal/proposals` (list), `portal/proposals/[id]` (view + sign), `portal/login`.

Platform (`platform/`, gated by `requirePlatformAdmin`): `platform`, `platform/agencies/new`, `platform/agencies/[id]`.

Auth (`(auth) /`): `login`, `register` (invite-only notice), `forgot-password`, `reset-password`. **Accept invite:** `invite/[token]`.

Public / docs: `i/[token]` (shareable itinerary, unauth), `p/[token]` (public signable proposal) + `p/[token]/pdf`, `proposal/[id]` (internal preview) + `proposal/[id]/pdf`, `booking-docs/[id]/voucher`, `booking-docs/[id]/invoice`.

API: `api/auth/[...all]` (Better Auth), `api/chat` (AI assistant), `api/export/[entity]` (BI export: CSV/XLSX per dataset + `workbook` = all sheets), `api/stripe/webhook` (subscription reconciliation), `api/stripe/connect-webhook` (Connect payment reconciliation), `api/stripe/connect` (initiate Connect onboarding), `api/stripe/connect/refresh` · `api/stripe/connect/return` (onboarding link callbacks), `api/portal/auth/request` · `verify` · `signout` (portal magic-link flow), `api/cron/cleanup` (daily maintenance sweep — expired idempotency/portal-session/ invite rows; see `deployment.md`).

Module layout

src/lib/

- `domain.ts` — roles, capabilities, enums (statuses, stages, item types, currencies + controlled

vocabularies: lead source, travel purpose, trip type, gender, title, lost reason, industry),
`roleHome`, `status/role` metadata.

- `analytics.ts` — pure dashboard helpers (see `analytics.md`).
- `reference/countries.ts` — ISO 3166-1 list (name, demonym, derived flag) + `normalizeCountry()` alias mapping.
- `export/` — `csv.ts` (RFC-4180 + UTF-8 BOM), `xlsx.ts` (exceljs workbook), `datasets.ts` (BI dataset registry with dual code/label columns).
- `permissions.ts` — auth guards + impersonation resolution.
- `auth.ts` / `auth-client.ts` — Better Auth config + client.
- `invites.ts` — create/find/accept invite tokens (7-day TTL).
- `queries.ts` — shared agency-scoped pickers.
- `activity.ts` — `logActivity` (agency-scoped audit).
- `db.ts` (validates env via `getServerEnv`), `schema.ts`, `env.ts`, `config.ts`, `format.ts`, `utils.ts`, `itinerary.ts`, `storage.ts`.
- `notifications/email.ts` (Resend adapter) + `notifications/templates.ts` (HTML)
 - `notifications/inbox.ts` (in-app inbox emitter — best-effort per-user notifications for the topbar bell; `createUserNotifications`, `staffToNotify`, event helpers for proposal accept/decline + payment received).
- `billing/stripe.ts` — SaaS subscriptions (distinct from `payments/stripe.ts`).
- `payments/stripe.ts` — Stripe Connect: account, onboarding link, destination charges.
- `suppliers/` — `index.ts` (`getFlightSupplier/getHotelSupplier` + `safeSearch`, `legacy`), `duffel.ts`, `hotelbeds.ts`, `content-cache.ts`, `amadeus.ts` (`legacy`), `mock.ts`, `types.ts`, `config.ts` (single entry point for supplier credential/hostname env resolution), `booking-service.ts` (quote → book → cancel orchestration over the registry). `suppliers/providers/` — the provider abstraction: `types.ts` (capability interfaces), `registry.ts` (`providerRegistry`, `capability` type-guards, `PROVIDER_CATALOG`), `register.ts` (`registerBuiltInProviders()`), `mock-provider.ts`, `duffel-provider.ts`, `hotelbeds-provider.ts`.
- `travel-platform/index.ts` — the Travel Platform facade: single entry point for `searchFlights`, `searchHotels`, `searchAirports`, `searchHotelDestinations` routed through the provider registry. See `api-integrations.md`.
- `documents/proposal-pdf.tsx` + `proposal-data.ts` — proposal PDF rendering.
- `portal-session.ts` — `getPortalSession` / `requirePortalSession` (`httpOnly` cookie).

src/lib/actions/ (server actions, all agency-scoped): `clients`, `opportunities`, `products`, `proposals-public`, `bookings`, `payments`, `notifications`, `team`, `invites`, `platform`, `billing`, `settings`, `search`, `suppliers`, `commissions`, `portal-payments`, `portal-proposals`, `portal-invite`, `onboarding`, `ai`, `user-notifications` (bell inbox: `getMyNotifications`, `markNotificationRead`, `markAllNotificationsRead` — double-scoped by recipient + agency).

src/components/ — `app/` (shell, page-header, stat-card, status-badge, getting-started-card), `charts/` (insight-charts incl. `HBarInsight`, `FunnelInsight`), `reference/` (country-combobox, city-input), `reports/`, `settings/`, `team/`, `platform/`, `auth/`, `bookings/` (lifecycle stepper, search sheet, board), `clients/` (portal invite, timeline), `products/` (convert-to-booking, AI quote builder),

opportunities/, billing/, commissions/, suppliers/, portal/, search/, documents/, ui/ (shadcn).

Internationalization

- **Locales:** `en`, `fr`, `ar` (Arabic = RTL). Cookie-based (`locale`), no URL-locale routing — all routes unchanged.
- `src/i18n/` — `config.ts` (`locales`, `metadata`, `dir`), `request.ts` (next-intl request config reading the `locale` cookie). Messages: `messages/{en,fr,ar}.json`.
- Root layout sets `<html lang dir>` and applies IBM Plex Sans Arabic for RTL (`html[dir="rtl"] body` in `globals.css`).
- Translated surfaces: `nav`, `login`, `dashboard`, `settings`. Others fall back to English. **To translate a string:** add the key to all three `messages/*.json` and swap to `t("...")` (`getTranslations` in `server`, `useTranslations` in `client`).
- Language is changed in **Settings**; choice is saved to `user.locale` and the cookie. **Known gap:** a fresh device without the cookie shows English until the user re-picks — `user.locale` isn't yet synced to the cookie on login (see `roadmap.md` open items).

Domain Model

The core business entities and how they relate. This is the **data/relationship** view; the process view (states, who does what) is the golden workflow. Tables and columns live in `database.md`; the full column-by-column listing is in `schema-reference.md`. All tables are declared in `src/lib/schema.ts`; the shared status/type vocabularies live in `src/lib/domain.ts`.

The chain

```
Lead → Client → Opportunity → Proposal → Booking → Supplier →
Invoice → Payment → Accounting → Travel → Feedback
```

The **client is the aggregate root** — everything hangs off it, and every business entity is scoped to an `agencyId` (founding principle #2). The chain is partly aspirational: three links (Invoice, Accounting, Feedback) are not yet first-class entities — see Where the chain is aspirational.

Entities

Domain concept	Table(s)	Status	Key relations
Lead	<code>client</code> (<code>status lead</code> , <code>source code</code>)	✓	becomes a Client; no separate table
Client	<code>client</code> (+ <code>client_contact</code>)	✓	root; owns opportunities, proposals, bookings, portal sessions
Opportunity	<code>opportunity</code>	✓	belongs to a client; <code>assignedToId</code> ; links to a proposal
Proposal	<code>product</code> (+ <code>product_item</code>)	✓	belongs to client + opportunity; e-sign; auto-converts → Booking on accept (<code>convertedBookingId</code> idempotency latch)
Booking	<code>booking</code> (+ <code>booking_traveller</code> , <code>booking_item</code> , <code>booking_day</code> , <code>payment</code> , <code>booking_supplier_ref</code> , <code>booking_event</code> , <code>booking_document</code> , <code>booking_idempotency</code>)	✓	belongs to client; lifecycle states; items reference suppliers; Sprint 1 tables power real booking, audit log, and idempotency
Supplier	<code>supplier</code> (+ <code>supplier_contract</code> , <code>supplier_rate</code>)	✓	referenced by <code>booking_item.supplierId</code> + <code>product_item.supplierId</code>

Invoice	— (on-demand PDF at <code>/booking-docs/[id]/invoice</code>)	●	no managed invoice entity yet (planned module)
Payment	<code>payment</code> (child of booking)	✓	deposits/installments/refunds; Stripe Connect
Accounting	<code>commission</code> ledger only	●	partial; no GL/accounting entity (planned module)
Travel	<code>booking</code> (state <code>completed</code>) + <code>booking_day</code> + <code>booking_document</code>	✓	itinerary timeline; vouchers/tickets
Feedback	— (<code>activity_log</code> / notes)	●	no review/feedback entity (planned module)

The supporting tables — `agency`, `agency_invite`, `user`, `session`, `account`, `verification`, `portal_session`, `notification`, `activity_log`, `hotel_content` — are documented in Supporting entities below.

Entity lifecycle chain (Lead → ... → Feedback)

Each hop advances the same client relationship through the golden workflow. The status/type vocabularies below are the canonical values from `src/lib/domain.ts`; they are stored as plain text codes (never renamed — only appended) so historical reporting stays stable.

Lead → Client

There is **no separate lead table**. A lead is a `client` row whose `status` is `lead` (`CLIENT_STATUSES` = `lead` | `active` | `inactive`). The `source` column records acquisition via a `LEAD_SOURCES` code (`referral`, `website`, `instagram`, `facebook`, `walk_in`, `partner`, `event`, `outbound`, `repeat`, `other`). A corporate client also carries an `industry` code (`INDUSTRIES`). Qualifying a lead is a status change on the same row — the relationship (and its `id`) is preserved.

Client → Opportunity

An `opportunity` is a deal in the sales pipeline. It **must** belong to a client (`clientId` is `NOT NULL`, `onDelete`: `cascade`) and progresses through `OPPORTUNITY_STAGES` (`lead` → `qualified` → `proposal` → `booked` → `won` → `lost`). Each stage carries a `defaultProbability`; the open pipeline is `OPEN_STAGES` (`lead`, `qualified`, `proposal`, `booked`). A lost deal records a `lostReason` code (`LOST_REASONS`) for win/loss analysis, and a `travelPurpose` code (`TRAVEL_PURPOSES`) drives segmentation. `value` + `currency` size the deal; `assignedToId` is the owning agent.

Opportunity → Proposal

A `product` is the proposal — a priced travel package assembled from supplier offers. It optionally links back to both a `clientId` and an `opportunityId` (both `onDelete`: `set null`), so a proposal survives if the deal is deleted. Statuses are `PRODUCT_STATUSES` (`draft` → `sent` → `accepted` →

rejected → expired). Line items live in `product_item` (`PRODUCT_ITEM_TYPES`: `flight`, `hotel`, `activity`, `transfer`, `insurance`, `other`), each optionally tied to a managed `supplierId`. **Pricing:** stored truth is `unitCost/unitPrice` per item; totals are plain sums ($\sum \text{unitCost} \times \text{qty} / \sum \text{unitPrice} \times \text{qty}$ — no multiplier). `markupPercent` on the parent is the proposal's **default margin**: it seeds newly added items (incl. AI-quote items) and the "apply to all items" control, but never retroactively reprices existing lines. In the builder, each line's margin is a two-way *input device* (margin ↔ sell price, derived — never stored; pure math in `src/lib/pricing.ts`), and an explicit **Apply to all items** button rewrites every line's price from its own cost (zero-cost lines skipped). The `reference` (e.g. PRD-1042) is unique per agency (`product_agency_reference_unique`).

E-signature acceptance. A proposal is shared via an unguessable `shareToken` at the public route `/p/[token]`. On accept/decline the row captures `acceptedAt` or `declinedAt` plus non-repudiation fields: `signerName`, `signerEmail`, `signatureData` (typed-name or drawn signature), `signerIp`, `signerUserAgent`.

convertedBookingId. A nullable `uuid` FK to `booking(id)` (`onDelete: set null`) that latches the booking a proposal was converted into. It is the **idempotency guard** for the accept → auto-booking flow: once set, the shared `createBookingFromAcceptedProposal` helper returns the existing booking id as a no-op, so a proposal can never spawn two bookings (re-accept, double-submit, or the manual convert button are all safe). The `productRelations.convertedBooking` relation exposes the link.

Proposal → Booking

A `booking` is the booking file for one trip. On proposal accept + e-sign it is **auto-created** at status `awaiting_payment` (best-effort, tenant-scoped to the proposal's `product.agencyId`, idempotent via `product.convertedBookingId`); the agent `convertProposalToBooking` action is the one-click fallback that delegates to the same helper. `clientId` is `onDelete: set null` (the file survives client deletion). The **operational lifecycle** is `BOOKING_LIFECYCLE` from `src/lib/domain.ts`:

Status	Meaning
<code>draft</code>	Being assembled
<code>awaiting_payment</code>	Priced, waiting on client payment
<code>confirmed</code>	Client committed; supplier booking in progress
<code>ticketed</code>	Supplier orders issued (e-tickets/vouchers)
<code>completed</code>	Travel done
<code>cancelled</code>	Terminated (outside the ordered lifecycle)

`nextBookingStatus()` advances one step along `BOOKING_LIFECYCLE`. Advancing to `ticketed` has a hard prerequisite — items are booked with the supplier first, and the advance aborts if confirmation fails (see `src/lib/actions/bookings.ts`). `travelPurpose` (`TRAVEL_PURPOSES`) and `tripType` (`TRIP_TYPES`: `one_way`, `round_trip`, `multi_city`) classify the trip. `reference` (e.g.

BKG-1042) is unique per agency; a `shareToken` powers the public itinerary at `/i/[token]`.

Doc note: the inline comment on `booking.status` in `schema.ts` (`draft | confirmed | paid | cancelled`) is stale — the live six-state vocabulary is the `BOOKING_STATUSES` constant in `src/lib/domain.ts`.

Booking children:

Child table	Purpose
<code>booking_traveller</code>	Travellers + passport data; <code>title/gender</code> codes (<code>TITLES/GENDERS</code>); <code>email/phone</code> required by supplier APIs on the lead pax (<code>isLead</code>)
<code>booking_item</code>	Purchased lines (<code>BOOKING_ITEM_TYPES</code> : <code>flight</code> , <code>hotel</code> , <code>transfer</code> , <code>excursion</code> , <code>insurance</code> , <code>fee</code> , <code>other</code>); <code>itemStatus = `pending`</code>
<code>booking_day</code>	Per-day title/notes for the itinerary timeline
<code>payment</code>	Deposits, installments, payments, refunds (see below)
<code>booking_supplier_ref</code>	Structured supplier confirmation (replaces untyped JSONB)
<code>booking_event</code>	Append-only event log (audit + analytics)
<code>booking_document</code>	Generated vouchers/tickets/invoices/itineraries/receipts
<code>booking_idempotency</code>	Idempotency-key registry for supplier calls

Booking → Supplier

Items reference the agency's managed `supplier` directory via `booking_item.supplierId` (and `product_item.supplierId`); a free-text `supplier` string remains for ad-hoc entries.

`SUPPLIER_TYPES` are `hotel`, `airline`, `car_rental`, `transfer`, `dmc`, `insurance`, `other`. Commercial terms live in `supplier_contract` (`commissionBasis = percent | fixed | net`, `validity window`, `fileUrl`) with structured `supplier_rate` rows (`netRate/sellRate`). Actual supplier confirmations are captured on `booking_supplier_ref:providerId` (`duffel`, `hotelbeds`, `mock`, ...), `confirmationNumber`, `pnr`, `supplierOrderId`, and the full `rawPayload`.

Booking → Invoice (aspirational)

There is **no managed invoice entity**. Invoices are rendered on-demand as PDFs at `/booking-docs/[id]/invoice` (voucher counterpart at `/booking-docs/[id]/voucher`). A generated document may be persisted as a `booking_document` row of `type invoice`. Turning this into a numbered, ledgered record is the Accounting module.

Booking → Payment

`payment` records money against a booking (`onDelete: cascade`). `kind = deposit | installment | payment | refund` (`PAYMENT_KINDS`); `method = manual | card | transfer | cash | stripe` (`PAYMENT_METHODS`); `status = pending | completed | failed | refunded`. **Stripe Connect**

(traveler → agency) fields — `stripeSessionId`, `checkoutUrl` — reconcile hosted Checkout via webhook. This is distinct from the agency's SaaS billing on `agency` (vendor → agency).

Payment → Accounting (partial)

The only accounting primitive is the `commission ledger` (`COMMISSION_TYPES`):

Type	Direction	Links
<code>supplier_to_agency</code>	Agency earns from a supplier for a booking item	<code>supplierId</code> + <code>bookingItemId</code>
<code>agency_to_agent</code>	Agent earns from the agency for a booking	<code>agentUserId</code> + <code>bookingId</code>

`basis = percent | fixed`; `status = pending | earned | invoiced | paid | void` (`COMMISSION_STATUSES`). Commission auto-generates on booking confirm/ticket; no general ledger exists.

Travel → Feedback (aspirational)

Travel is the `booking` reaching `completed`, with the itinerary rendered from `booking_day` + `booking_item` and vouchers/tickets in `booking_document`. **Feedback is not modelled** — "Travel completed → request review" is an unbuilt automation trigger. Today only `activity_log` and free-text `notes` capture post-trip signal.

Aggregates & ownership

An aggregate is a root plus the children that share its lifecycle and are only ever loaded/mutated through it.

- **Client aggregate** — `client` → `client_contact`; the spine that opportunities, proposals, bookings, and portal sessions all reference by `clientId`. `ownerId` is the agent who owns the relationship (`relationName: "client_owner"`).
- **Opportunity** — a first-class root scoped by `agencyId`, but conceptually a child of the client aggregate (its `clientId` cascade-deletes with the client).
- **Proposal aggregate** — `product` → `product_item`. Root scoped by `agencyId`; soft-linked (`set null`) to client + opportunity so it survives their deletion.
- **Booking aggregate** — `booking` → `booking_traveller`, `booking_item`, `booking_day`, `payment`, `booking_supplier_ref`, `booking_event`, `booking_document`, `booking_idempotency`. Most children carry **no** `agencyId`; they inherit tenancy through the parent booking (scoped via the parent on every query). The exception is `booking_event`, which **denormalizes** `agencyId` for direct tenant-scoped analytics queries. `booking_event` is append-only (audit + analytics); `booking_idempotency` (PK = the derived key) prevents duplicate supplier orders on retry, with a 24 h `expiresAt` TTL.
- **Supplier aggregate** — `supplier` → `supplier_contract` → `supplier_rate`. `supplier_contract` denormalizes `agencyId` for efficient agency-scoped queries;

`supplier_rate` inherits tenancy through its contract. `commission` rows link a booking item back to the earning supplier/agent — but are **not** part of the booking aggregate's cascade: `commission.bookingId/bookingItemId` are `onDelete: set null` (migration 0021), so the ledger survives a booking or item being deleted (financial history is never silently destroyed by a cascade). Its `agencyId` FK still cascades with the tenant. See decision 0007.

- **Reference data** — `hotel_content` is **global** (not tenant-scoped): its PK is the Hotelbeds hotel code (a string), and it is shared vendor reference data like a currency list. ISO countries/controlled vocabularies are canonical lookups in `src/lib/domain.ts`, not per-agency rows.

Supporting entities

Table	Role
<code>agency</code>	The tenant. <code>status</code> (<code>active/suspended</code>); SaaS-billing (Stripe customer/subscription) + Stripe Connect columns; <code>onboardingDismissedAt</code>
<code>agency_invite</code>	Invitation-only registration: <code>email</code> , <code>role</code> , <code>unguessable_token</code> , <code>status</code> (<code>pending/accepted/revoked</code>), <code>expiresAt</code>
<code>user</code>	Team member (Better Auth). <code>agencyId</code> (NULL only for the platform super-admin), <code>role</code> , <code>active</code> , <code>locale</code> , <code>commissionRatePercent</code> , <code>isPlatformAdmin</code>
<code>session</code> , <code>account</code> , <code>verification</code>	Better Auth staff sessions/credentials (text ids — managed by Better Auth)
<code>portal_session</code>	Passwordless client portal session (separate from Better Auth): 15 min magic token rotated to a 7-day session token; scoped to one <code>clientId</code>
<code>notification</code>	Communications log: <code>channel</code> (<code>email/sms/whatsapp/push</code>), <code>kind</code> , <code>status</code> , optional <code>bookingId</code> — outbound messages, distinct from the in-app inbox below
<code>user_notification</code>	Per-user in-app inbox (topbar bell): recipient <code>userId</code> , <code>type</code> <code>code</code> (<code>proposal_accepted/proposal_declined/payment_received/booking_created</code>), <code>title/body/href</code> , <code>readAt</code> (<code>null = unread</code>)
<code>activity_log</code>	Manager-oversight audit trail: <code>action</code> , <code>entityType</code> , <code>entityId</code> , <code>entityLabel</code> , <code>metadata</code>
<code>hotel_content</code>	Global Hotelbeds content cache (photos, facilities, coords); not tenant-scoped

Tenant scoping

Every business table (except the global `hotel_content`) carries a tenancy path back to `agency`, enforced at the application layer — Postgres has no row-level security here.

- **Root tables** carry `agencyId uuid NOT NULL REFERENCES agency(id) ON DELETE CASCADE`:

`client`, `opportunity`, `product`, `booking`, `supplier`, `notification`, `activity_log`, `commission`, `agency_invite`, plus the denormalized `agencyId` on `supplier_contract` and `booking_event`.

- **Child tables** inherit tenancy through their parent (no `agencyId` column) and cascade-delete with it: `client_contact`, `product_item`, `booking_traveller`, `booking_item`, `booking_day`, `payment`, `booking_supplier_ref`, `booking_document`, `booking_idempotency`, `supplier_rate`, `portal_session`.
- **`user.agencyId` is nullable** — NULL identifies the platform super-admin (vendor), who lives above all tenants (`isPlatformAdmin`).
- **Uniqueness is per-agency, not global:** `product` and `booking` references are enforced by composite unique constraints (`product_agency_reference_unique`, `booking_agency_reference_unique`); `supplier` names by `supplier_agency_name_unique`. Composite (`agencyId`, ...) indexes back the tenant-scoped list/filter/order queries.

Enforcement. Server-side, `requireAgencyUser()` in `src/lib/permissions.ts` resolves the session, guarantees a non-null `agencyId`, and blocks suspended agencies / lapsed subscriptions; every tenant-scoped query then filters by that `agencyId` (see `src/lib/queries.ts`). The platform admin can "view as" an agency or a specific user via the `platform_view_agency` / `platform_view_user` cookies. Within an agency, `FULL_VISIBILITY_ROLES` (`admin`, `manager`, `finance`, `support`) see all agency data while `agent` sees only their own work (`seesAllData()` in `src/lib/domain.ts`).

Where the chain is aspirational

Three links are not yet first-class domain entities — they're the bridge to the planned modules:

- **Invoice** — today an on-demand PDF; the Accounting module turns it into a numbered, ledgered record.
- **Accounting** — only the two-ledger `commission` exists; no general ledger.
- **Feedback** — not modelled; "Travel completed → request review" is an unbuilt automation trigger.

See the entity standard for the baseline fields every one of these should carry. Note the two open gaps recorded there: there is **no `deletedAt` soft-delete column** anywhere (deletes are currently real), and a human-facing `reference` exists only on `booking` and `product`.

Database

Schema lives in `src/lib/schema.ts`. PostgreSQL (Neon) + Drizzle ORM. Tenancy column shown where present; see `architecture.md` and `security.md` for how scoping is enforced.

Entity standard

Every business entity should carry this baseline. It operationalizes the never rules (one source of truth, no hard delete, no cross-tenant exposure) at the schema level.

Field	Purpose	Current state
UUID	Primary key, randomly generated (non-Better-Auth IDs)	✓ convention in place
Reference	Human-facing id unique per agency (BKG-..., PRD-...)	⚠ only <code>booking</code> + <code>product</code> today
createdAt	Creation timestamp	✓ widespread
updatedAt	Last-modified timestamp	⚠ on roots, missing on some children
createdBy	<code>createdById</code> → user	✓ on main entities
agencyId	Tenant scope (root) or inherited (child)	✓ enforced everywhere
Status	Lifecycle/state enum	⚠ per-entity, not universal
Activity log	Audit trail of changes	⚠ shared <code>activity_log</code> table, not wired for every entity
Soft delete	<code>deletedAt</code> — never hard delete	✗ no soft-delete column exists yet
Notes	Free-form internal notes	⚠ only some entities

Gap to close: soft delete is **not implemented** — there is no `deletedAt` anywhere, so the "never hard delete" rule is currently aspirational. Adding a nullable `deletedAt` + filtering it out of reads (and a `reference` on the remaining entities) would need a migration. Until then, deletes are real.

Tables

Table	Tenancy	Notes
-------	---------	-------

agency	(root)	name, slug, status (active/suspended); Stripe billing : stripeCustomerId, stripeSubscriptionId, subscriptionStatus, priceId, currentPeriodEnd, trialEndsAt; Connect : stripeConnectAccountId, stripeConnectOnboarded; onboardingDismissedAt (getting-started card); depositPercent numeric(5,2) NOT NULL default 50 — % of total required to reach confirmed (also drives the proposal deposit line)
agency_invite	agencyId	email, role, token, status, expiresAt
user	agencyId (nullable)	+ isPlatformAdmin, role, active, locale, commissionRatePercent (Better Auth)
session, account, verification	via user	Better Auth
portal_session	via client	passwordless client portal sessions; token + expiresAt; purpose ('magic' 'session', default 'session') distinguishes a short-lived magic-link token from a real session — a magic token can no longer be used as a session bearer; swept daily by the cleanup cron (see deployment.md)
client	agencyId	+ client_contact (child); source (lead-source code), industry (code), country stored as canonical ISO name
opportunity	agencyId	pipeline stage, value, currency; travel_purpose code; lost_reason code
product	agencyId	proposal; ref unique per agency; depositPercent nullable numeric(5,2) — per-deal deposit override (null = inherit agency default); + product_item (child; supplierId FK optional); e-sign : shareToken (unique), acceptedAt/declinedAt, signerName/Email, signatureData, signerIp/UserAgent
booking	agencyId	ref unique per agency; shareToken; travel_purpose + trip_type codes; depositPercent nullable numeric(5,2) — effective deposit % snapshotted at proposal→booking conversion (null = inherit agency default)
booking_traveller (+ title, gender codes), booking_item (+ supplierId FK), payment, booking_day	via booking	children; booking_traveller carries email + phone (required by supplier APIs for the lead passenger); booking_item.unit_cost nullable numeric(12,2) — net supplier cost per unit (null = unknown, sell price is amount, margin derived never stored; snapshotted from product_item.unitCost at proposal→booking conversion)

booking_supplier_ref	via booking + booking_item	structured supplier confirmation: <code>providerId</code> , <code>confirmationNumber</code> , <code>pnr</code> , <code>supplierOrderId</code> , <code>rawPayload</code> ; replaces untyped JSONB in <code>booking_item.details</code>
booking_event	bookingId + agencyId	append-only event log (audit + analytics); stable event codes: <code>search_initiated</code> , <code>offer_selected</code> , <code>price_validated</code> , <code>price_changed</code> , <code>booking_submitted</code> , <code>booking_confirmed</code> , <code>booking_failed</code> , <code>booking_cancelled</code> , <code>payment_started</code> , <code>payment_completed</code>
booking_document	via booking (+ optional booking_item)	documents: <code>type</code> (voucher/ticket/invoice/itinerary/receipt), <code>url</code> (Vercel Blob), <code>rawData</code> (supplier payload for re-generation)
booking_idempotency	via booking (+ optional booking_item)	idempotency key registry; key = <code>sha256(bookingId+itemId+offerId)</code> ; prevents duplicate supplier orders on retry; <code>expiresAt</code> (24 h TTL); a <code>pending</code> row is treated as in-flight (never re-called) and expired rows are excluded from cache reads; swept daily by the cleanup cron (<code>GET /api/cron/cleanup</code> , see deployment.md)
notification	agencyId	comms log (outbound email/sms — distinct from the in-app inbox below)
user_notification	agencyId	per-user in-app inbox (topbar bell): recipient <code>userId</code> , stable type codes (<code>proposal_accepted</code> <code>proposal_declined</code> <code>payment_received</code> <code>booking_created</code>), <code>title/body/href</code> , <code>readAt</code> (null = unread); indexes (<code>userId</code> , <code>readAt</code>) + (<code>agencyId</code> , <code>createdAt</code>)
activity_log	agencyId	audit trail
supplier	agencyId	managed supplier directory (hotels, airlines, DMC, etc.); <code>updatedAt</code> is app-managed via <code>\$onUpdate</code>
supplier_contract	agencyId	commission basis/rate, validity dates, file URL; <code>updatedAt</code> is app-managed via <code>\$onUpdate</code>
supplier_rate	via contract	structured per-product rates within a contract
commission	agencyId	earnings ledger: supplier→agency and agency→agent; <code>bookingId</code> , <code>supplierId</code> , <code>agentUserId</code> , <code>basis</code> , <code>rate</code> , <code>amount</code> , <code>status</code> . <code>bookingId/bookingItemId</code> are <code>onDelete: set null</code> (not cascade) — the ledger is financial history and survives booking/item deletion; the tenant <code>agencyId</code> FK still cascades. <code>updatedAt</code> is app-managed via <code>\$onUpdate</code>

<code>hotel_content</code>	(global)	Hotelbeds content cache (photos, facilities, coords) — shared reference data, not tenant-scoped; PK is the Hotelbeds hotel code
----------------------------	----------	--

ID convention

All ID columns **not** related to Better Auth use UUIDs, randomly generated. See coding-standards.md.

Migrations

Migration	What it adds
0006	Tenancy + backfill (<code>agencyId</code> on all tables)
0007	<code>agency_invite</code> table
0008	<code>user.locale</code>
0009	Agency Stripe billing columns
0010	Product e-signature columns
0011	<code>hotel_content</code> cache
0012	Stripe Connect columns on <code>agency</code>
0013	<code>portal_session</code> table
0014	<code>supplier</code> , <code>supplier_contract</code> , <code>supplier_rate</code> tables; <code>supplierId</code> FK on <code>booking_item</code> + <code>product_item</code>
0015	<code>commission</code> table; <code>user.commissionRatePercent</code>
0016	<code>agency.onboardingDismissedAt</code>
0017	Controlled-vocab columns: <code>client.industry</code> , <code>opportunity.travel_purpose</code> , <code>booking.travel_purpose/trip_type</code> , <code>booking_traveller.title/gender</code> (all nullable, additive)
0018	<code>commission.bookingItemId</code> FK re-pointed at <code>booking_item</code> (was left dangling after an earlier rename); perf indexes: <code>booking_agency_status_idx</code> , <code>booking_agency_created_idx</code> , <code>booking_item_supplier_idx</code> , <code>client_agency_created_idx</code> , <code>commission_agency_status_idx</code> , <code>commission_agency_created_idx</code> , <code>notification_agency_created_idx</code> , <code>opportunity_agency_stage_idx</code> , <code>opportunity_agency_created_idx</code> , <code>payment_created_idx</code> , <code>product_agency_status_idx</code> , <code>product_agency_created_idx</code> , <code>product_item_supplier_idx</code> , <code>verification_identifier_idx</code>

0019	Sprint 1 booking architecture: <code>booking_supplier_ref</code> , <code>booking_event</code> , <code>booking_document</code> , <code>booking_idempotency</code> tables ; <code>booking_traveller.email</code> + <code>.phone</code> columns
0020	<code>product.convertedBookingId</code> (FK → <code>booking</code> , set <code>null</code>) — the proposal→booking idempotency latch used by auto-booking on accept (see decision 0006)
0021	<code>commission.bookingId</code> + <code>commission.bookingItemId</code> FKs changed <code>cascade</code> → <code>set null</code> (the commission ledger now survives booking/item deletion — see decision 0007); new indexes <code>commission_item_idx</code> , <code>commission_supplier_idx</code> , <code>product_converted_booking_idx</code> ; <code>portal_session.purpose</code> ('magic' 'session', default 'session')
0022	<code>user_notification</code> table — per-user in-app notification inbox (topbar bell)
0023	<code>agency.depositPercent</code> — deposit-threshold booking confirmation (decision 0009)
0024	<code>Nullable product.depositPercent</code> + <code>booking.depositPercent</code> — per-deal deposit override chain, snapshotted onto the booking at conversion
0025	<code>Nullable booking_item.unit_cost</code> — margin visibility on directly created bookings (snapshotted from proposal items on conversion)

Migrations 0000–0005 predate the multi-tenant rework (the pre-tenancy base schema) and are not itemized here.

Workflow

```
db:generate → db:migrate # NEVER db:push
```

Run on prod after deploy: `POSTGRES_URL=<prod-url> npx drizzle-kit migrate`. Branches: `dev` `ep-wandering-sunset-aitlty78` · `prod` `ep-misty-thunder-aixz34vy`. Full setup in `development-guide.md`.

Schema Reference

Full per-table, per-column reference. Schema is Drizzle ORM + PostgreSQL (Neon). Source of truth: `src/lib/schema.ts`. High-level overview: `database.md`.

Convention — `id` is always `uuid PRIMARY KEY DEFAULT gen_random_uuid()` on business tables unless noted. Better Auth tables use `text` PKs. All timestamps are `timestampz`.

Auth — Better Auth managed

user

Column	Type	Constraints	Notes
<code>id</code>	<code>text</code>	PK	Better Auth managed
<code>name</code>	<code>text</code>	NOT NULL	Display name
<code>email</code>	<code>text</code>	NOT NULL UNIQUE	Login credential
<code>email_verified</code>	<code>boolean</code>	NOT NULL DEFAULT false	
<code>image</code>	<code>text</code>		Avatar URL
<code>agency_id</code>	<code>uuid</code>	FK → <code>agency</code> CASCADE	null = platform admin or not yet onboarded
<code>is_platform_admin</code>	<code>boolean</code>	NOT NULL DEFAULT false	Vendor console access
<code>role</code>	<code>text</code>	NOT NULL DEFAULT "agent"	admin manager finance support agent
<code>locale</code>	<code>text</code>		"en" "fr" "ar" — UI language preference
<code>active</code>	<code>boolean</code>	NOT NULL DEFAULT true	Soft-disable without deleting history
<code>commission_rate_percent</code>	<code>numeric(5,2)</code>		Default commission % this agent earns
<code>created_at</code>	<code>timestamp</code>	NOT NULL DEFAULT now()	
<code>updated_at</code>	<code>timestamp</code>	NOT NULL	Auto-set on update

Indexes: user_email_idx, user_agency_idx

session

Column	Type	Notes
id	text	PK
expires_at	timestamp	NOT NULL
token	text	NOT NULL UNIQUE
ip_address	text	
user_agent	text	
user_id	text	FK → user CASCADE
created_at	timestamp	NOT NULL
updated_at	timestamp	NOT NULL

account

OAuth/credential accounts linked to a user. Better Auth managed.

Column	Type
id	text PK
account_id	text NOT NULL
provider_id	text NOT NULL
user_id	text FK → user CASCADE
access_token, refresh_token, id_token	text
access_token_expires_at, refresh_token_expires_at	timestamp
scope, password	text

verification

Magic-link / email verification tokens. Better Auth managed.

Column	Type
id	text PK
identifier	text NOT NULL
value	text NOT NULL

`expires_at` `timestamp` NOT NULL

Agency & onboarding

agency

Column	Type	Notes
<code>id</code>	<code>uuid</code>	PK
<code>name</code>	<code>text</code>	NOT NULL
<code>slug</code>	<code>text</code>	NOT NULL UNIQUE
<code>status</code>	<code>text</code>	"active" "suspended" DEFAULT "active"
Stripe billing		
<code>stripe_customer_id</code>	<code>text</code>	
<code>stripe_subscription_id</code>	<code>text</code>	
<code>subscription_status</code>	<code>text</code>	Stripe status string
<code>price_id</code>	<code>text</code>	Stripe price for the current plan
<code>current_period_end</code>	<code>timestamp</code>	Subscription period end
<code>trial_ends_at</code>	<code>timestamp</code>	14-day trial expiry on first provision
Stripe Connect		
<code>stripe_connect_account_id</code>	<code>text</code>	Agency's Express account
<code>stripe_connect_onboarded</code>	<code>boolean</code>	DEFAULT false — true after Connect flow
Onboarding		
<code>onboarding_dismissed_at</code>	<code>timestamp</code>	Set when agency dismisses the getting-started card
<code>created_at</code>	<code>timestamp</code>	NOT NULL
<code>updated_at</code>	<code>timestamp</code>	NOT NULL

agency_invite

Column	Type	Notes
<code>id</code>	<code>uuid</code>	PK

agency_id	uuid	FK → agency CASCADE
email	text	NOT NULL
role	text	NOT NULL — target role after sign-up
token	text	NOT NULL UNIQUE — unguessable magic link
status	text	"pending" "accepted" "expired" DEFAULT "pending"
invited_by_id	text	FK → user SET NULL
expires_at	timestamp	NOT NULL
created_at	timestamp	NOT NULL

CRM

client

Column	Type	Notes
id	uuid	PK
agency_id	uuid	FK → agency CASCADE
name	text	NOT NULL
type	text	"individual" "corporate" DEFAULT "individual"
status	text	"lead" "active" "inactive" DEFAULT "active"
email	text	
phone	text	
company	text	Corporate name
address	text	
city	text	
country	text	Canonical ISO 3166-1 name (full name, not code)
source	text	Lead-source vocab code
industry	text	Industry vocab code (corporate clients)
notes	text	
owner_id	text	FK → user SET NULL — relationship owner

created_by_id	text	FK → user SET NULL
created_at	timestamp	NOT NULL
updated_at	timestamp	NOT NULL

Indexes: client_agency_idx, client_owner_idx, client_status_idx, client_name_idx, client_agency_created_idx **Unique:** none (names may repeat)

client_contact

Additional contacts for corporate clients.

Column	Type
id	uuid PK
client_id	uuid FK → client CASCADE NOT NULL
name	text NOT NULL
job_title	text
email	text
phone	text
is_primary	boolean DEFAULT false NOT NULL
created_at	timestamp NOT NULL

Sales pipeline

opportunity

Column	Type	Notes
id	uuid	PK
agency_id	uuid	FK → agency CASCADE
title	text	NOT NULL
client_id	uuid	FK → client CASCADE NOT NULL
stage	text	"lead" "qualified" "proposal" "booked" "won" "lost" DEFAULT "lead"
value	numeric(12,2)	DEFAULT 0

currency	text	DEFAULT "EUR"
probability	integer	0–100 DEFAULT 10
destination	text	
travel_start_date	timestamp	
travel_end_date	timestamp	
pax_count	integer	DEFAULT 1
travel_purpose	text	Vocab code
expected_close_date	timestamp	
lost_reason	text	Vocab code — only when stage = "lost"
notes	text	
assigned_to_id	text	FK → user SET NULL
created_by_id	text	FK → user SET NULL
created_at	timestamp	NOT NULL
updated_at	timestamp	NOT NULL

Indexes: opportunity_agency_idx, opportunity_client_idx, opportunity_stage_idx, opportunity_assigned_idx, opportunity_agency_stage_idx, opportunity_agency_created_idx

Proposals (Products)

product

Column	Type	Notes
id	uuid	PK
agency_id	uuid	FK → agency CASCADE
reference	text	NOT NULL — e.g. "PRD-1042" unique per agency
title	text	NOT NULL
client_id	uuid	FK → client SET NULL
opportunity_id	uuid	FK → opportunity SET NULL
status	text	"draft" "sent" "accepted" "rejected" "expired" DEFAULT "draft"

destination	text	
start_date, end_date	timestamp	
pax_count	integer	DEFAULT 1
currency	text	DEFAULT "EUR"
markup_percent	numeric(5,2)	DEFAULT 0
total_cost	numeric(12,2)	DEFAULT 0
total_price	numeric(12,2)	DEFAULT 0
summary	text	Client-facing narrative (AI-generated)
valid_until	timestamp	
E-signature		
share_token	text	UNIQUE — public /p/[token] link
accepted_at, declined_at	timestamp	
signer_name, signer_email	text	Non-repudiation
signature_data	text	Typed name or drawn-signature data URL
signer_ip, signer_user_agent	text	
created_by_id	text	FK → user SET NULL
created_at, updated_at	timestamp	NOT NULL

Unique: product_agency_reference_unique (agency_id, reference)

product_item

Line items within a proposal.

Column	Type	Notes
id	uuid	PK
product_id	uuid	FK → product CASCADE NOT NULL
supplier_id	uuid	FK → supplier SET NULL (optional)
type	text	"flight" "hotel" "activity" "transfer" "insurance" "other"

title	text	NOT NULL
description', supplier	text	
quantity	integer	DEFAULT 1 NOT NULL
unit_cost', unit_price	numeric(12,2)	DEFAULT 0 NOT NULL
currency	text	DEFAULT "EUR"
start_date', end_date	timestamp	
details	jsonb	Raw supplier offer payload
sort_order	integer	DEFAULT 0
created_at	timestamp	NOT NULL

Bookings

booking

Column	Type	Notes
id	uuid	PK
agency_id	uuid	FK → <code>agency</code> CASCADE
reference	text	NOT NULL — e.g. "BKG-1042" unique per agency
client_id	uuid	FK → <code>client</code> SET NULL
status	text	"draft" "awaiting_payment" "confirmed" "ticketed" "completed" "cancelled" DEFAULT "draft"
destination	text	
depart_date', return_date	timestamp	
travel_purpose	text	Vocab code
trip_type	text	"one_way" "round_trip" "multi_city"
currency	text	DEFAULT "EUR"
notes	text	

total_amount	numeric(12,2)	DEFAULT 0
share_token	text	UNIQUE — shareable itinerary /i/[token]
created_by_id	text	FK → <code>user</code> SET NULL
created_at, updated_at	timestamp	NOT NULL

Lifecycle prerequisite rules (server-side enforced):

- `confirmed` requires: trip items present AND zero outstanding balance.
- `ticketed` requires: trip items present.

Unique: `booking_agency_reference_unique` (`agency_id`, `reference`)

booking_traveller

Column	Type	Notes
id	uuid	PK
booking_id	uuid	FK → <code>booking</code> CASCADE NOT NULL
full_name	text	NOT NULL
title	text	Vocab code (<code>mr</code> <code>mrs</code> <code>ms</code> <code>dr</code> <code>prof</code>)
gender	text	Vocab code (<code>male</code> <code>female</code> <code>unspecified</code>)
passport_number	text	
passport_expiry	timestamp	
nationality	text	ISO country name
date_of_birth	timestamp	
passport_issue_date, passport_issue_place	timestamp / text	
email	text	Required by Duffel for lead passenger
phone	text	Required by Duffel for lead passenger
is_lead	boolean	NOT NULL DEFAULT false — first passenger = lead
sort_order	integer	DEFAULT 0
created_at	timestamp	NOT NULL

booking_item

A purchased line item (flight, hotel, transfer, fee, etc.).

Column	Type	Notes
id	uuid	PK
booking_id	uuid	FK → <code>booking</code> CASCADE NOT NULL
supplier_id	uuid	FK → <code>supplier</code> SET NULL (optional)
type	text	"flight" "hotel" "excursion" "transfer" "insurance" "fee" "other"
title	text	NOT NULL
description, supplier	text	
booking_ref	text	Supplier confirmation / PNR reference
start_date, end_date	timestamp	
quantity	integer	DEFAULT 1
amount	numeric(12,2)	DEFAULT 0 — price charged to client (per unit)
currency	text	DEFAULT "EUR"
item_status	text	"pending" "confirmed" "ticketed" "cancelled" DEFAULT "pending"
confirmation_number	text	Supplier confirmation number once booked
day_index	integer	Itinerary day (0-based from departure; null = auto by date)
details	jsonb	Raw supplier offer payload
sort_order	integer	DEFAULT 0
created_at	timestamp	NOT NULL

payment

Column	Type	Notes
id	uuid	PK
booking_id	uuid	FK → <code>booking</code> CASCADE NOT NULL
amount	numeric(12,2)	NOT NULL
currency	text	DEFAULT "EUR"

kind	text	"deposit" "installment" "payment" "refund" DEFAULT "payment"
method	text	"manual" "card" "transfer" "cash" "stripe" DEFAULT "manual"
status	text	"pending" "completed" "failed" "refunded" DEFAULT "completed"
reference	text	Stripe payment id or manual reference
stripe_session_id	text	Stripe Checkout Session id (Connect flow)
checkout_url	text	Hosted Checkout URL sent to client
note	text	
created_by_id	text	FK → user SET NULL
created_at	timestamp	NOT NULL

booking_day

Per-day itinerary titles and notes.

Column	Type	
id	uuid	PK
booking_id	uuid	FK → booking CASCADE NOT NULL
day_index	integer	NOT NULL (0-based)
title	text	
notes	text	

notification

Communications log.

Column	Type	Notes
id	uuid	PK
agency_id	uuid	FK → agency CASCADE
booking_id	uuid	FK → booking CASCADE (nullable)
channel	text	"email" "sms" "whatsapp" "push" DEFAULT "email"
recipient	text	NOT NULL — email address or phone
subject	text	

body	text	
kind	text	"confirmation" "voucher" "receipt" "custom" "invite" "proposal" DEFAULT "custom"
status	text	"sent" "failed" "logged" DEFAULT "sent"
error	text	Error message when status = "failed"
created_by_id	text	FK → user SET NULL
created_at	timestamp	NOT NULL

Sprint 1 — Booking lifecycle tables

booking_supplier_ref

Structured supplier confirmation. Replaces ad-hoc JSONB in `booking_item.details`.

Column	Type	Notes
id	uuid	PK
booking_id	uuid	FK → booking CASCADE NOT NULL
booking_item_id	uuid	FK → booking_item CASCADE NOT NULL
provider_id	text	NOT NULL — "duffel" "hotelbeds" "mock"
confirmation_number	text	NOT NULL — shown to client
pnr	text	Airline PNR / record locator
supplier_order_id	text	Provider's internal order id (e.g. Duffel <code>order_xxx</code>)
raw_payload	jsonb	Full raw response for debugging
created_at	timestamp	NOT NULL

Indexes: `booking_supplier_ref_booking_idx`, `booking_supplier_ref_item_idx`, `booking_supplier_ref_provider_idx`

booking_event

Append-only audit + analytics log. Never update or delete rows.

Column	Type	Notes
id	uuid	PK
booking_id	uuid	FK → booking CASCADE NOT NULL

agency_id	uuid	FK → agency CASCADE NOT NULL (for agency-scoped analytics)
event	text	NOT NULL — stable event code (see booking-architecture.md §6)
provider_id	text	"duffel", "hotelbeds", "mock", or null for UI events
correlation_id	text	Request-level trace id
metadata	jsonb	Structured payload (offer id, price, error code, etc.)
created_at	timestamp	NOT NULL

Event codes: search_initiated, offer_selected, price_validated, price_changed, booking_submitted, booking_confirmed, booking_failed, booking_cancelled, payment_started, payment_completed

Indexes: booking_event_booking_idx, booking_event_agency_idx, booking_event_event_idx, booking_event_created_idx, booking_event_booking_created_idx

booking_document

Documents generated for a booking (vouchers, tickets, invoices, itineraries).

Column	Type	Notes
id	uuid	PK
booking_id	uuid	FK → booking CASCADE NOT NULL
booking_item_id	uuid	FK → booking_item SET NULL (null = booking-level doc)
type	text	NOT NULL — "voucher" "ticket" "invoice" "itinerary" "receipt"
provider_id	text	Supplier that issued the document
url	text	Vercel Blob URL or supplier CDN link
raw_data	jsonb	Supplier payload for re-generation
generated_at	timestamp	When the document was issued
created_at	timestamp	NOT NULL

booking_idempotency

Key registry preventing duplicate supplier orders on retry.

Column	Type	Notes
key	text	PK — sha256(bookingId:itemId:offerId)
booking_id	uuid	FK → booking CASCADE NOT NULL

booking_item_id	uuid	FK → booking_item CASCADE
provider_id	text	NOT NULL — provider the key was sent to
status	text	"pending" "success" "failed" DEFAULT "pending"
supplier_ref	text	Confirmation number when status = "success"
created_at	timestamp	NOT NULL
expires_at	timestamp	NOT NULL — 24h TTL; safe to clean up after expiry

Indexes: booking_idempotency_booking_idx, booking_idempotency_expires_idx

Supplier management

supplier

Agency's managed supplier directory (hotels, airlines, DMCs, etc.).

Column	Type	Notes
id	uuid	PK
agency_id	uuid	FK → agency CASCADE
name	text	NOT NULL — unique per agency
type	text	"hotel" "airline" "car_rental" "transfer" "dmc" "insurance" "other"
status	text	"active" "inactive" DEFAULT "active"
email, phone, website	text	
address, city, country	text	
contact_name	text	
notes	text	
created_by_id	text	FK → user SET NULL
created_at, updated_at	timestamp	NOT NULL

Unique: supplier_agency_name_unique (agency_id, name)

supplier_contract

Commercial contracts with commission terms.

Column	Type	Notes
id	uuid	PK
supplier_id	uuid	FK → <code>supplier</code> CASCADE NOT NULL
agency_id	uuid	FK → <code>agency</code> CASCADE (denormalized for query efficiency)
name	text	NOT NULL
reference	text	Contract number
commission_basis	text	"percent" "fixed" "net"
commission_rate	numeric(5,2)	% or fixed amount depending on basis
valid_from, valid_to	timestamp	
file_url	text	Vercel Blob URL for the PDF contract
notes	text	
created_at, updated_at	timestamp	NOT NULL

supplier_rate

Structured per-product rates within a contract.

Column	Type	Notes
id	uuid	PK
contract_id	uuid	FK → <code>supplier_contract</code> CASCADE NOT NULL
name, description	text	
rate_type	text	Type of rate (product/season/tier)
amount	numeric(12,2)	
currency	text	
valid_from, valid_to	timestamp	
created_at	timestamp	NOT NULL

commission

Two-ledger earnings record. Auto-generated on booking confirm/ticket.

Column	Type	Notes
id	uuid	PK
agency_id	uuid	FK → <code>agency</code> CASCADE
booking_id	uuid	FK → <code>booking</code> SET NULL (nullable) — ledger survives booking deletion (migration <code>0021</code>)
booking_item_id	uuid	FK → <code>booking_item</code> SET NULL (nullable) — same rationale as <code>booking_id</code>
supplier_id	uuid	FK → <code>supplier</code> SET NULL
agent_user_id	text	FK → <code>user</code> SET NULL
ledger	text	"supplier_to_agency" "agency_to_agent"
basis	text	"percent" "fixed" "net"
rate	numeric(5,2)	
amount	numeric(12,2)	NOT NULL — computed commission
currency	text	
status	text	"pending" "confirmed" "paid" "cancelled"
notes	text	
created_at, updated_at	timestamp	NOT NULL

Client portal

portal_session

Passwordless sessions for the client-facing traveler portal. Entirely separate from the Better Auth staff session system.

Column	Type	Notes
id	uuid	PK
client_id	uuid	FK → <code>client</code> CASCADE NOT NULL
token	text	NOT NULL UNIQUE — magic-link token (15 min) → session token (7 days)
purpose	text	NOT NULL, default <code>'session'</code> — <code>'magic'</code> <code>'session'</code> (migration <code>0021</code>)

<code>expires_at</code>	<code>timestamp</code>	NOT NULL
<code>created_at</code>	<code>timestamp</code>	NOT NULL

Magic-link flow: token is short-lived (15 min); on verification the row is updated with a rotated long-lived (7-day) token **and** `purpose` flips from `'magic'` to `'session'`, stored in an `httpOnly` cookie. `purpose` discriminates the two so a magic-link row can never authenticate a normal portal request — only `purpose = 'session'` rows are accepted as session bearers, and only `purpose = 'magic'` rows are accepted by the verify step.

Reference data (global, not tenant-scoped)

`hotel_content`

Hotelbeds Content API cache. PK is the Hotelbeds hotel code (text), **not** UUID. Shared across all agencies — intentionally not scoped to `agency_id`.

Column	Type	Notes
<code>code</code>	<code>text</code>	PK — Hotelbeds hotel code (e.g. "12345")
<code>name</code>	<code>text</code>	NOT NULL
<code>stars</code>	<code>integer</code>	DEFAULT 0
<code>hotel_type</code>	<code>text</code>	
<code>description</code>	<code>text</code>	
<code>address, city, country, postal_code</code>	<code>text</code>	
<code>latitude, longitude</code>	<code>numeric(10,6)</code>	
<code>destination_code</code>	<code>text</code>	Hotelbeds destination code (e.g. "BCN")
<code>segments</code>	<code>jsonb</code>	Marketing segment tags: <code>string[]</code>
<code>facilities</code>	<code>jsonb</code>	Amenity names: <code>string[]</code>
<code>images</code>	<code>jsonb</code>	<code>{ url: string; roomCode?: string } []</code>
<code>updated_at</code>	<code>timestamp</code>	NOT NULL — sync timestamp

Index: `hotel_content_destination_idx` (`destination_code`)

Audit

`activity_log`

Records every meaningful action for manager oversight.

Column	Type	Notes
<code>id</code>	<code>uuid</code>	PK
<code>agency_id</code>	<code>uuid</code>	FK → <code>agency</code> CASCADE
<code>user_id</code>	<code>text</code>	FK → <code>user</code> SET NULL
<code>action</code>	<code>text</code>	"created" "updated" "deleted" "stage_changed" "sent" "status_changed"
<code>entity_type</code>	<code>text</code>	"client" "opportunity" "product" "user"
<code>entity_id</code>	<code>text</code>	
<code>entity_label</code>	<code>text</code>	Human-readable name kept even if entity is deleted
<code>metadata</code>	<code>jsonb</code>	Diff / context payload
<code>created_at</code>	<code>timestamp</code>	NOT NULL

Indexes: `activity_agency_idx`, `activity_user_idx`, `activity_entity_idx` (`entity_type`, `entity_id`), `activity_created_idx`

Migrations summary

Migration	Description
0000-0005	Pre-tenancy base schema (not itemized)
0006	Multi-tenant <code>agencyId</code> backfill
0007	<code>agency_invite</code>
0008	<code>user.locale</code>
0009	Agency Stripe billing columns
0010	Product e-signature columns
0011	<code>hotel_content</code> cache
0012	Stripe Connect columns on <code>agency</code>

0013	portal_session
0014	supplier, supplier_contract, supplier_rate; supplier_id FK on booking_item + product_item
0015	commission; user.commission_rate_percent
0016	agency.onboarding_dismissed_at
0017	Controlled-vocab columns (additive, nullable): client.industry, opportunity.travel_purpose, booking.travel_purpose/trip_type, booking_traveller.title/gender
0018	commission.booking_item_id FK re-pointed at booking_item; agency/status/created perf indexes across booking, client, commission, notification, opportunity, product, plus booking_item.supplier_id, product_item.supplier_id, verification.identifier
0019	Sprint 1: booking_supplier_ref, booking_event, booking_document, booking_idempotency; booking_traveller.email + .phone
0020	product.converted_booking_id (FK → booking, set null) — proposal→booking idempotency latch
0021	commission.booking_id + .booking_item_id FKs cascade → set null (ledger survives booking/item deletion); commission_item_idx, commission_supplier_idx, product_converted_booking_idx; portal_session.purpose ('magic' 'session', default 'session')

API Integrations

Every external integration **degrades gracefully** to sample or logged behaviour when its keys are unset, so the app runs end-to-end with only `POSTGRES_URL` + `BETTER_AUTH_SECRET`. Env vars are summarized in `development-guide.md`.

Provider architecture

Booking providers plug into **one abstraction** so adding Hotelbeds, Amadeus, TravelgateX, Booking.com, or Expedia never touches calling code. Interfaces + registry live in `src/lib/suppliers/providers/`. Environment resolution lives in `src/lib/suppliers/config.ts` — the single entry point for credential/hostname config; no other file reads `process.env` for supplier credentials directly.

- **Capability-segmented interfaces** (`types.ts`) — a provider implements only what it offers: `HotelSearchCapable`, `HotelBookingCapable`, `FlightSearchCapable`, `FlightBookingCapable`, `CancelCapable`, `ContentCapable` (hotel enrichment / name-search), `AutocompleteCapable` (airport suggestions). A hotels-only aggregator implements the hotel interfaces and nothing else (no empty stubs).
- **Full booking lifecycle** — `search` → `quote` (re-validate price/rate) → `book` (idempotent) → `cancel`. Real bedbank/GDS rates expire, so `quoteHotel/quoteFlight` re-price immediately before `book*`; book requests carry an `idempotencyKey` so replays never double-book. The idempotency cache read is gated on `expiresAt` (24 h TTL) — an **expired** row is treated as absent (the row is refreshed to `pending` on the next attempt), and a still-live `pending` row (a prior call that touched the supplier and never recorded a result, e.g. a serverless timeout) is treated as **in-flight**: the provider is never re-called, and a provisional needs-manual-reconciliation result is returned instead — this is what prevents a double PNR/charge on retry.
- **Normalized errors** — every provider throws `ProviderError` with a stable `code` (`rate_expired`, `sold_out`, `rate_limited`, `provider_unavailable`, ...) and a `retryable` flag, so callers branch on the code, not provider strings.
- **Tenant-aware context** — every call takes a `ProviderContext` (`agencyId`, `currency`, `locale`, `correlationId`, abort signal); providers never read globals.
- **Registry** (`registry.ts`) — adapters `register()` themselves at startup; callers resolve by vertical + capability + priority (`providerRegistry.pick("hotels", "search")`) with capability type-guards (`canSearchHotels`, `canBookFlights`, ...). Replaces the hardcoded `getFlightSupplier()/getHotelSupplier()` if/else. Selection (`forVertical/pick`) applies the runtime type-guard for the requested capability, not just the provider's declared `capabilities` array — a provider that *claims* a capability but doesn't implement the matching method is filtered out of selection rather than silently degrading bookings to provisional refs. `register()` also asserts catalog↔implementation parity at registration time (`assertCatalogParity`): a mismatch between a provider's declared capabilities and its actual methods logs a loud `console.error` in non-production so the gap is caught at startup, not at booking time.

- **Registration** (`register.ts`, Wave 2) — `registerBuiltInProviders()` wires the three shipped adapters into the registry: `MockBookingProvider` (both verticals, priority 0, always configured — the fallback), `DuffelBookingProvider` (flights, priority 50) and `HotelbedsBookingProvider` (hotels, priority 50). It is idempotent (module-level guard) and runs as a side effect when the providers barrel is imported, so the registry is self-wiring. The mock outranks nothing, so `pick()` returns a real provider whenever one is configured and falls back to the mock otherwise.
- **Provider catalog** (`PROVIDER_CATALOG`) — one logic-free source of truth for which providers exist, their verticals/capabilities, env vars, and status (`live` / `legacy` / `planned`):

Provider	Verticals	Status
Mock	flights + hotels	live
Duffel	flights	live
Amadeus	flights (+ hotels)	legacy
Hotelbeds	hotels	live
TravelgateX	hotels + flights	planned
Booking.com	hotels	planned
Expedia (Rapid)	hotels	planned

Travel Platform sprint complete (Sprint 2): search, content, autocomplete, and booking are all routed through the registry. Business logic no longer calls provider-specific functions; adding a new provider is a `register()` call only.

- **Booking** (`booking-service.ts`) — full `quote` → `book` → `cancel` lifecycle via `providerRegistry.pick(...)`, idempotent, writes `booking_supplier_ref` / `booking_event` / `booking_idempotency`.
- **Search + content** (`src/lib/travel-platform/index.ts`) — single facade for `searchFlights`, `searchHotels` (availability, name-search, content fallback, thumbnail enrichment), `searchAirports`, `searchHotelDestinations`. Server actions delegate here; the `getFlightSupplier` / `getHotelSupplier` / `safeSearch` layer is no longer imported by consumer code (kept for backward compatibility only).
- **New capabilities registered:** `ContentCapable` (Hotelbeds: name-search, enrichment, room rates) and `AutocompleteCapable` (Duffel: airport suggestions).

Open item #1 (real supplier booking) is closed — swap in production credentials and both search and booking flows go live.

Flights — Duffel

- `src/lib/suppliers/duffel.ts`, behind `getFlightSupplier()` + `safeSearch()`.
- Airport autocomplete, one-way/round-trip, flight codes, connecting airports.

- Falls back to sample data when `DUFFEL_API_TOKEN` is unset.
- Amadeus self-service (`amadeus.ts`) is kept **only as a legacy fallback**.
- **Sprint 1:** `DuffelBookingProvider` implements `FlightSearchCapable` + `FlightBookingCapable` through the registry, adding `quoteFlight` (price revalidation) + `bookFlight` (idempotent order creation) to the existing search capability.

Hotels — Hotelbeds (APITUDE)

- `src/lib/suppliers/hotelbeds.ts` + `content-cache.ts`.
- Availability + content APIs. Booking.com-style search/results/details, dynamic occupancy pricing, filters, room photos, compare.
- **Content cache** (`hotel_content` table, global/non-tenant) serves real photos, facilities and coords quota-free; synced via `scripts/sync-hotel-content.ts`.
- **Search by hotel name** resolves matching hotels via the Content API (`searchHotelbedsHotelsByName`) then prices them live; falls back to estimated rates.
- **Keys:** `HOTELBEDS_API_KEY` / `HOTELBEDS_SECRET`. **Hostname:** `HOTELBEDS_HOSTNAME` (defaults to test endpoint; set to `https://api.hotelbeds.com` for production). Falls back to sample data when keys are unset.
- **Sprint 1:** `HotelbedsBookingProvider` implements `HotelSearchCapable` + `HotelBookingCapable` through the registry, adding `quoteHotel` + `bookHotel`.

Travel Platform facade

`src/lib/travel-platform/index.ts` is the single entry point for all travel operations. Server actions import from here only; no supplier-specific code leaks into the action or page layer.

Key exports: `searchFlights`, `searchHotels`, `searchAirports`, `searchHotelDestinations`, `getActiveFlightProvider`, `getActiveHotelProvider`, `isFlightProviderConfigured`, `isHotelProviderConfigured`.

The hotel search path handles three cases internally:

1. Name search → `ContentCapable.searchHotelsByName`
2. Availability search → `HotelSearchCapable.searchHotels` + content enrichment via `ContentCapable.fetchHotelContent`
3. Content fallback (empty/degraded availability) → DB cache (`listHotelOffersCached`) → `ContentCapable.searchHotelsByName` with city as query

Supplier abstraction (legacy)

`src/lib/suppliers/index.ts` — `getFlightSupplier` / `getHotelSupplier` / `safeSearch` — is kept for backward compatibility. New code must use the Travel Platform facade instead. This is separate from the managed **supplier directory** (the `supplier` table — agency's own hotel/airline/DMC contacts; see `business-rules.md`).

Billing — Stripe subscriptions (vendor → agency)

- `src/lib/billing/stripe.ts`. Vendor bills agencies; 14-day trial on provision.
- Checkout, billing portal, and **manual webhook signature verification**.
- `api/stripe/webhook` reconciles subscription status; `requireAgencyUser` gates on a lapsed subscription.
- Keys: `STRIPE_SECRET_KEY`, `STRIPE_PRICE_ID`, `STRIPE_WEBHOOK_SECRET`.

Payments — Stripe Connect (traveler → agency)

- `src/lib/payments/stripe.ts` — **distinct** from `billing/stripe.ts`.
- Agencies onboard a connected Express account to receive traveler payments directly (**destination charges**); the platform takes a configurable fee.
- `api/stripe/connect-webhook` reconciles Connect payments. Powers the client portal "Pay now".
- Keys: `STRIPE_CONNECT_WEBHOOK_SECRET`, `STRIPE_PLATFORM_FEE_PERCENT`.

Email — Resend

- `src/lib/notifications/email.ts` (adapter) + `templates.ts` (HTML).
- Sends invite emails, password-reset, proposal acceptance, portal invites.
- When unconfigured, logs to console + the `notification` table; invite/portal links are made copyable so the flow still works.
- Keys: `RESEND_API_KEY`, `EMAIL_FROM`.

AI — Vercel AI SDK + Google Gemini (primary) / OpenRouter (fallback)

- Powers the assistant and inline AI features; see `ai.md`.
- Keys: `GEMINI_API_KEY` (primary; `GEMINI_MODEL`, default `gemini-2.5-flash`) and `OPENROUTER_API_KEY` (fallback; `OPENROUTER_MODEL`). Both optional — AI features no-op/degrade when neither is set.

Storage — Vercel Blob (optional)

- Supplier contract PDF uploads. Key: `BLOB_READ_WRITE_TOKEN`.

Booking Architecture (Sprint 1)

Detailed technical reference for the provider abstraction system, booking service, idempotency model, and event log introduced in Sprint 1. For a high-level overview see `api-integrations.md`.

1. Overview

The booking architecture introduces a **three-layer** stack on top of the existing search-only supplier adapters:



The server action collects intent (which booking item to fulfil), builds a `ProviderContext`, and calls `serviceBookFlight` or `serviceBookHotel`. The booking service runs the full lifecycle — idempotency check, quote, book, event logging, supplier-ref persistence — without any booking-specific code leaking into the action layer.

2. Provider abstraction

2.1 Capability-segmented interfaces

`src/lib/suppliers/providers/types.ts`

A provider implements **only the capabilities it actually offers**. There are no empty stubs. The capability contract:

Interface	Methods	Who implements
<code>ProviderDescriptor</code>	<code>id, label, verticals, capabilities, priority, isConfigured()</code>	every provider

HotelSearchCapable	searchHotels(params, ctx)	Hotelbeds, Mock
HotelBookingCapable	quoteHotel(offer, ctx), bookHotel(req, ctx)	Hotelbeds, Mock
FlightSearchCapable	searchFlights(params, ctx)	Duffel, Mock
FlightBookingCapable	quoteFlight(offer, ctx), bookFlight(req, ctx)	Duffel, Mock
CancelCapable	cancel(ref, ctx)	(future)

A concrete provider is typed as:

```
type BookingProvider = ProviderDescriptor &
  Partial<HotelSearchCapable & HotelBookingCapable &
    FlightSearchCapable & FlightBookingCapable & CancelCapable>
```

Callers narrow capabilities using type-guard functions:

```
canSearchHotels(p)    // p is HotelSearchCapable
canBookHotels(p)      // p is HotelBookingCapable
canSearchFlights(p)   // p is FlightSearchCapable
canBookFlights(p)     // p is FlightBookingCapable
```

2.2 ProviderContext

Passed to every provider call so providers never read globals:

```
type ProviderContext = {
  agencyId:      string;           // tenant scope + per-agency cred lookup
  currency?:    string;           // ISO code, e.g. "DZD"
  locale?:      string;           // e.g. "en"
  correlationId?: string;          // traces a request across providers and logs
  signal?:      AbortSignal;       // timeout / user cancellation
}
```

2.3 Booking lifecycle DTOs

RateQuote — returned by `quoteHotel` / `quoteFlight`:

Field	Type	Purpose
quoteId	string	Opaque token passed to <code>book*</code>
providerId	ProviderId	Which provider issued the quote
vertical	"flights" "hotels"	—

priceTotal	number	Price after re-validation
currency	string	—
refundable	boolean	—
expiresAt	string (ISO)	Quote expiry deadline
cancellationDeadline?	string (ISO)	Free-cancel deadline (when refundable)
priceChanged?	boolean	True when price differs from search result

HotelBookingRequest — sent to `bookHotel`:

Field	Type	Purpose
offer	HotelOffer	Original search offer
guests	GuestDetails[]	Guest roster (lead first)
idempotencyKey	string	Caller-generated; same key = no double-book
quoteId?	string	Fresh rate key from <code>quoteHotel</code>
agencyReference?	string	Cross-linking reference (e.g. BKG-1001)

FlightBookingRequest — sent to `bookFlight`:

Field	Type	Purpose
offer	FlightOffer	Original search offer
passengers	FlightPassenger[]	Passenger roster
idempotencyKey	string	Caller-generated; same key = no double-book
quoteId?	string	Fresh quote token
agencyReference?	string	Cross-linking reference

BookingResult — returned by `book*` on success:

Field	Type
ref.providerId	ProviderId
ref.confirmationNumber	string
ref.raw?	unknown (raw supplier response)
status	"confirmed" "pending"

<code>priceTotal</code>	<code>number</code>
<code>currency</code>	<code>string</code>

2.4 Normalized error model

All providers throw `ProviderError` with a stable `code`:

Code	Meaning	Retryable
<code>auth</code>	Bad/expired credentials	false
<code>validation</code>	Malformed request	false
<code>rate_expired</code>	Quote/rate no longer valid → re-quote	true
<code>sold_out</code>	Availability gone	false
<code>rate_limited</code>	Provider throttled us → backoff	true
<code>provider_unavailable</code>	Upstream down/timeout	true
<code>not_supported</code>	Capability not offered by this provider	false
<code>unknown</code>	Unclassified	false

Callers branch on `code`, never on provider-specific error strings:

```

} catch (err) {
  if (err instanceof ProviderError && err.code === "rate_expired") {
    // re-quote then retry
  }
}

```

3. Provider registry

`src/lib/suppliers/providers/registry.ts`

The registry is a singleton. Providers self-register at startup and callers resolve by vertical + capability.

3.1 API

```

// Register (called by each provider at startup):
providerRegistry.register(provider: BookingProvider): void

// Resolve (returns highest-priority configured provider for
vertical+capability):
providerRegistry.pick(vertical: ProviderVertical, capability:

```

```
ProviderCapability): BookingProvider | undefined

// Introspect:
providerRegistry.list(): BookingProvider[]
providerRegistry.all(vertical, capability): BookingProvider[]
```

`pick()` returns the highest-priority `isConfigured()` provider for the requested vertical + capability. If no configured provider exists, it returns `undefined` — the caller must handle that gracefully (the mock always fills this gap since it is always configured at priority 0).

3.2 Registration (register.ts)

`src/lib/suppliers/providers/register.ts` wires all three built-in adapters into the registry **as a module side effect** — importing the providers barrel triggers registration automatically:

```
export function registerBuiltInProviders(): void {
  if (registered) return; // module-level idempotency guard
  registered = true;
  providerRegistry.register(new MockBookingProvider()); // priority 0 —
  always
  providerRegistry.register(new DuffelBookingProvider()); // priority 50
  providerRegistry.register(new HotelbedsBookingProvider()); // priority 50
}
registerBuiltInProviders(); // runs on import
```

3.3 Provider catalog (PROVIDER_CATALOG)

Logic-free metadata record used to drive UI ("Available integrations") and future dynamic registration. The catalog is not the registry — it is a static description of what *could* be wired:

Provider	Verticals	Capabilities	Status	Env vars
mock	flights + hotels	search, quote, book	live	(none)
duffel	flights	search, quote, book	live	DUFFEL_API_TOKEN
amadeus	flights (+ hotels)	search	legacy	AMADEUS_CLIENT_ID, AMADEUS_CLIENT_SECRET
hotelbeds	hotels	search, quote, book	live	HOTELBEDS_API_KEY, HOTELBEDS_SECRET
travelgatemx	hotels + flights	search	planned	—
booking_com	hotels	search	planned	—
expedia	hotels	search	planned	—

3.4 Priority resolution

When multiple providers are configured for the same vertical:

- **Highest priority wins** (higher number = higher priority).
- `MockBookingProvider` is always priority **0** — the unconditional fallback.
- `DuffelBookingProvider` and `HotelbedsBookingProvider` are priority **50**.
- Add a future aggregator at **100** to override the defaults.

4. Booking service

`src/lib/suppliers/booking-service.ts`

The orchestration layer. Server actions call `serviceBookFlight` or `serviceBookHotel` and receive a `ServiceBookingResult`:

```
type ServiceBookingResult =
  | { confirmed: true;   confirmationNumber: string; providerId: string }
  | { confirmed: false; confirmationNumber: string; reason: string }
```

The booking service never throws. Failures are returned as `{ confirmed: false }` so the server action can write a provisional reference and surface a user-facing error without crashing the booking flow.

4.1 Flight booking flow (`serviceBookFlight`)

```
1. Derive idempotency key
   sha256("bookingId:bookingItemId:offerId")

2. Replay check
   IF booking_idempotency.status = "success" THEN return cached result (no
   supplier call)

3. Pick provider
   providerRegistry.pick("flights", "book") → DuffelBookingProvider (or mock)
   IF no provider → return { confirmed: false, reason: "No configured flight
   booking provider" }

4. Register idempotency key as "pending" (ON CONFLICT DO NOTHING)
   → prevents duplicate calls even if two requests race to the same booking

5. Quote (price revalidation) – non-fatal
   provider.quoteFlight(offer, ctx)
   → emit "price_validated" or "price_changed" event

6. Emit "booking_submitted" event

7. Book
   provider.bookFlight({ offer, passengers, idempotencyKey, agencyReference },
   ctx)
```

```

8. On success:
  - INSERT booking_supplier_ref (confirmationNumber, pnr, rawPayload)
  - UPDATE booking_idempotency → status = "success", supplierRef =
confirmationNumber
  - emit "booking_confirmed" event
  - return { confirmed: true, confirmationNumber, providerId }

9. On failure:
  - UPDATE booking_idempotency → status = "failed"
  - emit "booking_failed" event
  - return { confirmed: false, confirmationNumber: "REF-<timestamp>", reason }

```

4.2 Hotel booking flow (serviceBookHotel)

Same as 4.1 with two differences:

1. **offerId** is `offer.rateKey ?? offer.id` (Hotelbeds uses rate keys).
2. **Quote carries the fresh `rateKey`** — `quoteHotel` returns a new `quoteId` (the refreshed Hotelbeds `rateKey`), which is passed as `quoteId` in the `HotelBookingRequest`. This ensures the book call always uses the freshest rate, never the one from the initial search.

4.3 Quote step — non-fatal by design

If `quoteFlight/quoteHotel` throws, the booking service logs a warning and proceeds to book. This prevents a quote-API timeout from blocking an entire booking attempt — the supplier's `book*` endpoint will reject a stale rate explicitly if it expired, surfacing as a `ProviderError("rate_expired")` from the book step itself.

For hotels, if the quote fails the `quoteId` is `undefined` and the book request uses the original `rateKey` from the search offer.

5. Idempotency model

5.1 Key derivation

```
sha256(`${bookingId}:${bookingItemId}:${offerId}`)
```

- `offerId` is the supplier's offer/rate identifier (e.g. Duffel offer id, Hotelbeds `rateKey`). A proposal-converted booking item carries no supplier offer (`details` is null) — `offerId` is then `undefined`, and the booking service short-circuits to a provisional `REF-...` reference before ever deriving a key, rather than deref'ing a null offer.
- The same triple always produces the same 64-hex-char key.
- Keys are registered with `onConflictDoUpdate (registerPending())`: callers only reach the insert/upsert after ruling out a live (unexpired) success/pending row (see 5.3), so any PK conflict at that point can only be an **expired** row — it is reset to `pending` with a fresh TTL

rather than left alone, so an expired key is treated as absent for the new attempt.

5.2 Lifecycle states

State	Meaning
<code>pending</code>	Row inserted before the supplier call
<code>success</code>	Supplier confirmed; <code>supplierRef</code> is the confirmation number
<code>failed</code>	Supplier rejected; safe to retry with a fresh key (new offer/rate)

5.3 Replay semantics

Every read is scoped to **unexpired** rows only (`WHERE key = ... AND expiresAt > now()`) — an expired row is invisible to this check regardless of its status, so it falls through to a fresh attempt (see 5.1).

- On success (`status = "success"`, unexpired): return `supplierRef` immediately — no supplier call. This handles browser re-submits, network retries, and serverless double-invocations.
- On pending (`status = "pending"`, unexpired): the prior call is still in flight (or crashed mid-flight without recording a result). This is treated as **in-flight, not safe to retry** — the provider is **not** called again; the service returns a provisional `REF-...` reference with a reason asking to reconcile with the supplier before retrying. This is a deliberate change from an earlier version that let a pending row fall through to a fresh book call, which could double-book/double-charge on a slow-but-eventually-successful first attempt.
- On failed (`status = "failed"`): the key represents a bad offer; the agent should select a fresh offer (which will have a different `offerId`) and book again with a new key.

5.4 TTL

Keys expire after **24 hours** (`expiresAt` column). Expiry is enforced on every read (5.3), not merely advisory: an expired row can never be replayed as a cache hit, and registering a new attempt against an expired key refreshes it to `pending` in place. Expired rows (along with expired `portal_session` and stale-pending `agency_invite` rows) are swept daily by `GET /api/cron/cleanup`, scheduled via `vercel.json`'s `crons` — see `deployment.md`.

6. Event log (`booking_event`)

Append-only. Never update or delete rows.

6.1 Event types

Event	When	Metadata
-------	------	----------

search_initiated	Agent starts a search	{ destination, checkIn, nights, guests }
offer_selected	Agent picks an offer from results	{ offerId, priceTotal, currency }
price_validated	Quote returned same price	{ priceTotal, currency }
price_changed	Quote returned different price	{ priceTotal, currency }
booking_submitted	Just before calling book*	{ offerId, priceChanged }
booking_confirmed	book* returned success	{ confirmationNumber, status, priceTotal, currency }
booking_failed	book* threw	{ reason }
booking_cancelled	Cancellation succeeded	{ penaltyAmount, currency }
payment_started	Stripe Checkout session created	{ sessionId, amount }
payment_completed	Stripe webhook confirmed	{ sessionId, amount }

6.2 Dual purpose

- **Compliance audit trail** — every state transition is permanently recorded with timestamp, providerId, and correlationId. Never deleted (even if the booking itself is cancelled).
- **Analytics source** — booking funnel metrics (search→select, select→book, book→confirm, confirm→cancel) can be derived directly from this table without a third-party SDK.

6.3 Schema

Column	Type	Notes
id	uuid	PK
bookingId	uuid	FK → booking
agencyId	uuid	FK → agency (for efficient agency-scoped analytics)
event	text	Stable event code (see 6.1)
providerId	text?	"duffel", "hotelbeds", "mock", or null for UI events
correlationId	text?	Traces a request across logs
metadata	jsonb?	Structured payload (offer id, price, error, etc.)

<code>createdAt</code>	<code>timestamp</code>	Append timestamp
------------------------	------------------------	------------------

Indexes: `bookingId`, `agencyId`, `event`, `createdAt`, **composite** (`bookingId`, `createdAt`).

7. Supplier reference (`booking_supplier_ref`)

Structured record of the supplier's confirmation for a booking item. Replaces the ad-hoc JSONB in `booking_item.details` so references are **queryable** for status polling, cancellation, and voucher retrieval.

Column	Type	Notes
<code>id</code>	<code>uuid</code>	PK
<code>bookingId</code>	<code>uuid</code>	FK → <code>booking</code>
<code>bookingItemId</code>	<code>uuid</code>	FK → <code>booking_item</code>
<code>providerId</code>	<code>text</code>	"duffel", "hotelbeds", etc.
<code>confirmationNumber</code>	<code>text</code>	Shown to client
<code>pnr</code>	<code>text?</code>	Airline PNR / record locator
<code>supplierOrderId</code>	<code>text?</code>	Provider's internal order id
<code>rawPayload</code>	<code>jsonb?</code>	Full raw response for debugging + future re-parsing
<code>createdAt</code>	<code>timestamp</code>	—

Multiple refs per `bookingItemId` are possible (e.g. re-booking after a failure leaves both the failed attempt and the successful one).

8. Document store (`booking_document`)

Documents generated for a booking: vouchers, e-tickets, invoices, itineraries.

Column	Type	Notes
<code>id</code>	<code>uuid</code>	PK
<code>bookingId</code>	<code>uuid</code>	FK → <code>booking</code>
<code>bookingItemId</code>	<code>uuid?</code>	FK → <code>booking_item</code> (null = booking-level doc)
<code>type</code>	<code>text</code>	"voucher" "ticket" "invoice" "itinerary" "receipt"

<code>providerId</code>	<code>text?</code>	Supplier that issued the document
<code>url</code>	<code>text?</code>	Vercel Blob URL or supplier CDN link
<code>rawData</code>	<code>jsonb?</code>	Supplier's payload for re-generation
<code>generatedAt</code>	<code>timestamp?</code>	When the document was issued
<code>createdAt</code>	<code>timestamp</code>	—

9. Adding a new provider

1. **Create the provider class** in `src/lib/suppliers/providers/<name>-provider.ts`.

- Implement `ProviderDescriptor` on every provider.
- Implement only the capability interfaces the supplier supports.
- Throw `ProviderError` (never native `Error`) on failure.
- Return `true` from `isConfigured()` only when all required env vars are set.

2. **Export from the barrel** — add to `src/lib/suppliers/providers/index.ts`:

```
export * from "./<name>-provider";
```

3. **Register in `register.ts`**:

```
providerRegistry.register(new MyNewProvider());
```

4. **Add to PROVIDER_CATALOG** — one entry in `registry.ts` with `id`, `label`, `verticals`, `capabilities`, `envVars`, and `status: "live"`.

5. **Set priority** — choose a number relative to existing providers. If it should win over Duffel/Hotelbeds (priority 50), use `> 50`. Use `< 50` if it should only fill the gap when the primary provider is unconfigured.

No calling-code changes are needed — the registry and booking service automatically route to the new provider when it is configured.

10. Configuration and activation

All provider credentials are resolved in `src/lib/suppliers/config.ts`:

```
ATLAS_ENV=development → mock only (default when unset)
ATLAS_ENV=sandbox     → provider sandbox endpoints
ATLAS_ENV=production  → live endpoints
```

Provider	Required env vars	Optional
Duffel (flights)	DUFFEL_API_TOKEN	—
Hotelbeds (hotels)	HOTELBEDS_API_KEY, HOTELBEDS_SECRET	HOTELBEDS_HOSTNAME

When `ATLAS_ENV=production` and the credentials are set, `pick()` returns the real provider at priority 50. The mock continues to serve as a fallback only when the real provider's `isConfigured()` returns `false`.

Server Actions Reference

All server-side mutations are Next.js Server Actions under `src/lib/actions/`. Every action:

1. Calls `requireAgencyUser()` to authenticate + `get (userId, agencyId)`.
2. Validates input with Zod (most actions; see gap tracker in roadmap.md).
3. Returns `ActionResult<T> — { success: true; data: T } OR { success: false; error: string }`.

```
type ActionResult<T = void> =  
  | { success: true; data: T }  
  | { success: false; error: string }
```

bookings.ts

The largest action file (≈1000 lines). Orchestrates the full booking lifecycle.

`createBooking(input)`

Creates a new booking file.

Input: `{ clientId, destination?, departDate?, returnDate?, currency?, notes? }`

Guards: client must belong to the agency (tenant isolation). **Side effects:** auto-generates a human reference (`BKG-NNN`).

`updateBooking(bookingId, input)`

Updates booking header fields (destination, dates, currency, notes, etc.).

Guards: booking must belong to the agency.

`deleteBooking(bookingId)`

Hard-deletes a booking and all its children.

Guards: `canDeleteRecords(role)` (admin/manager only). **Note:** soft delete is not yet implemented — see database.md gap tracker.

`advanceStatus(bookingId)`

Advances the booking to the next lifecycle state.

Lifecycle: `draft → awaiting_payment → confirmed → ticketed → completed → cancelled`

Prerequisites enforced:

- `confirmed`: trip items present AND zero outstanding balance.
- `ticketed`: trip items present.

Side effects:

- On `confirmed` or `ticketed`: `autoGenerateCommissions(bookingId, agencyId)` (idempotent).
- On `ticketed`: fires `confirmItemBooking` for all unconfirmed trip items (flights, hotels, transfers)
→ calls `serviceBookFlight` / `serviceBookHotel` via the provider registry.

`bookItem(bookingItemId)`

Attempts to confirm a single booking item with the supplier.

Side effects: calls `serviceBookFlight` or `serviceBookHotel` in `booking-service.ts`. Returns {
`confirmed`: boolean; `confirmationNumber`: string; `reason?`: string }.

**`addTraveller(bookingId, input)` / `updateTraveller(travellerId, input)` /
`deleteTraveller(travellerId)`**

CRUD for `booking_traveller`. Input includes personal details (name, title, gender, passport, DOB, nationality) plus `email/phone` (required for the lead passenger by supplier APIs).

**`addBookingItem(bookingId, input)` / `updateBookingItem(itemId, input)` /
`deleteBookingItem(itemId)`**

CRUD for `booking_item`. `type` must be a `BOOKING_ITEM_TYPES` code.

`updateItemOrder(bookingId, orderedIds)`

Reorders booking items by updating `sort_order`.

`assignDay(itemId, dayIndex)`

Assigns a `booking_item` to a specific itinerary day.

`updateBookingDay(bookingId, dayIndex, input)`

Upserts a `booking_day` title/notes row for a given day index.

`shareBooking(bookingId)` / `unshareBooking(bookingId)`

Generates or clears the `booking.shareToken` (itinerary public link `/i/[token]`).

`generateVoucher(bookingId)` / `generateInvoice(bookingId)`

Server-rendered PDF generation. Guards: booking must have trip services. Sends the PDF link via `notification` log.

`sendConfirmationEmail(bookingId)`

Sends a booking confirmation email to the client via Resend. Falls back to logging when Resend is unconfigured.

clients.ts

`createClient(input)`

Creates a client record. `type` defaults to "individual". Validates country against the ISO reference list.

`updateClient(clientId, input)`

Updates client fields. Validates country, source, and industry codes.

`deleteClient(clientId)`

Hard-deletes the client and all descendant records. **Guards:** `canDeleteRecords(role)`.

`addContact(clientId, input) / updateContact(contactId, input) / deleteContact(contactId)`

CRUD for `client_contact`.

`convertToClient(clientId)`

Flips a lead (`status = "lead"`) to an active client.

opportunities.ts

`createOpportunity(input)`

Creates an opportunity. `stage` defaults to "lead".

`updateOpportunity(opportunityId, input)`

Updates opportunity fields including stage, value, and travel details.

`deleteOpportunity(opportunityId)`

Guards: `canDeleteRecords(role)`.

`advanceOpportunityStage(opportunityId)`

Advances through: `lead` → `qualified` → `proposal` → `booked` → `won`. `won` is terminal; `lost` can be set via `updateOpportunity`.

products.ts (proposals)

`createProduct(input)`

Creates a proposal draft. Auto-generates `PRD-NNN` reference.

```
updateProduct (productId, input)
```

Updates proposal fields including markup and dates.

```
deleteProduct (productId)
```

Guards: `canDeleteRecords (role)`.

```
addProductItem (productId, input) / updateProductItem (itemId, input) /  
deleteProductItem (itemId)
```

CRUD for `product_item` line items.

```
sendProposal (productId)
```

Sets `status = "sent"` and generates a `shareToken` for the public link. Sends a notification email to the client.

```
shareProposal (productId) / unshareProposal (productId)
```

Generates or clears the `product.shareToken`.

```
convertProposalToBooking (productId)
```

One-click conversion: creates a `booking` from an accepted `product`, copying client, dates, and line items. Flips the linked opportunity to "won".

proposals-public.ts

Actions callable from the **public proposal page** (`/p/[token]`) — no auth.

```
acceptProposal (token, input)
```

- Validates token; finds product.
- Sets `acceptedAt`, stamps signer name/email/IP/UA and `signatureData`.
- If `opportunityId` is linked, advances opportunity to "won".

```
declineProposal (token)
```

Sets `declinedAt`.

payments.ts

```
createPayment (bookingId, input)
```

Records a payment against a booking. `kind` must be `PAYMENT_KINDS` code. Re-calculates outstanding balance.

```
deletePayment (paymentId)
```

Guards: `canManagePayments (role)`.

```
createStripeCheckout (bookingId, amount)
```

Creates a Stripe Checkout Session (Connect: destination charge to agency). Returns {
`checkoutUrl` }.

commissions.ts

```
autoGenerateCommissions (bookingId, agencyId)
```

Idempotent — safe to call multiple times. Generates two commission rows per bookable item:

`supplier_to_agency` (from the supplier contract) and `agency_to_agent` (from
`user.commissionRatePercent`). Called automatically by `advanceStatus` on `confirmed` / `ticketed`.

```
updateCommission (commissionId, input)
```

Adjusts `status` (`confirmed` → `paid`, etc.) or `amount` / `notes`.

```
deleteCommission (commissionId)
```

Guards: `canViewFinance (role)`.

suppliers.ts

```
createSupplier (input) / updateSupplier (supplierId, input) / deleteSupplier (supplierId)
```

CRUD for the managed supplier directory.

```
createSupplierContract (supplierId, input) / updateSupplierContract (contractId, input)  
/ deleteSupplierContract (contractId)
```

CRUD for supplier contracts.

```
createSupplierRate (contractId, input) / updateSupplierRate (rateId, input) /  
deleteSupplierRate (rateId)
```

CRUD for per-product rates within a contract.

```
uploadContractFile (contractId, formData)
```

Uploads a PDF contract to Vercel Blob and saves the URL.

notifications.ts

sendNotification(bookingId, input)

Sends an email/SMS/WhatsApp/push notification to a recipient. Logs to `notification` table regardless of delivery status.

invites.ts

inviteTeamMember(input)

Creates an `agency_invite` row with a signed, expiring token. Sends an invitation email via Resend (or logs + returns the link when unconfigured).

revokeInvite(inviteId)

Marks the invite `status = "expired"`.

team.ts

updateTeamMember(userId, input)

Updates `role`, `active`, `commissionRatePercent`. Role changes are gated: only an admin can assign/change the `admin` role (`canAssignAdmin(role)`).

deactivateTeamMember(userId)

Sets `active = false`.

settings.ts

updateAgencySettings(input)

Updates agency `name`, `slug`.

updateUserProfile(input)

Updates the current user's `name`, `locale`.

onboarding.ts

dismissOnboarding()

Sets `agency.onboardingDismissedAt` to close the getting-started card.

search.ts

searchAirportsAction(query)

Routes through the Travel Platform facade (`src/lib/travel-platform`). Uses the registry's `AutocompleteCapable` provider (Duffel when configured); falls back to a built-in airport list in dev.

searchHotelDestinationsAction(query)

Filters the static `HOTEL_DESTINATIONS` list (no provider call).

searchFlightsAction(params)

Delegates to `searchFlights(params, ctx)` from the Travel Platform facade. Returns `{ ok, results, source, degraded }`. Degrades to mock on provider error.

searchHotelsAction(params)

Delegates to `searchHotels(params, ctx)` from the Travel Platform facade, which handles three paths internally: name-search (`ContentCapable`), availability search (`HotelSearchCapable` + thumbnail enrichment), and content fallback (DB cache → `ContentCapable` with city query). Returns `{ ok, results, source, degraded, estimatedPricing }`.

getHotelDetailsAction(code)

Loads rich content (photos, description, address) for one hotel via `getHotelContentCached` — DB content cache, quota-free.

searchHotelRoomsAction(params)

Re-prices every room of one hotel for a specific occupancy + date range. Calls `getHotelbedsRates` when `Hotelbeds` is configured; falls back to `mockHotelRates`. Returns `{ ok, rooms: HotelRoomRate[], degraded }`.

ai.ts

generateItinerary(bookingId, prompt)

Calls the configured AI provider (streamed) — Gemini if configured, else OpenRouter — to generate a day-by-day itinerary. Returns a `ReadableStream` — consumed by the assistant UI.

generateQuote(opportunityId, prompt)

AI-assisted quote: generates product items + summary text.

draftEmail(context, prompt)

Drafts a client communication email. Output is shown in a preview step before sending.

visaAssistant(destination, nationality)

Returns visa requirement guidance. Advisory only — never authoritative.

billing.ts

`createCheckoutSession()`

Starts a Stripe subscription Checkout for an agency. Redirect to Stripe.

`createBillingPortalSession()`

Opens the Stripe Billing Portal for subscription management.

platform.ts

Vendor-only (requires `is_platform_admin = true`).

`listAgencies()` / `getAgencyStats(agencyId)`

Vendor console: list all agencies with billing status and usage stats.

`suspendAgency(agencyId)` / `activateAgency(agencyId)`

Toggle `agency.status` ("suspended" \| "active"). Suspended agencies are locked out via `requireAgencyUser`.

portal-*.ts

Actions for the **client portal** (`/portal/...`). Authenticated via `portal_session.token` in an `httpOnly` cookie — entirely separate from Better Auth.

`portal-invite.ts`

`sendPortalInvite(clientId)`

Generates a magic-link token (`portal_session`), emails it to the client, and returns the URL (so the agent can copy it when email is unconfigured).

`portal-payments.ts`

`createPortalCheckout(bookingId, amount)`

Stripe Connect destination charge — same backend as `payments.ts`, but verifies the caller is the scoped client (via `portal_session.clientId`).

`portal-proposals.ts`

```
portalAcceptProposal (productId, input) / portalDeclineProposal (productId)
```

In-portal e-sign flow. Same logic as `proposals-public.ts` but verifies the caller is the client the proposal belongs to.

Development Guide

This guide takes you from a clean checkout to a running local Atlas instance, then documents every package script, the Drizzle database workflow, and the maintenance / seed scripts. Every command below is verified against `package.json`, `drizzle.config.ts`, `src/lib/env.ts`, and the files under `scripts/`.

Prerequisites

Requirement	Notes
Node.js	v20+ recommended. <code>scripts/setup.ts</code> enforces a v20 minimum; the repo ships a <code>.nvmrc</code> pinned to 20.
Package manager	<code>pnpm</code> is the canonical manager (the repo tracks <code>pnpm-lock.yaml</code> , and <code>build</code> shells out to <code>pnpm run db:migrate</code>). <code>npm/yarn</code> also work — substitute <code>npm run <script></code> .
PostgreSQL	A reachable Postgres instance (local, Neon, or Vercel Postgres). Its connection string goes in <code>POSTGRES_URL</code> .

Setup

From a clean checkout:

```
pnpm install           # installs deps from pnpm-lock.yaml
cp env.example .env    # then fill in POSTGRES_URL +
BETTER_AUTH_SECRET
pnpm db:migrate        # apply existing migrations to your database
pnpm dev               # http://localhost:3000 (Next.js + Turbopack)
```

`pnpm dev` runs `next dev --turbopack`. Open <http://localhost:3000>.

Alternatively, run the interactive wizard, which copies `env.example` → `.env`, checks Node/env vars, and offers to run migrations:

```
pnpm setup             # npx tsx scripts/setup.ts
```

After completing any change, run the quality gates:

```
pnpm check             # eslint . && tsc --noEmit
pnpm build:ci          # next build (no migrations)
```

See `coding-standards.md` for lint/typecheck conventions.

Package scripts

Every script in `package.json`, with exactly what it does:

Script	Command	Purpose
<code>dev</code>	<code>next dev --turbo</code>	Start the dev server with Turbopack at <code>:3000</code> .
<code>build</code>	<code>pnpm run db:migrate && next build</code>	Production build — runs migrations first , then <code>next build</code> .
<code>build:ci</code>	<code>next build</code>	Build without migrations (use in CI where migrations run as a separate step).
<code>start</code>	<code>next start</code>	Serve the production build.
<code>lint</code>	<code>eslint .</code>	Lint the repository.
<code>typecheck</code>	<code>tsc --noEmit</code>	Type-check without emitting output.
<code>check</code>	<code>pnpm lint && pnpm typecheck</code>	Combined lint + typecheck gate.
<code>format</code>	<code>prettier --write .</code>	Format all files in place.
<code>format:check</code>	<code>prettier --check .</code>	Verify formatting without writing.
<code>setup</code>	<code>npx tsx scripts/setup.ts</code>	Interactive setup wizard (env file + optional migration).
<code>env:check</code>	<code>node -e "...checkEnv()"</code>	Best-effort env sanity check; run under <code>tsx</code> for reliability (see below).
<code>db:generate</code>	<code>drizzle-kit generate</code>	Generate a new SQL migration from schema changes.
<code>db:migrate</code>	<code>drizzle-kit migrate</code>	Apply pending migrations to <code>POSTGRES_URL</code> .
<code>db:push</code>	<code>drizzle-kit push</code>	Push schema directly — do not use (see Database rules).
<code>db:studio</code>	<code>drizzle-kit studio</code>	Open Drizzle Studio to browse the DB.
<code>db:dev</code>	<code>drizzle-kit push</code>	Alias of <code>db:push</code> — same caveat, avoid.
<code>db:reset</code>	<code>drizzle-kit drop && drizzle-kit push</code>	Drop + re-push. Destructive; avoid on any shared DB.

The `env:check` script tries to `require` a `.ts` file directly under Node, which will fail in most setups. The required-env fail-fast at runtime is handled separately by `src/lib/db.ts`, which calls `getServerEnv()` (the Zod schema) at import time — `checkEnv()` itself is not wired into the running app. To run the `checkEnv()` validator manually:

```
npm run tsx -e "import { checkEnv } from './src/lib/env'; checkEnv();"
```

Environment

Copy `env.example` to `.env` and fill it in. Environment variables are validated by `src/lib/env.ts` (Zod schemas). `src/lib/db.ts` calls `getServerEnv()` at import time, so a missing or malformed `POSTGRES_URL/BETTER_AUTH_SECRET` fails fast with a clear message instead of a raw driver error.

Note: `env.example` still lists some legacy/alternate providers (Amadeus, SendGrid, Polar) that predate the current stack. The variables the running code actually reads are defined in `src/lib/env.ts` and documented below; that schema is the source of truth.

Required

Variable	Validation	Purpose
<code>POSTGRES_URL</code>	must be a valid URL	Postgres connection string (Drizzle + <code>postgres-js</code>).
<code>BETTER_AUTH_SECRET</code>	≥ 32 characters	Better Auth signing secret. Generate one per the Better Auth docs.

`checkEnv()` throws if either of these is unset. All other variables are optional and the app degrades gracefully when they are missing (see the per-variable behaviour below and `api-integrations.md`).

Optional

Variable	Default	When unset
<code>NEXT_PUBLIC_APP_URL</code>	<code>http://localhost:3000</code>	Client-exposed base URL (the only <code>NEXT_PUBLIC_*</code> var).
<code>NODE_ENV</code>	<code>development</code>	One of <code>development</code> / <code>production</code> / <code>test</code> .
<code>GOOGLE_CLIENT_ID</code> / <code>GOOGLE_CLIENT_SECRET</code>	—	Google social login is disabled.
<code>OPENROUTER_API_KEY</code>	—	AI chat / assistant features do not work.
<code>OPENROUTER_MODEL</code>	<code>openai/gpt-5-mini</code>	Overrides the default AI model.
<code>BLOB_READ_WRITE_TOKEN</code>	—	File uploads fall back to local storage instead of Vercel Blob.

<code>RESEND_API_KEY / EMAIL_FROM</code>	—	Emails are logged to the console instead of sent.
<code>STRIPE_SECRET_KEY</code>	—	SaaS subscription billing (vendor bills agencies) is disabled.
<code>STRIPE_WEBHOOK_SECRET</code>	—	Verifies Stripe subscription webhooks.
<code>STRIPE_PRICE_ID</code>	—	The subscription price used at checkout.
<code>STRIPE_CONNECT_WEBHOOK_SECRET</code>	—	Verifies Stripe Connect (traveler → agency) webhooks.
<code>STRIPE_PLATFORM_FEE_PERCENT</code>	5	Platform fee percentage taken on Connect payments.
<code>DUFFEL_API_TOKEN</code>	—	Flight search uses sample data instead of live Duffel results.
<code>DUFFEL_VERSION</code>	—	Pins the Duffel API version header.
<code>HOTELBEDS_API_KEY / HOTELBEDS_SECRET</code>	—	Hotel search uses sample data instead of live Hotelbeds results.
<code>HOTELBEDS_HOSTNAME</code>	—	Overrides the Hotelbeds API host.
<code>PROTECTED_DB_HOSTS</code>	—	Comma-separated host substrings that make destructive scripts refuse to run (see below).

`AMADEUS_CLIENT_ID / AMADEUS_CLIENT_SECRET / AMADEUS_HOSTNAME` remain in the schema as a **decommissioned** legacy flights provider (Amadeus self-service retires 2026-07-17); Duffel is the current flights provider. See [api-integrations.md](#).

When `NODE_ENV=development`, `checkEnv()` prints a warning for each unset optional integration so you can see at a glance what is running on sample/logged behaviour.

Database

Atlas uses **Drizzle ORM** over `postgres-js`. The schema lives in `src/lib/schema.ts`; migrations are emitted to `./drizzle` (`drizzle.config.ts`, `dialect postgresql`). The database connection is created once in `src/lib/db.ts`.

- Branches: `dev` `ep-wandering-sunset-aitlty78` · `prod` `ep-misty-thunder-aixz34vy`. See [database.md](#).

Migration workflow (generate → migrate, never push)

1. Edit `src/lib/schema.ts`.

2. `pnpm db:generate` — writes a new timestamped SQL file under `drizzle/`.
3. Review the generated SQL.
4. `pnpm db:migrate` — applies pending migrations to `POSTGRES_URL`.

```
pnpm db:generate      # after any schema change
pnpm db:migrate       # apply
pnpm db:studio        # browse the DB
```

Never run `db:push` (or `db:dev` / `db:reset`). Per project rules, all schema changes go through `db:generate` → `db:migrate` so migrations stay reviewable and reproducible. The `db:push` / `db:dev` / `db:reset` scripts exist from the upstream starter but must not be used against any shared database.

For UUID primary keys (all IDs except Better Auth's own tables), use randomly generated UUIDs. See `database.md` for the full schema and conventions.

Migrating production

Migrations do **not** apply themselves in `build:ci`. Always run migrations against prod after a deploy, pointing `POSTGRES_URL` at the prod branch:

```
POSTGRES_URL=<prod-url> npx drizzle-kit migrate
```

(The default `pnpm build` script already runs `db:migrate` before `next build`; `build:ci` skips it so hosts can run migrations as a discrete step.)

Maintenance & seed scripts

All scripts live in `scripts/` and are run with `tsx`, loading `.env` via `--env-file`. They import the app's `db/auth`, so `POSTGRES_URL` must point at the intended database.

```
# Promote an existing account to the platform super-admin (isPlatformAdmin=true,
# agencyId=null → routed to /platform). The account must already be registered.
npx tsx --env-file=.env scripts/make-platform-admin.ts <email>

# Seed / reset the Demo Agency (idempotent). Wipes the Demo Agency's business
data
# (clients, suppliers, opportunities, products, bookings + children, payments,
# commissions, notifications, activity), KEEPS the agency + users, then reseeds
a
# large curated set — all amounts in DZD. Requires the Demo Agency (UUID
# 00000000-...-0000000000001) and its base users to already exist; it also
(re)creates
# five agent logins (karim/lina/omar/yacine/nour @demo.test, password Agent!
2026).
npx tsx --env-file=.env scripts/seed-demo-data.ts

# Cross-tenant isolation harness — seeds two agencies, asserts no cross-tenant
```

```
# read/leak across root + child tables, then cascades cleanup of both.
npx tsx --env-file=.env scripts/test-tenant-isolation.ts

# Normalize legacy free-text country values on client/supplier rows to canonical
# ISO names (e.g. "USA" → "United States"). Idempotent; unmapped values are
# logged.
npx tsx --env-file=.env scripts/backfill-countries.ts

# Sync Hotelbeds hotel content (photos/facilities/coords) into the hotel_content
# cache table. Requires HOTELBEDS_API_KEY / HOTELBEDS_SECRET.
npx tsx --env-file=.env scripts/sync-hotel-content.ts          # curated
destinations
npx tsx --env-file=.env scripts/sync-hotel-content.ts BCN MAD  # specific
codes
SYNC_MAX=200 npx tsx --env-file=.env scripts/sync-hotel-content.ts BCN  # cap
per dest (default 80)
```

Destructive-script safety guard

The seed, isolation-test, and country-backfill scripts call `assertSafeDestructiveTarget()` (`scripts/guard.ts`) before touching data. A target is **protected** — and the script refuses to run — when either:

- `NODE_ENV === "production"`, or
- the `POSTGRES_URL` host matches any comma-separated substring in `PROTECTED_DB_HOSTS` (set this in prod/Vercel, never in local `.env`).

Override for an intentional prod run with `ALLOW_PROD=1`:

```
ALLOW_PROD=1 POSTGRES_URL=<prod-url> npx tsx scripts/backfill-countries.ts
```

See `security.md`.

Testing

Unit tests — Vitest. The unit-test runner is Vitest (`vitest@^3.2.6`, pinned to the 3.x line — vitest 4 pulls vite 8, whose transitive deps currently fail to install; `@vitest/coverage-v8` is available). Config lives in `vitest.config.ts` (node environment, `@` → `./src` alias, includes `src/**/*test.ts`).

```
pnpm test          # vitest run — single pass
pnpm test:watch    # vitest — watch mode
```

Conventions: tests are **colocated** next to the module (`src/lib/analytics.test.ts`), import `{ describe, it, expect }` explicitly from "vitest" (no globals), and cover **pure modules only** — nothing that imports `src/lib/db.ts` or triggers `env.ts` validation. Current suite: analytics, payments/summary, reports/period, status-tone, domain, export/csv, format.

Integration verification scripts. The project also ships targeted verification scripts — most

importantly `scripts/test-tenant-isolation.ts`, which seeds two agencies and asserts that no query, child-row lookup, or reference collides across tenants (then cleans up). Use these to verify changes; per project rules, never assume a change works, and if no tooling covers a change, ask whether testing should be skipped. See `coding-standards.md`.

Coding Standards

Source of truth: `AGENTS.md` (aliased by `CLAUDE.md`) holds the enforced working conventions. This doc summarizes and points there — it does not restate or override it.

Working conventions (from AGENTS.md)

- **Responses** — keep concise and to the point unless asked otherwise.
- **Planning** — always ask clarifying questions; never assume design, stack or features; use deep-dive sub-agents for research and to review plans.
- **Change/edit mode** — prefer sub-agents to implement features; act as a coordinator; parallelize independent work; use premium models for complex tasks (coding) and mid-tier for simpler ones (documentation).
- **Quality gates** — after completing features, run lint, type check and `next build` (`npm run check`, `npm run build:ci`). See `development-guide.md`.
- **Testing** — use available testing tools; never assume changes work; if no tooling exists for a change, ask whether to skip. Pure `src/lib/**` modules are unit-tested with **Vitest** (`pnpm test`; colocated `*.test.ts`, explicit `{ describe, it, expect }` imports — no globals). See `development-guide.md`.

Engineering rules

Hard rules for how code is written. Status reflects the current codebase (✓ holds · ● partial).

Rule	Status	Notes
Never duplicate logic	●	Shared helpers in <code>src/lib</code> (<code>queries</code> , <code>analytics</code> , <code>domain</code>); enforce by review.
Always use server actions	✓	20/21 files in <code>src/lib/actions</code> are <code>"use server"</code> ; mutations go through them.
Always validate input	●	Validated where it matters, not universally.
Always use Zod	●	Zod v4 present, used in ~13 files; not every action yet.
No business logic inside components	●	Logic lives in <code>actions/lib</code> ; enforce by review.
No direct SQL in UI	✓	Pages call Drizzle queries server-side / via actions, not raw SQL in client components.
Every action logs activity	●	<code>logActivity</code> in ~11/21 action files — not yet every mutation (see <code>security.md</code>).

**Every mutation is tenant
scoped**

✓

All actions go through `requireAgencyUser()` → `agencyId`
scope (see `business-rules.md`).

The ● rows (universal Zod validation, universal activity logging) are in the spec-vs-reality
gap tracker.

Database changes

- After any schema change, run **drizzle generate + migrate**. **Never** `db:push`. Workflow detail in `database.md`.
- All ID columns **not** related to Better Auth use **UUIDs**, randomly generated.

UI

- Follow the design system in `DESIGN.md` when creating or reviewing components/pages. See `ui-ux.md`.

UI / UX

Source of truth: `DESIGN.md` defines the full visual design system — colors (oklch tokens), typography, spacing, radius, shadows, animations, layout patterns, shadcn/ui component conventions, and dark mode. All new components and pages **must** follow it. This doc points there and captures product-level UX patterns only; it does not duplicate the token tables.

Atlas design principles

The UI-level expression of the ten founding Atlas principles. The product guardrails every page and feature is measured against:

1. **Never ask the user twice for the same information.** Reuse what's already captured (controlled vocabularies, reference data, client records) instead of re-prompting.
2. **Everything starts from the client.** The client record is the spine — leads, opportunities, proposals, bookings and payments all hang off it.
3. **Every action must be reversible.** Prefer soft states, confirmations and undo over destructive, irreversible operations.
4. **One source of truth.** No duplicated or conflicting data; canonical values (ISO countries, enums, references unique per agency) over free text.
5. **Automation before manual work.** Default to generating (commissions, itineraries, quotes, documents) rather than asking the user to do it by hand.
6. **Every page should answer "What should I do next?"** Surface the next action — stepper advance, CTA, empty-state prompt — never leave a dead end.
7. **Managers want insights; agents want speed.** Tailor each role's surface — analytics-dense for managers, fast and scoped for agents.
8. **No empty pages.** Every list/chart/detail has an empty state with a clear next step, never blank space.
9. **Data quality is more important than flexibility.** Constrain inputs (dropdowns, pickers, validation) to keep reporting clean.
10. **Everything should be mobile friendly.** Layouts work on small screens and in both LTR and RTL.

These principles inform the patterns below and the business rules. See also the product vision.

Design system summary

- **Stack:** Next.js + Tailwind v4 (CSS-first `@theme inline`, no config file), shadcn/ui (new-york, neutral), Lucide icons, Geist fonts, next-themes dark mode.
- **Tokens:** semantic oklch color tokens, `--radius 10px` base, Tailwind-default shadows, custom `fade-in/fade-up/scale-in` animations. Use `cn()` for class merging.

- See `DESIGN.md` for every value.

Performance budgets

Target response times — what "fast enough" means per surface. These back design principle #6 (agents want speed) and dictate where to lean on caching, streaming, and loading states.

Surface	Budget
Dashboard	< 1 s
Search (in-app lookup)	< 500 ms
Booking creation	< 2 s
Hotel search (live Hotelbeds)	< 5 s
Reports / export	< 10 s

Budgets are **targets, not yet measured** — there's no perf instrumentation in place. The slow surfaces (hotel search, reports) are I/O-bound on external APIs and the export workbook; the content cache already keeps hotel photos quota- and latency-free (api-integrations.md). Every surface over its budget must show a loading state.

Data table standard

The target capabilities for every data table (clients, bookings, opportunities, proposals, suppliers, commissions). These belong in **one reusable `DataTable` component**, not re-implemented per page:

- **Sorting** — click a column header to sort; multi-column where useful.
- **Filtering** — per-column filters plus the page-level search/filter bar.
- **Column chooser** — show/hide columns; remember the user's choice.
- **Export** — export the current view (respecting sort/filter) to CSV/Excel via the existing `/api/export` endpoint.
- **Infinite scrolling** — load more on scroll instead of hard `limit` truncation (fixes the silent-cap issue in the page audit).
- **Sticky header** — header stays visible while the body scrolls.
- **Keyboard shortcuts** — arrow-key row navigation, Enter to open, / to focus search.
- **Context menu** — right-click a row for quick actions (open, edit, duplicate, delete — gated by permissions).

Current state: tables today are plain `shadcn Table` markup with none of the above — no sorting, column chooser, infinite scroll, sticky header, shortcuts or context menu, and a hardcoded row `limit`. This standard is the target for a shared `DataTable`; building it also

clears most of the systemic gaps in the page audit (export, pagination, bulk actions).

Page requirements checklist

Every list/index page must ship with all of these — a page missing any of them is incomplete, not "v1":

- [] **Search** — free-text lookup over the page's records
- [] **Filters** — by the page's key dimensions (status, type, date range, owner)
- [] **Export** — CSV/Excel of the current view (see analytics.md)
- [] **Bulk actions** — multi-select + an action that applies to the selection
- [] **Empty state** — a prompt with a clear next step (never blank — principle #8)
- [] **Loading state** — skeleton/spinner while data resolves
- [] **Error state** — a recoverable message, not a crash
- [] **Pagination** — bounded result sets, never an unbounded dump
- [] **Mobile layout** — works on small screens, LTR and RTL (principle #10)
- [] **Permissions** — role-scoped data + role-gated actions (see business-rules.md)

This operationalizes the design principles at the page level. Detail pages additionally answer "what should I do next?" via a primary CTA or lifecycle action.

Product UX patterns

These are app-specific patterns layered on the design system:

- **Getting-started checklist** — dismissible 4-step card on the dashboard for new agencies; dismissed state persists in DB (`agency.onboardingDismissedAt`) across devices and team members.
- **Lifecycle stepper** — horizontal progress bar on booking detail with an "Advance to [next]" button and soft/hard prerequisite guards (see business-rules.md).
- **Role-gated nav** — locked nav items shown dimmed with a tooltip for non-admin roles; nav is filtered by a `show(role)` predicate.
- **List/Board toggle** — Bookings and Pipeline share data with a table↔kanban switch.
- **Inline search sheet** — "Search flights/hotels" on the trip-services panel opens a sheet pre-scoped to the booking's destination.
- **Consolidated sharing** — proposal sharing lives under one "Share with client ▼" dropdown.
- **Empty states** — charts and lists show empty-state badges/CTAs rather than blank space.

RTL & i18n

Arabic is RTL (IBM Plex Sans Arabic, `html[dir="rtl"]`). Layout must work in both directions. i18n plumbing is in architecture.md.

Atlas Travel Desk — Presentation & Brand Design Guide

Deliverable 4 · The visual language for the sales deck, mockups, and brand surface

"The Operating System for Modern Travel Agencies."

This is the design bible for everything a prospect sees: the pitch deck, the live product mockups, the one-pagers, the proposal PDFs, the website hero. It exists to make Atlas feel **inevitable** — a category-defining product built by people who sweat every pixel. Our north star for craft is the company a CEO already trusts: **Stripe** (restraint + density done right), **Linear** (speed, precision, dark surfaces), **Notion** (warmth, approachability), **Apple** (confidence through whitespace and typographic hierarchy).

A travel-agency CEO should look at one slide and think: *"This is more serious software than what my agency runs on today."* That feeling is the entire job of this document.

The tokens below are already implemented in `marketing/assets/atlas-ui.css` and the deck shell in `marketing/index.html`. This guide explains **how to wield them** — the rules, the rhythm, and the do/don't lines a designer can pick up and run with immediately.

Table of contents

1. Brand principles
2. Color palette
3. Typography
4. Iconography (Lucide)
5. Illustration & hero graphics
6. Chart & data-visualization style
7. Photography & destination imagery
8. Motion & transition system
9. Slide layout grid
10. Do / Don't quick reference
11. Designer checklist

1. Brand principles

Five principles govern every decision. When in doubt, return here.

Principle	What it means in practice
Calm confidence	We don't shout. No exclamation marks, no neon, no "🦋". Authority comes from restraint, precision, and whitespace — the Apple/Stripe move.
Density with air	Travel agencies live in dense operational screens. We show that density proudly, then frame it in generous margins so it reads as <i>organized</i> , not <i>cluttered</i> . Linear does this.
One accent, used sparingly	Atlas Blue is the only loud color. Everything else is ink, slate, and paper. A single accent makes the product feel designed, not decorated.
Real software, real data	Every mockup uses the demo agency (Atlas Travel Demo, agents Yasmine/Karim/Lina/Omar, DZD pricing, real routes like Algiers→Dubai). Never lorem ipsum. Realism is the proof.
Outcome over feature	Headlines speak the CEO's language — <i>time to booking, conversion, retention</i> — not the engineer's. The UI shows the feature; the words sell the result.

The brand voice in three words: *precise, warm, operational.*

2. Color palette

Atlas runs a **light-first** system (paper canvas, dark ink for emphasis surfaces). The palette is intentionally narrow: one brand blue, a disciplined neutral ramp, and a small functional set. Dark navy is reserved for moments of gravity — hero, section breaks, the closing slide.

2.1 Core palette

Token	Hex	Role & usage
Ink	#0E1525	The darkest surface. Sidebar, dark slides, hero gradients, table headers. The "weight" color.
Navy	#1B2239	Deep navy surfaces and hero backdrops; the second stop in the navy gradient.
Canvas	#F6F8FB	The default page/slide background. Cool paper — never pure white for large fields.
Surface	#FFFFFF	Cards, panels, table bodies, browser frames. The thing that floats on canvas.
Surface-2	#FBFCFE	Nested/zebra surfaces, table-head fill — a barely-there step down.
Border	#E6EAF1	Hairline borders and dividers. The most important neutral — it does the structural work so shadows can stay subtle.
Border-strong	#D4DAE6	Input borders, heavier dividers where a hairline reads too faint.
Text	#1B2333	Primary text. Near-black with a hint of blue so it sits in the same family as ink.

Muted	#6B7385	Secondary text, labels, descriptions, captions.
Faint	#9AA1B2	Tertiary text, placeholders, axis labels, disabled glyphs.

2.2 Brand & accent

Token	Hex	Role & usage
Brand (Atlas Blue)	#2B59C3	The one accent. Primary buttons, active nav, links, key chart series, focus rings, the logo mark. Use it <i>deliberately</i> — if everything is blue, nothing is.
Brand hover	#2249A6	Hover/pressed state for primary actions.
Brand soft	#EAF0FC	Brand-tinted backgrounds: icon chips, info panels, selection highlight, soft chart bars.
Brand ring	<code>rgba(43, 89, 195, 0.35)</code>	The 3px focus ring (<code>--ring</code>). Accessibility and "this is interactive" cue.
Hero accent (deck)	#F0A35E	Warm amber, used only on dark/navy slides for the eyebrow label — a single warm spark against deep blue. Echoes a horizon/sunset without being literal.

Atlas Blue is a confident, slightly desaturated cobalt — not a Facebook blue, not a Tailwind `blue-500`. It reads as "financial-grade software," adjacent to Stripe's indigo and Linear's accent. Do not brighten it for "energy." The restraint *is* the brand.

2.3 Functional / status colors

Each has a solid and a `-soft` tint. Soft tint is the background, solid is the text/icon — that pairing is how every badge and pill is built.

Meaning	Solid	Soft tint	Usage
Success	#1E9E6A	#E4F6EE	Confirmed bookings, paid invoices, positive deltas, won deals.
Warning	#C77800	#FBF1DF	Pending action, expiring holds, attention-needed.
Danger	#D14343	#FBE9E9	Errors, cancellations, overdue, destructive actions.
Amber	#B45313	#FBEEE2	Secondary emphasis, deadlines, "time" framing (ties to the travel/horizon motif).
Info	#2B59C3	#EAF0FC	Neutral informational notes (shares the brand hue intentionally).

2.4 Chart & avatar palette

A six-color categorical ramp. Brand blue always leads (the "hero" series); the rest are tuned to sit

in harmony and stay distinguishable for color-blind viewers and in grayscale print.

Token	Hex	Notes
--c1	#2B59C3	Brand blue — the primary/hero series, always first.
--c2	#1E9E6A	Green — second series, revenue/positive.
--c3	#B45313	Amber — third series.
--c4	#7C5CE6	Violet — fourth series.
--c5	#1FA2C7	Cyan — fifth series.
--c6	#D14343	Red — sixth / "churn or negative" series.

Avatars use a deterministic six-color set (--av1...--av6, sharing the chart hues plus a magenta #C2477F). Assign by hashing the person's name so Yasmine is *always* the same color across every screen — consistency reads as a real system.

2.5 Usage rules

- **60 / 30 / 10.** ~60% canvas + surface (paper/white), ~30% ink + neutrals (text, borders, the occasional dark surface), ~10% brand blue + functional color. If brand exceeds ~10% of a surface, pull it back.
- **Dark surfaces are punctuation, not paragraphs.** Use `ink/navy` for the hero, section dividers, and the close — moments that need gravity. A deck that is mostly dark feels heavy and "agency-template." Mostly light with dark accents feels like product.
- **Borders carry structure; shadows whisper.** Lean on `--border` for separation. Shadows are soft and low (see §2.6) — they lift cards a millimeter, never a centimeter.
- **Never invent a color.** If a hue isn't in this table, it doesn't ship. New need → propose a token, don't one-off a hex.
- **Contrast floor: WCAG AA.** Body text (`text` on `canvas/surface`) and `muted` on white both clear AA. Never put `faint` on anything but white/canvas, and never use it for anything a reader must read — it's for placeholders and axis ticks only.

2.6 Elevation (shadows & radii)

Shadows are part of the palette — they define "floating" without color.

Token	Value	Use
-- shadow- sm	0 1px 2px rgba(16,24,40,.05), 0 1px 1px rgba(16,24,40,.04)	Resting cards, inputs, badges.
-- shadow- md	0 4px 12px .../.08	Card hover, dropdowns, kanban lift.

```
--          0 12px 32px .../.12
shadow-
lg
```

Dialogs, popovers.

```
--          0 24px 60px .../.16
shadow-
xl
```

Device frames in the deck (the browser/ phone mockups) — the one place a big shadow is earned.

Radius scale: `--r-sm` 6px (tags, small chips) · `--r` 10px (buttons, inputs) · `--r-lg` 14px (cards, panels) · `--r-xl` 20px (hero containers, device frames) · `--r-full` (pills, avatars, dots). Keep radii consistent per element class — a card is always 14px; mixing 10/14/20 on sibling cards looks broken.

3. Typography

One typeface: Inter. It is the connective tissue between the product UI and the marketing surface — what the prospect sees in the deck is literally the font in the app. Inter at the marketing scale should feel like Stripe's and Linear's type: tight tracking on big sizes, generous line-height on body, tabular numerals on every figure.

```
font-family: 'Inter', -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto,
Helvetica, Arial, sans-serif;
```

Load the variable/optical-size axis and enable Inter's stylistic features for the polished look: `font-feature-settings: 'cv02', 'cv03', 'cv04', 'cv11';` (single-story a, open digits) and `tabular-nums` on all numeric data.

Monospace (`ui-monospace`, `SF Mono`, `Menlo`...) appears **only** in code, API snippets, IDs, and `kbd` chips — never for prose or numbers in a chart.

3.1 The type scale

A single ramp serves both deck and mockups. Bigger sizes get tighter tracking — that inverse relationship is the whole secret to "expensive" type.

Role	Size	Weight	Tracking	Line-height	Where
Hero display	54px (deck) / 48px (web)	800 / 720	−0.02 to −0.03em	1.04– 1.05	Slide 1 title, web hero
Slide title (h2)	34px	800	−0.02em	1.08	Section/feature slide headlines
Page title (h1)	30px	650	−0.022em	1.15	Mockup page headers, report titles

Section h2	24px	650	−0.02em	1.2	Card-cluster headings
Subhead h3	18px	650	−0.014em	1.3	Card titles, dialog titles
Lede / lead	17–20px	400	0	1.5–1.55	The sentence under a headline
Body	14px	400	0	1.5	Default UI/prose text
Body-sm	13–13.5px	400–500	0	1.45	Card body, table cells, descriptions
Label / caption	12.5px	500–580	0	1.4	Form labels, stat labels
Eyebrow	12–13px	600–700	+0.06 to +0.16em	1	UPPERCASE kicker above a title
Micro / kbd	11–11.5px	600	+0.05em	—	Table heads (uppercase), axis text, shortcuts

3.2 Weight discipline

Inter ships nine weights. We use **four**:

- **400 Regular** — body, ledes, descriptions.
- **500–550 Medium** — labels, buttons, nav items, table primary cells. (The CSS uses fractional weights like 550/580; round to the nearest available weight if a static font is used.)
- **650 Semibold** — headings, card titles, section labels, badges, the `.strong` emphasis.
- **800 Bold/Extrabold** — hero and slide titles, KPI numbers, the logo wordmark.

Never use 300 (too thin, breaks on projectors) or 900 (too heavy, looks like a discount banner). Two weights per screen is the target; three is the ceiling.

3.3 Typographic rules

- **Tracking is a function of size.** Display/title: negative tracking (`−0.02em`). Body: 0. UPPERCASE eyebrows and table heads: positive tracking (`+0.06` to `+0.16em`) or they look cramped. This single rule separates amateur from professional type.
- **Numbers are tabular, always.** Every stat, price, table figure, and chart label uses `font-variant-numeric: tabular-nums` so columns align and KPIs don't jitter. DZD/EUR/USD amounts especially.
- **One eyebrow, one title, one lede.** The canonical headline stack is: UPPERCASE amber/brand eyebrow → big tight title → muted lede. Don't stack two titles or two ledes.
- **Measure (line length) caps at ~70ch.** Ledes use `max-width: 780px`; body prose `max-w-2xl`. Long full-width lines read as a wall.
- **Sentence case for everything except eyebrows.** Titles and buttons are sentence case ("Close the booking," not "Close The Booking"). Only the eyebrow and table-head/section

labels are uppercase.

- **No italics for emphasis** — use weight (`.strong`) or the brand color (`.text-brand`). Italic is reserved for the occasional quote.

4. Iconography (Lucide)

Lucide React is the sole icon library — same set as the product, so deck and app stay identical. Lucide's 1.5–2px stroke, rounded joins, and 24px grid match Inter's geometry perfectly.

4.1 Sizing

Context	Size	Notes
Inline in badges / tags	12–13px (<code>h-3 w-3</code>)	Tiny status glyphs.
Buttons, nav, table actions	15–17px (<code>h-4 w-4</code>)	The default. Nav icons 17px, button icons 15px.
Stat / card icon chip	19–21px inside a 38–40px chip	The "feature icon" — glyph in a soft-brand rounded square.
Hero / section mark	21–34px	The compass mark, slide section icons.
Empty-state illustration	44px	Single faint glyph centered above empty-state copy.

4.2 Usage rules

- **Stroke icons only, never filled.** Keep Lucide's default outline style. A filled icon next to outline icons looks like a bug.
- **Consistent stroke weight.** Lucide default is `stroke-width: 2`. Don't mix in 1px or 3px icons; at small sizes 2px holds, at 21px+ you may drop to 1.75 for elegance — but be consistent within a screen.
- **Icons clarify, never decorate.** Every icon must name a real thing (a flight = `Plane`, a client = `User`, money = `Wallet/CreditCard`). No icon "for visual interest." If you can't name what it means in one word, remove it.
- **The icon-chip pattern.** Feature/stat icons live in a rounded square chip: 38–40px, `--r` to `--r-lg` radius, `brand-soft` background, `brand` glyph. On dark slides the chip becomes `rgba(91, 123, 180, .18)` background with `#9FC0FF` glyph. This is the single most-repeated component in the deck — keep it pixel-identical everywhere.
- **One metaphor per concept, forever.** Pick the icon for "proposal," "booking," "supplier," "commission" once and never swap. A stable icon vocabulary is what makes a product feel like a system.
- **Align to the optical center**, not the bounding box — Lucide glyphs are drawn on a 24px grid with ~2px padding; center the chip on the glyph's visual weight.

4.3 Canonical icon vocabulary (Atlas domain)

Concept	Lucide icon	Concept	Lucide icon
Dashboard	LayoutDashboard	Flights	Plane
Hotels / stays	BedDouble / Building2	Clients / CRM	Users / UserRound
Opportunities / pipeline	Kanban / Target	Proposals	FileText / ScrollText
Bookings	Ticket / CalendarCheck	Suppliers	Plug / Network
Finance / commission	Wallet / Receipt	Reports / analytics	BarChart3 / TrendingUp
AI assistant	Sparkles / Bot	Search	Search
Destinations / map	MapPin / Globe	Time-to-booking	Timer / Clock
Confirmation / success	CheckCircle2	Warning / hold	AlertTriangle

Lead with the first option; the alternate is for when two related concepts share a screen and need to be told apart.

5. Illustration & hero graphics

Atlas does **not** use cartoon/character illustration (no Stripe-style blobby people, no Notion doodles for the sales deck). The "illustration" is the **product itself, framed beautifully** — the Linear/Vercel move. This is deliberate: for a B2B operational tool, the screenshot *is* the most persuasive graphic.

5.1 The hero graphic = the product in a device frame

The repeating hero element is a **live mockup inside a browser frame** (`.browser-frame`), shown at a slight tilt or straight-on, floating on a navy gradient with a soft `--shadow-xl`. The frame includes the macOS traffic-light dots and a real URL bar (`app.atlastraveldes.com/...`). Rules:

- **Always real screens.** Pull from the eleven mockups in `marketing/mockups/` (dashboard, crm, flight-search, proposal-builder, reports, etc.). Never a fake wireframe.
- **One frame, one focus.** A hero shows a single screen, not a collage of twelve. If you must show breadth, use a 2×2 small-multiple of frames at reduced opacity behind a lead frame.
- **Float on navy, glow softly.** On the hero/section slides, the frame sits on the `navy` gradient with a large soft shadow and an optional 1px inner light border. The contrast of crisp light UI on deep navy is the signature shot.
- **Phone frame for the mobile story.** Use `.phone-frame` (notch, 9:19.5 screen) only when the narrative is "agents work from anywhere." One phone, optionally beside a browser frame.

5.2 The brand mark

The Atlas mark is a **compass / horizon glyph** in a rounded-square chip (the `.hero-mark / .logo-mark`): `linear-gradient(150deg, #2B59C3, #3B5B9A)`, white glyph, `--r-xl` radius, soft brand-blue glow. It evokes *navigation* and *operating system* without being a literal airplane or globe (those are clichés every travel tool already uses). Wordmark "Atlas Travel Desk" in Inter 800, `-0.01em` tracking, ink on light / white on dark.

- **Clear space:** keep a margin of at least the mark's height around the logo lockup.
- **Minimum size:** 22px mark in chrome, never smaller than 18px.
- **Don't** stretch, recolor, add a drop shadow beyond the defined glow, or place the mark on a busy photo without a scrim.

5.3 Abstract accents (use sparingly)

When a slide needs visual energy without a screenshot, use **geometric, on-brand accents**, never stock vector art:

- **Dotted-grid / faint guide lines** behind a hero (low-opacity, brand or border color) — the "blueprint of an operating system" motif.
- **Route lines:** a thin brand-blue arc connecting two `MapPins` (Algiers → Dubai) as a recurring decorative motif. Dashed `2 4`, rounded caps. This is the *only* travel-literal flourish we allow, and it ties to the product (real routes).
- **Soft radial glow** (`auth-bg-style` radial of `accent`) behind cards on light slides for depth.

Never: gradients-as-decoration overload, glassmorphism blur stacks, 3D isometric icon scenes, or AI-art textures. Restraint reads as expensive.

6. Chart & data-visualization style

Charts are a core selling surface — the Reports and Dashboard screens *are* the pitch for "Atlas runs your numbers." They must look like Stripe's dashboard: clean, honest, tabular, never a marketing infographic.

6.1 Style rules

- **Brand blue leads, every time.** The primary series is `--c1` Atlas Blue; secondary series step through `--c2...--c6` in order. Don't randomize series colors between charts.
- **Bars:** flat fill, **4px corner radius on top**, ~60–70% category width. Resting `--brand`; a single highlighted bar may use solid brand while the rest go `soft` (`#DCE6FA`) to direct the eye. Never gradient-fill bars in a data chart (gradients are for hero decoration only).
- **Lines:** 2.5px stroke, rounded caps/joins. Pair with a subtle **area fill** (brand at ~10–12% opacity, or a top-down gradient `id="atlasAreaGrad"`). Data points are white dots with a 2.5px colored ring — only on hover or for the latest point, not every node.
- **Donuts:** 16px stroke, rounded segment caps, a faint `--border` track behind. Center shows the total in 22px/700 ink with a 10px muted label. Max 4–5 segments; everything else is "Other."
- **Sparklines:** 2px brand stroke, no axis, no fill — inline trend beside a KPI.

- **Grid & axes:** horizontal gridlines only, `--border` color, dashed 2 4. Axis labels in `faint`, 10.5px, Inter, tabular. Drop the chart border/box — let whitespace contain it. No vertical gridlines, no chart-junk, no 3D, no shadows on data.
- **Legends** are inline chips: a 10px rounded swatch + muted label, optionally with the value in `text/600`. Place above or to the right, never a floating box over data.

6.2 KPI / stat cards

The dashboard's stat card is a recurring hero of its own: a small icon chip + muted label, a **27px/700 tabular value** (with a smaller muted unit, e.g. 1.2M DZD), and a colored delta (▲ 12% success / ▼ 4% danger) with a faint "vs last month." Keep four across on desktop, two on tablet. This is the first thing a CEO's eye lands on — make the number the loudest thing on the card.

6.3 Honesty rules

- **Y-axes start at zero** for bar charts. No truncated axes to exaggerate growth — a CEO will catch it and you lose all trust.
- **Round sensibly:** DZD 1.24M, not DZD 1,243,887.42 in a headline KPI. Full precision lives in the table below.
- **Label currency explicitly** (DZD / EUR / USD) — the demo agency is multi-currency; never show a bare number where the unit is ambiguous.

7. Photography & destination imagery

Photography is the **emotional counterweight** to the operational UI — used sparingly, at high quality, to remind the buyer *what business they're in*. The default deck is screenshot-driven; photography appears on the hero, section dividers, the "destinations" story, and the close.

7.1 Subject & selection

- **Editorial travel, not stock-travel.** Think a premium airline's brand film or Apple's "Shot on iPhone" — architectural, atmospheric, human-but-not-posed. The Atlas destination list is the shot list: **Algiers, Paris, Dubai, Istanbul, Casablanca, Marrakesh, Cairo, Hurghada, Sharm El Sheikh, Rome, Barcelona, Madrid, Antalya, Doha, Jeddah, Tunis, Maldives.**
- **Lead with our home market.** Algiers, Casablanca, Marrakesh, Tunis, Cairo first — this is a North-Africa-rooted product; showing the Casbah, Hassan II Mosque, or Jemaa el-Fnaa signals "built for agencies like yours," which beats a generic Eiffel Tower.
- **Avoid clichés:** no smiling-headset call-center stock, no thumbs-up tourists, no passport-on-a-map flat-lay, no airplane-window-wing selfie. These scream "template."

7.2 Treatment

- **Cool, slightly desaturated grade** that harmonizes with the navy/blue palette — pull warmth down a touch so a Dubai-gold shot doesn't fight Atlas Blue. Consistent grade across all photos so the deck feels authored, not assembled.

- **Always with a scrim when text overlays.** A navy gradient scrim (`linear-gradient(160deg, rgba(14,21,37,.85), rgba(36,48,86,.55))`) keeps white type AA-legible on any image. Text on the dark side, image breathing on the light side.
- **Generous crop, rule-of-thirds, room for type.** Shoot/crop wide; leave a quiet zone for the headline. Full-bleed on section slides; card-cropped (16:9 or 3:2, `--r-lg` corners) inside content.
- **Resolution & format:** 2× for retina, WebP/AVIF, never upscaled or visibly compressed. One soft, blurry, or watermarked photo undoes the whole premium impression.

7.3 When NOT to use photography

Inside product mockups, on KPI/data slides, and on dense feature slides — **no photos**. Mixing destination beauty with operational density muddies both. Photography sets the scene; the product closes the deal. Keep them on separate slides.

8. Motion & transition system

Motion is **calm, fast, and purposeful** — Linear-grade. It confirms state and guides the eye; it never performs. If an animation makes the audience wait, it's wrong.

8.1 Timing & easing

Token	Value	Use
Micro	120–150ms, <code>ease</code>	Hover color, nav background, table-row tint, button states.
Standard	200ms, <code>ease-out</code>	Card lift, dropdown/scale-in, switch toggle.
Slide transition	500ms, <code>ease</code>	Deck slide change (opacity + 14px rise + subtle 0.995→1 scale).
Reveal	300–400ms, <code>ease-out</code>	<code>fade-in</code> , <code>fade-up</code> (8px), <code>scale-in</code> (0.97→1) for entering content.

Easing language: default to `ease-out` (decelerate — content "arrives" and settles, the premium feel). Use a custom `cubic-bezier(0.16, 1, 0.3, 1)` ("ease-out-expo") for the slide transition if you want the signature Linear glide. Avoid `ease-in` (feels sluggish) and bouncy/elastic easings entirely — they read as toy-like.

8.2 The deck transition

Slides cross-fade with a gentle vertical rise and micro-scale (already in `index.html`): `opacity 0→1, translateY(14px)→0, scale(.995)→1, 500ms`. The top **progress bar** (`--brand, 3px`) and bottom **dot rail** (active dot widens to a 22px brand pill) animate in lockstep. This trio — fade-rise + progress + dots — is the entire deck motion vocabulary. Don't add slide-from-side, flips, or cube rotations.

8.3 In-slide choreography (build-ins)

When a slide has multiple elements (e.g. four feature cards), stagger their entrance: `fade-up` with **60–80ms stagger** between siblings, left-to-right / top-to-bottom. The eye lands where you intend, in order. Cap a stagger group at ~6 items; beyond that, fade the whole group as one.

8.4 Component motion

- **Cards:** `hover:translateY(-1px) + shadow-md`, 200ms. A 1px lift — felt, not seen.
- **Buttons:** background/border transition only, 150ms. No scale-on-press in marketing (save tactile press for the real app).
- **Kanban / draggable:** `translateY(-1px) + shadow-md` on hover to signal grabbability.
- **Charts:** animate the *draw* on first view — bars grow from baseline, lines stroke left-to-right (~600ms ease-out) — once. Never loop chart animations.

8.5 Rules

- **One thing moves at a time** in the viewer's focal area. Simultaneous unrelated motion = chaos.
- **Respect `prefers-reduced-motion`:** drop transforms, keep opacity-only fades. Accessibility is part of premium.
- **No autoplay loops, no infinite spinners as decoration, no parallax-on-scroll** in the deck. The only persistent motion allowed is a one-time chart draw and the loading `Spinner` in live mockups.

9. Slide layout grid

Every slide is a **16:9 canvas** with a strict, generous frame. The discipline of the grid is what makes a 30-slide deck feel like one object.

9.1 The canvas & margins

- **Aspect:** 16:9 (1920×1080 design space; the live deck is fluid/fullscreen).
- **Outer padding:** `48px top · 70px sides · 60px bottom` (from `.slide`). This wide side margin is non-negotiable — it's where the "Apple confidence" lives. Content never touches the edge.
- **Footer chrome:** a persistent bottom strip (`left/right: 88px, bottom: 30px`) with the brandmark on the left and slide context/page number on the right, in 12.5px muted. It anchors every slide and signals "this is a system."
- **Counter & progress:** top-right page counter (`N / 18`) and a full-width 3px brand progress bar; bottom-center dot rail. These frame the content without intruding.

9.2 The content grid

Inside the margins, lay out on a **12-column grid** with a 24px gutter (use the `gap-6/24px` rhythm).

Common slide archetypes:

Archetype	Layout	When
Hero	Centered stack, navy bg, compass mark → eyebrow → 54px title → lede → button.	Slide 1, close.
Section divider	Big number/eyebrow + title, dark or navy, minimal.	Chapter breaks.
Statement	Left: headline stack (5–6 cols). Right: device frame or single big visual (6–7 cols).	Most feature slides.
Split feature	Headline + <code>.flist</code> (3 feature items) on one side, browser-frame mockup on the other.	"Here's how it works" slides.
Card grid	Headline head + 3- or 4-up <code>.cards</code> row.	"Everything in one place" / capability slides.
Comparison	Headline + <code>.cmp</code> table (Old way vs Atlas).	The wedge slide.
Data	Headline + KPI row + one chart, on light canvas.	Proof/ROI slides.
Journey / roadmap	Horizontal step rail (<code>.journey</code> / <code>.road</code>) full width.	Workflow & timeline.

9.3 Vertical rhythm

Each slide reads top-to-bottom in three zones: **head** (eyebrow + title + lede, `flex: 0 0 auto`), **body** (the content, `flex: 1`, vertically centered), and **foot** (chrome). Keep the head consistent in height across consecutive slides so titles don't jump as you advance — alignment across slides is a hallmark of a designed deck.

9.4 Layout rules

- **One idea per slide.** If a slide needs two headlines, it's two slides. The deck is free; confusion is expensive.
- **Left-align by default.** Center only the hero and the close. Left-aligned headline stacks scan faster and look more editorial.
- **Optical alignment over mathematical.** Align to the visual edge of glyphs and icons (icon chips, big numbers) — nudge a few px so it *looks* aligned, even if the bounding boxes don't match.
- **Max one device frame per slide** (or a deliberate small-multiple set). Two competing frames split focus.
- **Whitespace is content.** A slide with one sentence and a lot of air is a *strong* slide, not an empty one. Resist the urge to fill.
- **Consistent foot, counter, and progress on every slide** — never hide the chrome on "special" slides; consistency is the point.

10. Do / Don't quick reference

Color

- ✓ Use Atlas Blue as the single accent; let neutrals carry the layout. ✗ Don't add a second bright accent "for variety."
- ✓ Keep dark navy for hero/section/close. ✗ Don't make the deck mostly dark — it reads heavy and template-y.
- ✓ Build badges as `soft tint background + solid text`. ✗ Don't put solid status colors as large fills.
- ✓ Stay inside the token table. ✗ Don't one-off a hex because "it's close enough."

Typography

- ✓ Tighten tracking as size grows; loosen it on uppercase eyebrows. ✗ Don't ship default-tracked 54px titles — they look unfinished.
- ✓ Use tabular numerals on every figure. ✗ Don't let KPI digits jitter between frames.
- ✓ Two weights per screen, max three. ✗ Don't mix 300/900 or four+ weights.
- ✓ Sentence case for titles/buttons. ✗ Don't Title-Case Everything Like A Banner.

Icons

- ✓ Lucide outline, consistent 2px stroke, in soft-brand chips. ✗ Don't mix filled + outline, or icons from another set.
- ✓ One stable icon per concept, forever. ✗ Don't swap the "booking" icon between slides.
- ✓ Every icon names a real thing. ✗ Don't add icons "for decoration."

Imagery & illustration

- ✓ Frame the real product in a browser/phone frame on navy. ✗ Don't use fake wireframes or cartoon characters.
- ✓ Editorial destination photography, cool-graded, with a scrim. ✗ Don't use headset call-center stock or passport-on-map flat-lays.
- ✓ Lead destination shots with our home market (Algiers, Casablanca, Marrakesh). ✗ Don't default to a generic Eiffel/Big-Ben establishing shot.
- ✓ Photos OR data on a slide, never both. ✗ Don't overlay a chart on a sunset.

Charts

- ✓ Brand-blue lead series, flat bars (4px top radius), zero-based axes. ✗ Don't gradient-fill data bars or truncate the y-axis.
- ✓ Horizontal dashed gridlines only, no box. ✗ Don't add 3D, shadows on data, or vertical grid clutter.

Motion

- ✓ 150ms micro / 200ms standard / 500ms slide, all ease-out. ✗ Don't use bounce/elastic or ease-in.
- ✓ Stagger build-ins 60–80ms; draw charts once. ✗ Don't loop animations or autoplay parallax.
- ✓ Honor `prefers-reduced-motion`. ✗ Don't make motion mandatory to understand a slide.

Layout

- ✓ One idea per slide, left-aligned, wide margins, consistent chrome. ✗ Don't cram two headlines or kill the side margins to fit more.
- ✓ Let whitespace breathe. ✗ Don't treat empty space as wasted space.

11. Designer checklist

Before any deck, mockup, or asset ships, confirm:

- [] **Font:** Inter everywhere; mono only in code/IDs; tabular-nums on all figures.
- [] **Color:** ≤10% brand blue per surface; only tokenized hexes; AA contrast on all text.
- [] **Type:** eyebrow → title → lede stack; negative tracking on titles, positive on eyebrows; ≤3 weights.
- [] **Icons:** Lucide outline, 2px stroke, soft-brand chips, one-concept-one-icon.
- [] **Product shots:** real mockups in a device frame on navy; never wireframes; one frame focus.
- [] **Charts:** brand-led series, flat bars, zero-based axes, dashed horizontal grid, no chart-junk.
- [] **Photos:** editorial + cool-graded + scrimmed; home-market-first; no clichés; not on data/feature slides.
- [] **Motion:** ease-out, 150/200/500ms; staggered build-ins; reduced-motion fallback; no loops.
- [] **Layout:** 16:9, wide margins, one idea/slide, left-aligned, persistent footer/counter/progress.
- [] **Data realism:** Atlas Travel Demo agency, real agents (Yasmine, Karim, Lina, Omar, Nadia, Sofiane), DZD/EUR/USD with explicit currency, real routes — never lorem ipsum.
- [] **Voice:** outcome-led headlines (time to booking / conversion / retention); calm, no exclamation marks, no emoji in the artifact.

Source of truth: this file plus `marketing/assets/atlas-ui.css` (implemented tokens) and the product `DESIGN.md`. When the product and marketing systems diverge, reconcile here first — the deck must always look like it was cut from the same cloth as the app, because it was.

Business Rules

Domain logic and constraints. Enums and capability helpers are defined in `src/lib/domain.ts`; server-side enforcement lives in the actions under `src/lib/actions/`.

Automation triggers

The event → action automation layer Atlas is targeting (design principle #5, *automation before manual work*). Each is a domain event that should fire a follow-up action:

Event	Action	Current state
Client created	Send welcome email	✗ not implemented (Resend only sends invites/reset/proposal/portal)
Opportunity won	Generate proposal	✗ manual (new proposal + AI quote builder)
Proposal accepted	Generate booking	✓ automatic — accepting + e-signing a proposal (client portal <i>or</i> public token link) auto-creates an <code>awaiting_payment</code> booking (best-effort, idempotent); the one-click <i>convert proposal → booking</i> remains for the agent path
Booking confirmed	Generate invoice	✗ invoices are on-demand PDFs, not auto-generated
Payment received / booking confirmed	Update accounting	⚠ partial — <code>autoGenerateCommissions</code> fires on confirm/ticket (bookings.ts); no full accounting
Proposal accepted / declined · online payment received · booking auto-created	Notify the team (in-app)	✓ automatic — best-effort in-app inbox notifications (topbar bell) to the client's owner + all active admins/managers, emitted from both accept/decline paths and the Stripe Connect webhook (<code>src/lib/notifications/inbox.ts</code>); failures never affect the business action
Travel completed	Request review	✗ not implemented

Current state: two event-driven automations run today: commission generation on booking confirm/ticket, and **auto-booking on proposal accept** (accept → e-sign spawns an `awaiting_payment` booking) — both idempotent. The rest are manual or on-demand. Building this trigger layer is the heart of the "Automation" pillar in the vision and the automated-quotation item in the roadmap.

Never rules (hard constraints)

Invariants Atlas must never violate. Each maps to an enforcement point — a violation is a bug, not a preference.

Never	Why / where enforced
Never use free text when a dropdown exists	Controlled vocabularies + reference pickers keep reporting clean (data quality > flexibility). See vocabularies.
Never duplicate client information	One source of truth — the client record is the spine; reuse it, don't re-enter it.
Never create a booking without a client	Every booking hangs off a client; the spine is mandatory.
Never hard delete data	Reversibility — prefer soft state / archival + the activity log; deletes are gated to admin/manager only.
Never expose another tenant	Every read/write is scoped by <code>agencyId</code> via <code>requireAgencyUser()</code> ; verified by <code>test-tenant-isolation.ts</code> . See security.md.
Never sum different currencies	No FX — group by currency with a DZD headline. See Currency and analytics.md.
Never make users navigate five pages	Surface work where it happens — inline search sheets, detail-page panels, "what next?" CTAs.
Never require more than three clicks	Key actions (advance lifecycle, convert proposal→booking, share, pay) are one-to-three clicks.

These are the enforcement edge of the design principles.

Golden workflow

The canonical end-to-end happy path Atlas is built around. Everything starts from the client (design principle #2) and ends by feeding the next trip — a loop, not a line.

```
Lead → Client → Opportunity → Proposal → Customer accepts → Booking →
Supplier reservation → Payment → Ticketing → Travel → Feedback → Repeat client
```

Step	In Atlas	Notes
Lead	<code>client</code> with a <code>source</code> (lead-source code)	captured as an early-stage client
Client	<code>client</code> (+ contacts)	the spine; all records hang off it
Opportunity	<code>opportunity</code> (pipeline stage, value, <code>travel_purpose</code>)	shown on the Pipeline board
Proposal	<code>product</code> (PDF + e-sign)	shareable <code>/p/[token]</code> or in-portal

Customer accepts	e-sign stamps signer + flips opportunity to won + auto-creates the booking	also acceptable in the client portal
Booking	<code>booking</code> — auto-created on accept (status <code>awaiting_payment</code>); agent one-click convert proposal → booking is the idempotent fallback	auto path starts at <code>awaiting_payment</code> , agent-drafted bookings at <code>draft</code>
Supplier reservation	<code>booking_item</code> + <code>supplier</code> picker + <code>booking_service.ts</code>	live Duffel/Hotelbeds search; real booking wired via provider registry (quote → book → idempotency → event log); activate with production credentials
Payment	<code>payment</code> (deposit/installments, Stripe Connect)	<code>confirmed</code> requires the deal's effective deposit threshold (per-proposal override → agency default 50%); zero balance required for <code>ticketed</code>
Ticketing	lifecycle <code>ticketed</code>	requires trip items; auto-generates commissions
Travel	lifecycle <code>completed</code> ; itinerary <code>/i/[token]</code>	day-by-day timeline, vouchers/invoices
Feedback	activity log / notes on the client timeline	closes the loop
Repeat client	back to Opportunity on the same client	retention, not re-acquisition

This maps onto the booking lifecycle below and the role landings in `architecture.md`.

Roles & capabilities

Five roles: **admin**, **manager**, **finance**, **support**, **agent**. The capability matrix is in `architecture.md`. Key rules:

- `seesAllData` — all roles except **agent** (agents see only their own records).
- `canManageTeam`, `canManagePayments` — admin/manager.
- `canViewFinance` — admin/manager/finance (gates `/finance`, `/commissions`, `/reports`).
- `canViewSupport` — admin/manager/support.
- `canDeleteRecords` — admin/manager.
- `canAssignAdmin` — **only an admin** can assign or change the admin role.

Onboarding & access

- **Invitation-only signup** — enforced at the auth layer (`user.create.before` hook). No invite ⇒ no account. See `security.md`.
- **Agency suspension** — a suspended agency locks out its users via `requireAgencyUser`.
- **Lapsed subscription** — gates access via `requireAgencyUser` (14-day trial on provision). See

api-integrations.md.

Currency

- Supported: **DZD (default)**, EUR, USD.
- **No FX conversion.** The agency operates in DZD; Atlas never sums across currencies. Analytics group **by currency** with a DZD headline. See analytics.md.

References

- BKG-... (bookings) and PRD-... (proposals) reference numbers are unique **per agency**, not globally.

Booking lifecycle

States: `draft` → `awaiting_payment` → `confirmed` → `ticketed` → `completed`. Two entry points can change status: a visual stepper's "Advance to [next]" button (`advanceStatus` — steps to the immediate next stage only) and a status dropdown that can jump to any status (`setBookingStatus`). **Both share the same server-side guards** — the dropdown is not a way around the stepper's rules:

- `confirmed`, `ticketed`, and `completed` all require at least one trip item.
- `confirmed` requires the **effective deposit threshold** to be paid — `paid ≥ effective deposit % of the booking total`. The percent resolves an **override chain**: `booking.depositPercent` (snapshotted at proposal → booking conversion) → `agency.depositPercent` (default **50**, configurable 0–100 in Settings → Agency) → 50. See decision 0009. This matches the promise on client proposals ("a {depositPercent}% deposit secures your dates"), which resolves the **same chain** (`product.depositPercent ?? agency.depositPercent`), so the sales copy and the lifecycle gate can never disagree. Each proposal may carry a per-deal override ("Deposit required (%)") in the builder — empty inherits the agency default, an explicit 0 means no deposit; acceptance **snapshots** the effective percent onto the booking so later default changes never alter signed terms. Agent-drafted bookings inherit the agency default until overridden on the booking edit form. A 0% deposit confirms without payment; 100% reproduces the old zero-balance behavior. Threshold math lives in the pure helper `src/lib/payments/deposit.ts` (cent-rounded comparison).
- `ticketed` and `completed` additionally require a **zero outstanding balance** — full payment is due before supplier orders are issued, not before confirmation.
- **Pricing on proposal-born bookings is locked to the signed total.** When a booking was created from an accepted (e-signed) proposal (`product.convertedBookingId` latched to the booking), each line item's sell price and quantity are read-only — enforced server-side in `updateBookingItemPricing`, with a "Pricing locked · from signed proposal" indicator in the trip-services panel. The net supplier **cost stays recordable** (it only refines the displayed margin, never the sell price). Directly created bookings keep fully editable pricing. *Known gap*: adding/

removing whole items still changes a locked booking's total.

- Reaching `ticketed` runs the supplier-confirmation flow for every item (`runTicketingConfirmation`): each item without an existing confirmation is booked via the provider registry; if any provider call fails, the whole transition **aborts** with an error — a booking is never ticketed off a fabricated confirmation. A proposal-converted item with no supplier offer attached (`details` is null) does not crash this flow: it falls back to a provisional `REF-...` reference with reason "No supplier offer attached to this item" and is left `pending`.
- Entering `confirmed` or `ticketed` (from either entry point) triggers `autoGenerateCommissions`.
- Backward moves and `cancelled` have no prerequisites — a booking must be reversible/cancellable from any state. The status dropdown only *offers* legal forward moves (current stage, one step forward, any backward stage, and always `cancelled`); a cancelled booking can only be reopened back to `draft`. The stale `paid` status option no longer appears — it was never a real lifecycle stage.
- Vouchers/invoices are **blocked** when a booking has no trip services.

Proposals & e-signature

- Server-rendered PDF; shareable via public tokenized `/p/[token]` (no login) and in-portal signing.
- E-sign stamps signer name/email/IP/UA, flips the linked opportunity to **won**, and **auto-creates a booking** (status `awaiting_payment`).
- **Accept/decline race safety** — both accept paths (`acceptProposalByToken`, `acceptProposalFromPortal`) and their decline counterparts perform an **atomic, conditional UPDATE**: `WHERE accepted_at IS NULL AND declined_at IS NULL`, using `.returning()` to detect whether the caller's write actually matched a row. Only the first writer's UPDATE matches; a concurrent/racing second call sees zero rows returned, so it never overwrites the first signer's stamped name/email/IP/signature — it is simply handed the already-recorded response instead, and runs no side effects (no second opportunity flip, no second auto-booking attempt). The recorded signer identity always reflects the true first signer, even under a race.
- **Auto-booking on accept** — both accept paths (in-portal `acceptProposalFromPortal` and public-token `acceptProposalByToken`) call the shared `createBookingFromAcceptedProposal` helper right after marking the proposal accepted. This is **best-effort**: the auto-booking is wrapped in try/catch so a booking error never fails the client's acceptance (the agent can convert manually later).
- **Idempotency** — `product.convertedBookingId` latches the created booking. The helper no-ops (returns the existing booking id) when it is already set, so re-accepting, double-submits, and the manual convert button can never spawn a second booking.
- **Tenant safety** — the helper runs on the unauthenticated client path, so it scopes every read/insert to the proposal's own `product.agencyId` (never a caller-supplied agency); the booking + items inherit that agency.
- **Convert accepted proposal → booking** in one click on the agent side (`convertProposalToBooking`) keeps `requireAgencyUser` as the guard and delegates to the

same idempotent helper.

Commissions

- **Two ledgers:** `supplier_to_agency` (agency earns from a supplier per booking item) and `agency_to_agent` (agent earns from the agency per booking).
- **Auto-generated** when a booking is confirmed or ticketed; **idempotent**.
- Agent rate from `user.commissionRatePercent`; supplier basis from the supplier contract (percent / fixed / net).

Controlled vocabularies

Seven fields are enum dropdowns (codes stored, labels shown) for clean reporting: lead source, travel purpose, trip type, gender, title, lost reason, industry. Country/nationality use canonical ISO 3166-1 values (full name stored) so spellings never drift; city uses curated autocomplete suggestions.

Client portal

- Magic-link passwordless sessions scoped to **one client**, separate from staff auth (`portal_session`).
- Agents send the invite from a client or booking detail page; the link is copyable when email is unconfigured.
- Online "Pay now" appears only when the agency has onboarded Stripe Connect. It offers the traveler **two server-computed options**: pay the booking's effective deposit % (the remainder needed to reach the deposit threshold, recorded as kind `deposit`) or the full outstanding balance. Amounts are recomputed server-side from the booking total, completed payments, and the deposit override chain (`booking.depositPercent ?? agency.depositPercent`) — a client-supplied amount is never trusted. Once the deposit is covered, only "Pay balance" is shown.
- Clients can accept/decline proposals in-portal with the same e-sign audit as the public flow.

Deployment

Atlas ships as a single Next.js app to **Vercel**, backed by a **Neon** Postgres database. Deploys are Git-driven: pushing to `main` triggers a production build that runs pending Drizzle migrations and then compiles the app. This document covers the Vercel pipeline, the dev-vs-prod database split, the full production environment, migration-on-deploy behaviour, and the demo/seed accounts.

Stack at a glance

Piece	Value	Source
Framework	Next.js 16.1.6 (App Router, Turbopack dev)	package.json
Runtime	React 19.2.4 / Node.js 20+	package.json, scripts/setup.ts
ORM / migrations	Drizzle ORM ^0.44.7 + drizzle-kit ^0.31.10	package.json
DB driver	postgres (postgres-js) ^3.4.9	src/lib/db.ts
Host	Vercel (framework: nextjs)	vercel.json
Database	Neon Postgres	docs/database.md

Vercel

- Project `atlasproject/agence_tool`, connected to GitHub.
- **Auto-deploy:** `git push origin main` → builds & ships production.
- **Manual:** `npm run vercel deploy --prod --yes`.
- Configuration lives in `vercel.json`:

```
{
  "$schema": "https://openapi.vercel.sh/vercel.json",
  "framework": "nextjs",
  "buildCommand": "npm run db:migrate && npm run build:ci",
  "installCommand": "npm install --legacy-peer-deps",
  "crons": [{ "path": "/api/cron/cleanup", "schedule": "0 3 * * *" } ]
}
```

- `installCommand` uses `--legacy-peer-deps` (also pinned via `.npmrc` → `legacy-peer-deps=true`) so React 19 peer-dependency mismatches in the dependency tree don't fail the install.
- Migrations run automatically on every deploy — no manual step needed for schema changes (see Migrations on prod).

- **Cron:** `crons` schedules a daily `GET /api/cron/cleanup` at 03:00 UTC — see [Scheduled cleanup](#) below.

Build pipeline

The Vercel `buildCommand` chains two npm scripts (from `package.json`):

Step	Script	Command	Purpose
1	<code>db:migrate</code>	<code>drizzle-kit migrate</code>	Apply pending migrations to the target DB
2	<code>build:ci</code>	<code>next build</code>	Compile the production bundle

Because the two run in sequence, a failed migration aborts the build before `next build` — the database and the deployed code never drift apart. Note that `build:ci` is deliberately migration-free; the top-level `build` script (`pnpm run db:migrate && next build`) also migrates and is used for local production builds. Only `build:ci` is invoked on Vercel, with `db:migrate` prepended explicitly by `vercel.json`.

next.config.ts

Production behaviour that ships with the build (`next.config.ts`):

- **Image optimization** — remote patterns allow-list: `lh3.googleusercontent.com`, `avatars.githubusercontent.com`, `*.public.blob.vercel-storage.com`, `photos.hotelbeds.com` (live hotel/room photos), and `picsum.photos / fastly.picsum.photos` (sample fallbacks).
- **Compression** — `compress: true`.
- **URL canonicalization** — `rewrites()` `serve /proposals*` from `/products*` and `/sourcing/hotels*` from `/hotels*`; `redirects()` send old paths to the canonical ones (`/products* → /proposals*` permanent; `/search, /hotels*, /operations → new paths`, temporary).
- **Security headers** applied to all routes: `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, `Referrer-Policy: strict-origin-when-cross-origin`, `X-XSS-Protection: 1; mode=block`, and `Permissions-Policy: camera=(), microphone=(), geolocation=()`.
- **next-intl** — the config is wrapped with `withNextIntl()` for i18n (English / Français / العربية).

Vercel environment

Environment variables are validated at runtime by a Zod schema in `src/lib/env.ts` (`getServerEnv()` / `getClientEnv()`). `POSTGRES_URL` and `BETTER_AUTH_SECRET` are the only hard requirements; everything else is optional and the app degrades gracefully when a key is absent (`checkEnv()` logs a warning in development for each missing optional group).

Required

Variable	Validation	Purpose
----------	------------	---------

POSTGRES_URL	must be a valid URL	Neon Postgres connection string (consumed by <code>src/lib/db.ts</code> and <code>drizzle.config.ts</code>)
BETTER_AUTH_SECRET	min 32 chars	Better Auth session/cookie signing secret

App / auth

Variable	Default	Purpose
NEXT_PUBLIC_APP_URL	<code>http://localhost:3000</code>	Public base URL (client-exposed)
BETTER_AUTH_URL	—	Base URL Better Auth uses for callbacks; set to the production origin
NODE_ENV	<code>development</code>	<code>development</code> <code>production</code> <code>test</code>
GOOGLE_CLIENT_ID	optional	Google OAuth — social login disabled when unset
GOOGLE_CLIENT_SECRET	optional	Google OAuth secret

Database safety

Variable	Purpose
PROTECTED_DB_HOSTS	Comma-separated host substrings that destructive scripts refuse to touch. Set to <code>ep-misty-thunder-aixz34vy</code> (prod host) in Vercel; never in local <code>.env</code> .
CRON_SECRET	Optional bearer secret Vercel injects into the <code>Authorization</code> header when calling scheduled cron routes. Without it, <code>GET /api/cron/cleanup</code> returns <code>503</code> (see below).

Integrations (optional)

Consumed across `src/lib/*` and documented in `api-integrations.md`. Each group no-ops or falls back to sample data when unset.

Variable	Default	Feature
OPENROUTER_API_KEY	optional	AI chat / assistant (disabled when unset)
OPENROUTER_MODEL	<code>openai/gpt-5-mini</code>	Default AI model
BLOB_READ_WRITE_TOKEN	optional	Vercel Blob file storage (falls back to local storage)
RESEND_API_KEY	optional	Transactional email via Resend

EMAIL_FROM	optional	Sender address; without <code>RESEND_API_KEY</code> + <code>EMAIL_FROM</code> , emails are logged to the console
STRIPE_SECRET_KEY	optional	SaaS subscription billing (vendor bills agencies)
STRIPE_WEBHOOK_SECRET	optional	Billing webhook signature verification
STRIPE_PRICE_ID	optional	Subscription price
STRIPE_CONNECT_WEBHOOK_SECRET	optional	Stripe Connect (traveler → agency) webhook secret
STRIPE_PLATFORM_FEE_PERCENT	5	Platform fee percentage on Connect payments
DUFFEL_API_TOKEN	optional	Live flight search (sample data when unset)
DUFFEL_VERSION	optional	Duffel API version header
HOTELBEDS_API_KEY	optional	Live hotel search (sample data when unset)
HOTELBEDS_SECRET	optional	Hotelbeds signature secret
HOTELBEDS_HOSTNAME	optional	Hotelbeds API host
AMADEUS_CLIENT_ID	optional	Legacy flights provider (Amadeus self-service decommissioned 2026-07-17)
AMADEUS_CLIENT_SECRET	optional	Legacy Amadeus secret
AMADEUS_HOSTNAME	optional	Legacy Amadeus host

Note: `env.example` at the repo root is a stale starter-kit template (it references Polar, SendGrid, and Amadeus-only setup). The authoritative list of variables the app actually reads is the Zod schema in `src/lib/env.ts` — use the tables above, not `env.example`, when provisioning production.

Databases (Neon)

Atlas uses two Neon branches off the same project — one for local/dev work and one for production. The dev database is treated as scratch; production is guarded so a stray local script can never wipe it.

Branch	Neon endpoint	Used by
dev	ep-wandering-sunset-aitlty78	Local development, throwaway data
prod	ep-misty-thunder-aixz34vy	Vercel production deploy

Each environment points `POSTGRES_URL` at its own branch. The production host substring is also set as `PROTECTED_DB_HOSTS` in Vercel so the destructive-script guard (below) refuses to run against

it.

The DB connection is created in `src/lib/db.ts` via `drizzle(postgres(POSTGRES_URL))` using the `postgres-js` driver, with the schema imported from `src/lib/schema.ts`. See `database.md` for the schema and tenancy model.

Migrations on prod

Migrations run automatically as part of every Vercel deploy (`npm run db:migrate` — i.e. `drizzle-kit migrate` — is prepended to the build command). No manual step is required. Migration SQL files live in `drizzle/` and are generated from `src/lib/schema.ts` via `drizzle-kit generate` (per `drizzle.config.ts`, dialect `postgresql`, output `./drizzle`).

Per project rules, schema changes must go through **generate** → **migrate**; never `drizzle-kit push` against a shared/prod database.

```
# Local workflow after editing src/lib/schema.ts
pnpm db:generate      # create a new migration in drizzle/
pnpm db:migrate       # apply it to the DB in POSTGRES_URL
```

For a one-off manual run (e.g. emergency hotfix outside a deploy):

```
POSTGRES_URL=<prod-url> npx drizzle-kit migrate
```

Destructive-script guard

Seeding, reset, and isolation-test scripts call `assertSafeDestructiveTarget()` from `scripts/guard.ts`. A target is **protected** — and the script refuses to run — when either:

- `NODE_ENV === "production"`, **or**
- the `POSTGRES_URL` host matches any entry in `PROTECTED_DB_HOSTS` (comma-separated substrings).

Override an intentional run with `ALLOW_PROD=1`. This is why `PROTECTED_DB_HOSTS=ep-misty-thunder-aixz34vy` is set in Vercel and must **not** be set in local `.env`. See `security.md` and `database.md`.

Scheduled cleanup

`GET /api/cron/cleanup` (`src/app/api/cron/cleanup/route.ts`, `runtime = "nodejs"`) runs daily at 03:00 UTC via the `crons` entry in `vercel.json`. It deletes rows that grow unbounded and have no value once expired: `booking_idempotency` rows past `expiresAt`, `portal_session` rows past `expiresAt`, and `agency_invite` rows that are still `pending` **and** expired (accepted/revoked invites are left as audit history). Auth is a shared-secret bearer check against `CRON_SECRET` (Vercel injects `Authorization: Bearer ${CRON_SECRET}` automatically for cron invocations): the route returns `503` when `CRON_SECRET` is unset and `401` on a missing/incorrect bearer token.

Deploy checklist

1. Confirm required env vars (`POSTGRES_URL`, `BETTER_AUTH_SECRET`) and any integration keys are set in the Vercel project.
2. Commit schema changes with their generated `drizzle/*.sql` migration.
3. `git push origin main` (or `npx vercel deploy --prod --yes`).
4. The build runs `drizzle-kit migrate` then `next build`; a migration failure aborts the deploy.
5. Verify the live site and, if needed, reseed the Demo Agency (below).

Demo accounts

Throwaway accounts on the **Demo Agency** (agencyId 00000000-0000-0000-0000-000000000001), all on the live site.

Role	Email	Password
Platform admin (vendor)	ouksili.abdelmalek@gmail.com	Atlas!2026
Manager	yasmine@agence.test	Agency!2026
Finance	finance@demo.test	Finance!2026
Support	support@demo.test	Support!2026
Agent	karim@demo.test	Agent!2026
Agent	lina@demo.test	Agent!2026
Agent	omar@demo.test	Agent!2026
Agent	yacine@demo.test	Agent!2026
Agent	nour@demo.test	Agent!2026

The platform admin is provisioned by promoting an existing account with `scripts/make-platform-admin.ts <email>` (sets `isPlatformAdmin = true`, `agencyId = null`, so they route to `/platform`). The agent accounts and their `Agent!2026` password are (re)created by the seed script; the manager, finance, and support accounts are pre-existing users on the Demo Agency that the seed keeps intact.

🔒 DEMO CREDENTIALS ONLY — rotate or delete before any real production use. These live passwords are checked into the repo; treat as a known risk (see `security.md`).

Suggested demo flow: sign in as vendor → `/platform` → View as Demo Agency → manager dashboard → switch to finance/support/agent views → Settings → switch to Français or العربية.

Reseeding demo data

`scripts/seed-demo-data.ts` rebuilds a rich, realistic dataset for the Demo Agency. It is **idempotent**: it wipes the Demo Agency's business data (clients, suppliers, opportunities, products, bookings + children, payments, commissions, notifications, activity) but **keeps the agency and its users**, then reseeds. All amounts are denominated in DZD (Algerian Dinar). The seed is gated by the destructive-script guard, so it refuses to run against the protected prod host without `ALLOW_PROD=1`.

```
npx tsx --env-file=.env scripts/seed-demo-data.ts
```

Roadmap & Changelog

Phase status

Phase 1 — Sellable ✓ COMPLETE Email (Resend) · Stripe SaaS billing · Separate Neon DB branches (dev/prod) · PDF proposals · E-signature · Live flights (Duffel) · Live hotels (Hotelbeds + content cache).

Phase 2 — Competitive ✓ COMPLETE Supplier management (`/suppliers`, contracts, rates, picker) · Commissions tracking (auto-generated, two-ledger, `/commissions`) · Client portal (magic-link login, trip view, online payments via Stripe Connect, in-portal proposal signing + portal invite from agent).

Phase 3 — AI differentiation ✓ COMPLETE AI itinerary generation · AI quote builder · AI email drafting · AI visa assistant — all inline, embedded where the work happens. See `ai.md`.

UX overhaul ✓ COMPLETE (20 changes across 4 size categories) Getting-started checklist · Lifecycle stepper · Board view toggle · Inline search · Portal invite · Convert proposal→booking · Client funnel timeline · Booking hard guards · Vocabulary pass · Role nav tooltips · Empty-state badges.

Data quality & BI ✓ COMPLETE (4 phases) DZD-only currency (no FX) · **P1** richer analytics (revenue/conversion/pipeline funnel/AR aging/margin) · **P2** controlled-vocabulary enums (7 fields, codes+labels) · **P3** ISO country/nationality reference data + city suggestions · **P4** standardized CSV/Excel export (`/reports`, dual code+label columns). Hotel search-by-name added. See `analytics.md`.

Module maturity

Maturity levels: **Production** (stable, in daily use) · **Beta** (works, hardening) · **Alpha** (functional, evolving) · **Planned** (not built).

Module	Status
CRM	Production
Sales	Production
Hotels	Beta
Flights	Beta
Accounting	Planned
AI	Alpha

Supplier Portal

Planned

Flights/Hotels nuance: *live search* is shipped for both (Duffel + Hotelbeds); the maturity above reflects the end-to-end **booking** flow — Hotels is further along (Beta), real Flights booking is Planned (Duffel orders). See `api-integrations.md` and Open item #1.

UI redesign (phased) — ✓ COMPLETE

Brought the **live app** up to the marketing-grade standard set by the sales deck and mockups under `marketing/` — a full visual/UX upgrade of **every one of the 55 routes**, shipped by phase, **preserving all functionality**. Follow-on polish added a unified `StatStrip` KPI band app-wide. See the plan at `specs/ui-redesign/PLAN.md` and the decision at `docs/decisions/0005-app-ui-redesign.md`.

Phase	Scope	Status
0	Design foundation (tokens + primitives) + seed full demo data	✓ done
1	Dashboard · CRM (list + profile) · Pipeline	✓ done
2	Proposal builder · Flight search · Hotel search	✓ done
3	Bookings & Booking details · Finance · Commissions	✓ done
4	Customer portal · Reports · AI assistant	✓ done
5	Responsive, empty/loading/error states, polish	✓ done

Since the redesign

- **AI → Google Gemini** (primary; OpenRouter fallback). The `/assistant` gained tenant-safe read tools across **every data domain** (bookings, clients, proposals, pipeline, finance, commissions), a live client/booking context rail, and platform-admin "view as" scoping; provider/multi-turn/currency fixes along the way. See `ai.md`.
- **Auto-booking on proposal accept** — accepting + signing a proposal auto-creates a booking (`awaiting_payment`), idempotent + tenant-safe. `product.convertedBookingId` (migration `0020`). See `business-rules.md` and decision `0006`.
- **Dashboard** deduped (removed 6 KPIs/panels that repeated the manager Insights) + an operational "act-today" band (departures 7d, proposals awaiting response, overdue, new clients) + a "closing this month" chart.

Marketing assets (deck + 11 mockups + demo dataset + design guide) live under `marketing/` and are the visual target.

Phase 1 delivery (command center). Rebuilt Dashboard, CRM list + client profile, and the Pipeline board to the mockup standard, reusing the Phase 0 primitives (StatCard deltas, insight charts) + three new shadcn primitives (Tabs, Breadcrumb, Tooltip). **Delivery policy — redesign,**

not rewrite: no schema changes; any mockup field without a backing column is **derived from real data** (lifetime value, trips, win rate, spend-by-year, MoM deltas, derived follow-ups from balances/close-dates/passport alerts) or **omitted** — never fabricated. Notable omissions (no backing data): client tags/loyalty/grade/ travel-preferences/Documents/Passport/Family tabs; opportunity reference ids and per-stage micro-labels; the dashboard "Atlas suggests" AI card and free-text tasks. This same derive-or-omit policy governs Phases 2–5.

Phases 2–4 delivery (sell · operate · client experience). Applied the command- center system across the revenue, fulfilment, and client-facing surfaces, all redesign-not-rewrite (no schema/logic changes; presentational + new-scaffold only):

- **Phase 2** — proposal builder (breadcrumb + sectioned card; line items got per-category colour tiles, per-line margin, a totals panel); flight search (airline logo tiles, elevated panel, loading skeleton, EmptyState); hotel search (same treatment). *Omitted:* the mockup's live proposal preview pane (net-new renderer — public `/p/[token]` view + server PDF already exist) and the flight sidebar-filter layout.
- **Phase 3** — bookings list KPI strip (counts only, no FX summing) + `loading.tsx/ error.tsx`; booking-details (breadcrumb, derived KPI strip, card-elevated, finance hero with payment progress bar) via a map→plan→implement→verify workflow; finance (card-elevated charts/ AR, EmptyState, row hover) + commissions (elevated table).
- **Phase 4** — customer portal (elevated/interactive cards, EmptyStates, balance hero) via a 4-page parallel workflow; reports export hub (per-dataset icon tiles); AI assistant (brand Bot tile, role icons, shadow Input).
- **Verification:** build-gated (lint + tsc) on every commit; the heavier screens ran adversarial multi-agent review (regression + design + policy lenses) — which caught and fixed a real client-facing regression in the portal (a negative balance could be shown to a client). Visual/ browser QA is still pending (local dev DB has no seeded user; signup is invite-only).
- **Docs:** DESIGN.md gained the amber "warning / amount-due" status colour (was undocumented despite wide use). i18n of new English labels on deeper pages is tracked under open-item #2 (translate deeper pages), consistent with those pages' pre-existing hardcoded strings.

Open items

1. **Real supplier booking** — ✓ architecture complete (Sprint 1 Waves 1–3). Provider registry, quote→book→cancel lifecycle, idempotency, event log, and supplier-ref tables are all wired. Activate by setting `DUFFEL_API_TOKEN` (flights) and `HOTELBEDS_API_KEY/HOTELBEDS_SECRET + ATLAS_ENV=production` (hotels).
2. **Translate deeper pages** — bookings, clients, finance, support, platform (i18n plumbing ready).
3. **Cross-device locale** — sync `user.locale` → cookie on login.
4. **WhatsApp integration** — Meta Cloud API adapter (skeleton ready; Meta Business account needed).
5. **Automated quotations** — trigger quote generation from opportunity stage change.

6. **Agent performance scoring** — leaderboard from commission + booking counts.

Known issues (non-blocking)

- **Topbar hydration warning (dev-only).** A React 19 + Radix `useId` mismatch logs on the topbar menus (`AppShell` → `Topbar` dropdowns) in the dev console. **Benign:** React recovers automatically, every menu/theme/nav works, and it does not surface in the production build. Root-caused as far as reasonable: the mobile `Sheet` (the only `useId` consumer before the topbar) was proven *not* responsible — the mismatch is byte-identical even with the `Sheet` absent during hydration — and the root providers already set `suppressHydrationWarning`. It appears to be a React streaming/Suspense × Radix `useId` interaction; most likely resolved by a future Radix/Next minor bump. **Do not** re-chase via app-code edits to the shell (already attempted; no fix, and it risks the verified-working chrome). Related: [docs/decisions/0005-app-ui-redesign.md](#).

Planned modules

The longer-horizon module backlog, grouped. Some already have partial foundations (noted) — the rest are net-new.

Accounting

- Accounting · General Ledger · Payroll — net-new finance suite.
- Invoices —  on-demand invoice PDFs exist today; this is the move to managed, numbered, ledgered invoices.

Marketing

- Marketing · Email campaigns — net-new (transactional email via Resend exists; campaigns do not).
- WhatsApp —  adapter skeleton ready (open item #4); needs a Meta Business account.
- SMS — net-new channel adapter.

Documents

- Document management — net-new (today: PDF generation + Vercel Blob uploads for supplier contracts).

Travel products

- eVisa · Visa tracking —  an AI visa assistant exists; these add structured application + tracking.
- Insurance —  exists as a trip-service item type; this is a managed insurance product/flow.
- Vehicle rental · Cruises — net-new verticals (alongside flights/hotels).
- Package builder — net-new (bundle multi-service trips).

Ecosystem

- Supplier portal — net-new (suppliers self-serve, parallel to the client portal).

- Customer loyalty · Affiliate program — net-new growth modules.
- API marketplace — net-new (expose/consume third-party integrations).

These are directional, not scheduled. Phase status and the active gap list are above; this is the "what's next after the core is hardened" backlog.

Spec vs. reality gap tracker

The design system (principles, never-rules, page/table/entity standards, AI guardrails, automation triggers, perf budgets) describes a **target state**. This table consolidates every item where the spec is ahead of the code, so the gaps are one punch list instead of scattered "current state" notes.

Status: ✓ done · ● partial · ● not started

Item	Spec	Status	Notes
Agent visibility scoped to own records on list pages	security	✓	Fixed — clients/bookings/opportunities/products now scope by owner column (commit 134ceb3).
Quality gate <code>npm run check</code> works (pnpm v11)	development-guide	✓	Fixed — config moved to <code>pnpm-workspace.yaml</code> ; 0 lint/type errors (commit 7d5f4e1).
Soft delete (<code>deletedAt</code> , filtered reads)	database	●	No soft-delete column anywhere; "never hard delete" is currently aspirational. Needs migration.
<code>reference</code> on all entities	database	●	Only <code>booking</code> + <code>product</code> ; missing on clients/opportunities/suppliers. Needs migration.
Consistent <code>updatedAt</code> / <code>status</code> / <code>notes</code> on children	database	●	Present on roots, inconsistent on child tables.
Page systemic-five: per-page Export	ui-ux	●	Endpoint exists (<code>/api/export</code>); list pages lack buttons.
Page systemic-five: Bulk actions	ui-ux	●	The <code>Checkbox</code> primitive exists (<code>src/components/ui/checkbox.tsx</code> , used in hotel/flight search) but no list page wires it up for row selection + bulk operations yet.
Page systemic-five: Loading states	ui-ux	●	24 <code>loading.tsx</code> under <code>src/app/(app) /</code> (29 repo-wide, incl. auth/portal/platform). Most list + detail pages covered; not universal.

Page systemic-five: Error states	ui-ux	●	8 <code>error.tsx</code> repo-wide: route-level for <code>root</code> , <code>(app)</code> , <code>assistant</code> , <code>bookings</code> , plus <code>(auth)</code> , <code>portal</code> , <code>portal/proposals</code> , <code>platform</code> . Most other <code>(app)</code> sub-routes still fall through to the shared <code>(app) / error.tsx</code> .
Page systemic-five: Pagination	ui-ux	●	Hardcoded <code>limit</code> (200/500) silently truncates.
Shared DataTable (sort, filter, column chooser, infinite scroll, sticky header, shortcuts, context menu)	ui-ux	●	Tables are plain shadcn markup. Building this clears most of the systemic-five.
Mobile-friendly tables (<code>overflow-x-auto</code>)	ui-ux	●	Tables overflow on phones; only <code>assistant</code> wraps for scroll.
Automation triggers (welcome email, won→proposal, accepted→booking, confirmed→invoice, completed→review)	business-rules	●	Only commission auto-gen on confirm/ticket fires; rest manual/on-demand.
AI mutation behind a hard confirm step	ai	●	<code>createBooking</code> is intent-gated by prompt only, not an enforced confirm gate.
Performance instrumentation vs budgets	ui-ux	●	Budgets defined; nothing measured.
Real supplier booking (Duffel orders, Hotelbeds book)	api-integrations	✓	Architecture complete (Sprint 1 + 2). Activate with production credentials (see open item #1).
Cross-device locale sync	architecture	●	English until re-pick on a fresh device (also open item #3 above).
Rate limiting (auth + API)	security	●	No throttling anywhere.
GDPR subject export/erasure + consent	security	●	No flow; erasure also blocked by missing soft delete.
Disaster recovery runbook + RTO/RPO	security	●	Neon has provider backups; no documented DR plan.
Universal audit-log coverage	security · coding-standards	●	<code>logActivity</code> in ~11/21 action files; not all mutations logged.
Universal Zod input validation	coding-standards	●	Zod in ~13/21 action files; not every action validates.

<code>activity_log</code> partitioning + retention (500M-row target)	architecture	●	Single unpartitioned table; no archival.
Search concurrency/backpressure (100 concurrent)	architecture	●	No queue or rate control on hotel/flight search.

Changelog

Commit	Summary
1163ab9	Auto-booking on proposal accept — accepting + signing a proposal auto-creates an <code>awaiting_payment</code> booking (idempotent, tenant-safe); <code>product.convertedBookingId</code> migration 0020; decision 0006
b76c911	Dashboard deduped vs manager Insights (removed 6 repeated KPIs/panels) + operational "act-today" band (departures 7d, proposals awaiting, overdue, new clients) + "closing this month" chart
3cec99d ea3c760	AI assistant → analyst : tenant-safe read tools across every data domain (bookings, clients, proposals, pipeline, finance, commissions) + a live client/booking context rail
2f08368	Assistant honors platform-admin "view as" agency scoping (empty-CRM bug)
f088abc... 16e9b17	AI provider → Google Gemini (primary; OpenRouter fallback + runtime chain); multi-turn tool-call fix, DZD (not EUR) internal figures
c7b9acc	Unified StatStrip KPI band rolled out app-wide
f3564ec... b41c8c2	UI redesign complete — all 55 routes to deck standard (tokens/primitives, 🎨 palette, mobile tab bar, per-screen rebuilds)
5d9ab6d	UI redesign deck-fidelity rebuild : flight search, booking details, customer portal & AI assistant rebuilt to their mockups (audit found phases 2–4 were token-skinned only, fidelity 1–2/5 → now 4–5/5, zero functional regressions)
eadc1b8	UI redesign: Reports → analytics dashboard (deck slide 12) — 6-KPI row, revenue trend, weighted-pipeline forecast card, "period at a glance"; CSV/Excel export preserved
ffa3dd4	UI redesign: deck identity adopted app-wide — Atlas Blue primary, cool-paper canvas, white cards, dark ink sidebar, 14px cards, deck shadows/charts (<code>globals.css</code> + <code>app-shell</code>); Geist kept. See decision 0005
b48c626	UI redesign Phase 1 : Dashboard, CRM list + profile, Pipeline (command center) — StatCard deltas, +Tabs/Breadcrumb/Tooltip primitives
84aab5e	UI redesign Phase 0 : design foundation — chart palette, brand accent, elevation tokens, StatCard deltas
9b151f3	UI redesign Phase 4 (skin) : AI assistant polish (brand Bot tile, role icons, shadow Input) — later fully rebuilt in 5d9ab6d

522c742	UI redesign Phase 4 : customer portal (4 pages, parallel workflow) — elevated cards, EmptyStates, balance hero; +DESIGN.md amber status colour
9b3f392	UI redesign Phase 4 : reports export hub — elevated cards + per-dataset icon tiles
853e195	UI redesign Phase 3 : finance/commissions polish + bookings-list <code>loading.tsx/error.tsx</code> (workflow)
ebaba97	UI redesign Phase 3 : booking-details — breadcrumb, KPI strip, finance hero (map→plan→implement→verify workflow)
7c9b378	UI redesign Phase 3 : bookings-list derived KPI strip (counts, no FX summing)
70d52a5	UI redesign Phase 2 : hotel search — elevated panel, loading skeleton, EmptyState
b754e77	UI redesign Phase 2 : flight search — elevated panel, loading skeleton, EmptyState
8b25e19	UI redesign Phase 2 : flight result cards — airline logo tiles + interactive hover
7f21420	UI redesign Phase 2 : proposal line items — category tiles, per-line margin, totals panel
403781e	UI redesign Phase 2 : proposal-builder shell — breadcrumb + sectioned card
b48c626	UI redesign Phase 1 : Dashboard, CRM list + profile, Pipeline rebuilt to mockup standard; +Tabs/Breadcrumb/Tooltip primitives; derive-or-omit policy (no schema changes)
84aab5e	UI redesign Phase 0 : design foundation — refined chart palette, brand accent, elevation tokens, StatCard deltas
(docs)	UI redesign initiative: decision (ADR 0005) + phased plan (<code>specs/ui-redesign/</code>)
e842828	Marketing: full demo dataset (150 clients), design guide + mockup consistency fixes
3f58668	Marketing: premium 18-slide sales deck + 11 high-fidelity UI mockups (<code>marketing/</code>)
95e0c1b	fix: run db:migrate on every Vercel deploy (migration 0019 was missing from prod)
00c6885	Sprint 2 Waves 1–4: Travel Platform facade — ContentCapable, AutocompleteCapable, <code>src/lib/travel-platform/index.ts</code> , consumer migration
b7eab1b	Sprint 1 Wave 3: booking-service — quote→book lifecycle, idempotency, event log, supplier ref
cc65ce1	Sprint 1 Wave 2: DuffelBookingProvider, HotelbedsBookingProvider, MockBookingProvider, registry wiring
bb571d2	Sprint 1 Wave 1: booking_supplier_ref/event/document/idempotency schema + config module (migration 0019)
a6def26	Phase 4: standardized BI data export (CSV + Excel) at <code>/reports</code>
148d1fb	Phase 3: country/nationality reference data + city suggestions
e09289f	Phase 2: controlled vocabularies (enums) for cleaner reporting

0f6840f	Phase 1 analytics: revenue/conversion/pipeline insights (DZD)
b7f71024	Enrich demo seed: 100 clients, suppliers, commissions, notifications (DZD)
0b5de21	Restrict currencies to DZD/EUR/USD, default DZD; add hotel name search
441fd6e	UX large: persistent onboarding (DB), client timeline, booking hard guards
284d00e	UX medium: lifecycle stepper, board view, inline search, getting-started card
805dbc8	UX small: portal invite, convert proposal→booking, client funnel view, share consolidation
9c7cc98	UX quick wins: vocabulary, nav labels, empty charts, dashboard CTA
23ddc35	Phase 2 & 3: supplier mgmt, commissions, client portal, AI features
94aaf08	Phase 2: real bookings, Stripe Connect, traveler portal, full i18n
2dd7650	Embed live flight/hotel search in booking Add dialog
a677b77	Fix duplicate-reference crash on create booking/proposal
169ee74	Split atlas.md into focused docs under docs/atlas/
8c1b24e	Hotel module: full search/results/details, dynamic occupancy pricing, content cache
b67cfa0	Phase 1: email (Resend), Stripe billing, e-sign, PDF, suppliers
a233d32	View-as-user + i18n (EN/FR/AR + RTL) + Settings hub
1896596	Per-role workspaces (Finance + Support) + role-based landing/nav
9e8fb4b	Multi-tenant architecture + vendor platform console

Migrations: 26 (latest 0025 — booking-item net cost). Prod DB: `ep-misty-thunder-aixz34vy`.

Security

Security controls

The control areas Atlas is held to, with honest current status (✓ in place · ● partial / provider-level · ○ not implemented). Details for the implemented ones follow in the sections below.

Control	Status	Notes
Tenant isolation	✓	<code>agencyId</code> scoping via <code>requireAgencyUser()</code> ; verified by <code>scripts/test-tenant-isolation.ts</code> .
RBAC	✓	5 roles + capability helpers in <code>src/lib/domain.ts</code> ; agent visibility scoped on list pages too.
Webhook verification	✓	Stripe billing + Connect webhooks verify the <code>Stripe-Signature</code> HMAC (with replay-window + constant-time compare) before acting.
Audit logs	●	<code>activity_log</code> + <code>logActivity</code> used in ~11 action files; not yet universal.
Encryption	●	TLS in transit + Neon at-rest (provider); Better Auth hashes passwords. No app-level field encryption.
Password policy	●	Better Auth defaults only; no enforced strength/rotation policy.
Secrets management	●	Vercel env vars; demo credentials committed in the repo (rotate before prod). No vault.
Backups	●	Neon automatic backups + branching (provider-level); not app-managed or restore-tested.
Rate limiting	●	None on auth, portal magic-link, or API routes.
GDPR	●	No subject data-export/erasure flow or consent tracking; right-to-erasure also blocked by missing soft delete.
Disaster recovery	●	No documented DR runbook or RTO/RPO targets.

The ●/● rows are tracked in the spec-vs-reality gap tracker.

Tenant isolation

Every business table carries `agencyId` (root) or inherits it via a parent (children). **All** reads/writes are scoped by agency through `requireAgencyUser()` → `user.agencyId`. Reference numbers (`BKG-...`, `PRD-...`) are unique per agency, not globally. See `architecture.md`.

How scoping is enforced

There is `no middleware.ts` performing tenant routing — isolation is enforced in-code at the top of every protected Server Component, layout, and Server Action:

1. The page/action calls `requireAgencyUser()` (`src/lib/permissions.ts`), which returns an `AgencyUser` whose `agencyId` is guaranteed non-null.
2. Every subsequent Drizzle query filters on that `agencyId`. Root tables filter directly; child rows use a **double constraint** (the child's own id *and* the parent's `agencyId`) so a guessed child id from another tenant returns nothing.

`agencyId` appears on 51 references across `src/lib/schema.ts` — every tenant root table and audit/notification table carries it, with `onDelete: "cascade"` back to `agency` so deleting a tenant removes all of its data.

Verification harness

`scripts/test-tenant-isolation.ts` (Wave 6) seeds two full agencies (A and B) with rows in every tenant table plus child tables, then asserts:

1. Every agency-root query scoped to A excludes B's rows (and vice-versa).
2. The exact `findFirst(id=B, agencyId=A)` pattern the actions/pages use returns nothing (no cross-tenant read by guessed id).
3. Child rows can't be reached via the other agency's parent (the double-constraint pattern used by booking/product child mutations).
4. References are unique **per** agency — the same `BKG-5001` inserts cleanly into both agencies (would collide under a global unique).
5. The activity log and notifications are agency-scoped.

It always cleans up its two test agencies (cascade) at the end. Run it against a non-prod DB:

```
npx tsx --env-file=.env scripts/test-tenant-isolation.ts
```

Authentication

Staff authentication is **Better Auth** (`better-auth@1.6.11`) with email/password, configured in `src/lib/auth.ts` over the Drizzle Postgres adapter. Sessions are resolved server-side with `auth.api.getSession()` inside `requireUser()`.

Setting	Value / source	Purpose
<code>emailAndPassword.enabled</code>	<code>true</code>	Email + password sign-in (Better Auth hashes passwords).
<code>emailVerification.sendOnSignUp</code>	<code>true</code>	Sends a verification email on registration.

<code>sendResetPassword</code>	Resend (or console fallback)	Password-reset email flow.
<code>baseUrl / trustedOrigins</code>	<code>BETTER_AUTH_URL</code> → <code>NEXT_PUBLIC_APP_URL</code> → <code>http://localhost:3000</code>	Restricts trusted origins to the deployed domain.
<code>BETTER_AUTH_SECRET</code>	<code>env (validated_min(32) in src/lib/env.ts)</code>	Session signing secret; startup fails if missing.

There are **no social/OAuth providers wired up** in `auth.ts` — although `GOOGLE_CLIENT_ID` / `GOOGLE_CLIENT_SECRET` are declared *optional* in `src/lib/env.ts`, no `socialProviders` block exists, so Google sign-in is not active.

Invitation-only signup

Registration is **invitation-only**, enforced in the Better Auth `user.create.before` database hook (`src/lib/auth.ts`):

- The hook calls `findPendingInviteByEmail()` (`src/lib/invites.ts`). If no pending, unexpired `agency_invite` matches the email, it throws `APIError("FORBIDDEN")` — this is the single enforcement point, so it also blocks direct calls to the public sign-up endpoint.
- On a match it stamps `agencyId` + `role` from the invite and forces `isPlatformAdmin: false, active: true`.
- The `user.create.after` hook consumes the invite via `markInviteAccepted()`.

The tenant/role columns (`agencyId`, `isPlatformAdmin`, `role`, `active`) are declared as Better Auth `additionalFields` with `input: false`, so a client **cannot** set them during sign-up — only the server hook can.

Invite tokens are generated with `randomBytes(24).toString("base64url")` and have a **7-day TTL** (`INVITE_TTL_DAYS`, `src/lib/invites.ts`). Lookups require the invite to be `status: "pending"` and `expiresAt > now`.

Account deactivation

A deactivated team member (`active === false`) is signed out on their next request: `requireUser()` redirects them to `/login?error=account_disabled`.

Authorization guards

`src/lib/permissions.ts` provides the guard set. Each runs at the top of a protected page/layout/action and **redirects** (never silently continues) when the check fails.

Guard	Requirement	On failure
-------	-------------	------------

<code>requireUser()</code>	Valid session; not deactivated	<code>/login</code> (or <code>/login?error=account_disabled</code>)
<code>requireAgencyUser()</code>	Belongs to an agency; agency <code>active</code> ; subscription not blocking	<code>/platform</code> , or <code>/login?error=agency_suspended / ?error=subscription_inactive / ?error=no_agency</code>
<code>requireManager()</code>	<code>canManageTeam(role)</code> (admin/manager)	<code>/dashboard</code>
<code>requireCapability(check, redirectTo?)</code>	Arbitrary capability helper from <code>@/lib/domain</code>	<code>redirectTo</code> (default <code>/dashboard</code>)
<code>requirePlatformAdmin()</code>	<code>isPlatformAdmin === true</code>	<code>/dashboard</code>
<code>getOptionalUser()</code>	— (never redirects)	returns <code>null</code>

`requireAgencyUser()` additionally re-reads the agency's `status` and `subscriptionStatus` on every call: a **suspended** agency (`status !== "active"`) or one whose subscription is in a blocking state locks out its real members. Blocking subscription states are `canceled`, `unpaid`, and `incomplete_expired` (`BLOCKING_SUBSCRIPTION_STATUSES` in `src/lib/billing/stripe.ts`); `NULL` / `trialing` / `active` all pass. The platform admin is exempt from these locks so they can still "view as" the agency for support.

Roles & capabilities

Five roles are defined in `src/lib/domain.ts` (`USER_ROLES`): `admin`, `manager`, `finance`, `support`, `agent`. Capability helpers gate features:

Helper	Roles allowed	Gates
<code>seesAllData</code>	admin, manager, finance, support	Agency-wide visibility; agents see only their own rows (applied on the clients / products / opportunities / bookings / operations / dashboard list pages)
<code>canManageTeam</code>	admin, manager	Team & role management (<code>requireManager</code>)
<code>canAssignAdmin</code>	admin	Grant/change the admin role (prevents privilege escalation)
<code>canManagePayments / canViewFinance</code>	admin, manager, finance	Payments & finance — <code>recordPayment/deletePayment/createPaymentLink</code> require <code>canManagePayments</code> ; <code>getCommissions/getCommissionsByBooking/getCommissionSummary</code> require <code>canViewFinance</code> (an agent gets an empty result, not an error)

<code>canDeleteRecords</code>	admin, manager	Deleting records
<code>canViewSupport</code>	admin, manager, support	Support workspace

Capability rules are documented in `business-rules.md`. Only an **admin** can assign/change the admin role.

Platform admin & impersonation

- The platform admin (vendor) has `isPlatformAdmin = true` and `agencyId = null`; they sit above all tenants and are routed to `/platform`. Because they have no agency, `requireAgencyUser()` sends them to their console and they **cannot enter a tenant except via impersonation**. Promotion is a deliberate, out-of-band step via `scripts/make-platform-admin.ts <email>`.
- Impersonation is **cookie-driven and platform-admin only**, set by Server Actions in `src/lib/actions/platform.ts` (each first re-checks `requirePlatformAdmin()`):

Action	Cookie set	Effect resolved in <code>requireUser()</code>
<code>viewAsAgency(agencyId)</code>	<code>platform_view_agency</code>	Adopt that <code>agencyId</code> with <code>role = "admin"</code>
<code>viewAsUser(userId)</code>	<code>platform_view_user</code>	Adopt the target user's identity, <code>agencyId</code> and role (full fidelity)
<code>exitAgencyView()</code>	deletes both	Return to <code>/platform</code>

Both cookies are `httpOnly`, `sameSite: "lax"`, and `secure` in production. `platform_view_user` takes precedence over `platform_view_agency`. `isPlatformAdmin` stays `true` throughout so `/platform` still recognizes them and the app shows a persistent "viewing as" exit banner. See `architecture.md`.

Client portal sessions

The Traveler Portal (`/portal`) is a **passwordless magic-link** system entirely separate from Better Auth (staff) sessions. Helpers live in `src/lib/portal-session.ts`; the flow spans `src/app/api/portal/auth/{request,verify,signout}`.

Property	Value	Source
Session store	<code>portal_session</code> row (<code>portalSession</code>), scoped to one <code>client</code>	<code>src/lib/schema.ts</code>

Cookie	<code>portalSessionToken</code> — <code>httpOnly</code> , <code>sameSite: "lax"</code> , <code>secure</code> in prod	<code>portal-session.ts</code>
Magic-link token	<code>randomBytes(32).hex</code> , 15-minute TTL	<code>api/portal/auth/request</code>
Session token	<code>randomBytes(32).hex</code> , 7-day TTL	<code>api/portal/auth/verify</code>
purpose	<code>'magic' 'session'</code> (default <code>'session'</code>)	<code>portal_session.purpose</code> (migration 0021)

Security properties of the flow:

- **No account enumeration.** `POST /api/portal/auth/request` always responds `{ ok: true }` whether or not the email matched a client, so it never reveals which addresses have a portal account.
- **Single-use links.** On `GET /api/portal/auth/verify`, a valid magic token is **rotated** into a fresh long-lived session token (the row's `token` is replaced), so the original link can't be replayed. Invalid/expired tokens bounce to `/portal/login?error=...`
- **Purpose-scoped tokens.** `portal_session.purpose` discriminates a short-lived magic-link row (`'magic'`) from a real session row (`'session'`). `verify` only accepts a row with `purpose = 'magic'`; `getPortalSession()` (the session lookup used on every portal request) only accepts `purpose = 'session'`. A magic token can therefore no longer double as a session bearer for its 15-minute window. Requesting a new magic link also retires any prior pending magic-link rows for that client, so old links can't accumulate as still-valid bypasses.
- **Tenant scoping.** A portal session points at exactly one `clientId`; the client's `agencyId` is reached through the relation, so portal reads stay within that client's agency. Deleting a client cascades to its portal sessions.
- **Sign-out** clears the browser cookie (`clearPortalSession()` / the signout route).

Webhooks

The only inbound webhooks are **Stripe**. Duffel (flights) and Hotelbeds (hotels) are consumed as request/response REST APIs — there are no supplier webhook endpoints.

Both Stripe routes verify the signature **before** acting and read the **raw** request body via `req.text()` (never `req.json()`), which is required for HMAC verification:

Route	Plane	Endpoint secret	Acts on
<code>api/stripe/webhook</code>	SaaS subscriptions (vendor → agency)	<code>STRIPE_WEBHOOK_SECRET</code>	<code>customer.subscription</code> . { <code>created</code> , <code>updated</code> , <code>deleted</code> } → reconcile <code>agency</code> subscription fields

api/ stripe/ connect- webhook	Booking payments (traveler → agency)	STRIPE_CONNECT_WEBHOOK_SECRET	checkout.session.completed (paid) → mark payment completed
--	---	-------------------------------	--

Verification is done **manually with Node crypto** (no Stripe SDK), in `verifyWebhookSignature()` (`src/lib/billing/stripe.ts`):

- Parses the `Stripe-Signature` header (`t` timestamp + one or more `v1` signatures).
- **Replay protection:** rejects the event if `|now - t| > 300s` (the tolerance window).
- Recomputes `HMAC-SHA256(secret, "{t}.{rawBody}")` and compares against each `v1` signature with `timingSafeEqual` (constant-time) after a length check.
- Returns the parsed event only on success, otherwise `null`.

Handler responses are chosen so Stripe retries appropriately: a bad/missing signature returns **400** (`Invalid signature`), a missing endpoint secret returns **503** (`not configured` — transient, so Stripe keeps retrying), and a transient DB write failure returns **500** so the idempotent update is retried. Non-targeted event types are acked with **200** so Stripe stops retrying them.

Database safety

Prod and dev historically shared one Neon database, so destructive scripts (seeding, the isolation test, resets) are gated by `assertSafeDestructiveTarget()` in `scripts/guard.ts`. A target is **protected** when:

- `NODE_ENV === "production"`, **or**
- the `POSTGRES_URL` host matches any comma-separated substring in `PROTECTED_DB_HOSTS` (set in the prod/Vercel env — the prod Neon branch host — never in local `.env`).

When the target is protected the script **throws and refuses to run** unless explicitly overridden with `ALLOW_PROD=1`:

```
# Intentional prod run (rare, deliberate):
ALLOW_PROD=1 POSTGRES_URL=<prod> npx tsx scripts/backfill-countries.ts
```

Schema changes go through `db:generate` → `db:migrate`; **never** `db:push` (per `AGENTS.md`). All non-BetterAuth IDs are randomly-generated UUIDs (`uuid().primaryKey().defaultRandom()`).

Audit logging

Sensitive mutations write to the `activity_log` table (`activityLog` in `src/lib/schema.ts`) via `logActivity`. Each row is **agency-scoped** (`agencyId`, `onDelete: "cascade"`), records the actor (`userId`, set `null` on user delete), plus `action`, `entityType`, `entityId`, an `entityLabel` retained even after the entity is deleted, and optional `metadata`. Coverage is partial — `logActivity` is called from ~11 action files, not universally.

Known risks

- **Demo credentials in the repo.** `deployment.md` lists live demo passwords — including the platform super-admin account. They must be rotated or deleted before any real production use. See `deployment.md`.
- **No rate limiting.** Neither staff auth, the portal magic-link request endpoint, nor any API route is rate-limited, so they are exposed to brute-force / email-flooding.
- **Partial audit coverage.** Not every mutation writes to `activity_log`.
- **No app-level field encryption or GDPR export/erasure flow** (right-to-erasure is also blocked by the missing soft-delete). Tracked in the gap tracker.
- **Locale cookie gap** — cosmetic only; a fresh device shows English until the user re-picks (see `roadmap.md` open items).

Analytics & BI

Decision-oriented dashboards plus standardized data export, all currency-safe. Every metric is computed **server-side, agency-scoped, and in DZD only** — the chart primitives receive plain `{ label, value }[]` arrays and never touch the database or perform currency conversion.

Surfaces

Analytics appears on four routes. All are agency-scoped via `requireAgencyUser()` (`src/lib/permissions.ts`); the finance-grade surfaces are additionally gated by `canViewFinance` (`src/lib/domain.ts` → `admin, manager, finance`).

Route	File	Access	Focus
<code>/dashboard</code>	<code>src/app/(app)/dashboard/page.tsx</code> + <code>dashboard-insights.tsx</code>	any agency user	Command center: hero KPIs, pipeline funnel, bookings-by-status, and the streamed Insights section
<code>/finance</code>	<code>src/app/(app)/finance/page.tsx</code>	<code>canViewFinance</code> (redirects to <code>roleHome</code> otherwise)	AR, collected/outstanding, aging, commissions
<code>/reports</code>	<code>src/app/(app)/reports/page.tsx</code> + <code>src/components/reports/reports-analytics.tsx</code>	<code>canViewFinance</code> (redirects otherwise)	Period-windowed KPI deck, revenue trend, weighted forecast, BI export
<code>/opportunities</code>	<code>src/app/(app)/opportunities/page.tsx</code>	any agency user	Pipeline KPIs + conversion funnel by value

Dashboard (`/dashboard`)

The insights section (`DashboardInsights` in `dashboard-insights.tsx`) is rendered inside a `Suspense` boundary — the page shell streams first, then the insights-specific queries resolve behind `DashboardInsightsSkeleton`. It renders:

- **Finance KPIs (4 tiles, all-time, labelled "· all time"):** Total revenue (post-confirmation bookings — `confirmed` | `paid` | `ticketed` | `completed`, DZD), Collected (completed payments net of refunds), Won pipeline, and Gross margin ($\Sigma \text{totalPrice} - \text{totalCost}$ over accepted DZD proposals — same basis as `/reports` gross profit — with a "% of proposal revenue" note). The former "MoM revenue growth" tile was removed — the hero `StatStrip`'s this-month delta is the page's single growth metric (it renders a neutral "no confirmed revenue yet this month" note instead of a red `-100%` chip when the current month is still zero). The

Insights section renders only for admin/manager (other roles are redirected to their role home before reaching it).

- **Act-today band:** Departures next 7 days, Proposals awaiting response, Overdue, **To collect next 30 days** (DZD balances on non-cancelled, non-completed bookings departing within 30 days — a cash-inflow forecast, warning-toned with an "across N bookings" note), New clients this month.
- **Charts:** Revenue evolution (last 12 months), Top destinations by revenue (last 12 months), Top clients by revenue, Revenue per agent (first names title-cased), Bookings by status (count donut — center shows the **active** count; subtitle reads "N total · M active" when cancellations exist), Lead sources (client count), Top source markets (clients by country).

Dashboard revenue basis: everywhere the dashboard says "revenue" it now means bookings in a post-confirmation lifecycle status (`confirmed` | `paid` | `ticketed` | `completed`), DZD-only — previously `confirmed` only, which made "Collected" exceed "Total revenue". The `/reports` and `/finance` definitions are unchanged.

The page itself (`dashboard/page.tsx`) also builds a **pipeline funnel by value** across `OPPORTUNITY_STAGES` (excluding `lost`), summing DZD `opportunity.value` per stage, plus a win-rate footer.

Finance (`/finance`)

Derives accounts-receivable from a single agency-scoped `booking` query (with its `payments` relation) — balances are computed in code via `paymentSummary`, never in SQL. Surfaces:

- **KPIs:** Outstanding balance, Collected, Confirmed revenue, Overdue count.
- **Charts:** Collected over time (6-month area, net of refunds), Payments by method (donut), Outstanding vs collected (bar), AR aging (bar), Commission earned by supplier (horizontal bar, `supplier_to_agency`, DZD, top 8).
- **Tables:** Accounts receivable (outstanding balances, soonest departure first), Recent payments (15 most recent completed, refunds shown negative/red).
- **Revenue summary:** Confirmed revenue, Collected, Agency margin ($\sum \text{product.totalPrice} - \text{product.totalCost}$), Margin % of price, Won opportunities.
- **Commissions:** Pending / Earned / Paid KPI cards. These use the commission ledger's own currency (preferring EUR, else the first currency present) — the one place a non-DZD headline is shown, because commissions are frequently supplier-settled in EUR.

Reports & analytics (`/reports`)

The primary view is `ReportsAnalytics` (`src/components/reports/reports-analytics.tsx`), computed entirely server-side and windowed by a `?period=` search param. It is wrapped in `Suspense` with `ReportsAnalyticsSkeleton` while the agency-wide queries resolve. Below it sits the **Export data** hub (`#export` anchor).

KPI deck (6 tiles, each with a "vs previous period" delta pill):

KPI	Definition	Notes
Revenue	$\Sigma \text{totalAmount}$ of confirmed/paid DZD bookings created in-window	Compact DZD
Gross profit	$\Sigma (\text{totalPrice} - \text{totalCost})$ of accepted DZD proposals in-window	Only shown when in-window proposal revenue > 0
Bookings	Count of all bookings created in-window	—
Conversion	Won ÷ (won + lost) opportunities created in-window	Delta in pp
Avg margin	Gross profit ÷ proposal revenue	Only shown when a cost basis exists; delta in pp
Repeat rate	Clients with ≥2 bookings ÷ clients with ≥1, in-window	— when no clients booked

Windowing is by `createdAt` (cohort semantics), not cash-recognition. Revenue/funnel KPIs window on when a booking or opportunity was *created*, not when it was confirmed or paid — a booking created in one period but confirmed/ paid later is counted in its creation-period cohort, not the period money actually moved. This is a deliberate, documented choice (cohort-by-creation), not a bug; a long sales-cycle booking can be "invisible" in the period it's actually settled if you're expecting cash-basis reporting.

Revenue trend: trailing 12-month area chart of confirmed DZD revenue, plus a 4-up mini-stat strip (Peak month, 12-mo total, Avg/month, Best route by revenue).

Next-month forecast card: the weighted pipeline — $\Sigma (\text{open-stage DZD deal value} \times \text{probability}/100)$ — with open-pipeline, win-rate, and forecast-vs-this-month sub-stats.

This period at a glance: Open opportunities, Avg booking value, Outstanding balance, Proposals accepted (%).

Reporting periods

Period windows live in `src/lib/reports/period.ts`. Pills (`PeriodPills`) are plain `?period=...` links — no client state; the Server Component re-scopes the metrics on navigation.

Period	Label	Window
30d	30D	Last 30 days
mtd	MTD	Month to date
qtd	QTD	Quarter to date
12m	12M	Trailing 12 whole months (default: <code>DEFAULT_PERIOD</code>)
ytd	YTD	Year to date

`resolvePeriodWindow(period, anchor)` returns a `PeriodWindow` with matching `from/to` **and** an equal-length `prevFrom/prevTo` baseline so deltas are like-for-like. To-date windows (MTD/QTDT/YTD) compare against the *same elapsed length* of the prior period, so a partial month isn't judged against a full one. All windows are half-open `from, to` (`inWindow`), matching how `analytics.ts` buckets `createdAt`.

Currency-safe by design

No FX conversion (see [business-rules.md]). The agency operates in DZD (`DEFAULT_CURRENCY = "DZD"` in `src/lib/domain.ts`). Monetary aggregations therefore work on **DZD records only**; any row in another currency is either reported separately (`sumByCurrency`) or excluded from money charts — a stray EUR/USD figure is never silently added to a DZD total. Count-based metrics (funnels, status breakdowns, lead sources) are currency-agnostic and include every row.

Rounding is defensive throughout: monetary reductions round to 2 dp, and "other-currency" chips ignore residuals under 0.005.

`src/lib/analytics.ts`

Pure helpers (no I/O), unit-testable; every function returns plain numbers or `{ label, value }[]` so a Server Component can compute and hand the result straight to the (client) chart primitives.

Helper	Purpose
<code>num(v)</code>	Coerces a numeric string / nullish value to a finite number (0 fallback)
<code>sumByCurrency(rows, amountFn, currencyFn)</code>	Totals grouped by currency code, e.g. <code>{ DZD: 1200000, EUR: 800 }</code>
<code>headlineTotal(byCurrency)</code>	The DZD entry from a <code>sumByCurrency</code> map
<code>otherCurrencies(byCurrency)</code>	Non-DZD currencies with a non-zero total (for "stray currency" chips)
<code>growthPct(current, previous)</code>	Period-over-period growth %, rounded to 1 dp; <code>null</code> when no baseline
<code>conversionRate(part, whole)</code>	0–100 rate, rounded to 1 dp; 0 when <code>whole</code> is 0
<code>monthlyBuckets(rows, dateFn, valueFn, monthsBack?, anchor?)</code>	Time-series bucketing over the last N calendar months; empty months render at 0 so trends never gap
<code>topN(rows, keyFn, valueFn, n?, fallbackLabel?)</code>	Top-N aggregation by string key, summed and sorted desc (zero-value entries dropped)
<code>countBy(rows, keyFn, n?, fallbackLabel?)</code>	<code>topN</code> with a unit value — tallies rows by key

```
agingBuckets(rows, anchor?)
```

AR aging into `Not due / 0-30d / 31-60d / 61d+`
by days past a reference (departure) date

`monthlyBuckets` and `agingBuckets` accept an `anchor` `"now"` so they can be tested against a fixed date. `paymentSummary` (`src/lib/payments/summary.ts`) computes `{ paid, balance }` from completed payments (refunds subtracted) and is reused across finance, reports, dashboard, and the bookings export.

Chart component system (`src/components/charts/insight-charts.tsx`)

Lightweight on-brand primitives built directly on **recharts 3** (`recharts@^3.8.0`) and the design-system `--chart-*` / `--popover` tokens. All props are **serializable** (no function props), so Server Components compute the data and render these client components directly. Formatting is selected via a `format` enum (`"number" | "currency"`) resolved client-side by `makeFormatter` (`Intl.NumberFormat`), with `currency` defaulting to `"DZD"`.

Component	Type	Typical use
<code>BarInsight</code>	Vertical bars	Bookings by country, team performance
<code>HBarInsight</code>	Horizontal bars	Long labels — top destinations/clients/suppliers
<code>AreaInsight</code>	Gradient area trend	Revenue / bookings over time
<code>DonutInsight</code>	Donut + legend	Bookings by status, payments by method
<code>FunnelInsight</code>	Pure-CSS stage bars	Pipeline conversion (by value)

Shared behaviour:

- Colors come from a 6-entry `CHART_COLORS` palette (`--chart-1 ... --chart-6`), cycled via `colorAt(i)`. Tooltips use `--popover` / `--border` / `--radius-md`; axis ticks use `--muted-foreground`.
- Axis ticks are compact** (`makeAxisTickFormatter`): large values render as lowercase compact units (`"1.2m"`, `"600k"`) with no currency code — the full `Intl` precision lives in the tooltip. This keeps 7–8-digit DZD figures from clipping the Y axis.
- Empty/zero data renders an `EmptyChart` (`"Not enough data yet."`).
- `FunnelInsight` is CSS-only (no recharts) so it stays crisp at any width; each row shows the stage label, formatted value, and **per-step conversion** — the stage's value as a % of the *previous* stage (standard funnel semantics). The first row shows no %, a zero-value predecessor renders `"—"`, and `>100%` is possible when a later stage's value legitimately exceeds the prior stage's. It is consumed by both `dashboard/page.tsx` and `opportunities/page.tsx` (the "Conversion funnel (by value)" card).

The `Point` type (`{ label: string; value: number }`) and the `ChartFormat` enum (`"number" | "currency"`) are exported from this module; `analytics.ts` imports the `Point` type to type its

helper return values.

BI export

Gated by `canViewFinance` — several datasets carry revenue and commission figures. Served by `GET /api/export/[entity]` (`src/app/api/export/[entity]/route.ts`, `runtime = "nodejs"`); the UI lives at the `/reports` **Export data** section (`ReportsExport`, `src/components/reports/reports-export.tsx`).

Endpoints

```
GET /api/export/clients?format=csv
GET /api/export/bookings?format=xlsx&from=2026-01-01&to=2026-06-30
GET /api/export/workbook?format=xlsx          # all datasets, one workbook
```

- `format` — `csv` or `xlsx` (default `xlsx`). `workbook` is Excel-only.
- `from` / `to` — optional ISO dates; invalid values are ignored. The range filters by each dataset's creation date (`createdAt`) where one exists.
- Non-finance roles get `403`; an unknown entity gets `404`.
- Filenames are stamped with the export date, e.g. `bookings-2026-07-01.xlsx`, `atlas-export-2026-07-01.xlsx`.

Datasets (`src/lib/export/datasets.ts`)

Each dataset is agency-scoped and produces a uniform `{ columns, rows }` shape consumed by both writers. Every controlled/enum field is emitted as **both** a stable `*_code` column (for pivots/joins in Power BI or SQL) **and** a human-readable `*_label` column; amounts are in DZD, dates are ISO (YYYY-MM-DD).

Key	Label	Financial	Source
<code>clients</code>	Clients	no	<code>client</code> (+ owner name)
<code>opportunities</code>	Opportunities	no	<code>opportunity</code> (+ client, assignee)
<code>bookings</code>	Bookings	yes	<code>booking</code> (+ payments → paid/balance via <code>paymentSummary</code>)
<code>booking_items</code>	Booking items	yes	<code>bookingItem</code> ↗ <code>booking</code>
<code>travellers</code>	Travellers	no	<code>bookingTraveller</code> ↗ <code>booking</code>
<code>payments</code>	Payments	yes	<code>payment</code> ↗ <code>booking</code>
<code>commissions</code>	Commissions	yes	<code>commission</code> (+ booking, supplier, agent)
<code>suppliers</code>	Suppliers	no	<code>supplier</code>

`DATASETS` is the server registry (`DATASET_KEYS` = its keys); the export UI mirrors it as a small hand-maintained list. The `financial` flag documents which datasets carry revenue/commission data — access to the whole endpoint is already restricted to `canViewFinance` roles.

Writers

- **CSV** (`src/lib/export/csv.ts`) — minimal RFC-4180 writer. Prepends a UTF-8 BOM so Excel opens accented text (é, ç, ü) correctly, quotes any field containing ", ,, or a newline, and joins rows with `\r\n`. Power-BI friendly.
- **XLSX** (`src/lib/export/xlsx.ts`) — builds a workbook via **exceljs** (`exceljs@^4.4.0`, Node-only). One sheet per dataset; header row bold and frozen (`ySplit: 1`), columns widened to 20, sheet names sanitized to Excel's 31-char / reserved-char limits. The `workbook` entity emits every dataset as its own sheet in a single file.

AI Features

Built on the **Vercel AI SDK (ai v5)**. **Google Gemini is the primary provider** (`@ai-sdk/google`, `GEMINI_API_KEY`), with **OpenRouter** (`@openrouter/ai-sdk-provider`, `OPENROUTER_API_KEY`) as the automatic fallback. All features degrade/no-op gracefully when no provider key is set. See `api-integrations.md`.

There are two surfaces:

- The **conversational assistant** at `/assistant`, backed by the streaming route `src/app/api/chat/route.ts` with agency-scoped tools.
- Four **inline AI actions** — server actions in `src/lib/actions/ai.ts` — embedded where the work happens (itinerary page, new-proposal form, booking communications, booking detail).

Provider & models

Gemini is tried first, OpenRouter second. Both keys are optional in `src/lib/env.ts`; set at least one. `env.ts` warns "No AI provider configured..." only when *both* are missing. The two surfaces differ in *how* they fall back:

Surface	Where	Primary → fallback	Model env (default)
Assistant chat (streaming)	<code>src/app/api/chat/route.ts</code>	Static — Gemini if <code>GEMINI_API_KEY</code> set, else OpenRouter (streams can't fall back mid-response)	<code>GEMINI_MODEL</code> (<code>gemini-2.5-flash</code>) / <code>OPENROUTER_MODEL</code> (<code>openai/gpt-5-mini</code>)
Inline actions	<code>src/lib/actions/ai.ts</code>	Runtime chain — every call runs through <code>withAiFallback()</code> over an ordered candidate list: primary Gemini → lighter Gemini models → OpenRouter. On <i>any</i> error it rolls to the next candidate	<code>GEMINI_MODEL</code> (<code>gemini-2.5-flash</code>) / <code>OPENROUTER_MODEL</code> (<code>openai/gpt-4.1-mini</code>)

The runtime fallback exists chiefly for **Gemini's free tier**, which has real rate/quota limits. Because each free-tier Gemini model has its **own** quota bucket, `aiModels()` builds the candidate list as `GEMINI_MODEL` → the `GEMINI_FALLBACK_CHAIN` (`gemini-2.5-flash` → `gemini-2.5-flash-lite`, deduped) → OpenRouter (only if `OPENROUTER_API_KEY` is set). `withAiFallback()` iterates it, so a throttled model rolls over to the next one — staying on Gemini if possible, crossing to OpenRouter only as a last resort. If no provider key is set at all, both surfaces error clearly ("No AI provider configured...").

AI SDK primitives used

Primitive	Import	Used by
-----------	--------	---------

<code>streamText</code>	<code>ai</code>	Assistant chat route (streamed, tool-calling)
<code>generateObject</code>	<code>ai</code>	<code>generateItinerary</code> , <code>buildQuote</code> , <code>checkVisa</code> (Zod-schema-constrained output)
<code>generateText</code>	<code>ai</code>	<code>draftEmail</code> (free-text, hand-parsed)
<code>useChat</code>	<code>@ai-sdk/ react</code>	Assistant UI (<code>/assistant</code>)
<code>tool</code> , <code>convertToModelMessages</code> , <code>stepCountIs</code>	<code>ai</code>	Assistant tool loop

Package versions (per `package.json`): `ai@^5.0.188`, `@ai-sdk/react@^2.0.190`, `@ai-sdk/google@^2.0.77` (Gemini, must stay on the 2.x line to match the `@ai-sdk/provider@2` that `ai@5` uses — `@ai-sdk/google@4` pulls `provider@4` and is incompatible), `@openrouter/ai-sdk-provider@^1.5.4`.

AI must never

Hard guardrails for every AI feature. These extend the never rules to the AI surface:

- **Never invent prices** — quote only live supplier rates or stored values, never a guessed number.
- **Never invent bookings** — only create a booking when explicitly asked.
- **Never invent clients** — never fabricate a client record.
- **Never invent payments** — never record or imply a payment that didn't happen.
- **Never modify data without confirmation** — mutations are explicit and reviewable, not silent side effects of a chat reply.
- **Never hide errors** — surface failures to the user; don't paper over a failed tool call with a plausible answer.
- **Never perform destructive actions silently** — no deletes/overwrites without a clear, confirmed user action.

Current state: all assistant tools are agency-scoped (via `resolveEffectiveAgencyId`, which honors platform-admin "view as" impersonation). Of them only `createBooking` mutates — it creates a **draft** booking (zero balance, unconfirmed), and the system prompt forbids inventing supplier confirmations or PNRs and restricts creation to explicit requests. Everything else is **read-only**:

- **Sourcing:** `searchFlights`, `searchHotels` (live suppliers).
- **CRM/ops read tools** (`src/lib/assistant/tools/{bookings,clients,sales,finance}.ts`, each a `make<Domain>Tools({ agencyId })` factory spread into the route when an agency is in scope): `findClients` + `getClientDetails` (profile, stats, history); `listBookings` (filter by status/client/destination/date — answers "who cancelled", "departing this week") +

```
getBookingDetails; listProposals + pipelineOverview; bookingsSummary +
financeOverview + commissionsOverview (all currency-safe, DZD headline). Every
query is hard-scoped to ctx.agencyId.
```

Gap: creation is intent-gated by the prompt, not yet behind a hard confirm step — the "modify without confirmation" guard is a convention, not an enforced gate.

AI assistant (/assistant)

The chat UI (`src/app/(app)/assistant/page.tsx`) uses `useChat` from `@ai-sdk/react`, renders assistant replies through `react-markdown`, and persists the thread to `localStorage` (key `chat-messages`) — there is no server-side chat history yet (the History button is disabled). The context rail seeds ready-made prompts into the composer (quote / follow-up / visa / flight search) rather than calling the inline actions directly.

Route: `POST /api/chat`

`src/app/api/chat/route.ts` handles the request:

1. **Auth** — resolves the Better Auth session; returns 401 if absent.
2. **Tenant scope** — reads `agencyId` off the session user. It is `NULL` only for the platform super-admin, in which case the data tools return empty results rather than querying globally, so the assistant can never read across agencies.
3. **Input validation** — the body is parsed with a Zod schema (`chatRequestSchema`): at most 100 messages, each message text capped at 10,000 chars. Message `role` is restricted to `user/assistant` — `role:"system"` is deliberately rejected so a caller cannot inject their own system prompt (and through it drive tool calls).
4. **Stream** — `streamText` is called with the server-controlled system prompt, `convertToModelMessages(messages)`, `stopWhen: stepCountIs(6)` (max 6 tool/model steps), and the tool set below. The response is returned via `toUIMessageStreamResponse()`. Any setup/streaming throw is converted to a structured JSON error (HTTP 502).

The system prompt is built by `SYSTEM_PROMPT(today, agentName)` in the same file. It addresses the agent by name, sets today's date, and states that internal agency figures (bookings, revenue, pipeline, quotes) are reported in **DZD** — live flight/hotel search results keep their own supplier-quoted currency and are never blended into a DZD total. It tells the model to infer IATA codes but state them, to disclose when figures come from sample data, and — as the hard rule — **never to invent supplier confirmations or PNRs and to create a booking only when explicitly asked**.

Assistant tools

14 tools are wired into the route: 5 defined inline (sourcing, CRM lookup, booking summary, the one mutation) plus 9 read tools spread in from four per-domain factory files. All are agency-scoped wrappers over existing server actions / DB queries, so the AI cannot bypass security.md tenant isolation. Tool inputs are Zod-validated.

Inline (defined directly in `route.ts`):

Tool	Kind	What it does	Scope / notes
<code>searchFlights</code>	read	Flight search by IATA codes + dates via <code>safeSearch(getFlightSupplier, ...)</code> ; returns up to 5 cheapest offers, currency forced to EUR	Returns <code>source</code> + <code>degraded</code> ; <code>degraded:true</code> means mock/sample prices, forwarded so the assistant can disclose it
<code>searchHotels</code>	read	Hotel search by city + dates via <code>safeSearch(getHotelSupplier, ...)</code> ; returns up to 6 offers with star ratings, EUR	Same <code>source</code> /degraded disclosure
<code>findClients</code>	read	Looks up clients in the agency CRM by name (or lists recent)	Always ANDs <code>eq(client.agencyId, agencyId)</code> ; platform admin (no agency) gets <code>[]</code> ; limit 10
<code>bookingsSummary</code>	read	Counts bookings by status and sums active (non-cancelled) value in DZD	Filtered to <code>booking.agencyId</code> ; platform admin gets an empty summary
<code>createBooking</code>	write	Creates a draft booking from items (flight/hotel/transfer/excursion/insurance/fee/other) + optional travellers; returns a <code>bookings/{id}</code> link	Only mutating tool. Rejects when no agency context; a supplied <code>clientId</code> is verified to belong to the agency before use; per-row traveller/item failures are collected and reported (<code>failedTravellers/failedItems</code>), not swallowed

`createBooking` delegates to `createBooking`, `addTraveller`, and `addBookingItem` from `src/lib/actions/bookings.ts`. Flight/hotel search goes through `safeSearch` in `src/lib/suppliers` (real provider first, mock fallback tagged `degraded:true`) — see `api-integrations.md`.

Factory read tools — `src/lib/assistant/tools/{bookings,clients,sales,finance}.ts`, each a `make<Domain>Tools({ agencyId })` factory spread into the route's `tools` object only when an agency is in scope (a platform admin not viewing any agency gets none of them, so it can never read cross-tenant):

Tool	File	What it does	Scope / notes
<code>listBookings</code>	<code>tools/bookings.ts</code>	Filter bookings by status, client name, destination, or departure-date window — answers "which were cancelled", "what departs next week"	Returns reference, client, status, destination, dates, pax count, total + currency (never summed across currencies)

<code>getBookingDetails</code>	<code>tools/ bookings.ts</code>	Open one booking's full file by reference (e.g. <code>BKG-1042</code>) or id — client, status, dates, travellers, items, payment summary	Answers "what's the balance due", "who's travelling on..."
<code>getClientDetails</code>	<code>tools/ clients.ts</code>	Full profile of one client by id or name: contact details, owner, source, status, derived lifetime value + open balance (DZD), recent bookings/proposals/opportunities	Resolves by name when no id is known; returns disambiguation options on multiple matches
<code>listProposals</code>	<code>tools/ sales.ts</code>	Filter proposals/quotes by status (<code>draft/sent/accepted/rejected/expired</code>) and/or client name	—
<code>pipelineOverview</code>	<code>tools/ sales.ts</code>	Pipeline breakdown by stage (count + DZD value) plus the biggest open deals; optional stage filter	Stages: <code>lead/qualified/proposal/booked/won/lost</code>
<code>financeOverview</code>	<code>tools/ finance.ts</code>	Confirmed revenue, cash collected, outstanding balance, overdue amount + count, all in DZD	Non-DZD amounts listed separately, never blended in
<code>commissionsOverview</code>	<code>tools/ finance.ts</code>	Commission ledger grouped by status (<code>pending/earned/invoiced/paid</code>) with count + DZD amount; optional top-earning-agents list	<code>includeTopAgents</code> input flag

`findClients` is the inline lookup used to resolve a client before mutating (`createBooking`); `getClientDetails` (factory) is the read-only deep profile. Every query in every factory file is hard-scoped to `ctx.agencyId`.

Inline features

Embedded where the work happens, not in a separate AI section. All four are server actions in `src/lib/actions/ai.ts`, each guarded by `requireAgencyUser()` (from `src/lib/permissions`) and scoped to `user.agencyId`. On failure they return `{ ok: false, error }` with the message "AI generation failed. Check that OPENROUTER_API_KEY is set."

Feature	Action	SDK call	Trigger UI	C
Itinerary generation	<code>generateItinerary(bookingId)</code>	<code>generateObject</code>	<code>GenerateItineraryButton</code>	/ [i
Quote builder	<code>buildQuote(brief, currency, paxCount)</code>	<code>generateObject</code>	<code>AiQuoteBuilder</code>	r f
Email drafting	<code>draftEmail(bookingId, kind, customInstruction?)</code>	<code>generateText</code>	<code>AiEmailDraftButton</code>	t c f
Visa assistant	<code>checkVisa(bookingId)</code>	<code>generateObject</code>	<code>VisaAssistant</code>	/ l

AI itinerary generation

`generateItinerary` loads the agency-scoped booking with its items (ordered by `sortOrder`), refuses if there are no items, and asks the model — via `generateObject` constrained to `itinerarySchema` (`days[]` of `{ dayIndex, title, notes }`) — for a concise day-by-day plan referencing the actual items. Each returned day is **upserted** into `booking_day` rows (existing rows with the same `dayIndex` have their title/notes overwritten). It then revalidates `/bookings/{id}/itinerary` and returns `{ dayCount }`. The saved rows render on the booking's itinerary page and on the public shareable itinerary at `/i/[token]` (via `booking.shareToken` and `buildItinerary` in `src/lib/itinerary.ts`).

- **Trigger:** `src/components/bookings/generate-itinerary-button.tsx`, mounted in `src/app/(app)/bookings/[id]/itinerary/page.tsx`.

AI quote builder

`buildQuote` takes a natural-language brief plus currency and traveller count and returns structured proposal line items via `generateObject` constrained to `quoteSchema` (`title`, optional `destination`, and `items[]` of `type/title/supplier/quantity/unitCost/description`, where `type` is one of `flight/hotel/activity/transfer/insurance/other`). The prompt instructs the model to use realistic market estimates and per-person quantities. **It does not persist anything** — the result pre-fills the new-proposal form for the agent to review; line items are added after the proposal is created.

- **Trigger:** `src/components/products/ai-quote-builder.tsx`, wrapped by `src/components/products/new-product-with-ai.tsx` (which applies the quote into the form via `applyQuote`).

AI email drafting

`draftEmail` loads the agency-scoped booking (with client, items, travellers) and generates a client email with `generateText`. The `kind` argument selects the brief: `confirmation`, `voucher`, `followup`, or `custom` (the latter using `customInstruction`). The model is asked to emit `SUBJECT: ... then ---`

then the body; the action splits on `\n---\n`, strips the `SUBJECT:` prefix, and returns `{ subject, body }` — returning a parse error if either half is empty. It writes no data; the accepted draft is lifted into the compose form.

- **Trigger:** `src/components/bookings/ai-email-draft-button.tsx`, mounted in `src/components/bookings/communications-manager.tsx`.

AI visa assistant

`checkVisa` loads the agency-scoped booking with travellers, requires a booking `destination` and at least one traveller `nationality`, then de-duplicates the nationalities and asks the model — via `generateObject` constrained to a `{ destination, requirements[], disclaimer }` schema — for a per-nationality visa summary plus a disclaimer to verify with official sources. Results are **based on the model's training data**, not a live visa database, and are read-only.

- **Trigger:** `src/components/bookings/visa-assistant.tsx`, mounted in `src/app/(app)/bookings/[id]/page.tsx`.

Implementation

- Assistant route: `src/app/api/chat/route.ts` (streaming, tools, system prompt).
- Assistant UI: `src/app/(app)/assistant/page.tsx` (+ `src/components/assistant/*`).
- Inline server actions: `src/lib/actions/ai.ts`.
- Itinerary rendering helpers: `src/lib/itinerary.ts`.
- Supplier search used by the assistant: `src/lib/suppliers` (`safeSearch`, `getFlightSupplier`, `getHotelSupplier`).
- Env schema: `src/lib/env.ts` (`OPENROUTER_API_KEY`, `OPENROUTER_MODEL`).
- The assistant's tool definitions and every inline action are agency-scoped (session `agencyId` / `requireAgencyUser`), so AI cannot bypass `security.md` tenant isolation.

PART II

Project Root Docs

5 documents

Agentic Coding Starter Kit

A production-oriented starter kit for building AI-powered web apps with an agentic development workflow. It gives you a working Next.js app, authentication, PostgreSQL, Drizzle ORM, AI SDK integration, shadcn/ui components, and project instructions that help coding agents plan, split, implement, review, and verify changes.

The goal is simple: install the starter, describe the product you want to build, and let your coding agent help turn the boilerplate into your actual POC, MVP, or internal tool.

What You Get

- **Next.js 16 and React 19** with the App Router
- **TypeScript** and a strict project setup
- **Better Auth** with email/password enabled by default
- **PostgreSQL and Drizzle ORM** for schema and migrations
- **AI SDK and OpenRouter** for chat and AI features
- **shadcn/ui, Tailwind CSS, and Lucide icons** for the UI foundation
- **Local or Vercel Blob file storage** through one storage abstraction
- **Agent instructions** through `AGENTS.md` and `CLAUDE.md`
- **Agent skills** for specs, implementation, reviews, security scans, UI work, and shipping

Quick Start

Create a new app with the CLI:

```
npx create-agentic-app@latest my-app
cd my-app
```

Or create the app in the current directory:

```
npx create-agentic-app@latest .
```

Then configure and run the app:

```
cp env.example .env
docker compose up -d
pnpm db:migrate
pnpm dev
```

Open <http://localhost:3000>.

The CLI copies the starter files, installs dependencies with your selected package manager, and

prepares the environment file. If you use `npm`, replace the `pnpm` commands above with `npm run`.

Guided Setup with Claude Code (Optional)

If you use Claude Code, you can install the `create-agentic-app` skill and have Claude walk you through the entire setup — folder strategy, package manager, PostgreSQL (Docker / Neon / Vercel / BYO), `.env` config, migrations, optional integrations (OpenRouter, Vercel Blob, Polar, email), build verification, and dev-server check — ending at a verified `http://localhost:3000`.

Install the skill:

```
npx skills add leonvanzyl/agentic-coding-starter-kit@create-agentic-app --agent
claude-code
```

The `--agent claude-code` flag is required. Without it, the installer drops the skill into 37+ IDE adapter folders at the project root.

Once installed, ask Claude something like:

```
Scaffold a new Agentic Coding Starter Kit project here.
```

Claude will run the skill end-to-end and ask you the few decisions it actually needs to make.

Prerequisites

- Node.js 18 or newer
- Git
- PostgreSQL, either through the included Docker Compose file or a hosted provider
- A package manager: `pnpm`, `npm`, or `yarn`
- Optional: an OpenRouter API key for AI chat features
- Optional: a Vercel account for deployment, hosted Postgres, and Blob storage

Environment Variables

Start from `env.example` and update values for your environment:

```
# Database
POSTGRES_URL=postgresql://dev_user:dev_password@localhost:5432/postgres_dev

# Authentication - Better Auth
BETTER_AUTH_SECRET=your-random-secret

# AI Integration via OpenRouter
OPENROUTER_API_KEY=
OPENROUTER_MODEL="openai/gpt-5-mini"
```

```
# Optional - for vector search only
OPENAI_EMBEDDING_MODEL="text-embedding-3-large"

# App URL
NEXT_PUBLIC_APP_URL="http://localhost:3000"

# File storage
BLOB_READ_WRITE_TOKEN=

# Polar payment processing
POLAR_WEBHOOK_SECRET=polar_
POLAR_ACCESS_TOKEN=polar_
```

For local development, the default database URL works with the included `docker-compose.yml`. For production, use the database URL from your hosting provider.

Generate a strong `BETTER_AUTH_SECRET` before deploying. The starter ships with a development value only so you can get moving quickly.

Default Auth

The starter now defaults to **email and password authentication** through Better Auth. This keeps the first setup small and helps you start building POCs and MVPs without creating OAuth credentials up front.

The current auth setup includes:

- user registration
- email/password login
- protected routes
- password reset flow
- email verification flow

In development, verification and password reset links are logged to the terminal instead of being sent through an email provider. When you are ready for production, ask your coding agent to connect an email service and update the Better Auth email callbacks.

Adding Google OAuth

Google OAuth is no longer the default, but adding it back is straightforward. Ask your coding agent:

```
Add Google OAuth to this Better Auth setup. Keep email/password login enabled,
add the Google provider, update the auth UI, and document the required Google
environment variables.
```

Your agent should update the Better Auth config, add the required `GOOGLE_CLIENT_ID` and `GOOGLE_CLIENT_SECRET` variables, and adjust the login UI.

Build With an Agent

This starter is designed to be used with coding agents. The generated project includes instructions that tell agents how to plan, ask questions, split work, use sub-agents when useful, follow the design system, and verify changes.

- `AGENTS.md` is the main instruction file for Codex, Cursor, and other agent-compatible tools.
- `CLAUDE.md` points Claude users to the same project guidance.
- `.agents/skills/` and `.claude/skills/` include optional workflows for more specialized tasks.
- `DESIGN.md` defines the UI design system agents should follow.

The default workflow does not require slash commands or a separate spec file.

Recommended Default Workflow

1. Install the starter and open the project in your coding-agent environment.
2. Switch your agent tool to planning mode.
3. Describe the app you want to build in plain language.
4. Let the agent ask clarifying questions and shape a clear plan.
5. Confirm the plan once the goal, scope, constraints, and success criteria are clear.
6. Switch your agent tool to edit mode.
7. Ask the agent to implement the approved plan.
8. The main agent should split the work into parallel streams, silos, or feature chunks that fit within context.
9. The agent should use sub-agents to implement those chunks in parallel where useful, then coordinate the results.
10. The agent should run quality checks such as lint, typecheck, and build.
11. Review the result in the browser and iterate.

You do not need a special command for this default workflow. The project instructions already tell the agent how to plan, split implementation work, use sub-agents, and verify the result.

Starter Prompt

Use this as a first message to your coding agent after installing the starter:

```
I am using the Agentic Coding Starter Kit. Treat the existing app as boilerplate that should be replaced by the product I describe.
```

```
Use the project instructions in AGENTS.md or CLAUDE.md. During planning, ask clarifying questions before making assumptions. During implementation, split the work into small chunks, use sub-agents where useful, follow DESIGN.md for UI, preserve the existing tech stack unless there is a good reason to change it, and run lint, typecheck, and build before finishing.
```

```
What I want to build:
```

```
[Describe your app here]
```

For example:

```
What I want to build:  
A lightweight CRM for solo consultants. It should let users manage clients,  
track deals, write notes, set follow-up reminders, and view a simple dashboard  
of open opportunities.
```

When to Use Specs

For most POCs and MVPs, the normal agent workflow is enough. Use a spec when the feature is large, long-running, risky, or needs to be split across multiple implementation sessions.

The starter includes two skills for that workflow:

- `create-spec`: turns a planning conversation into `specs/{feature}/` with requirements, task files, dependency waves, and manual action notes.
- `implement-feature`: reads a spec folder and coordinates implementation wave by wave with review gates.

Use this workflow when:

- the feature spans many files or modules
- multiple agents should work in parallel
- the implementation may take more than one session
- you need resumable progress tracking
- you want a written implementation record before coding starts

Example agent request:

```
Create a spec for the billing and subscriptions feature we just planned. Break  
it into parallel implementation waves and include any manual setup steps.
```

Then:

```
Implement the billing and subscriptions spec from specs/billing-subscriptions.
```

Project Structure

```
src/  
├─ app/  
│   └─ (auth) /  
│       └─ forgot-password /  
│       └─ login /  
│       └─ register /
```

```
| | └─ reset-password/
| | └─ api/
| |   └─ auth/
| |   └─ chat/
| |   └─ diagnostics/
| | └─ chat/
| | └─ dashboard/
| | └─ profile/
| | └─ layout.tsx
| | └─ page.tsx
└─ components/
  └─ auth/
  └─ ui/
  └─ site-footer.tsx
  └─ site-header.tsx
└─ hooks/
└─ lib/
  └─ auth.ts
  └─ auth-client.ts
  └─ db.ts
  └─ env.ts
  └─ schema.ts
  └─ session.ts
  └─ storage.ts
  └─ utils.ts
```

Important root files:

- `AGENTS.md`: coding-agent behavior rules
- `CLAUDE.md`: Claude entrypoint for the same guidance
- `DESIGN.md`: UI design system and component guidance
- `drizzle.config.ts`: Drizzle migration configuration
- `docker-compose.yml`: local PostgreSQL service
- `env.example`: environment variable template
- `components.json`: shadcn/ui configuration

Available Scripts

```
pnpm dev          # Start the development server with Turbopack
pnpm build         # Run migrations, then build for production
pnpm build:ci      # Build without running migrations
pnpm start         # Start the production server
pnpm lint          # Run ESLint
pnpm typecheck     # Run TypeScript without emitting files
pnpm check         # Run lint and typecheck
pnpm format        # Format the repository
pnpm format:check  # Check formatting
pnpm setup         # Run the setup script
pnpm db:generate   # Generate Drizzle migrations
pnpm db:migrate    # Run Drizzle migrations
pnpm db:studio     # Open Drizzle Studio
```

The repository also contains Drizzle push/reset helper scripts for local experimentation. For schema changes you intend to keep, prefer:

```
pnpm db:generate
pnpm db:migrate
```

Do not use schema push as a replacement for migrations in real project work.

Database Workflow

For local development:

```
docker compose up -d
pnpm db:migrate
```

When your app needs schema changes, ask your agent to update `src/lib/schema.ts`, generate a migration, and run it:

```
pnpm db:generate
pnpm db:migrate
```

If you deploy to Vercel or another hosted environment, set `POSTGRES_URL` in that environment before running migrations or building the app.

AI Features

The starter uses the Vercel AI SDK with OpenRouter. Set these variables to enable AI chat:

```
OPENROUTER_API_KEY=sk-or-v1-your-key
OPENROUTER_MODEL="openai/gpt-5-mini"
```

OpenRouter lets you switch models without changing the application code. Update `OPENROUTER_MODEL` when you want to try a different model.

File Storage

The starter includes a storage abstraction that can use local storage in development or Vercel Blob in production.

For local development, leave `BLOB_READ_WRITE_TOKEN` empty. Files are stored under `public/uploads/`.

For Vercel Blob:

1. Create a Blob store in Vercel.

2. Copy the `BLOB_READ_WRITE_TOKEN`.
3. Add it to your production environment variables.

The app chooses the storage backend based on whether `BLOB_READ_WRITE_TOKEN` is configured.

Deployment

Vercel is the recommended deployment target.

```
npm install -g vercel
vercel --prod
```

Set the required production environment variables:

- `POSTGRES_URL`
- `BETTER_AUTH_SECRET`
- `NEXT_PUBLIC_APP_URL`
- `OPENROUTER_API_KEY`, if using AI features
- `OPENROUTER_MODEL`, if using AI features
- `BLOB_READ_WRITE_TOKEN`, if using Vercel Blob
- `POLAR_WEBHOOK_SECRET` and `POLAR_ACCESS_TOKEN`, if using Polar payments

The default `pnpm build` script runs database migrations before `next build`. If your CI or host should not run migrations during build, use `pnpm build:ci` and run migrations as a separate deployment step.

Troubleshooting

The app cannot connect to Postgres

Confirm Docker is running and start the database:

```
docker compose up -d
```

Then check that `POSTGRES_URL` in `.env` matches the database connection string.

Auth reset or verification emails are not arriving

In development, links are logged to the terminal. This is intentional. Connect an email provider before using password reset or verification in production.

AI chat is not working

Set `OPENROUTER_API_KEY` and restart the dev server. Also confirm `OPENROUTER_MODEL` is a model available to your OpenRouter account.

My agent is preserving too much boilerplate

Tell the agent directly that the starter UI is scaffolding and should be replaced:

```
Replace the starter UI with the actual product UI. Do not keep setup checklists,
placeholder navigation, demo content, or boilerplate copy unless I explicitly
ask for it.
```

I need Google login

Ask your agent to add Google OAuth through Better Auth while keeping email/password enabled. You will need Google OAuth credentials and production callback URLs.

Video Tutorial

Watch the original walkthrough:

[Agentic Coding Boilerplate Tutorial](#)

Some details in older videos may differ from the current starter. This README is the source of truth for the current default workflow.

Support This Project

If this starter kit helped you build something useful, you can support the project here:

[Buy me a coffee](#)

Contributing

1. Fork this repository.
2. Create a feature branch.
3. Make your changes.
4. Run the relevant checks.
5. Open a pull request.

License

This project is licensed under the MIT License.

Need Help?

- Check the repository issues: github.com/leonvanzyl/agentic-coding-starter-kit/issues
- Review `AGENTS.md`, `CLAUDE.md`, and `DESIGN.md`

- Open a new issue with the exact setup steps, error output, and environment details

Atlas — Travel Agency SaaS

A multi-tenant SaaS for travel agencies: each agency runs its bookings, clients, proposals and finance in an isolated workspace, while the vendor manages all agencies from a platform console. Built on the Agentic Coding Starter Kit.

Live: <https://agencetool.vercel.app>

Tech stack

- **Framework:** Next.js 16 (App Router) + React 19 + TypeScript
- **Styling:** Tailwind CSS v4 (CSS-first, `@theme inline`) + shadcn/ui (new-york)
- **Auth:** Better Auth (email/password, invitation-gated signup)
- **DB:** PostgreSQL (Neon) + Drizzle ORM
- **i18n:** next-intl (EN / FR / AR, cookie-based, RTL)
- **Charts:** recharts (tokenized to the design system)
- **AI:** Vercel AI SDK + OpenRouter (assistant — needs `OPENROUTER_API_KEY`)
- **Hosting:** Vercel (GitHub auto-deploy on `main`)

See `DESIGN.md` for the design system and `AGENTS.md`/`CLAUDE.md` for working conventions.

Architecture

Multi-tenancy

Every business table carries an `agencyId` and **all** queries are scoped by it, so agencies never see each other's data. References (`BKG-...`, `PRD-...`) are unique **per agency**. Tenancy is enforced at the data layer and re-verified by a test harness (`scripts/test-tenant-isolation.ts`).

Tenant-root tables: `agency`, `client`, `opportunity`, `product`, `booking`, `notification`, `activity_log`, `agency_invite`, plus `user.agencyId`. Child tables (`travellers`, `items`, `payments`, `days`, `contacts`) inherit tenancy through their parent.

Roles (RBAC)

Five roles, defined in `src/lib/domain.ts` with capability helpers:

Role	Sees all agency data	Manages team	Manages payments	Deletes records	Home
admin	✓	✓	✓	✓	/dashboard

manager	✓	✓	✓	✓	/dashboard
finance	✓	—	✓	—	/finance
support	✓	—	—	—	/support
agent	own work only	—	—	—	/dashboard (scoped)

Helpers: `seesAllData`, `canManageTeam`, `canAssignAdmin`, `canManagePayments`, `canViewFinance`, `canViewSupport`, `canDeleteRecords`, `roleHome`.

Auth & onboarding

- **Invitation-only signup**: the Better Auth create hook (`src/lib/auth.ts`) rejects any signup without a matching pending invite and stamps `agencyId` + role from it. Accept flow lives at `/invite/[token]`.
- Guards (`src/lib/permissions.ts`): `requireUser`, `requireAgencyUser` (tenant + suspension lockout), `requireManager`, `requireCapability`, `requirePlatformAdmin`.

Platform (vendor) console — `/platform`

The vendor is a user with `isPlatformAdmin = true` and `agencyId = null`, sitting above all tenants. From `/platform` they can:

- Create agencies (generates the first-admin invite link)
- Suspend / reactivate agencies (suspended → users locked out)
- **View as agency** (act as agency admin) or **View as user** (act as a specific user with *their* role + scoped data) — with an exit banner.

Impersonation is cookie-driven (`platform_view_agency` / `platform_view_user`) and only takes effect for a platform admin (resolved in `requireUser`).

Per-role workspaces

- `/dashboard` — agency overview (admin/manager) or "Your work" (agent); manager view includes analytics charts (bookings by country, team performance, status, monthly trend) + finance KPIs.
- `/finance` — payments / accounts-receivable + revenue, with charts.
- `/support` — action queue (bookings needing attention) + clients + ops.
- `/team`, `/clients`, `/bookings`, `/opportunities`, `/products`, `/operations`, `/search`, `/assistant`, `/settings`, `/profile`.

i18n

- English / French / Arabic, **cookie-based** (no URL-locale routing — routes unchanged). Arabic is full RTL with **IBM Plex Sans Arabic**.
- Config: `src/i18n/config.ts` + `src/i18n/request.ts`; messages in `messages/`

`{en,fr,ar}.json`. Change language in **Settings**.

- Core surfaces (nav, login, dashboard, settings) are translated; deeper pages fall back to English. To translate more, add keys to **all three** message files and swap strings to `t("...")`.

Local development

```
npm install --legacy-peer-deps      # better-auth peer range needs this
npm run dev                         # http://localhost:3000
npm run check                       # lint + typecheck
npm run build:ci                    # next build (no migrate)
```

Required env (`.env`): `POSTGRES_URL`, `BETTER_AUTH_SECRET`. Optional (features off until set): `OPENROUTER_API_KEY` (AI chat), `BLOB_READ_WRITE_TOKEN` (uploads), Google OAuth.

Database

```
npm run db:generate    # after editing src/lib/schema.ts
npm run db:migrate     # apply migrations (NEVER db:push)
npm run db:studio
```

Migrations in `drizzle/`. Notable: `0006` (tenancy + backfill), `0007` (invites), `0008` (user.locale).

Scripts

```
# Promote an existing account to platform super-admin (vendor)
npx tsx --env-file=.env scripts/make-platform-admin.ts <email>

# Seed / reset the Demo Agency with rich demo data (idempotent)
npx tsx --env-file=.env scripts/seed-demo-data.ts

# Verify cross-tenant isolation (seeds 2 agencies, asserts no leak)
npx tsx --env-file=.env scripts/test-tenant-isolation.ts
```

Deployment

Hosted on **Vercel** (`atlasproject/agence_tool`), connected to GitHub (`ouks1993/agence_tool`).

- **Auto-deploy:** `git push origin main` builds & ships production.
- **Manual:** `npx vercel deploy --prod --yes`
- Config in `vercel.json` (`buildCommand: npm run build:ci`) + `.npmrc` (`legacy-peer-deps=true`).
- Env vars on Vercel: `POSTGRES_URL`, `BETTER_AUTH_SECRET`, `NEXT_PUBLIC_APP_URL`, `BETTER_AUTH_URL` (the last fixes `INVALID_ORIGIN` on the deployed domain).

⚠ Production and local dev currently **share one Neon database**. Split prod onto its own Neon branch before onboarding real customers.

Demo

The **Demo Agency** is seeded for customer demos (re-run the seed script to reset). Accounts (Demo Agency unless noted):

Role	Email
Platform admin (vendor)	ouksili.abdelmalek@gmail.com
Manager	yasmine@agence.test
Finance	finance@demo.test
Support	support@demo.test
Agents	karim@demo.test, lina@demo.test, omar@demo.test

Demo credentials are throwaway and set by the seed/admin scripts — **rotate or remove them before any real production use**. (Agent accounts use the seed default; reset any password via the scripts above.)

Suggested demo flow: sign in as the vendor → `/platform` → create an agency (or *View as Demo Agency*) → tour the manager dashboard charts → switch logins to show the finance, support and agent views → open Settings and switch to Français or العربية (RTL).

Roadmap / open items

- **Split prod database** onto a dedicated Neon branch (highest priority).
- **Translate deeper pages** (bookings, clients, finance, support, platform).
- **Cross-device locale:** apply a user's saved `locale` on a fresh device (sync `user.locale` → cookie on login).
- **Email delivery** for invite links (currently copy-paste; module logs to console).
- **Billing / subscriptions** (per-agency plans).
- **Traveler portal** (end-customer login to view their trips).

CRITICAL RULES - MUST FOLLOW

RESPONSES

- Keep responses concise and to the point - unless the user asks otherwise

PLANNING MODE

- Always ask clarifying questions
- Never assume design, tech stack or features
- Use deep-dive sub-agents to assist with research
- Use deep-dive sub-agents to review the different aspects of your plan before presenting to the user

CHANGE / EDIT MODE

- Never implement features yourself when possible - use sub-agents!
- Identify changes from the plan that can be implemented in parallel, and use sub-agents to implement the features efficiently
- When using sub-agents to implement features, act as a coordinator only
- Use the best model for the task - premium models for complex tasks (like coding) and mid-tier models for simpler tasks, like documentation
- After completing features (large or small), always run commands like lint, type check and next build to check code quality

DATABASE SCHEMA CHANGES

- Whenever you make changes to the database schema, ALWAYS run the drizzle generate and migrate commands
- NEVER run drizzle push!
- For all ID columns NOT related to BetterAuth, use UUID for the ID columns and be randomly generated

TESTING

- Use any testing tools, libraries available to the project for testing your changes
- Never assume your changes simply work, always test!
- If the project does not have any testing tools, scripts, MCP tools, skills, etc. available for testing, ask the user whether testing should be skipped.

UI DESIGN

- Always follow the UI design system when creating or reviewing components or pages.
- Design System: @DESIGN.md

FEATURE FILTER RULE

Before any feature is committed to the roadmap, it must satisfy at least one of:

1. **Reduces time to booking** — the agent completes the booking workflow faster
2. **Increases booking conversion** — more proposals become bookings, more searches become proposals
3. **Increases agency retention** — the agency becomes more operationally dependent on Atlas

If a proposed feature cannot clearly satisfy at least one criterion, it does not move forward. Apply this rule before design, before estimation, before any engineering discussion.

Examples:

- Supplier booking ✓ (reduces time to booking + increases conversion)
- Client portal depth ✓ (increases conversion + increases retention)
- Marketing campaigns module ✗ (does not directly satisfy any criterion — do not build)
- WhatsApp messaging platform ✗ (build as output channel/integration, not a messaging system)
- Accounting workflows (invoice generation + export) ✓ (reduces time to booking)
- Full GL accounting module ✗ (does not satisfy any criterion — partner instead)

PRE-COMMIT DOCUMENTATION RULE

Before every commit, you MUST:

1. **Review all modified files** — understand the full scope of what changed.
2. **Update affected documentation** — if the change touches behaviour, architecture, business rules, APIs, DB schema, UI patterns, or config, update the relevant file(s) in `docs/`, `atlas.md`, `DESIGN.md`, or `AGENTS.md` to stay in sync.
3. **Ensure no contradictions** — the docs must not describe something that the code no longer does. If they conflict, fix the docs before committing.
4. **Record architectural/business decisions** — if the change introduces a new architectural pattern, a significant tech choice, or a business-rule change, create a record under `docs/decisions/` using the template at `docs/decisions/_template.md`.
5. **Summarize doc updates in the commit message** — include a "Docs:" line listing which documentation files were updated and why.

Example commit message footer:

Docs: updated docs/business-rules.md (new booking prerequisite),
docs/decisions/0001-soft-delete-strategy.md (new decision record)

Design System

This document defines the visual design system for the project. All new components and pages **must** follow these tokens, patterns, and conventions.

Stack

- **Framework:** Next.js (App Router) + React + TypeScript
- **Styling:** Tailwind CSS v4 (CSS-first config via `@theme inline` in `globals.css` — no `tailwind.config.ts`)
- **Components:** shadcn/ui (new-york style, neutral base)
- **Icons:** Lucide React
- **Fonts:** Geist (sans) + Geist Mono (mono) via `next/font/google`
- **Dark mode:** next-themes (class-based, system default)
- **Utilities:** `cn()` from `@/lib/utils` (clsx + tailwind-merge)

Colors

The app adopts the **Atlas marketing-deck identity** (`marketing/assets/atlas-ui.css`): an Atlas Blue (`#2B59C3`) primary accent, a cool-paper canvas (`#F6F8FB`), white cards, and a dark ink (`#0E1525`) sidebar rail — while keeping the Geist font.

Light-theme tokens are authored as exact deck **hex** values (hex is valid in CSS custom properties; the deck values are matched verbatim rather than approximated in `oklch`). The dark theme keeps a coherent neutral-`oklch` scaffold (Linear-like surfaces) and only swaps the **accent family + sidebar** to the deck identity. All colors are defined as CSS custom properties in `globals.css` and bridged to Tailwind via `@theme inline`.

Semantic Tokens

Token	Light	Dark	Usage
background	#F6F8FB	oklch(0.141 0.005 285.823)	Page background (deck canvas)
foreground	#1B2333	oklch(0.985 0 0)	Primary text
primary	#2B59C3	#4F7BD9	Buttons, links, accents (Atlas Blue)
primary-foreground	#FFFFFF	#FFFFFF	Text on primary

secondary	#EEF1F6	oklch(0.274 0.006 286.033)	Secondary buttons, subtle bg
secondary-foreground	#1B2333	oklch(0.985 0 0)	Text on secondary
muted	#EEF1F6	oklch(0.274 0.006 286.033)	Subdued backgrounds
muted-foreground	#6B7385	oklch(0.705 0.015 286.067)	Subdued text, placeholders
accent	#EAF0FC	oklch(0.30 0.04 264)	Hover backgrounds, highlights (brand tint)
accent-foreground	#2249A6	#DCE6FA	Text on accent
destructive	#D14343	oklch(0.704 0.191 22.216)	Error states, delete actions
destructive-foreground	#FFFFFF	#FFFFFF	Text on <code>destructive</code> (destructive Badge fill)
surface-2	#FBFCFE	oklch(0.25 0.006 285.885)	Raised neutral surface — nested panels, zebra rows, table-head fill. Tailwind: <code>bg-surface-2</code> .
card	#FFFFFF	oklch(0.21 0.006 285.885)	Card backgrounds
card-foreground	#1B2333	oklch(0.985 0 0)	Card text
popover	#FFFFFF	oklch(0.21 0.006 285.885)	Popover/dropdown bg
popover-foreground	#1B2333	oklch(0.985 0 0)	Popover/dropdown text
border	#E6EAF1	oklch(1 0 0 / 10%)	Borders, dividers (hairline)
border-strong	#D4DAE6	oklch(1 0 0 / 18%)	Heavier divider — input borders, dense tables where a hairline reads too faint. Tailwind: <code>border-border-strong</code> .
input	#E1E6EF	oklch(1 0 0 / 15%)	Input borders
ring	#2B59C3	#4F7BD9	Focus rings

Brand Accent

Token	Light	Dark	Usage
<code>brand</code>	<code>#2B59C3</code>	<code>#5B85DE</code>	Atlas Blue accent for marketing-grade highlights (charts, KPI emphasis). Exposed to Tailwind as <code>*_brand</code> .
<code>brand-foreground</code>	<code>#FFFFFF</code>	<code>#0E1525</code>	Text on <code>brand</code> .
<code>--brand-ring</code>	<code>rgba(43,89,195,0.35)</code>	<code>rgba(91,133,222,0.45)</code>	Canonical 3px keyboard-focus ring. Consumed by the <code>.focus-ring / .skip-link</code> utilities (raw CSS var, not a Tailwind color).

Functional / Status Palette

The deck's tuned, financial-grade status colors — each a **base** (solid fill / icon / strong text), a **soft** low-contrast tint (pill & row backgrounds), and a **-foreground** (text on the solid base). Prefer these over raw Tailwind palette values (`text-green-600`, `bg-amber-500/10`, ...) so status color stays on-brand. All are bridged to Tailwind, so `bg-success` / `text-success` / `bg-success-soft` / `text-success-foreground` (and the `warning/danger/info` equivalents) all resolve.

Token	Light	Dark	Usage
<code>success</code>	<code>#1E9E6A</code>	<code>oklch(0.75 0.14 162)</code>	Paid, confirmed, positive delta
<code>success-soft</code>	<code>#E4F6EE</code>	<code>oklch(0.75 0.14 162 / 15%)</code>	Success pill / row bg
<code>success-foreground</code>	<code>#FFFFFF</code>	<code>#FFFFFF</code>	Text on <code>success</code>
<code>warning</code>	<code>#B45313</code>	<code>oklch(0.78 0.14 66)</code>	Amount-due, passport-expiry, at-risk
<code>warning-soft</code>	<code>#FBEEE2</code>	<code>oklch(0.78 0.14 66 / 15%)</code>	Warning pill / row bg
<code>warning-foreground</code>	<code>#FFFFFF</code>	<code>#0E1525</code>	Text on <code>warning</code>
<code>danger</code>	<code>#D14343</code>	<code>oklch(0.704 0.191 22.216)</code>	Overdue, error, cancelled (alias of <code>destructive</code>)
<code>danger-soft</code>	<code>#FBE9E9</code>	<code>oklch(0.704 0.191 22.216 / 15%)</code>	Danger pill / row bg

danger-foreground	#FFFFFF	#FFFFFF	Text on <code>danger</code>
info	#2B59C3	#4F7BD9	Neutral/informational status (Atlas Blue)
info-soft	#EAF0FC	oklch(0.55 0.14 264 / 20%)	Info pill / row bg
info-foreground	#FFFFFF	#FFFFFF	Text on <code>info</code>

Chart Colors

Aligned to the marketing-deck palette (blue · green · amber · violet · cyan · red). Consumed by `src/components/charts/insight-charts.tsx`. The dark theme keeps the brighter Phase-0 `oklch` values for legibility on dark surfaces.

Token	Light	Dark
chart-1 (blue)	#2B59C3	oklch(0.7 0.14 264)
chart-2 (green)	#1E9E6A	oklch(0.78 0.14 184)
chart-3 (amber)	#B45313	oklch(0.8 0.15 70)
chart-4 (violet)	#7C5CE6	oklch(0.72 0.17 295)
chart-5 (cyan)	#1FA2C7	oklch(0.74 0.18 9)
chart-6 (red)	#D14343	oklch(0.78 0.13 240)

Sidebar Colors

The sidebar is a **dark ink rail in both light and dark themes** (deck identity). Every element inside the rail must use sidebar-scoped tokens (`text-sidebar-foreground`, `bg-sidebar-accent`, `bg-sidebar-primary`, `border-sidebar-border`, ...) for legibility on the dark surface.

Token	Light	Dark
sidebar	#0E1525	#0E1525
sidebar-foreground	#C7CEDC	#C7CEDC
sidebar-primary	#2B59C3	#4F7BD9
sidebar-primary-foreground	#FFFFFF	#FFFFFF
sidebar-accent	rgba(255,255,255,0.07)	rgba(255,255,255,0.07)
sidebar-accent-foreground	#FFFFFF	#FFFFFF
sidebar-border	rgba(255,255,255,0.08)	rgba(255,255,255,0.08)

sidebar-ring

#2B59C3

#4F7BD9

Status Colors — functional tokens are the standard

For **any** status/semantic indicator (badges, pills, amount-due chips, deltas, alerts, timeline dots, progress bars, verified/failed markers) the **functional palette tokens** are the documented standard. Never author raw Tailwind palette values for status meaning.

Meaning	Text	Soft bg / pill	Solid fill / dot	Border
Success — paid, confirmed, positive delta, verified	<code>text-success</code>	<code>bg-success-soft</code>	<code>bg-success</code>	<code>border-success/30</code>
Warning — amount-due, expiry, at-risk, pending money	<code>text-warning</code>	<code>bg-warning-soft</code>	<code>bg-warning</code>	<code>border-warning/30</code>
Danger — overdue, error, failed, cancelled	<code>text-danger</code> (or <code>text-destructive</code>)	<code>bg-danger-soft</code>	<code>bg-danger</code>	<code>border-danger/30</code>
Info — in-flight, acknowledged, mid-lifecycle	<code>text-info</code>	<code>bg-info-soft</code>	<code>bg-info</code>	<code>border-info/30</code>
Neutral — draft, inactive, not-started	<code>text-muted-foreground</code>	<code>bg-secondary / bg-muted</code>	<code>bg-muted-foreground/40</code>	<code>border</code>

Prefer the shared primitives that already resolve to these tokens:

- `<StatusBadge variant={...} />` / `<StatusPill domain status />` with `statusTone(domain, status)` (`src/lib/status-tone.ts`) — the single source of truth for status → tone.
- **Soft-tint badges** (`bg-*-soft text-*`) for pills; **solid tokens** (`bg-*`) for dots and progress fills.

Deprecated / legacy: raw Tailwind status utilities — `text-green-600`, `bg-green-500/15`, `text-amber-600`, `bg-amber-500/10`, `text-red-600`, `text-blue-500`, `dark:* status variants`, etc. — are **legacy and must not be used for new status meaning**. They are being removed in favour of the tokens above. (Raw palette colors remain acceptable **only** for genuinely decorative / categorical use — e.g. amber star-rating icons, brand gradients, chart series `chart-1..6`, and print-doc brand colors — which are *not* status.)

Typography

Font Families

Token	Font	Usage
--font-geist-sans	Geist	All UI text (applied to body)
--font-geist-mono	Geist Mono	Code, monospace content

Body has `font-feature-settings: "rlig" 1, "calt" 1` and `antialiased` enabled.

Type Scale

Class	Size	Usage
text-xs	12px	Timestamps, shortcuts, helper text, code
text-sm	14px	Descriptions, labels, body copy, card descriptions
text-base	16px	Base text, inputs (mobile)
text-lg	18px	Card titles, dialog/sheet titles, sub-headings
text-xl	20px	Section titles, header logo
text-2xl	24px	Page titles
text-3xl	30px	Dashboard/profile headings
text-4xl	36px	Large display text
text-5xl	48px	Hero title

Font Weights

Class	Weight	Usage
font-medium	500	Buttons, labels, nav items
font-semibold	600	Card titles, section headings, badges, dialog titles
font-bold	700	Page titles, hero heading

Line Heights & Tracking

Class	Usage
leading-none	Labels, card titles
leading-5	Code blocks
leading-6	List items
leading-7	Paragraphs (markdown)

`tracking-tight`

Hero/display text

`tracking-widest`

Keyboard shortcuts

Spacing

Container Pattern

```
container mx-auto px-4
```

Responsive overrides where needed:

- Header: `px-3 sm:px-4`
- Footer: `px-4 sm:px-6 lg:px-8`

Max Widths

Class	Value	Usage
<code>max-w-sm</code>	24rem	Auth forms
<code>max-w-md</code>	28rem	Login/register cards, error pages
<code>max-w-lg</code>	32rem	Dialog content (sm+)
<code>max-w-2xl</code>	42rem	Large dialogs
<code>max-w-3xl</code>	48rem	Embeds, protected state
<code>max-w-4xl</code>	56rem	Main content pages

Vertical Spacing (space-y)

Class	Usage
<code>space-y-1</code>	Tight lists, inline stacks
<code>space-y-1.5</code>	Card header
<code>space-y-2</code>	Form field groups, small stacks
<code>space-y-3</code>	Footer stacks
<code>space-y-4</code>	Form sections, dialog content
<code>space-y-6</code>	Card content sections
<code>space-y-8</code>	Page-level sections

Padding

Class	Usage
p-1	Dropdown content, icon buttons
p-2	Code blocks, muted backgrounds
p-3	Chat bubbles, inputs
p-4	Grid items, action buttons, list items
p-6	Cards, dialog content

Page Vertical Padding

Class	Usage
py-3 sm:py-4	Header
py-4 sm:py-6	Footer
py-8	Standard content pages
py-12	Home page, dashboard
py-16	Error/not-found pages

Border Radius

Deck radius scale: cards/panels are **14px** (`--r-lg`), buttons/inputs 10px, chips 6px. The base `--radius` is set to `0.875rem` (14px) so `rounded-lg` cards match the deck.

Token	Value	Class
<code>--radius</code>	<code>0.875rem</code> (14px)	Base
<code>--radius-sm</code>	<code>calc(--radius - 4px) = 10px</code>	<code>rounded-sm</code>
<code>--radius-md</code>	<code>calc(--radius - 2px) = 12px</code>	<code>rounded-md</code>
<code>--radius-lg</code>	<code>var(--radius) = 14px</code>	<code>rounded-lg</code>
<code>--radius-xl</code>	<code>calc(--radius + 4px) = 18px</code>	<code>rounded-xl</code>
—	9999px	<code>rounded-full</code>

Usage:

- `rounded-md` — Buttons, inputs, textarea, code blocks, dropdowns

- `rounded-lg` — Cards, dialogs, feature cards, chat bubbles
- `rounded-xl` — Hero logo container
- `rounded-full` — Badges, avatars

Shadows

Class	Usage
<code>shadow-xs</code>	Inputs, textarea, secondary/outline buttons
<code>shadow-sm</code>	Card base
<code>shadow-md</code>	Card hover, dropdown content
<code>shadow-lg</code>	Dialogs, dropdown sub-content

Custom elevation tokens: layered shadows for premium dashboard / KPI cards, aligned to the marketing-deck `--shadow-sm` (resting) / `--shadow-md` (hover) values.

Token	Value	Usage
<code>--shadow-card</code>	<code>0 1px 2px rgba(16,24,40,.05), 0 1px 1px rgba(16,24,40,.04)</code>	Resting elevation for KPI / dashboard cards (via <code>.card-elevated</code>)
<code>--shadow-card-hover</code>	<code>0 4px 12px rgba(16,24,40,.08), 0 2px 4px rgba(16,24,40,.04)</code>	Hover elevation for interactive cards (via <code>.card-interactive: hover</code>)
<code>--shadow-lg</code>	<code>0 12px 32px rgba(16,24,40,.12), 0 4px 8px rgba(16,24,40,.05)</code>	Dialogs, popovers, dropdowns (deck lg). Tailwind: <code>shadow-lg</code> → bridged to this token.
<code>--shadow-xl</code>	<code>0 24px 60px rgba(16,24,40,.16), 0 8px 16px rgba(16,24,40,.06)</code>	Device frames, hero / floating cards (deck xl). Tailwind: <code>shadow-xl</code> .

Dark theme keeps a coherent scaffold: `--shadow-lg` → `0 12px 32px oklch(0 0 0 / 45%), 0 4px 8px oklch(0 0 0 / 35%);` `--shadow-xl` → `0 24px 60px oklch(0 0 0 / 55%), 0 8px 16px oklch(0 0 0 / 40%)`.

Animations

Custom Keyframes

Name	Effect	Duration	Easing
------	--------	----------	--------

<code>fade-in</code>	Opacity 0 → 1	0.3s	ease-out
<code>fade-up</code>	Opacity 0 → 1 + translateY(8px → 0)	0.4s	ease-out
<code>scale-in</code>	Opacity 0 → 1 + scale(0.97 → 1)	0.2s	ease-out

Use via: `animate-fade-in`, `animate-fade-up`, `animate-scale-in`

tw-animate-css Animations

Used on dialogs and dropdowns:

- `animate-in` / `animate-out`
- `fade-in-0` / `fade-out-0`
- `zoom-in-95` / `zoom-out-95`
- `slide-in-from-{top|bottom|left|right}-2`

Transition Classes

Class	Usage
<code>transition-colors</code>	Links, hover color changes
<code>transition-opacity</code>	Avatar hover, reveal-on-hover
<code>transition-all duration-200</code>	Card interactive hover, buttons
<code>transition-[color,box-shadow]</code>	Input/textarea focus

Utility Classes

```
.card-interactive {
  @apply transition-all duration-200 ease-out;
}
.card-interactive:hover {
  box-shadow: var(--shadow-card-hover);
  @apply -translate-y-0.5;
}
/* Premium resting elevation for dashboard / KPI cards. */
.card-elevated {
  box-shadow: var(--shadow-card);
}
```

```
.auth-bg {
  background-image: radial-gradient(
    circle at 50% 0%,
    var(--accent) 0%,
    transparent 50%
  );
}
```

Numeric alignment — `.tnum` / `.tabular-nums`: opt-in `font-variant-numeric: tabular-nums` for numeric columns, KPI values, and currency figures (DZD/EUR/USD) so digits align and totals don't jitter. Apply to table cells and stat values.

```
.tnum,  
.tabular-nums {  
  font-variant-numeric: tabular-nums;  
}
```

Brand focus ring — `.focus-ring`: the canonical 3px Atlas-Blue focus affordance (`box-shadow: 0 0 3px var(--brand-ring)`). Pair with `focus-visible`.

Skip link — `.skip-link`: visually hidden (`sr-only`) until focused, then fixed top-left with the brand ring. Used by the app shell for skip-to-content.

Reduced motion: a global `@media (prefers-reduced-motion: reduce)` block collapses animation/transition durations to `~0` and disables smooth scroll for users who opt out.

Layout

Root Structure

```
<body class="antialiased min-h-screen flex flex-col">  
  <SiteHeader />  
  <main id="main-content" class="flex-1">{children}</main>  
  <SiteFooter />  
  <Toaster />  
</body>
```

Page Layout Patterns

Auth pages:

```
flex min-h-[calc(100vh-4rem)] items-center justify-center p-4  
  → Card w-full max-w-md
```

Standard content pages:

```
container mx-auto px-4 py-8  
  → max-w-4xl mx-auto
```

Error/not-found pages:

```
container mx-auto px-4 py-16  
  → max-w-md mx-auto text-center
```

Grid Patterns

Pattern	Usage
<code>grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-6</code>	Feature cards (4-col)
<code>grid grid-cols-1 md:grid-cols-2 gap-6</code>	Dashboard cards
<code>grid grid-cols-1 md:grid-cols-2 gap-4</code>	Profile info
<code>grid grid-cols-1 md:grid-cols-3 gap-4</code>	Quick actions

Responsive Breakpoints

Standard Tailwind breakpoints:

- `sm`: (640px) — Padding adjustments, text alignment, button sizing
- `md`: (768px) — Grid column changes (→ 2 col), input font size
- `lg`: (1024px) — Grid column changes (→ 4 col), wide padding

Icons

Library: Lucide React

Sizing Convention

Size	Classes	Usage
XS	<code>h-3 w-3</code>	Inline badge icons
SM	<code>h-3.5 w-3.5</code>	Copy buttons
Default	<code>h-4 w-4</code> or <code>size-4</code>	Standard UI icons
MD	<code>h-5 w-5</code>	Header logo icon
LG	<code>h-7 w-7</code>	Hero logo icon
XL	<code>h-16 w-16</code>	Error/empty state illustrations

Commonly Used Icons

Bot, User, Lock, Shield, Mail, Calendar, Copy, Check, Loader2, LogOut, Sun, Moon, Github, ArrowLeft, RefreshCw, AlertCircle, FileQuestion, Database, Palette, Video

Components (shadcn/ui)

All components live in `src/components/ui/`. They use `data-slot` attributes, accept `className` for overrides via `cn()`, and follow either `React.forwardRef` or functional component patterns.

Button

6 variants, 4 sizes (CVA-based):

Variant	Usage
<code>default</code>	Primary actions
<code>secondary</code>	Secondary actions
<code>outline</code>	Tertiary actions
<code>ghost</code>	Subtle/icon actions
<code>destructive</code>	Delete/danger actions
<code>link</code>	Inline text links

Size	Height	Padding
<code>sm</code>	<code>h-8</code>	<code>px-3</code>
<code>default</code>	<code>h-9</code>	<code>px-4</code>
<code>lg</code>	<code>h-10</code>	<code>px-6</code>
<code>icon</code>	<code>size-9</code>	<code>—</code>

Card

6 sub-components: `Card`, `CardHeader`, `CardTitle`, `CardDescription`, `CardContent`, `CardFooter`

Base: `rounded-lg border bg-card text-card-foreground shadow-sm`

`CardTitle` is `text-lg font-semibold leading-none tracking-tight` (18px subhead scale — matches `DialogTitle` / `SheetTitle`; reserve `text-2xl/text-3xl` for page-level headings).

Elevation composition: the base card sits at `shadow-sm`. To lift a card to the premium dashboard/KPI resting elevation, add `className="card-elevated"` (applies `--shadow-card`). For interactive cards that lift on hover, add `className="card-interactive"` (200ms ease-out, `--shadow-card-hover + -translate-y-0.5`). Both are utility classes in `globals.css` and compose cleanly via `cn()` — no wrapper or prop needed.

Input / Textarea

- **Height:** `h-9` (input), `min-h-16` (textarea)
- **Border:** `border bg-transparent rounded-md shadow-xs`
- **Focus:** `focus-visible:border-ring focus-visible:ring-ring/50 focus-visible:ring-`

[3px]

- Validation: `aria-invalid:border-destructive aria-invalid:ring-destructive/20`
- Responsive font: `text-base md:text-sm`

Checkbox

Radix-based (`radix-ui` unified import; `data-slot="checkbox"`). A `size-4`, `rounded-[4px]`, `border-input` box; checked state is `bg-primary + border-primary` with a white Lucide Check (`size-3.5, strokeWidth={3}`) indicator. Shares the input focus ring (`focus-visible:border-ring + ring-ring/50 ring-[3px]`), `aria-invalid` (`border-destructive + ring-destructive/20`), and disabled (`opacity-50, cursor-not-allowed`) treatments. Replaces native `<input type="checkbox">` (hotel-search, flight-results, contacts). Exports `Checkbox`.

Badge

7 variants: default, secondary, destructive, outline, plus the deck **status variants** `success`, `warning`, `info`.

Base: `rounded-full border px-2.5 py-0.5 text-xs font-semibold` with the canonical brand focus ring (`focus-visible:ring-ring/50 focus-visible:ring-[3px]`).

Status variants follow the deck's pill look — a soft tinted background + solid token text — using the functional tokens:

Variant	Background	Text
success	<code>bg-success-soft</code>	<code>text-success</code>
warning	<code>bg-warning-soft</code>	<code>text-warning</code>
info	<code>bg-info-soft</code>	<code>text-info</code>

`destructive` now pairs the solid `bg-destructive` fill with the real `text-destructive-foreground` (white) token.

Leading status dot — pass `dot` to render a small `size-1.5 rounded-full` span before the label. It defaults to `bg-current`, inheriting the variant's solid text color (e.g. an Atlas-green dot on a `success` badge). Override with `dotClassName` for a fixed hue. The dot is `aria-hidden`. Example: `<Badge variant="success" dot>Confirmed</Badge>`.

StatusBadge / StatusPill

App-level status pill (`src/components/app/status-badge.tsx`) used for domain statuses (booking / payment / proposal / opportunity / supplier / commission / ...). It is the status-coloured counterpart to the shadcn `Badge` and shares its soft-tint+-solid-text look.

Colour it semantically. Prefer the `variant` prop — one of the five **semantic tones** backed by the Wave-1 functional tokens:

variant	Background	Text	Meaning
neutral	bg-secondary	text-secondary-foreground	Draft / inactive / not-yet-started
success	bg-success-soft	text-success	Settled / active / won
warning	bg-warning-soft	text-warning	Needs attention / pending / part-paid
info	bg-info-soft	text-info	In-flight / acknowledged / mid-lifecycle
danger	bg-danger-soft	text-danger	Failed / cancelled / lost / overdue

Never re-introduce a raw Tailwind colour string at the call site. Derive the tone from the status via the shared helper `statusTone(domain, status)` in `src/lib/status-tone.ts`, which maps every real status vocabulary (see `domain.ts`) to a tone. Domains: `opportunity` · `client` · `product` · `booking` · `bookingItem` · `paymentRecord` · `paymentSummary` · `supplier` · `contract` · `commission` · `subscription` · `generic`. Unknown codes fall back to `generic` then `neutral` (never crash).

- `<StatusBadge variant={statusTone("booking", status)} label={meta.label} dot />` — explicit tone.
- `<StatusPill domain="booking" status={booking.status} label={meta.label} dot />` — convenience wrapper that runs `statusTone` for you (`label` defaults to the raw code).

`dot` / `dotClassName` behave exactly as on `Badge`.

No raw tone escape hatch. The legacy `tone?: string` prop (a raw Tailwind colour string like `"bg-green-100 text-green-700"`) has been **removed entirely** from `StatusBadge`/`StatusPill` — `variant` (or `statusTone()`/`StatusPill`'s `domain`) is the only way to colour a status pill now. `src/lib/domain.ts` no longer carries any colour metadata either: its `*_META` maps are label-only, so every status pill in the app is derived through `statusTone()` at render time, not stored alongside the status code.

Role badges — `USER_ROLE_TONE`. Roles are categorical, not a status lifecycle, so they get their own fixed mapping in `src/lib/domain.ts` rather than running through `statusTone()`: `admin` / `manager` → `info`, `finance` → `success`, `support` → `warning`, `agent` → `neutral`. Use `<StatusBadge variant={USER_ROLE_TONE[role]} label={USER_ROLE_META[role].label} />` for role pills (team list, user profile) instead of a bespoke colour.

Dialog

Radix-based with overlay (`bg-black/50`), fade + zoom animations, optional close button.

DropDownMenu

Radix-based. Content: `rounded-md border p-1 shadow-md min-w-[8rem]`. Items support a destructive variant.

Table

The deck **data-table** treatment (`marketing/assets/atlas-ui.css .table`). Base exports (`Table`, `TableHeader`, `TableBody`, `TableRow`, `TableHead`, `TableCell`) keep their native element signatures — the deck upgrades are **additive / opt-in** so existing markup is unchanged.

Header band (default): every `TableHead` now renders the deck header chrome — `bg-surface-2`, `text-[11px]` `font-semibold` `uppercase` `tracking-wider` `text-muted-foreground`, `h-10` `px-4`, `whitespace-nowrap` — matching `.table thead th` verbatim. Body cells use `px-4` `py-3` for the deck's 16px horizontal rhythm.

Opt-in props:

Prop	On	Effect
<code>zebra</code>	<code>Table</code>	Stripes even body rows (<code>bg-surface-2/60</code>) — deck <code>.table.zebra</code> .
<code>sticky</code>	<code>TableHeader</code>	Pins the header (<code>sticky top-0 z-10</code> , header cells keep <code>bg-surface-2</code>). Pair with a <code>max-h-* / overflow-auto</code> wrapper.
<code>numeric</code>	<code>TableHead</code> / <code>TableCell</code>	Right-aligns + applies <code>tabular-nums</code> for money/counts.
<code>align</code>	<code>TableHead</code> / <code>TableCell</code>	"left" "center" "right" — overrides <code>numeric</code> 's default right-align.
<code>sortable</code>	<code>TableHead</code>	Renders a direction-aware chevron and sets <code>aria-sort</code> ; wire <code>onClick</code> to re-order.
<code>sortDirection</code>	<code>TableHead</code>	"asc" "desc" <code>false</code> — drives the chevron (<code>ChevronUp</code> / <code>ChevronDown</code> / faint <code>ArrowUpDown</code>) and <code>aria-sort</code> (<code>ascending</code> / <code>descending</code> / <code>none</code>).

TableSortButton (exported helper) — a standalone sortable-header trigger for when a `TableHead` must wrap a real `<button>` (router / server-action sorting). Renders the label + a `SortDirection`-aware chevron, carries the `.focus-ring`, and auto-labels (`Sort by {label}`). The `SortDirection` type ("asc" | "desc" | `false`) is also exported.

Mobile card-reflow pattern: tables don't reflow under ~640px — hide the table and render list cards instead of horizontal-scrolling. Wrap the `<Table>` in `hidden sm:block` and render a `sm:hidden <ul>` of `Card` items with the same data + links (documented as a comment in `table.tsx`).

Tabs

Radix-based (added in the UI-redesign Phase 1). `TabsList` is a `rounded-md` `bg-muted` `p-1`

segmented control; the active `TabTrigger` gets `bg-background + shadow-sm`. Used for the client-profile sections (Overview / Trips / Opportunities / Proposals / Contacts / Notes) and the pipeline scope switcher (All / My deals / Closing soon).

Breadcrumb

Markup-only (added in Phase 1). `Breadcrumb` → `BreadcrumbList` → `BreadcrumbItem` with `BreadcrumbLink` / `BreadcrumbPage` and a `BreadcrumbSeparator` (`ChevronRight`). Used at the top of detail/section pages (client profile, pipeline) for location context.

Tooltip

Radix-based (added in Phase 1). `TooltipProvider` + `Tooltip` / `TooltipTrigger` / `TooltipContent` (`bg-primary text-primary-foreground, fade + zoom in`). For terse hover hints on icon-only affordances.

Sheet

Radix-Dialog-based side drawer — the canonical drawer other screens compose. Slides from any side (`side="right"` default, `left/top/bottom`), overlay `bg-black/50`, panel `bg-background` with the deck's tuned `shadow-[var(--shadow-lg)]`. Motion is **ease-out only** (`ease-out`, open 300ms / close 200ms — no ease-in or bounce, per the motion rule). Slots: `Sheet`, `SheetTrigger`, `SheetClose`, `SheetContent`, `SheetHeader`, `SheetFooter`, `SheetTitle` (`text-lg font-semibold`), `SheetDescription` (`text-sm text-muted-foreground`). Built-in close button (top-right `XIcon`) uses the canonical `.focus-ring`.

Skeleton

Neutral loading placeholder: `bg-muted animate-pulse` (muted grey — reads as absent content, not a brand-tinted/selected block), `aria-hidden` so screen readers skip the decorative bars. Optional `variant` prop (backward-compatible, defaults to `default`):

Variant	Shape	Usage
default	rounded-md	Block placeholders (cards, thumbnails, buttons)
text	h-4 w-full rounded-full	Inline text-line placeholder
circle	rounded-full	Avatar / icon placeholder

`className` still overrides sizing/shape as before. Exports `Skeleton` + `SkeletonVariant` type.

Spinner

Sizes: `sm` (h-4 w-4), `md` (h-6 w-6), `lg` (h-8 w-8). Uses `Loader2` with `animate-spin`.

Toast (Sonner)

Custom icons per state (success, info, warning, error, loading). Themed via CSS variable

overrides.

Focus & Interaction States

Focus Ring (Global)

```
outline-2 outline-offset-2 outline-ring/70
```

Component-level override:

```
focus-visible:ring-ring/50 focus-visible:ring-[3px]
```

Canonical brand ring: the `--brand-ring` token (`rgba(43,89,195,0.35)` light / `rgba(91,133,222,0.45)` dark) backs the `.focus-ring` utility — a single 3px Atlas-Blue ring for keyboard focus across interactive primitives. New focusable components should converge on this rather than inventing a fourth focus treatment.

Disabled

```
disabled:pointer-events-none disabled:opacity-50
```

Interactive Card Hover

```
transition-all duration-200 ease-out  
hover:shadow-md hover:-translate-y-0.5
```

Dark Mode

- **Method:** Class-based via `next-themes` with `attribute="class"` and `disableTransitionOnChange`
- **Default:** System preference
- **Toggle:** 3-way dropdown — Light / Dark / System
- All semantic color tokens swap automatically via `.dark` CSS selector
- Use `dark:` prefix for component-specific overrides (e.g., `dark:bg-input/30`)

Branding

Logo Text

```
bg-gradient-to-r from-primary to-primary/70 bg-clip-text text-transparent
```

Logo Icon Container

```
w-8 h-8 rounded-lg bg-primary/10 flex items-center justify-center
```

Hero variant: `w-12 h-12 rounded-xl`

@AGENTS.md

PART III

Audits & Reviews

4 documents

Atlas — Production Readiness Audit Report

Date: 2026-06-28

Remediation status (applied 2026-06-28): The safe high-priority tranche (both Criticals + all 13 Highs + several low-risk Mediums/Lows) has been implemented via coordinated sub-agents and verified with `tsc --noEmit` (clean), `npm run lint` (0 errors), and `next build` (all routes compile). See § **Remediation Applied** at the end of this report for the full list of fixes, files changed, and remaining items. **One blocker is environmental:** the DB index migration `drizzle/0018_regular_karnak.sql` is generated but **not yet applied** — the dev Neon credentials in `.env` return `28P01 password authentication failed (password rotated)`. Refresh `POSTGRES_URL` and run `npx drizzle-kit migrate` to apply it.

Executive Summary

Atlas has a fundamentally sound architecture: tenant isolation via `requireAgencyUser()` and double-condition `and(eq(agencyId, ...), ...)` scoping is consistent across all 20 server-action files, all inputs are Zod-validated, and there is no SQL injection or `dangerouslySetInnerHTML` exposure. However, the application is **not yet production-ready** without remediation. The two genuine blockers are silent fake-PNR fallback on failed airline bookings (agents see a confirmation for a flight that was never booked) and missing try/catch on Stripe webhook DB writes (crashes cause endless Stripe retries with duplicate side-effects). Counting across all six audits: **2 critical blockers, 11 high-severity, 21 medium-severity, and ~20 low-severity** issues. The high-severity cluster is dominated by missing error handling (no try/catch in 10 action files, no `error.tsx` in client-facing segments), absent `fetch` timeouts on all external supplier calls, RTL breakage in the mobile drawer, and a prompt-injection vector in the AI chat endpoint.

Critical Issues (must fix before production)

#	Category	File	Issue	Fix
C1	Error Handling	<code>src/lib/suppliers/duffel.ts:278</code> <code>src/lib/actions/bookings.ts:382</code>	Failed Duffel order silently falls back to a fake <code>DF-xxxxxxx</code> reference and marks booking <code>confirmed</code> — agent sees success, no real PNR exists	Return discriminated provisional/confirmed result; never set <code>status: "confirmed"</code> on a provisional; surface a visible UI warning

C2	Error Handling	<code>src/app/api/stripe/webhook/ route.ts:75-86</code> , <code>src/app/api/stripe/connect-webhook/ route.ts:40-46</code>	No try/catch around webhook DB updates; a DB throw returns 500 and Stripe retries indefinitely → duplicate side-effects. <code>connect-webhook</code> also returns 400 (stops retries) when temporarily misconfigured instead of 503	Wrap DB writes in try/catch returning 500 only on genuine transient errors; return 503 (not 400) for misconfiguration so Stripe retries cleanly
----	----------------	---	---	---

High Severity Issues

#	Category	File	Issue	Fix
H1	Security	<code>src/app/api/chat/ route.ts:28-36</code>	Zod schema accepts <code>role: "system"</code> from client → prompt injection / jailbreak of the AI agent, including spurious <code>createBooking</code> calls	Drop <code>"system"</code> from the cli
H2	Security	<code>src/app/api/portal/auth/signout/ route.ts:2,15</code> ; <code>src/app/portal/layout.tsx:39</code>	Portal signout is <code>GET</code> via <code><a href></code> → CSRF forced-logout (CWE-352)	Change handler to <code>POST</code> ; us
H3	Error Handling	10 action files (<code>clients.ts:45</code> , <code>team.ts</code> , <code>suppliers.ts</code> , <code>invites.ts</code> , <code>opportunities.ts</code> , <code>portal-proposals.ts</code> , <code>notifications.ts</code> , <code>proposals-public.ts</code> , <code>portal-invite.ts</code> , <code>onboarding.ts</code>)	Zero try/catch around DB calls → unhandled exceptions crash the action with a generic 500, no logging	Wrap DB writes; return <code>{ ok</code> <code>console.error("[actio</code>
H4	Error Handling	<code>src/app/api/chat/ route.ts:77-277</code>	<code>streamText</code> and all tool <code>execute</code> bodies have no try/catch; <code>createBooking</code> item loop silently skips failed items	Wrap <code>streamText</code> and eac

H5	Error Handling	<code>src/lib/suppliers/duffel.ts:39</code> , <code>hotelbeds.ts:159</code> , <code>payments/stripe.ts</code>	No <code>fetch</code> timeout (AbortController) on any external call → function hangs to Vercel's 30s timeout	Add <code>AbortController</code> + <code>AbortSignal</code> to base fetch wrapper
H6	Error Handling	<code>src/app/portal/</code> , <code>src/app/platform/</code> , <code>src/app/(app)/bookings/</code> , <code>src/app/(app)/clients/</code>	No segment-level <code>error.tsx</code> ; errors bubble to root boundary whose "Go home" link points to <code>/</code> (inaccessible to portal clients)	Add <code>error.tsx</code> per segment (<code>portal/login</code>)
H7	Performance	<code>src/app/(app)/dashboard/page.tsx:83–85</code>	Counts done by fetching every row's <code>id</code> then <code>.length</code> — 3 full-table scans for 3 integers	Use <code>db.\$count(table, v</code>
H8	Performance	<code>src/lib/actions/bookings.ts:53</code> , <code>src/lib/actions/products.ts:36</code>	<code>nextReference()</code> fetches all references in the agency and finds max in JS on every create	Single <code>max(...)</code> aggregation
H9	Performance	<code>src/lib/actions/products.ts:57–67</code>	<code>recalcTotals()</code> issues one sequential UPDATE per item (N+1 writes)	Batch CASE-WHEN update
H10	Performance	<code>src/app/(app)/dashboard/page.tsx</code> , <code>src/app/(app)/finance/page.tsx</code>	No Suspense boundaries; user sees blank screen until all DB queries finish (finance loads 1,000 bookings first)	Split into shell + Suspense a
H11	UX / RTL	<code>src/components/app/app-shell.tsx:223</code>	Mobile nav drawer hardcodes <code>slide-in-from-left-2 / left-0 / border-r</code> → appears from wrong side in Arabic (RTL)	Conditional <code>rtl</code> : variants or
H12	UX / A11y	All auth forms (<code>sign-in-button.tsx:83</code> , <code>sign-up-form.tsx:107</code> , <code>forgot-password-form.tsx:70</code> , <code>reset-password-form.tsx:25,100</code> , <code>accept-invite-form.tsx:99</code>)	Error <code><p></code> has no <code>role="alert"/aria-live</code> → screen readers never announce login failures	Add <code>role="alert"</code> to error

H13	Database	<code>src/lib/ schema.ts:188-198 (verification table)</code>	No index on <code>verification.identifier</code> → full scan on every login/OTP check	Add <code>index("verification_i + migration</code>
-----	----------	--	--	---

Medium Severity Issues

#	Category	File	Issue	Fix
M1	Security	<code>src/lib/actions/ commissions.ts:28-31, 74-76</code>	<code>recordCommission</code> does not cross-tenant validate <code>supplierId</code> , <code>bookingItemId</code> , <code>agentUserId</code>	Add ownership, <code>user</code> scoped to <code>user</code> before INSERT
M2	Security	<code>src/lib/notifications/ templates.ts:14, 52-72</code>	User strings (<code>clientName</code> , <code>agencyName</code> , <code>roleLabel</code>) interpolated into HTML email without escaping	Add <code>h()</code> HTML- helper, apply to sourced values
M3	Security	<code>src/lib/storage.ts:117</code>	File upload validates extension only, not MIME magic bytes; SVG payloads accepted; risky on local-fs fallback path	Add <code>file-type</code> , byte check; reje user images
M4	Error Handling	<code>src/lib/actions/ commissions.ts:505-508</code>	<code>autoGenerateCommissions</code> failure swallowed with <code>console.error</code> ; booking advances with no commissions and no warning	Return structure surface non-bl warning
M5	Error Handling	<code>src/lib/actions/ platform.ts:134-136</code>	Stripe customer-create failure only logged; agency saved without <code>stripeCustomerId</code> → later "Agency not found" or duplicate customers	Structured log + for reconciliation
M6	Error Handling	<code>src/lib/suppliers/ index.ts:78-97 (safeSearch)</code>	<code>degraded: true</code> from mock fallback not forwarded to AI in chat route (<code>api/chat/ route.ts:96-101</code>) → mock prices presented as real	Include <code>source</code> in tool result
M7	Error Handling	entire <code>src/</code>	No structured logging library; only <code>console.error(string)</code> → cannot query/alert per tenant or correlate digests	Adopt Pino/Sen line logging con

M8	Performance	<code>src/lib/actions/bookings.ts:459-468</code>	<code>advanceStatus</code> ticketing loop: sequential external API + DB write per item	<code>Promise.all()</code> calls; batch DB
M9	Performance	<code>src/lib/actions/commissions.ts:258-277</code>	<code>getCommissions()</code> has no <code>.limit()</code> → unbounded result set	Add <code>.limit(50)</code> pagination
M10	Performance	<code>src/app/(app)/dashboard/page.tsx:109</code>	Loads up to 500 bookings with payments+travellers to compute KPIs in JS	Move aggregation scope traveller & passport alerts
M11	Performance	<code>src/app/(app)/finance/page.tsx:74</code>	Loads up to 1,000 bookings+payments; KPIs computed via JS reductions	Replace with SC aggregates; load only when balance
M12	Performance	<code>src/lib/schema.ts</code>	Missing composite indexes for <code>(agencyId, status)</code> / <code>(agencyId, createdAt)</code> on booking, opportunity, product, commission	Add composite index & migrate
M13	Performance	<code>src/lib/export/datasets.ts:74+</code>	Export loaders are unbounded <code>findMany</code> → OOM/timeout on large agencies	Per-dataset limit streaming export
M14	Performance	<code>src/components/app/app-shell.tsx:1</code>	Entire nav shell is <code>"use client"</code> only for <code>usePathname()</code>	Extract active-link tiny <code><NavLink></code> as server component
M15	Performance	<code>src/app/(app)/dashboard/page.tsx:33</code>	Recharts (~150KB) eagerly imported for all users	<code>next/dynamic({ssr:false})</code> imports
M16	Performance	<code>src/components/hotels/hotel-details-view.tsx:221</code> , <code>src/components/search/hotel-details-dialog.tsx:123</code>	Raw <code></code> bypassing <code>next/image</code> optimization	Use <code>next/image</code> Hotelbeds CDN <code>images.remote</code>
M17	Code Quality	9+ files (<code>billing.ts</code> , <code>invites.ts</code> , <code>payments.ts</code> , <code>portal-payments.ts</code> , <code>portal-invite.ts</code> , <code>api/portal/auth/request/route.ts</code> , <code>robots.ts</code> , <code>sitemap.ts</code> , <code>auth-client.ts</code>)	<code>NEXT_PUBLIC_APP_URL ?? "localhost:3000"</code> duplicated, bypassing <code>env.ts</code>	Export single <code>API config.ts</code>
M18	Code Quality	<code>src/lib/suppliers/</code> vs <code>src/lib/suppliers/providers/</code>	Two parallel supplier abstractions coexist; <code>providers/</code> registry is empty and unused	Mark <code>provider</code> progress or completion migration and re-layer

M19	Code Quality	<code>src/lib/actions/**/*.ts</code>	62 of 94 exported server actions lack explicit return-type annotations	Annotate with <code>: Promise<Action></code>
M20	UX / A11y	<code>client-form.tsx:62</code> , <code>booking-form.tsx:87</code> , <code>opportunity-form.tsx</code>	Server errors only via auto-dismissing toast, no persistent inline error	Add <code>serverError</code> role="alert" message
M21	UX / A11y	All forms	<code>aria-invalid</code> never set on fields after failed validation	Set <code>aria-invalid</code> empty/invalid re
M22	UX / A11y	<code>clients/page.tsx:92</code> , <code>suppliers/page.tsx:85</code>	Search <code><Input></code> has no <code><Label></code> (placeholder is not an accessible name)	Add <code><Label></code> <code>htmlFor="q"</code> <code>className="s</code>
M23	UX / A11y	<code>mode-toggle.tsx:19</code> , <code>create-agency-form.tsx:66</code> , <code>travellers-manager.tsx:303</code> , <code>payments-manager.tsx:227</code>	Icon-only buttons missing <code>aria-label/aria-haspopup</code> ; <code>save</code> buttons lack <code>aria-busy</code>	Add <code>aria-label</code> <code>haspopup="me</code> <code>busy</code>
M24	UX / RTL	<code>~62 mr-/ml- + ~94 pl-/pr-/border-l/border-r</code> across components	Physical CSS directions don't flip in RTL	Refactor to logic <code>ps-/pe-/border-e</code>
M25	UX / RTL	List pages with tables (<code>clients</code> , <code>bookings</code> , <code>products</code> , <code>suppliers</code>)	<code>text-right</code> numeric columns don't flip in RTL	Replace with <code>te</code>
M26	Database	<code>src/lib/schema.ts</code>	No <code>createdAt</code> index on booking/client/opportunity/product/payment/notification	Add <code>createdAt</code> (priority: booking/opportunity > pr payment)
M27	Database	<code>src/lib/schema.ts:423,510-536</code>	<code>bookingItem.supplierId</code> / <code>productItem.supplierId</code> FKs have no index → full scans in commission/supplier joins	Add <code>supplierId</code> on both tables
M28	Database	<code>src/lib/schema.ts</code>	No soft-delete (<code>deletedAt</code>) anywhere; hard deletes orphan commissions, no undelete/audit	Add <code>deletedAt</code> booking/opportunity/product; filter lis
M29	Database	<code>src/lib/schema.ts:985-1001</code>	<code>supplierRate</code> has no denormalized <code>agencyId</code> → direct agency-scoped queries need double join	Add <code>agencyId</code> (matches <code>supplierCont</code> pattern)

M30	Database	<code>src/lib/actions/bookings.ts:65-78</code>	<code>recalcTotal(bookingId) queries bookingItem by bookingId with no agency scope (footgun if called with attacker-controlled id)</code>	Pass <code>agencyId</code> filter <code>booking.</code> or accept pre-lo
-----	----------	--	---	--

Low Severity Issues / Recommendations

#	Category	File	Issue
L1	Security	<code>src/proxy.ts</code> matcher	<code>/suppliers, /finance, /reports, /settings, /billing, /commissions</code> not in matcher (still safe via <code>requireAgencyUser()</code>)
L2	Security	<code>src/app/api/portal/auth/request/route.ts</code>	No rate limiting on magic-link request → resource exhaustion / enumeration
L3	Security	<code>src/lib/actions/settings.ts:21-27</code>	Locale cookie <code>httpOnly:false</code> (intentional; validated via <code>isLocale()</code>)
L4	Error Handling	<code>src/lib/actions/settings.ts:34,65</code>	Empty <code>catch {}</code> with no logging
L5	Error Handling	<code>src/lib/suppliers/hotelbeds.ts:467</code>	Silent <code>catch {}</code> on availability fallback
L6	Error Handling	<code>src/lib/suppliers/content-cache.ts:110,121</code>	<code>.catch(() => {})</code> silent cache-write swallow → cache never heals
L7	Error Handling	clipboard handlers <code>ai-quote-builder.tsx:79, itinerary-builder.tsx:306,340, payments-manager.tsx:95</code>	Silent clipboard catch after success toast → "copied!" shown even on failure
L8	Error Handling	<code>src/app/error.tsx:16-17</code>	<code>error.digest</code> shown in UI but not logged server-side → can't correlate
L9	Performance	<code>src/lib/actions/commissions.ts:284-309</code>	<code>getCommissionsByBooking()</code> does two sequential queries
L10	Performance	<code>src/app/layout.tsx:54</code>	All <code>i18n</code> messages shipped to every client

L11	Performance	<code>src/app/proposal/[id]/page.tsx</code> , <code>public token pages</code>	No ISR/revalidate on public proposal pages
L12	Code Quality	<code>src/lib/session.ts</code>	Entire file is dead code (<code>requireAuth</code> , <code>getOptionalSession</code> , <code>stale protectedRoutes</code>)
L13	Code Quality	<code>src/proxy.ts</code>	Middleware never executes (not <code>middleware.ts</code> , no <code>middleware/default export</code>) → manifest empty
L14	Code Quality	<code>src/components/ui/github-stars.tsx</code>	Exported component with zero consumers
L15	Code Quality	<code>portal-session.ts:20</code> , <code>settings.ts:24, 60</code> , <code>platform.ts:200, 228</code>	<code>secure: NODE_ENV=== "production"</code> duplicated 5×
L16	Code Quality	<code>analytics.ts:62</code> , <code>finance/page.tsx:176</code>	<code>Intl.DateTimeFormat("en-GB", {month: "short"})</code> duplicated
L17	Code Quality	<code>permissions.ts:53, 193</code>	<code>session.user as unknown as {...}</code> cast duplicated
L18	Code Quality	<code>hotelbeds.ts:81</code> , <code>add-to-booking-dialog.tsx:67</code> , <code>add-to-proposal-dialog.tsx:57</code> , <code>payments-manager.tsx:97</code> , <code>hotel-details-view.tsx:297</code>	Non-null assertions without guard
L19	Code Quality	<code>api/portal/auth/request/route.ts:67</code> , <code>notifications/email.ts:30</code>	<code>console.log</code> violates <code>no-console</code> ESLint rule (warn/error only)
L20	UX / A11y	<code>hotel-search-experience.tsx:222-228</code>	Search button loses label text + no <code>aria-label</code> while loading
L21	UX / Mobile	Table wrappers in list pages	No visual scroll hint on overflowing tables
L22	UX / Mobile	<code>pipeline-board.tsx:101-108</code>	Stage-move trigger ~20px touch target (< WCAG 44px)
L23	UX / A11y	<code>hotel-details-view.tsx:221</code> , <code>hotel-details-dialog.tsx:123</code>	Gallery images use empty <code>alt=""</code> though not purely decorative
L24	UX / A11y	<code>portal/layout.tsx:25</code>	<code><nav></code> missing <code>aria-label</code>

L25	Code Quality	billing/stripe.ts:17, payments/stripe.ts:9	isBillingConfigured()/isStripeConfigured() identical
L26	Database	src/lib/schema.ts:1023-1025	commission.bookingItemId uses ON DELETE SET NULL → orphaned, unreconcilable commissions
L27	Database	src/lib/schema.ts:652-868	Missing Drizzle relations for supplier, supplierContract, supplierRate, commission → breaks with: eager loads
L28	Database	drizzle/0006_add_agency_tenancy.sql:343	Unconditional backfill could set platform-admin agencyId to seed agency (non-issue in practice, all is_platform_admin=false then)
L29	Database	src/lib/schema.ts:882-906	hotelContent uses text ("code") PK (justified natural key)
L30	Database	payment, bookingTraveller and other child tables	No denormalized agencyId (defense-in-depth gap)

Category Findings

1. Security

Overall posture is solid: `agencyId` is always server-derived from `session.user.agencyId`, every action calls `requireAgencyUser()/requireUser()/requirePlatformAdmin()/requireManager()` first, all mutations use `and(eq(id), eq(agencyId))` double-conditions, all inputs are Zod-validated, Drizzle parameterizes all queries, no secrets are in `NEXT_PUBLIC_`, no `dangerouslySetInnerHTML`, Stripe webhooks verify signatures, impersonation gates on `requirePlatformAdmin()` with `httpOnly` cookies, portal actions verify `clientId + agencyId`, and invite registration ties `agencyId/role` to the invite row (`input:false`).

- **H1 — Prompt injection via `role:"system"`** (`api/chat/route.ts:28-36`): client can POST `system` messages that flow into `convertToModelMessages` and override the server system prompt, including triggering `createBooking`. Restrict the enum to `["user", "assistant"]`.
- **H2 — CSRF forced-logout** (`api/portal/auth/signout/route.ts:2,15, portal/layout.tsx:39`): GET signout via `<a href>` is exploitable with `<img src=...>`. Switch to POST + form. Impact limited to DoS but CWE-352.

- **M1 — Commission FK cross-tenant** (`commissions.ts:28-31,74-76`): `bookingId` is validated but `supplierId/bookingItemId/agentUserId` are not, allowing foreign-tenant FK references and an existence oracle.
- **M2 — Unescaped HTML email** (`notifications/templates.ts`): `clientName/agencyName/roleLabel` interpolated raw; low risk in mail clients but the same names render on admin pages.
- **M3 — Extension-only upload validation** (`storage.ts:117`): no magic-byte check; SVG/script payloads accepted; risky on local-fs fallback.
- **L1** missing middleware matcher routes, **L2** no rate limit on magic-link request, **L3** intentional `non-httpOnly` locale cookie (validated).

2. Error Handling

This is the weakest category and the source of both blockers.

- **C1 — Fake PNR fallback** (`duffel.ts:278, bookings.ts:382`): the single most dangerous behavior in the app. A failed airline booking is recorded as `confirmed` with a provisional `DF-` reference and no agent-visible warning — agents could issue tickets against a non-existent booking.
- **C2 — Webhook DB writes unguarded** (`stripe/webhook/route.ts:75-86, connect-webhook/route.ts:40-46`): DB throw → 500 → Stripe retry storm with duplicate side-effects; `connect-webhook` returns 400 on misconfig (stops legitimate retries).
- **H3 — No try/catch in 10 action files; H4 — chat route + tools unguarded; H5 — no fetch timeouts** on Duffel/Hotelbeds/Stripe (hang to Vercel 30s); **H6 — missing `error.tsx`** in portal/platform/bookings/clients (root boundary sends portal clients to inaccessible `/`).
- Medium: **M4** silent commission auto-gen failure, **M5** Stripe customer-create failure unsurfaced, **M6** `degraded` flag not forwarded to AI, **M7** no structured logging.
- Low: **L4-L8** silent/empty catches and missing digest logging.

3. Performance

No catastrophic issues but several scale time-bombs as agencies grow.

- **H7** dashboard counts via full fetch + `.length`; **H8** `nextReference()` fetches all references per create; **H9** `recalcTotals()` N+1 UPDATES; **H10** no Suspense → blank screen on dashboard/finance (finance loads 1,000 bookings first, has no `loading.tsx`).
- Medium: **M8** serial ticketing API/DB loop, **M9** unbounded `getCommissions()`, **M10/M11** large in-memory KPI computation on dashboard (500 bookings) and finance (1,000 bookings) that should be SQL aggregates, **M12** missing composite indexes, **M13** unbounded export → OOM, **M14** unnecessarily-client AppShell, **M15** eager Recharts ~150KB, **M16** raw `<img>`.
- Low: **L9** sequential commission-by-booking queries, **L10** full i18n payload per client, **L11** no ISR on public proposals.

4. Code Quality

Clean of TODO/FIXME/HACK markers; main issues are dead code and duplication.

- Dead code: **L12** `session.ts` (entire file), **L13** `proxy.ts` (middleware never executes — wrong filename/export), **L14** `github-stars.tsx`.
- Duplication: **M17** `APP_URL` fallback in 9+ files, **L15** `secure` cookie option `×5`, **L16** `SHORT_MONTH` formatter, **L17** `session.user` cast `×2`, **L25** identical Stripe-configured sentinels.
- Architecture: **M18** two parallel supplier systems (`suppliers/` live vs empty `suppliers/providers/`).
- Types: **M19** 62/94 actions lack return-type annotations; **L18** 5 unguarded non-null assertions; **L19** 2 `console.log` ESLint violations.

5. UX & Accessibility

Strongest gaps are RTL (Arabic) breakage and screen-reader announcements.

- **H11/H12** — mobile drawer slides from the wrong side in RTL (`app-shell.tsx:223`); auth error `<p>s` lack `role="alert"`.
- RTL: **M24** ~156 physical-direction utilities should be logical (`me-/ms-/ps-/pe-/border-s/border-e`); **M25** `text-right` → `text-end` on numeric table columns.
- Forms: **M20** errors only via dismissible toast, **M21** `aria-invalid` never set, **M22** search inputs missing `<Label>`.
- A11y: **M23** icon-only buttons missing `aria-label/aria-haspopup/aria-busy` (`mode-toggle`, `create-agency-form`, `travellers-manager`, `payments-manager`).
- Loading: **L1.1** 6 list pages have no `loading.tsx` skeleton; low-severity **L20-L24** scroll hint, touch target, gallery alt, nav label.

6. Database

Schema is well-structured with consistent UUID PKs and tenant scoping; gaps are indexing and soft-delete.

- **H13** — no index on `verification.identifier` (full scan every login).
- Indexing: **M26** missing `createdAt` indexes on high-volume tables, **M27** missing `supplierId` indexes on `bookingItem/productItem`, **M12** missing `(agencyId, status)` composites.
- Tenant isolation: **M29** `supplierRate` missing denormalized `agencyId`; **M30** `recalcTotal` queries `bookingItem` without agency scope (footgun); **L30** child tables (`payment`, `bookingTraveller`) lack defense-in-depth `agencyId`.
- Data lifecycle: **M28** no soft-delete anywhere; **L26** `commission.bookingItemId` SET NULL orphans commissions.
- Hygiene: **L27** missing relations for supplier/commission tables, **L28** migration 0006 backfill edge case, **L29** `hotelContent` text PK (justified).

Files That Need Immediate Attention

1. `src/lib/suppliers/duffel.ts` — C1 (fake PNR), H5 (no fetch timeout), L5/L18 — the highest-risk file; fake-booking behavior is a business blocker.
2. `src/app/api/stripe/webhook/route.ts` + `connect-webhook/route.ts` — C2 (unguarded DB

writes → retry storm).

3. `src/lib/actions/bookings.ts` — C1 (confirmed status on provisional), H8 (`nextReference`), M8 (serial ticketing), M30 (`recalcTotal` scope).
4. `src/app/api/chat/route.ts` — H1 (prompt injection), H4 (no try/catch), M6 (degraded flag).
5. `src/components/app/app-shell.tsx` — H11 (RTL drawer), M14 (client component), M24 (logical props).
6. `src/app/(app)/dashboard/page.tsx` — H7 (count via fetch), H10 (no Suspense), M10/M15.
7. `src/app/(app)/finance/page.tsx` — H10 (no Suspense/loading), M11 (1,000-row JS KPIs).
8. `src/lib/schema.ts` — H13 (verification index), M12/M26/M27/M28/M29, L26/L27.
9. `src/lib/actions/commissions.ts` — M1 (cross-tenant FK), M4 (silent failure), M9 (no limit), L9.
10. `src/lib/actions/products.ts` — H8 (`nextReference`), H9 (`N+1 recalcTotals`).
11. **Auth form components** (sign-in/sign-up/forgot/reset/accept-invite) — H12 (`role="alert"`), M21.

Remaining Recommendations

- Adopt a structured logging / observability layer (Pino + Vercel log drain, or Sentry) so per-tenant error rates can be alerted on and user-visible `error.digest` values correlate with server logs (**M7, L8**).
- Add rate limiting (Upstash or token bucket) to the magic-link request endpoint and any other unauthenticated POST endpoints (**L2**).
- Introduce a soft-delete strategy (`deletedAt`) on core entities for audit/undelete before data volume makes retrofitting expensive (**M28**).
- Complete or formally retire the `suppliers/providers/` registry to remove the dual-abstraction ambiguity (**M18**).
- Centralize repeated config (`APP_URL`, `isProduction`, `formatShortMonth`) in `config.ts/format.ts` and delete confirmed dead code (`session.ts`, `proxy.ts` or `rename`, `github-stars.tsx`) (**M17, L12-L17**).
- Run a single RTL refactor pass converting physical to logical Tailwind utilities across components, and add `loading.tsx` skeletons to the six list pages lacking them (**M24, L1.1**).
- Annotate all server actions with explicit `Promise<ActionResult>` return types to catch accidental return-shape drift in CI (**M19**).
- After schema index/FK changes, run `drizzle-kit generate && migrate` (never push, per AGENTS.md).

Remediation Applied (2026-06-28)

Implemented via parallel coder sub-agents, each owning a disjoint file set. Verification: `npx tsc --noEmit clean · npm run lint` 0 errors (48 pre-existing import-order warnings only) · `npx next build` all routes compile.

Critical

- **C1 — Fake-PNR fallback** — `duffel.ts` no longer fabricates a `DF-...` reference; `bookFlight` throws on provider failure. `bookings.ts` `confirmItemBooking` returns a discriminated result; failed items are set to `pending` (existing enum value), never `confirmed`. `advanceStatus` aborts the ticketing advance and returns `{ ok:false, reasons }` if any item failed. Files: `src/lib/suppliers/duffel.ts`, `src/lib/actions/bookings.ts`.
- **C2 — Stripe webhook retry storm** — DB writes wrapped in `try/catch` in both `webhook/route.ts` and `connect-webhook/route.ts`; misconfiguration in `connect-webhook` now returns 503 (was 400) so Stripe retries; signature failures stay 400; transient DB errors return 500 with logging.

High

- **H1** chat role enum restricted to `["user", "assistant"]` (no client system).
- **H2** portal signout changed `GET→POST` + `<form method="POST">` (303 redirect).
- **H3** `try/catch` + `console.error` + friendly error returns added across 10 action files (clients, team, suppliers, invites, opportunities, portal-proposals, notifications, proposals-public, portal-invite, onboarding) and commissions.
- **H4** chat `streamText` + every tool `execute` wrapped; `createBooking` collects failed items/travellers instead of silently skipping.
- **H5** 15s `AbortController` fetch timeouts on Duffel, Hotelbeds, Stripe wrappers.
- **H6** segment `error.tsx` added for `portal/` (`→ /portal/login`), `(app) /` (`→ /dashboard`), `platform/` (`→ /platform`).
- **H7** dashboard counts now use `db.$count(...)` instead of `fetch + .length`.
- **H8** `nextReference()` uses a single `max(...)` aggregate.
- **H9** `recalcTotals()` `N+1` `UPDATES` replaced with `Promise.all`.
- **H10** Suspense around dashboard insights + new `finance/loading.tsx` skeleton.
- **H11** mobile drawer made RTL-aware via Tailwind `rtl: variants`.
- **H12** `role="alert"` added to all auth-form error messages.
- **H13** `verification_identifier_idx` added (in migration 0018, pending apply).

Medium / Low (included in tranche)

- **M1** commissions `recordCommission` now cross-tenant-validates `supplierId/bookingItemId/agentUserId` before insert.
- **M2** `escapeHtml()` applied to user-sourced values in email templates.
- **M6** `degraded/source` flag now part of `safeSearch` result type and forwarded to the AI chat tools (mock prices disclosable).
- **M12/M26/M27/L26/L27** composite + `createdAt` + `supplierId` indexes, commission cascade, and supplier/commission Drizzle relations — all in `drizzle/0018_regular_karnak.sql` (**pending apply** — see blocker above).
- **M30** `recalcTotal` now agency-scoped via `join on booking.agencyId`.
- **L12/L13/L14** deleted dead code: `src/lib/session.ts`, `src/proxy.ts` (dormant non-executing)

middleware; protections already enforced by `requireAgencyUser()`, `src/components/ui/github-stars.tsx`.

Deferred (explicitly out of this tranche — recommend follow-up)

- **M28** soft-delete (`deletedAt`) strategy across core entities.
- **M18** retire/complete the dual `suppliers/providers/` abstraction.
- **M24/M25** full RTL refactor (~156 physical→logical Tailwind utilities).
- **M7** structured logging/observability (Pino/Sentry).
- **L2** rate limiting on the magic-link request endpoint.
- Remaining Medium/Low UX, a11y, and perf items per the tables above.

Action required to fully close

1. Refresh `POSTGRES_URL` in `.env` (dev) with a valid Neon password, then run `npx drizzle-kit migrate` to apply `0018_regular_karnak.sql`.
2. Apply the same migration to **prod**: `POSTGRES_URL=<prod-url> npx drizzle-kit migrate`.

Production-Readiness Audit

Date: 2026-06-28 · **Auditor:** Lead Software Architect · **Scope:** security, error handling, performance, code quality, UX, database. No new features.

Note: the brief mentioned "optimize Prisma" — Atlas uses **Drizzle ORM**, not Prisma. Drizzle findings are reported instead.

Summary

Atlas has **strong security foundations** — tenant isolation, authorization, and input safety are largely correct. The gaps are in **operational hardening** (rate limiting, error handling, loading states) and **reversibility** (no soft delete). One genuine horizontal-privilege-escalation issue was found **and fixed** during this audit.

Area	Verdict
Security	● Solid; 1 escalation bug fixed, 2 hardening gaps
Error handling	● Inconsistent try/catch; few error boundaries
Performance	● Well-indexed; pagination missing
Code quality	● Clean; minor lint/dup
UX	● Loading/error/mobile gaps
Database	● Good FKs/indexes; no soft delete

1. Security

#	Issue	Severity	Proposed fix	Status
S1	Horizontal privilege escalation — detail pages (<code>bookings/clients/opportunities/products/[id]</code>) were tenant-scoped (<code>agencyId</code>) but not agent-scoped, so an agent could open a colleague's record by direct URL.	High	Add agent own-scope (<code>createdById/ownerId/assignedToId</code>) to each detail query, gated by <code>seesAllData(role)</code> .	✓ Fixed (4 files this audit; list pages fixed earlier in 134ceb3)

S2	No rate limiting on login, the public proposal token endpoints (<code>/p/[token]</code> , <code>acceptProposalByToken</code>), or API routes — brute-force / abuse vector.	Medium	Add an IP+route limiter (e.g. Upstash ratelimit) in middleware for auth + public token routes.	☐ Recommended
S3	Demo credentials committed in <code>docs/deployment.md</code> (incl. <code>platform-admin</code>).	Medium	Rotate the demo passwords; move them out of the repo or behind a private note.	☐ Recommended
S4	Supplier-detail booking-item count query not explicitly agency-scoped (logically safe via FK ownership).	Low	Add <code>agencyId</code> to the count <code>where</code> for defense-in-depth.	☐ Optional

Verified GOOD (no action):

- **Tenant isolation** — `requireAgencyUser()` on every `(app)` page (enforced in the layout) and every server action; `getSupplierById`, all `[id]` detail queries, and exports filter by `agencyId`. `scripts/test-tenant-isolation.ts` exists.
- **Authorization** — all 20 action files guard their exports (billing via a `requireBillingAdmin` wrapper; platform via `requirePlatformAdmin`; `platform.exitAgencyView` only clears cookies; `proposals-public` is token-authorized by design).
- **No SQL injection** — Drizzle parameterizes; no `sql.raw`/string-built queries.
- **No XSS sinks** — zero `dangerouslySetInnerHTML`; React auto-escapes.
- **Secrets** — `.env*` gitignored; no `.env` tracked; Stripe webhooks verify signatures.

2. Error Handling

#	Issue	Severity	Proposed fix
E1	Only 10/20 action files use <code>try/catch</code> ; uncaught throws surface a generic Next error.	Medium	Standardize on the <code>ActionResult</code> return shape with user-friendly messages; wrap external calls (Stripe/Duffel/Hotelbeds/Resend) in <code>try/catch</code> .
E2	Only <code>assistant/</code> has an <code>error.tsx</code> ; most routes have no error boundary.	Medium	Add <code>error.tsx</code> per route group (or one in <code>(app)/</code>) with a retry action.
E3	No structured logging; <code>logActivity</code> covers ~11/21 action files.	Low	Add a logger; extend <code>logActivity</code> to every mutation.

3. Performance

#	Issue	Severity	Proposed fix
---	-------	----------	--------------

P1	Hardcoded <code>limit</code> (200/500) on list queries — silent truncation and unbounded scan as data grows.	Medium	Real pagination (cursor/offset) — pairs with the planned <code>DataTable</code> .
P2	No obvious N+1 — queries use <code>with</code> joins and grouped counts.	—	None.
P3	Mostly Server Components (good); bundle not analyzed; no <code>next/dynamic</code> for heavy client islands (charts, search sheet).	Low	Run <code>@next/bundle-analyzer</code> ; lazy-load chart/search client components.

DB is well-indexed (**53 indexes**) and Drizzle is parameterized — no slow-query red flags in static review (runtime profiling recommended).

4. Code Quality

#	Issue	Severity	Proposed fix
Q1	Redundant auth: <code>suppliers/[id]</code> calls <code>requireAgencyUser()</code> then <code>getSupplierById</code> calls it again.	Low	Drop the page-level call (helper already guards).
Q2	47 ESLint <code>import/order</code> warnings.	Low	<code>pnpm lint --fix</code> (auto-fixable).
Q3	No dead code or duplicated logic found; <code>src/lib</code> (domain/queries/analytics) and domain-foldered components are clean.	—	None.

5. UX

#	Issue	Severity	Proposed fix
U1	Loading states missing on most routes (only <code>assistant/dashboard</code>).	Medium	Add <code>loading.tsx</code> per route using the existing <code>skeleton.tsx</code> .
U2	Error states missing (see E2).	Medium	Add <code>error.tsx</code> .
U3	Tables overflow on mobile (no <code>overflow-x-auto</code>).	Medium	Wrap tables; ship the responsive <code>DataTable</code> .
U4	Accessibility not deeply audited.	Low	Run the web-interface-guidelines pass (focus order, labels, contrast).
U5	Empty states are present and filter-aware.	—	None (good).

6. Database

#	Issue	Severity	Proposed fix
D1	No soft delete (<code>deletedAt</code>) anywhere; FK cascades perform hard deletes — conflicts with the "everything is reversible" principle.	Medium	Add nullable <code>deletedAt</code> to core tables + filter reads (needs migration).
D2	<code>reference</code> only on <code>booking/product</code> ; <code>updatedAt/status/notes</code> inconsistent on child tables.	Low	Backfill per the entity standard (needs migration).
D3	FKs (45 <code>onDelete</code> : 25 cascade / 20 set null), indexes (53), sequential migrations (latest <code>0017</code>), <code>generate-migrate</code> workflow.	—	Verified good.

7. Files modified (this audit)

- `src/app/(app)/bookings/[id]/page.tsx` — agent own-scope on detail query
- `src/app/(app)/clients/[id]/page.tsx` — agent own-scope on detail query
- `src/app/(app)/opportunities/[id]/page.tsx` — agent own-scope on detail query
- `src/app/(app)/products/[id]/page.tsx` — agent own-scope on detail query

(Earlier this session: list-page agent scoping `134ceb3`; quality-gate repair `7d5f4e1`.) Verified: `npm run check` (`lint + tsc`) passes clean.

Remaining recommendations (priority order)

1. **Rate limiting** (S2) — highest-value security add; small, middleware-level.
2. **Soft delete migration** (D1) — unblocks reversibility *and* GDPR erasure.
3. **Error boundaries + loading states** (E2/U1/U2) — mechanical, big UX win; `skeleton.tsx` already exists.
4. **Real pagination** (P1) — fold into the shared `DataTable` build.
5. **Rotate committed demo credentials** (S3).
6. **Standardize action error handling** (E1) on the `ActionResult` shape.
7. **Lint --fix + bundle analysis** (Q2/P3).

All items are tracked in the spec-vs-reality gap tracker.

Owner's-Eye UX Audit

Date: 2026-06-28 · **Lens:** "I own this travel agency and use Atlas all day." Walking every workflow looking for friction. **Recommendations only — nothing implemented.**

How to read severity: **P0** = trips me up daily / costs real time · **P1** = recurring annoyance · **P2** = polish.

Top findings (prioritized)

#	Issue	Sev	Why it hurts
1	Opportunities page is orphaned from the nav	P0	My core sales list isn't in the sidebar; I can only reach deals via a client or the "Operations" board. I lose my pipeline.
2	"Operations" is really the sales pipeline	P0	The label says fulfillment/ops; it's actually the deal board. I waste clicks hunting for where my pipeline lives — and it overlaps Opportunities + the Bookings board.
3	Can't create a booking without first creating a client	P0	A walk-in/phone booking forces me to stop, go create a client, come back. Two screens for one sale.
4	Nav order contradicts daily work	P1	Finance/Commissions/Reports sit <i>above</i> Bookings/Clients. My most-used screens are buried under money screens I open weekly.
5	Three money destinations (Finance, Commissions, Reports)	P1	I never know which to open for "how much did we make?" Overlapping, unlabeled difference.
6	Fragmented sourcing (Search vs Hotels vs Assistant vs inline)	P1	Four ways to look up inventory. I forget which finds what.
7	Sales→Proposal is manual	P1	Won an opportunity? I re-enter trip details into a new proposal. Duplicated work.
8	Inconsistent terminology (Proposal/Product, Opportunity, Operations)	P2	Same thing has different names across URL, page, and conversation.
9	No nav grouping (15 flat items)	P2	A wall of links; no Sales / Ops / Finance / Admin sections to scan.

Workflow walkthroughs

Finding new business → my pipeline (P0)

What I see: The sidebar has **Operations** (which opens a Pipeline kanban) but **no Opportunities link** — even though there's a full `/opportunities` page. So my deals live in two places with two names, and the "real" one isn't in the menu.

Why it hurts: My pipeline is the heartbeat of the agency. Making me guess between "Operations" and a hidden Opportunities page costs clicks every single day and makes the tool feel unfinished.

Better workflow: One nav item — **"Sales"** or **"Pipeline"** — that opens the board with a list/board toggle (the toggle already exists on Bookings). Retire the "Operations" name. One pipeline, one entry point.

Taking a booking (P0)

What I see: On *New booking*, if the client doesn't exist I get "No clients yet. Add a client first" — I must leave, create the client, and return.

Why it hurts: Half my bookings start as a phone call from someone not yet in the system. The client-first rule is right for *data*, but it shouldn't be a *detour*. That's two screens and a lost context switch for one sale.

Better workflow: Inline "+ New client" inside the booking form (a quick create that returns the new client selected), or a combined "New booking" that captures a minimal client on the same screen. Create the client record *as a side effect* of the sale, not a prerequisite step.

Building the quote (P1)

What I see: I win an opportunity, then open *New proposal* and re-type the destination, dates, and value I already captured on the opportunity. Convert proposal→booking is one click (great) — but opportunity→proposal isn't.

Why it hurts: Re-entering the same trip details is duplicated work and a place for transcription errors. It breaks the "never ask twice" principle right in the middle of my money-making flow.

Better workflow: A **"Create proposal" button on the opportunity** that pre-fills client, destination, dates, and value (the AI quote builder already exists — wire it to the opportunity). Won → proposal in one click, mirroring proposal → booking.

Seeing the money (P1)

What I see: Finance, Commissions, and Reports are three separate nav items, consecutive, with no stated difference.

Why it hurts: As the owner the #1 question is "how are we doing?" Three doors for that answer means I learn-and-forget which is which.

Better workflow: Group them under one **"Finance"** section with sub-tabs (Overview · Commissions · Export). One mental model, drill in for detail.

Sourcing flights/hotels (P1)

What I see: A **Search** page, a separate **Hotels** module, the **Assistant**, and an inline search sheet on the booking — four entry points.

Why it hurts: I can't form a habit. "Where do I price a hotel?" has four answers depending on context.

Better workflow: Make the **in-booking search the primary path** (I'm almost always sourcing *for a trip*), and frame standalone Search/Hotels as "explore without a booking." Fold the Assistant's search into the same sheet.

Terminology inconsistencies (P2)

Same concept, different words across URL / page title / everyday speech:

Concept	URL	UI label	Agency speak	Fix
Quote/ offer	<code>/products</code>	"Proposals"	"quote" / "proposal"	Rename route+code to <code>proposals</code> ; pick one word everywhere.
Sales pipeline	<code>/operations + / opportunities</code>	"Operations" + (none)	"pipeline" / "deals" / "enquiries"	One name ("Pipeline"), one page.
Deal	<code>/opportunities</code>	"Opportunity"	often "enquiry" / "lead"	Confirm the agency's word; use it consistently.

Mismatched names make training new staff slower and make the product feel stitched together.

Missing automation (P1 — productivity multipliers)

As an owner I want the system to do the obvious next step:

- New client → **welcome email** (not sent today)
- Opportunity won → **draft proposal** (manual today)
- Booking confirmed → **invoice** (on-demand PDF today)
- Travel completed → **review request** (not built)

Each is a manual chore I'll forget under load. These are the automation triggers — building them is the difference between a record-keeping tool and one that runs the agency.

What already feels good (keep it)

- **Getting-started checklist** — I knew what to do on day one.
 - **Booking lifecycle stepper** with "Advance to next" — I always know the state and the next action. This is exactly the "what do I do next?" principle done right.
 - **Convert proposal** → **booking** in one click — more of this, please.
 - **Client timeline** (activity + payments + notifications in one view) — this is how I think about a client.
 - **Empty states with a CTA** — never a dead end.
-

Recommended sequence

1. **Fix the pipeline confusion** (#1, #2) — rename/merge Operations + Opportunities into one "Pipeline" nav item. Cheap, high daily relief.
2. **Inline client create in booking** (#3) — removes the worst detour.
3. **Reorder + group the nav** (#4, #9) — daily work on top, grouped sections.
4. **Opportunity** → **proposal generation** (#7) — wire the existing AI quote builder.
5. **Consolidate Finance** (#5) and **terminology pass** (#8).
6. **Automation triggers** (welcome email, invoice, review) as capacity allows.

These are UX/workflow recommendations; the engineering gaps behind some of them (automation, pagination, DataTable) are tracked in the gap tracker.

CTO Review — Atlas

Date: 2026-06-28 · **Reviewer:** CTO (pre-first enterprise customer) **Verdict:** Atlas is a well-structured, ambitious product with real engineering quality — but it is not enterprise-ready today. Several issues would cause visible failures under real-world load or scrutiny. This report is honest, not diplomatic.

Executive Summary

Dimension	Grade	One-line verdict
Architecture	B+	Solid Next.js 16/Drizzle/Neon foundation; DB connection model wrong for serverless
Security	B	Tenant isolation real; no middleware, no CSP, no rate limiting
Database	B+	Good schema; soft delete missing; 11 tables missing <code>updatedAt</code>
Folder structure	A-	Clean and predictable; minor naming confusion (products = proposals)
UI	B	Design system consistent; no loading/error states; mobile overflow
UX	C+	Core flows work; 3 P0 navigation problems; no automation
Business logic	B	Core RBAC solid; reference generation has race condition
Performance	C+	Finance page does 4 sequential awaits; DB connection wrong driver
Scalability	C	No connection pooler; no caching layer; no queue; single DB point of failure
API design	B-	REST-style routes adequate; no versioning; chat endpoint has no body size limit
Developer experience	C+	No CI/CD; one test file; no seed script for fresh dev; two DB drivers in <code>package.json</code>
Maintainability	B	<code>bookings.ts</code> at 976 lines needs splitting; products/proposals naming confusing
Testing	D	One test file in the entire codebase. Not acceptable for enterprise.
Deployment	B-	Vercel is fine; no health endpoint; no rollback plan documented
CI/CD	F	Zero — no <code>.github/workflows/</code> . No automated checks before merge.
Documentation	A-	Unusually thorough for this stage; gap tracker is a standout
Code quality	B+	Consistent patterns; 45 raw <code>console</code> calls instead of a real logger

CRITICAL ISSUES

Must be fixed before the first enterprise customer goes live.

C1 — No database connection pooler

File: `src/lib/db.ts`

```
const client = postgres(connectionString); // wrong driver for serverless
```

Atlas uses `postgres` (the `postgres.js` driver) on Vercel serverless functions. Every function invocation opens a **new TCP connection** to Neon. At 50 concurrent users that's 50 open connections. At 200 it exhausts Neon's connection limit and every query starts failing with "too many clients."

Neon provides a **serverless pooler** (`-pooler` suffix in the connection string) that multiplexes thousands of connections through ~10 persistent ones — but you need to switch to the `@neondatabase/serverless` driver to use it correctly over HTTP (not TCP), which is mandatory for Vercel edge/serverless.

You also have **two DB drivers** (`postgres` + `pg`) in `package.json` with no clear owner — `pg` is for Drizzle migrations, `postgres` for runtime. This needs to be explicit and documented.

Impact: 200 concurrent users = database down. Enterprise customer demos in front of their IT team = connection pool exhaustion at the worst moment.

Fix:

```
npm install @neondatabase/serverless
```

```
// src/lib/db.ts
import { neon } from "@neondatabase/serverless";
import { drizzle } from "drizzle-orm/neon-http";
const sql = neon(process.env.POSTGRES_URL!);
export const db = drizzle(sql, { schema });
```

Keep `pg` only for `drizzle-kit` migrations (it needs a real TCP connection).

C2 — Zero CI/CD pipeline

Finding: No `.github/workflows/` directory. Nothing runs automatically on `git push`.

This means:

- A broken build can be pushed to `main` and deployed to production instantly

- No automated type checking, linting, or tests before deploy
- The quality gate (`npm run check`) only runs if someone remembers to run it
- No way to enforce code review before merge

Impact: The first enterprise customer will eventually hit a regression caused by an unreviewed push. At that point you have no safety net and no audit trail.

Fix: Create `.github/workflows/ci.yml`:

```
name: CI
on: [push, pull_request]
jobs:
  check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npm install --legacy-peer-deps
      - run: npm run check          # lint + typecheck
      - run: npm run build:ci       # next build
```

Vercel already deploys on push — CI just adds the gate before it reaches Vercel.

C3 — No test coverage

Finding: One test file exists: `.claude/worktrees/*/src/lib/references.test.ts` — not even in the main source tree. Zero tests for:

- Server actions (RBAC, tenant scoping, business logic)
- API routes (auth, input validation)
- Domain logic (`domain.ts` — roles, capabilities, booking lifecycle rules)
- Utility functions

Impact: Any refactor silently breaks business rules. Enterprise customers expect vendors to have a test suite. Without it, you cannot safely hire additional engineers (they will introduce bugs you won't catch).

Fix (prioritized, not everything at once):

1. Integration tests for the 5 most critical server actions: `createBooking`, `advanceBookingStatus`, `acceptProposalByToken`, `createCommissions`, `createAgency`
2. Unit tests for `domain.ts` (RBAC logic is pure functions — trivial to test)
3. Tenant isolation test already exists as a script (`test-tenant-isolation.ts`) — promote it to the CI pipeline

C4 — No middleware (rate limiting, security headers enforcement)

Finding: No `src/middleware.ts`. This means:

- Login endpoint (`/api/auth/sign-in`) has no rate limiting — brute-forceable
- Public proposal endpoint (`/p/[token]`) has no rate limiting
- Portal magic-link request has no rate limiting — email flooding
- Security headers (set in `next.config.ts`) work but a middleware could short-circuit abuse before it hits server functions

CSP (Content-Security-Policy) is also absent from `next.config.ts`. Every major browser security audit flags missing CSP — an enterprise customer's security team will catch this immediately.

Impact: Any motivated attacker can brute-force login, flood the magic-link email, or scrape tokenized public proposals. An enterprise security review fails.

Fix:

```
// src/middleware.ts
import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";
// Use Upstash ratelimit (Redis-backed, works on Vercel Edge)
export function middleware(req: NextRequest) {
  // 1. Rate limit /api/auth and /api/portal/auth/request
  // 2. Add Content-Security-Policy header
  return NextResponse.next();
}
export const config = { matcher: ["/api/auth/:path*", "/api/portal/:path*"] };
```

C5 — Race condition in booking/proposal reference generation

File: `src/lib/actions/bookings.ts:55-68`

```
// Fetches ALL booking references for the agency, iterates in JS to find max
const rows = await db.select({ reference: booking.reference })
  .from(booking).where(eq(booking.agencyId, agencyId));
let max = 1000;
for (const r of rows) {
  const n = Number.parseInt(r.reference.replace(/\D/g, ""), 10);
  if (Number.isFinite(n) && n > max) max = n;
}
return `BKG-${max + 1}`;
```

At 10,000 bookings per agency this loads the full reference list into memory on every creation. Worse: two simultaneous booking creations can read the same max and produce duplicate references (unique constraint catches it, but the UX is a silent retry loop, not an atomic increment).

Impact: Agencies with high booking volume (enterprise customers) hit slow booking creation and occasional silent retries. Could silently fail under load.

Fix: Atomic SQL max:

```
import { max as sqlMax } from "drizzle-orm";
const [row] = await db
  .select({ max: sqlMax(booking.numericReference) })
  .from(booking)
  .where(eq(booking.agencyId, agencyId));
const next = (row?.max ?? 1000) + 1;
return `BKG-${next}`;
```

Requires adding a `numericReference` integer column (same migration as soft delete).

HIGH PRIORITY

Fix within the first sprint after launch.

H1 — No soft delete

File: `src/lib/schema.ts`

No `deletedAt` column exists anywhere. The "everything is reversible" founding principle is aspirational only. An agency that accidentally deletes a booking has no recovery path. This also blocks GDPR erasure (you can't "erase" what you can't soft-delete and then hard-purge on schedule).

Fix: Migration adding `deletedAt: timestamp("deleted_at")` to core tables (booking, client, opportunity, product), filter all reads with `.where(isNull(entity.deletedAt))`, and a restore action per entity.

H2 — `requireAgencyUser` makes a DB query on EVERY page render

File: `src/lib/permissions.ts:117-132`

```
const ag = await db.query.agency.findFirst({
  where: eq(agency.id, user.agencyId),
  columns: { status: true, subscriptionStatus: true },
});
```

This query fires on every authenticated server render. At 1,000 concurrent users that's 1,000 extra agency-status queries per second. The agency status doesn't change often — it's a perfect cache candidate.

Fix: Cache the agency status in the session token (updated on suspension/ reactivation) or use `unstable_cache` with a short TTL and `revalidateTag` on suspension events.

H3 — Finance page does 4 sequential DB queries (633 lines)

File: `src/app/(app)/finance/page.tsx`

The page is enormous (633 lines) and makes 4+ DB queries, some sequential. It will be slow as the dataset grows and impossible to maintain.

Fix: Split into smaller async components using React Suspense, parallelizing queries with `Promise.all` or per-section server components. Cap the page at 150 lines of layout; extract `<PaymentsSummary>`, `<CommissionsSummary>`, etc.

H4 — No structured logging (45 raw `console.*` calls in actions)

Finding: 45 `console.log/error/warn` calls scattered across action files. On Vercel these show up as unstructured text in Function Logs with no correlation IDs, no severity levels, no timestamps, no request tracing.

Impact: When an enterprise customer files a support ticket saying "booking creation failed at 14:32 UTC," you cannot find the relevant log entry.

Fix: A one-file logger:

```
// src/lib/logger.ts
export const logger = {
  info: (msg: string, meta?: object) =>
    console.log(JSON.stringify({ level: "info", msg, ...meta, ts: new
Date().toISOString() })),
  error: (msg: string, meta?: object) =>
    console.error(JSON.stringify({ level: "error", msg, ...meta, ts: new
Date().toISOString() })),
};
```

Replace all `console.*` calls. Ship a correlation ID from the request context.

H5 — `bookings.ts` is 976 lines — unmaintainable

File: `src/lib/actions/bookings.ts`

Nearly 1,000 lines covering booking CRUD, lifecycle advancement, traveller management, item management, itinerary, and commission generation. A new engineer cannot reason about it. Every bug fix risks breaking something else in the file.

Fix: Split into:

- `bookings/crud.ts` — create, update, delete
- `bookings/lifecycle.ts` — advance status, prerequisites
- `bookings/travellers.ts` — passenger CRUD

- `bookings/items.ts` — trip service items
- `bookings/commissions.ts` — commission generation

H6 — No health check endpoint

Finding: No `/api/health` route. Vercel's uptime monitors, load balancers, and the enterprise customer's infrastructure team expect one.

Fix:

```
// src/app/api/health/route.ts
export async function GET() {
  return Response.json({ status: "ok", ts: new Date().toISOString() });
}
```

H7 — Products are called "proposals" everywhere but the URL and the DB

Finding: The concept is **Proposal** in all UI, docs, and conversation. But the database table is `product`, the URL is `/products`, the actions file is `products.ts`, the type is `ProductInput`. An enterprise customer's developer integrating via the API will be confused, and new engineers will be too.

Fix: Rename — DB table to `proposal`, route to `/proposals`, action file to `proposals.ts`. One migration, one find-replace. Do it now before the codebase gets larger.

H8 — No input validation on chat endpoint (body size / message length)

File: `src/app/api/chat/route.ts`

No body size limit, no message length cap, no token budget per request. A user (or attacker) can send a 100KB message that gets passed to the LLM, costing significant API spend.

Fix: Add `export const maxDuration = 30` (Vercel), validate message array length and individual message size before calling the LLM, and add the rate limiter from C4.

MEDIUM PRIORITY

Address in the first 60 days.

M1 — 11 tables missing `updatedAt`

11 of 24 tables have no `updatedAt` — impossible to answer "when was this record last changed?" for audit, support, or sync scenarios.

M2 — No caching layer whatsoever

Dashboard, finance, analytics — every render hits the DB fresh. `unstable_cache` with `revalidateTag` on mutations would cut DB load by ~60% for read-heavy pages.

M3 — Opportunities page not in navigation

The core sales pipeline (`/opportunities`) is not in the sidebar — agents can only reach it via the operations board or a client page. Discovered in the owner UX audit.

M4 — "Operations" label wrong

`/operations` is the pipeline kanban — "Operations" implies fulfillment/logistics. Enterprise buyers will be confused during demos.

M5 — No background jobs / queue

Sending emails, generating PDFs, syncing hotel content, processing webhooks — all done synchronously in request handlers. Slow operations block the user. Vercel serverless has a 10s default timeout.

M6 — Amadeus marked as "legacy" but still in the active provider selection

`getFlightSupplier()` still falls back to Amadeus. The codebase comment says "decommissioned 2026-07-17". Remove it before it becomes a dead code maintenance burden.

M7 — No pagination on any list page

All list queries use hardcoded `LIMIT 200` or `LIMIT 500`. An agency with 5,000 clients or 10,000 bookings silently gets a truncated list with no indication that data is missing. This is a data integrity perception problem for enterprise.

M8 — Portal session expiry not enforced on the server

The portal session expiry check (`gt(portalSession.expiresAt, new Date())`) exists but the TTL is not clearly documented and there's no cleanup job to remove expired sessions. Old sessions accumulate forever.

M9 — No GDPR compliance path

No data export per client, no erasure flow, no data retention policy, no consent tracking. EU travel agencies (your primary market) are legally required to provide these. Blocked by missing soft delete (H1).

M10 — Two PostgreSQL drivers in package.json

`postgres` (runtime) and `pg` (drizzle-kit migrations) coexist with no explanation. Confusing for new engineers and a potential version conflict risk.

NICE-TO-HAVE

Post-stabilization improvements.

N1 — No PR template or contribution guidelines

No `PULL_REQUEST_TEMPLATE.md`, no `CONTRIBUTING.md`. Enterprise customers who want to contribute integrations or customizations have no guidance.

N2 — `picsum.photos` in production image whitelist

`next.config.ts` whitelists `picsum.photos` (lorem ipsum placeholder images) in production. This is a dev-only domain — remove it from production config.

N3 — `OPENROUTER_MODEL` defaults to `openai/gpt-5-mini`

The env default in `env.ts` references `gpt-5-mini` — a model that may not exist or may be deprecated. This should be an explicit, versioned model ID.

N4 — No API versioning

All API routes are unversioned (`/api/chat`, `/api/export`). When you change the chat protocol or export format, existing integrations break with no migration path. Add `/api/v1/` prefixing now while there are no external consumers.

N5 — Finance page at 633 lines is both a performance and maintainability problem

Covered in H3 — also a nice-to-have style cleanup even after the Suspense split.

N6 — No Storybook or component documentation

UI components have no visual documentation. When a designer or new frontend engineer joins, they have no way to browse the component library.

N7 — `PROTECTED_DB_HOSTS` is a soft guard

The protection against running destructive scripts against prod is a hostname string comparison. It can be bypassed. Not critical today, but consider a proper DB IAM role with no DROP/TRUNCATE in production.

What is genuinely good

I want to be clear: most codebases I review at this stage are far messier. These are the things that

show real engineering judgment:

- **Tenant isolation** is correctly implemented and tested — `agencyId` on every table, enforced in every query, with a script that verifies no leaks.
- **ActionResult<T>** is a clean, consistent error-handling contract.
- **Zod validation** exists and is used, even if not universal.
- **The domain model** (`domain.ts`) is well-organized — roles, capabilities, enums, status metadata in one place.
- **The provider abstraction** (`src/lib/suppliers/`) is well-designed — the new `providers/` layer with capability-segmented interfaces is production-quality.
- **Security headers** in `next.config.ts` (X-Frame-Options, X-Content-Type-Options, Referrer-Policy) — most startups forget these.
- **The documentation** is the best I have seen at this stage. The gap tracker, the owner UX audit, the architecture decisions — this saves weeks of onboarding.
- **env.ts** validates all environment variables at startup with Zod — catches misconfigured deploys immediately.

Priority matrix

Priority	Issue	Effort	Risk if ignored
Critical	C1: No connection pooler	1 day	DB down at 200 users
Critical	C2: No CI/CD	1 day	Broken builds reach prod
Critical	C3: No tests	2 weeks	Regressions on every change
Critical	C4: No middleware/rate limiting	2 days	Brute force, abuse, CSP fail
Critical	C5: Reference race condition	2 hours	Duplicate refs under load
High	H1: No soft delete	3 days	Unrecoverable deletes, GDPR block
High	H2: Agency query on every render	4 hours	Slow at scale
High	H3: Finance page 633 lines	2 days	Slow, unmaintainable
High	H4: No structured logging	4 hours	Blind in production
High	H5: bookings.ts 976 lines	2 days	Bugs, slow PRs
High	H6: No health endpoint	30 min	Monitoring blind
High	H7: products = proposals naming	1 day	Developer confusion forever
High	H8: Chat no input limit	2 hours	LLM cost abuse
Medium	M1–M10	Various	Quality and compliance debt

Recommended 30-day plan before enterprise onboarding

Week 1 — Non-negotiable hardening

- C1: Switch to `@neondatabase/serverless` driver
- C2: Add GitHub Actions CI (lint + typecheck + build)
- C4: Create `middleware.ts` with rate limiting + CSP header
- C5: Fix reference generation with atomic SQL max
- H6: Add `/api/health` endpoint

Week 2 — Reliability

- H1: Soft delete migration for core tables
- H4: Replace `console.*` with structured logger
- H8: Chat endpoint body size + rate limit
- H2: Cache agency status

Week 3 — Code health

- H5: Split `bookings.ts` into 5 files
- H7: Rename products→proposals (DB + routes + components)
- M3/M4: Fix navigation (add Opportunities, rename Operations)
- C3: Start: integration tests for 5 critical actions

Week 4 — Enterprise readiness

- M9: Skeleton GDPR data export per client
- M5: Evaluate Vercel background tasks or QStash for async work
- H3: Split finance page with Suspense
- C3: Continue test coverage

This plan gets Atlas from "impressive prototype" to "defensible enterprise product" in 30 days. The bones are good. The hardening is what separates a product people trust with their revenue from one they pilot and abandon.

PART IV

Architecture Decision Records

9 documents

Decision 0001 — Navigation IA restructure

Date: 2026-06-28 **Status:** accepted **Deciders:** Product owner + CTO review

Context

Atlas had 15 flat navigation items in an order that contradicted the golden workflow (Finance appeared above Clients; Opportunities was entirely missing from the nav; Search and Hotels were two separate items for the same user task; Operations was a duplicate of the Bookings board view). An IA audit (docs/owner-ia-audit.md) identified these as P0/P1 usability issues.

Decision

Restructure navigation into 5 labelled sections following the golden workflow: **WORK** (Clients → Pipeline → Proposals → Bookings) · **SOURCING** (Flights · Hotels) · **FINANCE** · **TOOLS** · **ADMIN**. Rename `/products` → `/proposals` canonically (URL rewrite). Retire Operations as a nav item. Create `/sourcing/flights` as a dedicated page.

Reasoning

- Section-grouped nav reduces cognitive load from 15 flat items to 4-5 scannable groups
- Workflow order (Clients first, Finance last) matches how agents think
- Sourcing section scales cleanly to Cars, Cruises, Packages without nav restructure
- `/proposals` label matches what agents, clients and every competitor call the concept
- Operations was a duplicate of the Bookings board toggle — removing it eliminates confusion

Alternatives considered

Option	Why rejected
Keep flat nav, just reorder	Doesn't solve the 15-item cognitive load or the Search/Hotels duplication
Merge Search + Hotels into one page	Reasonable but defers the Sourcing section scaling problem
Rename Operations → Pipeline	The real pipeline is Opportunities; Operations was a board view, not a pipeline

Consequences

- Easier: adding new sourcing verticals (Cars, Cruises) — one line in NAV_SECTIONS
- Easier: onboarding new agents — nav reads like the job
- Harder: old `/products` and `/search` bookmarks need redirects (added to next.config.ts)
- Old URLs preserved via 301 redirects (products) and 307 redirects (search, hotels, operations)

Related

- docs/owner-ia-audit.md (full IA analysis)
- src/components/app/app-shell.tsx (implementation)
- next.config.ts (rewrites + redirects)
- src/app/(app)/sourcing/flights/page.tsx (new page)

Decision 0002 — Agency Network is a Stage 3 Direction

Date: 2026-06-28 **Status:** accepted (direction), deferred (implementation) **Deciders:** Founding team + board

Context

Board feedback on the strategic audit introduced the concept of Atlas as infrastructure connecting agencies to each other, not just software serving agencies individually. Agency A cannot fulfil a complex request; Atlas surfaces Agency B — a specialist — who can. Revenue sharing, inventory exchange, trusted subcontractor relationships, shared local suppliers. Network effects from participant relationships, not code.

This was not in the original strategic audit and represents a meaningfully stronger moat than data accumulation alone. Network effects in B2B do not decay; an agency earning revenue through Atlas referrals will not leave the platform.

Decision

Name this as the Stage 3 strategic direction. Begin designing for it in the data model now at minimal cost. Delay the product surface until Atlas has 300+ agencies (Stage 3 threshold).

Reasoning

Network effects require critical mass before generating value. At 50 agencies, the network is too thin for meaningful referral flow. Building the product surface before critical mass exists creates engineering overhead without corresponding user value and fragments attention from the supplier booking and automation work that must happen first to reach the scale where the network becomes possible.

Designing for it in the data model costs almost nothing. Agency records should carry specialisation attributes, destination expertise tags, and capacity signals from the beginning — these enable future matching without requiring a current product surface.

What this means now (Stage 1–2)

- Agency schema should include `specialisations`, `destinations_served`, and `agency_type` as structured fields (not free text) to enable future matching
- Activity and booking data should be structured to allow anonymised capability inference over time

- Do not build any product surface for inter-agency connection yet
- Do not market this capability yet — announcing a network that does not exist creates expectation debt

What this means at Stage 3 (300+ agencies)

- Surface: "Agency B specialises in this destination and has capacity"
- Revenue sharing mechanism for fulfilled referrals
- Trusted subcontractor relationships formalised in the platform
- Inventory and resource exchange between agencies

Alternatives considered

Option	Why deferred/rejected
Build the network now	Critical mass is required; building before scale creates overhead without value
Never build the network	This is the strongest long-term moat available; intentionally deferring is different from discarding
Build it as a marketplace immediately	Marketplace dynamics require both sides; agencies are not yet numerous enough to form a functioning marketplace

Consequences

- Stronger: when Atlas reaches 300 agencies, the network surface can be activated on data infrastructure already in place
- Harder: resisting the temptation to build this early when it is clearly a good idea
- Unchanged: no current engineering work required beyond schema attributes

Related

- docs/strategic-audit.md — the original moat analysis this extends
- docs/strategy-board-response.md — the board feedback that introduced this direction
- docs/domain.md — agency entity where specialisation attributes should be added

Decision 0003 — Sprint 1 Booking Architecture

Date: 2026-06-28 **Status:** accepted **Deciders:** engineering (Atlas Sprint 1)

Context

Atlas had functional search (Duffel flights, Hotelbeds hotels) but no complete booking workflow. Supplier references were stored as untyped JSONB. There was no price revalidation, no idempotency, no analytics event stream, and no clear environment separation between development, sandbox, and production. The `BookingProvider` interface existed in `providers/types.ts` but nothing used it.

The goal: build the full booking architecture in sandbox so that production credentials can be plugged in with configuration changes only — no code changes.

Decision

Build a layered booking architecture: **config** → **registry** → **provider adapters** → **booking services** → **server actions** → **UI**. Each layer is independent and testable. The UI never sees supplier types. Environment is resolved once in a config module.

Reasoning

Why consolidate to the new `BookingProvider` interface now

The legacy `SupplierProvider` (search-only, both verticals in one class) was a stepping stone. The new interface is capability-segmented and multi-step (search → quote → book → cancel). Building Sprint 1 on top of the new interface means provider adapters written now are the same ones production uses — no "sandwich migration" later.

Why four new database tables

`booking_supplier_ref` — supplier references must be queryable (status polling, cancellation, reporting). Storing them in JSONB is schema-less and unindexable.

`booking_event` — `activity_log` is too coarse for a booking funnel. A dedicated append-only event table serves as both the compliance audit trail and the analytics source without a third-party SDK dependency. One write, two consumers.

`booking_document` — vouchers and e-tickets are not line items. Separating them into their own table allows typed retrieval and future Vercel Blob integration.

`booking_idempotency` — without key tracking, a browser retry or a server-action re-execution after a timeout can create duplicate supplier orders. The key is derived deterministically so it is stable across retries for the same booking attempt.

Why a single config module (`suppliers/config.ts`)

Scattered `isDuffelConfigured()` / `process.env.*` checks across multiple files make it impossible to reason about which provider is active. A single config module resolves `EnvironmentTier` once and returns typed `FlightProviderConfig` / `HotelProviderConfig` objects. Providers receive config objects — they never read `process.env` directly.

Why in-process retry (no job queue)

Sprint 1 is sandbox-only. Async job queues (pg-boss, BullMQ) add operational complexity that is not justified until production traffic. The `BookingService` retry handler is synchronous and typed by error code (retryable errors get exponential backoff; non-retryable errors surface immediately to the UI).

Why post-booking stubs (not omitted)

Defining `cancel`, `refreshStatus`, `getVoucher`, `modifyPassengers` as typed interfaces with `NotImplementedError` now means Sprint 2 only fills in implementations — the UI is already wired against stable types. Omitting the interfaces forces Sprint 2 to design and implement simultaneously.

Alternatives considered

Option	Why rejected
Continue using <code>SupplierProvider</code> (legacy)	Would require a second migration when production arrives; two abstraction layers for the same concern
Store supplier refs in <code>booking_item.details</code> JSONB	Unqueryable, no index, no schema guarantee, impossible to cancel/poll
Third-party analytics SDK (PostHog, Segment)	External dependency with no offline fallback; <code>booking_event</code> table is queryable and sufficient for Sprint 1
Redis for idempotency keys	Redis is not in the stack; DB table with TTL expiry achieves the same guarantee without a new service
Async job queue for retry	Premature for sandbox; adds ops burden before production traffic justifies it

Consequences

Easier:

- Adding a new provider (TravelgateX, Expedia) requires one new adapter file and one `register()` call — no changes to services, actions, or UI.
- Switching sandbox → production is `ATLAS_ENV=production` + production tokens.
- Booking funnel analytics are queryable from day one via `booking_event`.
- Duplicate booking prevention is guaranteed at the DB level (idempotency PK).

Harder:

- The `booking_idempotency` table needs a periodic cleanup job (expired rows). Acceptable trade-off; a simple cron (`DELETE WHERE expires_at < NOW()`) suffices.
- Post-booking stubs surface `NotImplementedError` in the UI — must be surfaced gracefully ("Coming soon") rather than silently swallowed.

Related

- `src/lib/suppliers/config.ts` — environment config module (new, Sprint 1)
- `src/lib/suppliers/providers/types.ts` — `BookingProvider` interface (existing)
- `src/lib/suppliers/providers/registry.ts` — provider registry (existing, now wired)
- `src/lib/schema.ts` — schema additions (migration 0019)
- `docs/database.md` — updated table and migration entries
- `docs/api-integrations.md` — updated provider status

Decision 0004 — Travel Platform facade over direct supplier access

Date: 2026-06-29 **Status:** accepted **Deciders:** Engineering

Context

After Sprint 1 wired the booking lifecycle through the provider registry, the search / content / autocomplete path still called provider-specific functions directly from server actions and page components:

- `getFlightSupplier()`, `getHotelSupplier()`, `safeSearch()` hardcoded in `src/lib/actions/search.ts` (444 lines of Hotelbeds-specific orchestration)
- `isDuffelConfigured()`, `isHotelbedsConfigured()` scattered across 5 page files
- `searchDuffelPlaces`, `getHotelbedsContentBatch`, `searchHotelbedsHotelsByName`, etc. imported directly into the action layer

Adding a second search provider (RateHawk, Expedia, Booking.com) would require editing every consumer file. There was also no single home for the "availability search → content fallback → thumbnail enrichment" orchestration logic.

Decision

Introduce `src/lib/travel-platform/index.ts` as the single facade for all travel operations. Server actions and pages import from this module only; no provider-specific code leaks into the action or page layer.

Reasoning

- Keeps the registry pattern consistent: booking was already fully abstracted; search should be too.
- One module to update when a new provider is added. Zero consumer changes.
- The hotel search orchestration (availability → content fallback → enrichment) was duplicated and implicit in `search.ts`; moving it to the facade makes it explicit, testable, and shared.
- Preserves the exact same return shapes (`source`, `degraded`, `estimatedPricing`) so all UI components required zero changes.

Alternatives considered

Option	Why rejected
--------	--------------

Move Hotelbeds orchestration into <code>HotelbedsBookingProvider.searchHotels()</code>	Cleaner long-term but larger surface area; the availability→content→enrichment flow spans three Hotelbeds APIs with separate quota budgets — better owned by the facade, which can route each step to the best provider
Keep <code>safeSearch</code> and just add guards	Provider-specific code stays in the action layer; adding provider #3 still requires editing <code>search.ts</code>
Full rewrite of legacy <code>suppliers/index.ts</code>	Sprint constraint: preserve working code, only add the abstraction layer

Consequences

Easier:

- Adding a new provider (Expedia, RateHawk, Booking.com) requires only: implement the relevant capability interfaces, call `providerRegistry.register()` — zero consumer changes.
- Hotel search orchestration (content fallback, enrichment) is in one place and can be unit-tested.
- `isFlightProviderConfigured()` / `getActiveFlightProvider().label` replace provider-specific checks in pages.

Harder / trade-offs:

- The content fallback path still calls `listHotelOffersCached` (DB cache) and falls back to `ContentCapable.searchHotelsByName` with city-as-query, which is an approximation. A future `ContentCapable.searchHotelsByDestination` would be more precise.
- The legacy `src/lib/suppliers/index.ts` layer (`getFlightSupplier` / `safeSearch`) remains in the codebase for backward compatibility — two layers until it is removed in a future cleanup sprint.

Related

- `src/lib/travel-platform/index.ts` — the facade
- `src/lib/suppliers/providers/` — registry, types, adapters (Duffel, Hotelbeds, Mock)
- `docs/api-integrations.md` — updated with Travel Platform section
- `docs/decisions/0003-booking-architecture-sprint1.md` — Sprint 1 booking registry

Decision 0005 — Marketing-grade UI redesign of the live app

Date: 2026-06-29 **Status:** accepted **Deciders:** Owner (product), Engineering

Context

A premium sales deck and a set of high-fidelity UI mockups were produced under `marketing/` (deck `marketing/index.html`, 11 screens in `marketing/mockups/`, target design language in `marketing/DESIGN-RECOMMENDATIONS.md`). The mockups are idealized, fully-populated renders of how Atlas should look at its best.

The live app at `agencetool.vercel.app` does **not** match them. The gap is not in the shell — the live app already has the dark "Atlas · Travel Desk" sidebar, the same nav groups and branding — but in the **page content**: the live screens use the baseline shadow/neutral system with sparse real data and plain empty states, while the mockups show rich spacing, custom charts, populated tables, and a more premium visual treatment.

Showing prospects the deck and then the live app creates a credibility gap. The owner has chosen to close it by **bringing the live product up to the deck**, not by tempering the deck.

Decision

Undertake a **full, phased visual/UX redesign of the live application** to the marketing-grade standard set by the deck and `marketing/DESIGN-RECOMMENDATIONS.md`. The work ships incrementally to production, one phase at a time. The phased plan lives at `specs/ui-redesign/PLAN.md`.

Reasoning

- **Feature-filter rule** (`AGENTS.md`) is satisfied:
 - *Increases booking conversion* — a more polished proposal builder, portal, and quoting flow makes more proposals become bookings.
 - *Increases agency retention* — a product that looks and feels premium is one the agency is prouder to use daily and slower to leave.
- **Honest selling** — what the prospect sees in a demo should be what they buy.
- **Foundation already exists** — the deck, the 11 mockups, the canonical demo dataset (`marketing/demo-data.json`) and the design guide give a concrete, agreed target, so the redesign is execution against a spec, not exploration.
- **Phased, not a rewrite** — each phase is a visual/UX upgrade of existing screens that

preserves all current functionality, routes, and server actions. No business logic is rewritten to change the look.

Alternatives considered

Option	Why not chosen
Make the deck honest (rebuild it around real screenshots)	Fastest, but caps the product at today's polish and abandons the agreed target.
Seed rich demo data only	Closes the "looks empty" gap but not the visual-polish gap; folded into the redesign as Phase 0 instead.
Polish only a few hero screens	Higher leverage short-term, but the owner wants the whole app brought up; hero screens are simply Phase 1 here.
Big-bang redesign	Too risky for a live multi-tenant product; phased delivery keeps production stable.

Consequences

Easier / better

- The product matches the sales narrative; demos are honest.
- `DESIGN.md` becomes a richer, marketing-grade design system as Phase 0 lands, benefiting every future screen.
- New reusable primitives (KPI/stat cards, chart components, populated empty states, skeletons) are built once and reused everywhere.

Harder / trade-offs

- Sustained multi-phase effort touching most screens.
- `DESIGN.md` must be kept in lockstep as tokens/components evolve (the design mockups are the *target*, but `DESIGN.md` stays the in-code source of truth).
- The marketing mockups are standalone HTML, not Next.js/shadcn — they are a **visual target to match, not code to copy**.

Related

- `specs/ui-redesign/PLAN.md` — the phased plan
- `marketing/DESIGN-RECOMMENDATIONS.md` — target design language (Deliverable 4)
- `marketing/index.html` + `marketing/mockups/*.html` — the visual targets
- `marketing/demo-data.json` — canonical demo dataset for populating screens
- `DESIGN.md` — current in-code design system (evolves with Phase 0)
- `roadmap.md` — initiative + module status

Decision 0006 — Auto-create a booking when a client accepts a proposal

Date: 2026-07-01 **Status:** accepted **Deciders:** Engineering (Atlas)

Context

Accepting + e-signing a proposal is the client's commitment moment, yet the booking was created only when an agent later clicked "Convert to booking". That leaves a gap between the client committing and Atlas having a booking file to collect payment against — slower time-to-booking and a manual step that can be forgotten.

The acceptance happens in the **client context** — the authenticated client portal (`acceptProposalFromPortal`) and the unauthenticated public token link (`acceptProposalByToken`). Neither has an agent session, so the conversion cannot use `requireAgencyUser()`. Three hard constraints:

1. **Tenant safety** — the public path is a tenant boundary; a client action must never be trusted to supply an `agencyId`.
2. **Idempotency** — a proposal must never spawn two bookings across re-accepts, double-submits, and the manual convert button.
3. **Best-effort** — a booking error must never make the client's acceptance fail.

Decision

Extract a shared, auth-agnostic `createBookingFromAcceptedProposal` (`productId`, `{ actorUserId }`) helper in `src/lib/actions/bookings.ts` that both accept actions call (best-effort, in try/catch) right after marking the proposal accepted, and that the agent `convertProposalToBooking` delegates to. A new nullable `product.convertedBookingId` FK latches the created booking and enforces idempotency.

Reasoning

- The helper derives its tenant scope from the loaded `product.agencyId` — the only trusted source on the public path — and inserts the booking + items under that agency, satisfying tenant safety without `requireAgencyUser`.
- `convertedBookingId` is a single-column, additive guard: the helper returns the existing booking id as a no-op when it is set, and the linking `UPDATE` is guarded on the column still being `NULL`, so concurrent/duplicate calls converge on one booking.
- Wrapping the call in try/catch on the client path keeps acceptance resilient: the client always succeeds; the agent can convert manually if the auto-booking throws.

- Reusing the same helper for the agent path removes duplicated conversion logic and makes the manual convert button idempotent for free.

Alternatives considered

Option	Why rejected
Keep manual-only conversion	Leaves the time-to-booking gap the Feature Filter rule targets.
Reuse <code>createBooking + addBookingItem</code> (which call <code>requireAgencyUser</code>)	Impossible on the public path — no agent session; would break tenant derivation.
A separate <code>proposal_conversion</code> idempotency table	Heavier than needed; a single nullable FK on <code>product</code> is the natural latch and doubles as a queryable link.
Make acceptance fail if booking creation errors	Violates the best-effort rule — a client must never be blocked by an internal booking error.
New booking status <code>draft</code>	The client has committed, so <code>awaiting_payment</code> is the correct lifecycle entry.

Consequences

- Accepting a proposal (portal or public link) now yields an `awaiting_payment` booking automatically, tenant-scoped and idempotent.
- `booking.createdById` is the proposal's owner/creator when known, else null (system), since there is no agent actor on the client path.
- The agent convert button is now idempotent (safe to double-click).
- One more thing to keep in sync: the auto-booking carries item pricing at `unitPrice`; downstream commission/accounting still runs on booking confirm.

Related

- **Migration:** `drizzle/0020_clever_khan.sql` (adds `product.converted_booking_id`).
- **Code:** `src/lib/actions/bookings.ts` (`createBookingFromAcceptedProposal`, refactored `convertProposalToBooking`), `src/lib/actions/portal-proposals.ts`, `src/lib/actions/proposals-public.ts`, `src/lib/schema.ts`.
- **Docs:** `docs/business-rules.md`, `docs/domain.md`.

Decision 0007 — Commission ledger survives booking/item deletion

Date: 2026-07-02 **Status:** accepted **Deciders:** Engineering (Atlas)

Context

A codebase audit ([docs/full-audit-2026-07-01.md](#), punch-list #5) found that `commission.bookingId` and `commission.bookingItemId` were declared `onDelete: cascade`. Since Atlas has no soft delete anywhere yet (see [database.md](#)), a hard delete of a `booking` or `booking_item` — the "never hard delete" rule is currently aspirational for these tables — silently destroyed the matching `commission` rows along with it. Commission rows are earnings history (`supplier_to_agency` and `agency_to_agent`), frequently already `paid`; losing them on a booking delete is a financial-record-loss bug, not an acceptable side effect of tidying up a booking.

Decision

Change `commission.bookingId` and `commission.bookingItemId` foreign keys from `onDelete: cascade` to `onDelete: set null` ([migration drizzle/0021_peaceful_vertigo.sql](#)). The `commission` row itself is never deleted when its booking/item is; the link columns just go `NULL`. The tenant-scoping `commission.agencyId` FK is unaffected and still cascades with the agency.

Reasoning

- A commission row is a financial ledger entry, not a child record of the booking in the "belongs to and dies with" sense — it documents money that was actually earned/paid, independent of whether the originating booking record still exists.
- `set null` preserves the row (amount, status, supplier, agent, rate, basis) for reporting and audit while honestly reflecting that its originating booking/item is gone — a `NULL` link is queryable and doesn't fabricate a reference to a row that no longer exists.
- This is the smallest possible fix: a constraint-only migration, no new table, no application-code changes required to keep existing commission reads working (all report/finance queries already tolerate a nullable join).
- Proper soft-delete on `booking/booking_item` would remove the need for this entirely, but that's a larger, separately-tracked gap ([roadmap.md](#)); `set null` is the correct interim (and arguably permanent, regardless of soft delete) posture for a ledger table specifically.

Alternatives considered

Option	Why rejected
Leave <code>cascade</code> as-is	Confirmed data-loss bug; unacceptable for financial history.
<code>onDelete: restrict</code> (block the delete)	Would make it impossible to delete a booking that has ever generated a commission — too restrictive for legitimate cleanup/test data, and doesn't match how the rest of the schema handles this pattern (<code>product</code> → client/opportunity is already <code>set null</code>).
Snapshot commission fields onto a separate immutable ledger table	Heavier than needed; the existing <code>commission</code> row already carries the durable fields (amount, status, rate, basis) — only the <i>link</i> needs to survive, not a full denormalized copy.
Implement soft delete on <code>booking/booking_item</code> first, then this becomes moot	Correct long-term fix but a much bigger, separately-tracked change; <code>set null</code> closes the immediate financial-data-loss risk without waiting on it.

Consequences

- Deleting a booking or booking item no longer deletes any `commission` rows that reference it; those rows keep their amount/status/supplier/agent but their `bookingId/bookingItemId` become `NULL`.
- Commission reports/exports that join through `bookingId` must already handle (or now need to handle) a `NULL` link gracefully — e.g. showing "booking deleted" rather than crashing on a missing join.
- New indexes `commission_item_idx` (`bookingItemId`) and `commission_supplier_idx` (`supplierId`) were added in the same migration to keep those (now more frequently `NULL`-scanned) lookups fast.
- Closes punch-list item #5 from the 2026-07-01 audit.

Related

- Migration: `drizzle/0021_peaceful_vertigo.sql`.
- Docs: `database.md` (commission table row + migration table), `domain.md` (supplier aggregate section).
- Audit: `docs/full-audit-2026-07-01.md` punch-list #5.

Decision 0008 — One set of booking lifecycle guards, shared by both status-change entry points

Date: 2026-07-02 **Status:** accepted **Deciders:** Engineering (Atlas)

Context

A booking's status can be changed two ways: the lifecycle stepper's "Advance to [next]" button (`advanceStatus`, which only ever steps to the immediate next stage) and a status dropdown that can jump directly to any status (`setBookingStatus`). The 2026-07-01 audit (punch-list #2) found that `setBookingStatus` enforced **none** of `advanceStatus`'s prerequisites: a zero-item, fully-unpaid booking could be set straight to `ticketed` or `completed` from the dropdown, bypassing the items-required and zero-balance-required rules, and — because the commission auto-generation call was also only wired into `advanceStatus` — silently skipping commission generation for bookings that reached `confirmed/ticketed` via the dropdown. Any future third entry point (a bulk-status API, an admin tool) would have faced the same choice: duplicate the rules again, or drift.

Separately, ticketing a proposal-converted booking crashed (punch-list #1): auto-created booking items from `createBookingFromAcceptedProposal` have no `details` (no supplier offer was ever selected for them), and `serviceBookFlight/serviceBookHotel` dereferenced `offer.rawOfferId/offer.rateKey` directly, throwing on a null `offer`. Both entry points call the same ticketing path, so this needed a single fix, not two.

Decision

Extract the lifecycle prerequisites into two shared helpers in `src/lib/actions/bookings.ts` — `checkStatusPrerequisites(existing, target)` and `runTicketingConfirmation(existing, agencyId, userId)` — and have **both** `setBookingStatus` and `advanceStatus` call them, in the same order, with the same commission-generation trigger. `setBookingStatus` becomes exactly as strict as `advanceStatus`, just able to target any status instead of only the next one. `serviceBookFlight/serviceBookHotel` in `booking-service.ts` gain an explicit null-offer guard that returns a provisional reference (reason: "No supplier offer attached to this item") instead of throwing.

Reasoning

- A single shared helper pair is the only way to guarantee the dropdown and the stepper can never drift apart on what's "legal" — there is exactly one place that defines "can this booking become `confirmed/ticketed/completed` right now", and every current and future caller goes

through it.

- `checkStatusPrerequisites` keys off the **target** status, not the transition path, so it naturally covers `setBookingStatus`'s any-status jumps: reaching `confirmed` requires items + zero balance regardless of which status the booking is jumping *from*.
- `runTicketingConfirmation` already had the right shape (loop items, book via the provider registry, abort on any failure) — lifting it out and calling it from both places means the "never ticket off a fabricated confirmation" rule is enforced identically everywhere, not just on the stepper's path.
- The UI-level fix (narrowing the status dropdown's offered options to legal transitions, and dropping the stale `paid` option) is a UX improvement layered on top of the real fix — the server-side guard is the actual invariant; the dropdown narrowing just avoids agents hitting an error message for an obviously-illegal jump.
- The null-offer guard in `booking-service.ts` is the correct place for the fix (not a special-case in `bookings.ts`) because "no offer attached" is a property of the booking item, and any caller of `serviceBookFlight/ serviceBookHotel` — not just the ticketing flow — needs the same protection.

Alternatives considered

Option	Why rejected
Duplicate the guard logic into <code>setBookingStatus</code> inline	Exactly the drift risk this decision exists to prevent — two copies of "what makes a status legal" will diverge the next time either is touched.
Only fix the ticketing crash, leave <code>setBookingStatus</code> unguarded	Leaves the more severe bug (punch-list #2, a High-severity guard bypass) in place; the crash and the missing-guards bug share a root cause (the two entry points never shared code) and are naturally fixed together.
Remove <code>setBookingStatus</code> / the dropdown entirely, keep only <code>advanceStatus</code>	Removes a genuinely useful capability (correcting a mis-set status, reopening a cancelled booking) for agents/admins; the fix is to guard it properly, not remove it.
Throw on a null offer instead of falling back to a provisional reference	Breaks the existing "never crash, always return a reviewable provisional result" contract <code>confirmItemBooking/ItemBookingResult</code> already establish for every other failure mode — a missing offer should be handled the same way as a provider outage.

Consequences

- `setBookingStatus` and `advanceStatus` can never again enforce different rules for the same target status — a change to the prerequisites is a one-function edit.
- Ticketing a proposal-converted booking (no supplier offer on its items) now completes with provisional references instead of throwing; the agent sees the item marked `pending` with a clear reason instead of a crashed request.

- The status dropdown only offers transitions that can actually succeed server-side (current + backward + one step forward + cancelled), reducing wasted round-trips to a rejected `setBookingStatus` call.
- Closes punch-list items #1 and #2 from the 2026-07-01 audit.

Related

- **Code:** `src/lib/actions/bookings.ts` (`checkStatusPrerequisites`, `runTicketingConfirmation`, `setBookingStatus`, `advanceStatus`), `src/components/bookings/booking-status-control.tsx` (`allowedTransitions`), `src/lib/suppliers/booking-service.ts` (`serviceBookFlight/serviceBookHotel` null-offer guard).
- **Docs:** `business-rules.md`.
- **Audit:** `docs/full-audit-2026-07-01.md` punch-list #1 and #2.

Decision 0009 — Deposit-threshold booking confirmation

Date: 2026-07-02 **Status:** accepted **Deciders:** Product owner + Engineering (Atlas)

Context

The public proposal signing page promised clients "a 50% deposit (DZD ...) secures your dates", while the booking lifecycle required a **zero outstanding balance** to reach `confirmed`. A client who did exactly what the proposal asked — paid the deposit — left the booking stuck in `awaiting_payment`, blocking everything downstream (supplier booking, ticketing, commissions). The sales promise and the operational rule contradicted each other at the workflow's money moment. The delayed supplier booking also amplified rate-expiry risk: bedbank/GDS rates drift between acceptance and full payment, which is why the provider layer re-quotes before booking.

Decision

Make confirmation deposit-based and agency-configurable:

- New column `agency.depositPercent` (`numeric(5,2)`, NOT NULL, default 50) — migration `drizzle/0023_steep_wild_pack.sql`; editable 0–100 in Settings → Agency (admin/manager).
- `confirmed` now requires `paid ≥ depositPercent % of the booking total` (pure helper `src/lib/payments/deposit.ts`, cent-rounded so float noise can't block a threshold-exact payment).
- The **zero-balance gate moves to `ticketed`** (and remains on `completed`): full payment is due before supplier orders are issued, not before confirmation.
- The deposit sentence on client-facing proposals (public token page, portal signing view, builder preview) is sourced from the **same column**, so the promise and the gate can never diverge again. A 0% deposit hides the deposit clause ("full payment due at booking").

Reasoning

- Real agencies confirm on deposit and settle before ticketing; requiring full payment to confirm blocked the workflow at the client's moment of highest commitment (fails the "reduces time to booking" feature filter in reverse).
- 100% collapses exactly to the previous behavior, so conservative agencies lose nothing; 0% supports invoice-later corporate relationships.
- One source of truth for the percentage (principle #3, no duplicated data): proposal copy, lifecycle guard, and settings all read `agency.depositPercent`.

Consequences

- `autoGenerateCommissions` still fires on entering `confirmed/ticketed` (unchanged) — commissions can now be generated with a balance outstanding, which matches the earned-not-yet-settled semantics of the ledger's `pending` status.
- The booking-detail UI (advance button + status dropdown) shows a deposit soft-warning for `confirmed` and balance warnings only for `ticketed+` (server-side guards remain the source of truth).
- Unit coverage: `src/lib/payments/deposit.test.ts` (18 cases — thresholds, clamping, rounding, float noise).

Amendment (2026-07-02) — per-deal override chain

The deposit percent is no longer agency-global at read time. It resolves an override chain — `booking.depositPercent` → `agency.depositPercent` → 50 — with nullable `product.depositPercent` / `booking.depositPercent` columns added in migration 0024. The proposal builder exposes "Deposit required (%)" per deal (empty = inherit the agency default; an explicit 0 = no deposit, confirm immediately; 100 = full payment at acceptance). At proposal → booking conversion the **effective** percent is snapshotted onto the booking, freezing the signed terms against later changes to the agency default or the proposal. Every consumer (confirmed gate, proposal copy on all four surfaces, portal deposit-vs-full payment options, booking-detail soft warnings) reads the same `effectiveDepositPercent()` helper (`src/lib/payments/deposit.ts`, unit-tested).

PART V

Specifications

16 documents

Phase 1 — "Sellable" Implementation Plan

Goal: take Atlas from demo to sellable. Seven features, dependency-ordered.

Decisions locked:

- **Billing:** vendor bills agencies (SaaS subscriptions) **now**; agencies bill travelers (Stripe Connect) **later**.
- **Email:** Resend · **Flights:** Amadeus · **Hotels:** Hotelbeds.

Key findings (much is already partially built)

- **Email** — a SendGrid adapter already exists at `src/lib/notifications/email.ts` with the right `sendEmail()/EmailResult` contract. Resend = one-file swap. Gaps: invites never actually send (toast is cosmetic, delivery is copy-paste); both Better Auth hooks (`auth.ts:85-97`) only `console.log`.
- **Stripe** — a fetch-based Checkout integration already exists for *traveler* booking payments (`src/lib/payments/stripe.ts`); `STRIPE_SECRET_KEY` is in `.env`. SaaS subscriptions are a **separate billing plane** on the `agency` table.
- **Travel APIs** — `SupplierProvider` abstraction exists with a working **Amadeus implementation for flights AND hotels** (`src/lib/suppliers/`). Flights ~done.
- **Bug** — `products/[id]/page.tsx:61` links to `/products/${id}/proposal`, a route that doesn't exist (real one is `/proposal/[id]`). Fix during PDF/e-sign work.

Sequence

1. DB split — foundation; before billing & real customers
2. Email (Resend) — unblocks invites, billing receipts, e-sign, proposals
3. Stripe SaaS billing — needs email + agency gating
4. PDF proposals — pair
5. E-signature — e-sign needs public token + email
6. Hotelbeds + flights — parallel track, independent of 1-5

1. Separate databases — S (operational)

- Route `src/lib/db.ts:5-12` through `getServerEnv()` (currently bypasses zod).
- Add prod guard to destructive scripts (`seed-demo-data.ts`, `db:reset`, `test-tenant-isolation.ts`) — refuse unless `ALLOW_PROD=1` / `host ≠ prod`.
- User action: create a dev Neon branch; prod `POSTGRES_URL` only in Vercel env.
- Vercel `build:ci` skips migrations — confirm out-of-band migrate process.

2. Email delivery (Resend) — S

- Rewrite `src/lib/notifications/email.ts` to call Resend, keep the signature (zero caller changes); add optional `html?` for React Email later.
- Add `RESEND_API_KEY + EMAIL_FROM` to `src/lib/env.ts` (schema + `checkEnv`).
- Wire invites: capture `createInvite()` at `actions/invites.ts:67`, build link from `NEXT_PUBLIC_APP_URL + /invite/${token}`, send, persist to `notification`.
- Fill the two auth hooks (`auth.ts:85-97`): password reset + verification.
- Extend `notification.kind` (`invite`, `password_reset`, `proposal`) as an outbox.
- Watch: auth emails may have no `agencyId` (NOT NULL on `notification`).

3. Stripe SaaS billing (vendor → agencies) — M

- Migration 0009: add to `agency` — `stripeCustomerId`, `stripeSubscriptionId`, `subscriptionStatus`, `priceId`, `currentPeriodEnd`, `trialEndsAt`.
- Gate in `requireAgencyUser` (`permissions.ts:136-144`) — already checks `agency.status`; add subscription-state block (`past_due/canceled`).
- `createAgency` (`actions/platform.ts:74`) creates Stripe customer + trial sub.
- New route `src/app/api/stripe/webhook/route.ts` (POST, raw `req.text()` body).
- Add the official `stripe` npm SDK (subscriptions + signature verification).
- Env: `STRIPE_WEBHOOK_SECRET` + price IDs. Surface status on platform detail page.
- Stripe Connect (traveler payments) = deferred to Phase 1.5, separate module on the `payment/booking` plane.

4. PDF proposals — M

- Today = HTML + `window.print()`. Add real renderer: Puppeteer + `@sparticuz/chromium` on Vercel serverless, store via `@vercel/blob` (`src/lib/storage.ts`). Alt: `react-pdf`. Fix the broken proposal link.

5. E-signature acceptance — M (greenfield)

- Add to `product`: `shareToken` (unique), `acceptedAt`, `declinedAt`, `signerName`, `signerEmail`, `signatureData`, `signerIp`, `signerUserAgent`.
- Public token route mirroring `/i/[token]:generateProposalLink` (copy `generateShareLink` `bookings.ts:383`), new unauth `/p/[token]`.
- New unauth `acceptProposalByToken` (first token-authenticated write) → set signature, `product.status=accepted`, `opportunity.stage=won`, send confirmation email.

6. Hotelbeds (hotels) + Amadeus (flights) — M

- New `src/lib/suppliers/hotelbeds.ts` mirroring `amadeus.ts` (Api-key + SHA-256).
- Split `getSupplier()` (`suppliers/index.ts:20`) into `getFlightSupplier()` / `getHotelSupplier()`; update `safeSearch` + call sites (`actions/search.ts`, `api/chat/route.ts`).
- Add `rateKey` to `HotelOffer`; persist supplier offers into `bookingItem.details` (AI `createBooking` currently drops them; `confirmItemBooking` expects them).
- Add `HOTELBEDS_API_KEY/HOTELBEDS_SECRET` to `env.ts`.

Effort

#	Feature	Effort	Status
1	DB split (code parts)	S	✓ done — Neon branch is a manual step
2	Email (Resend)	S	✓ done
3	Stripe SaaS billing	M	✓ done (subscriptions); Connect deferred
4	PDF proposals	M	✓ done (@react-pdf, server-rendered)
5	E-signature	M	✓ done (public token + sign + audit)
6	Hotelbeds + flights	M	✓ done (per-vertical providers + rateKey)

Manual setup required (you)

- **Neon dev branch:** create a separate Neon branch/database for local dev; put its URL in local `.env`, keep prod `POSTGRES_URL` only in Vercel env. Set `PROTECTED_DB_HOSTS=<prod-neon-host>` in prod env so destructive scripts refuse to touch it (override with `ALLOW_PROD=1`).
- **Resend:** add `RESEND_API_KEY` + `EMAIL_FROM` (verified domain) to `.env` and Vercel. Without them, emails log to console (status logged).
- **Stripe billing:** add `STRIPE_SECRET_KEY`, `STRIPE_PRICE_ID` (the SaaS plan price), and `STRIPE_WEBHOOK_SECRET` to `.env` and Vercel. Register the webhook endpoint `POST /api/stripe/webhook` in the Stripe dashboard, subscribing to `customer.subscription.{created,updated,deleted}`. Enable the Billing Portal.
- **Suppliers:** add `AMADEUS_CLIENT_ID/AMADEUS_CLIENT_SECRET` (flights) and `HOTELBEDS_API_KEY/HOTELBEDS_SECRET` (hotels); optional `*_HOSTNAME=production`. Without them, search falls back to sample data. Hotelbeds keys availability by its own destination codes — pass a resolved `cityCode` for live hotel search (city name works against the mock).

Known follow-ups (not blocking Phase 1)

- AI `createBooking` tool (`api/chat/route.ts`) still doesn't persist the supplier offer into `bookingItem.details`, so AI-built items can't be auto-booked by `confirmItemBooking`. The

human search → "Add to booking" flow does persist it.

- Hotelbeds destination-code resolution (content API) for city-name → code.

UI Polish & Responsive Improvements

Overview

Improve the visual quality, component consistency, and responsive behavior of the boilerplate UI across all pages. This includes refreshing the color palette with a subtle blue accent, replacing bare HTML elements with shadcn components, adding hover/transition effects via a reusable utility class, and introducing `sm:` breakpoints throughout for better tablet/mobile scaling. All custom styling goes in `globals.css` — no inline or custom CSS on components.

Quick Links

- Requirements — full requirements and acceptance criteria
- Action Required — manual steps needing human action

Dependency Graph

```
graph TD
  task-01-globals-css["01: globals.css Foundation"]
  task-02-layout["02: Layout Sticky Footer"]
  task-03-site-header["03: Site Header Responsive"]
  task-04-site-footer["04: Site Footer Responsive"]
  task-05-home-page["05: Home Page Overhaul"]
  task-06-dashboard["06: Dashboard Page"]
  task-07-chat-page["07: Chat Page"]
  task-08-profile-page["08: Profile Page"]
  task-09-auth-pages["09: Auth Pages"]
  task-10-setup-checklist["10: Setup Checklist"]
  task-11-starter-prompt-modal["11: Starter Prompt Modal"]
  task-01-globals-css --> task-05-home-page
  task-01-globals-css --> task-06-dashboard
  task-01-globals-css --> task-07-chat-page
  task-01-globals-css --> task-08-profile-page
  task-01-globals-css --> task-09-auth-pages
  task-01-globals-css --> task-10-setup-checklist
  task-01-globals-css --> task-11-starter-prompt-modal
  task-02-layout --> task-09-auth-pages
```

Waves

Wave	Tasks	Description
------	-------	-------------

1	task-01, task-02, task-03, task-04	Foundation: globals.css enhancements, layout flex structure, header/footer responsive
2	task-05, task-06, task-07, task-08, task-09, task-10, task-11	All pages: Card adoption, responsive breakpoints, component swaps

Task Status

Wave 1

- ☒ task-01-globals-css — globals.css color palette, animations, and utility classes
- ☒ task-02-layout — Root layout sticky footer with flex column
- ☒ task-03-site-header — Header responsive padding and sizing
- ☒ task-04-site-footer — Footer responsive padding

Wave 2

- ☐ task-05-home-page — Home page Card adoption + responsive typography
- ☐ task-06-dashboard — Dashboard Card adoption + responsive grid
- ☐ task-07-chat-page — Chat Input swap + sticky form + responsive
- ☐ task-08-profile-page — Profile responsive stacking + grid fixes
- ☐ task-09-auth-pages — Auth pages animation, shadow, and background
- ☐ task-10-setup-checklist — Setup checklist Card adoption
- ☐ task-11-starter-prompt-modal — Textarea swap + responsive buttons

Action Required: UI Polish & Responsive Improvements

No manual steps required for this feature. All tasks can be implemented automatically.

Requirements: UI Polish & Responsive Improvements

Summary

The boilerplate UI is functional but visually flat — bare `<div>` elements instead of shadcn Cards, a monotone neutral-gray palette, no hover/transition effects, and minimal responsive breakpoints (most pages only use `md:`). The goal is to meaningfully improve visual quality, component consistency, and responsive behavior across all pages and components.

All changes must use Tailwind utilities and shadcn standards. Custom styling (animations, utilities, color tokens) goes in `globals.css`. No custom inline styles or CSS-in-JS on individual components. The result should look like a polished, high-quality default that can be customized by users of the boilerplate.

Goals

- Refresh the color palette with a subtle cool blue accent (shift primary/ring/accent from flat gray to hue 270)
- Replace all bare `<div>` cards with shadcn Card components for consistency
- Replace native `<input>`, `<button>`, and `<textarea>` elements with their shadcn equivalents
- Add hover/transition effects to interactive cards via a reusable `card-interactive` utility
- Introduce `sm:` breakpoints throughout for proper tablet/mobile scaling
- Add entrance animations to auth pages
- Fix the sticky footer layout so it works on short-content pages
- Ensure all pages scale well from 320px to 1536px+ viewports

Non-Goals

- No hamburger/mobile menu for the header — the current content (avatar + theme toggle) is minimal enough
- No new shadcn component installations — all needed components are already installed
- No changes to business logic, API routes, or authentication flows
- No new pages or routes
- No changes to the Geist font choice
- No Storybook or component documentation

Acceptance Criteria

- [] All pages render correctly in both light and dark modes
- [] No horizontal scroll on any page at 320px viewport width
- [] Feature cards, dashboard cards, and next steps cards use shadcn Card components
- [] Chat input uses shadcn Input, not a native `<input>`
- [] Footer sticks to the bottom on short-content pages (auth pages)
- [] Primary color has a visible cool-blue tint (not flat gray)
- [] Cards have hover lift effect with shadow transition
- [] All grids have `sm:` breakpoints for tablet-size viewports
- [] `pnpm lint` and `pnpm typecheck` pass with no errors

Assumptions

- Tailwind CSS v4 is in use (confirmed: `@import "tailwindcss" syntax, @theme inline block`)
- `tw-animate-css` is installed in dependencies but currently unused (confirmed in `package.json`)
- The ThemeProvider from next-themes renders children directly without a wrapping `<div>` (confirmed by reading the component)
- All shadcn components needed (Card, Input, Button, Textarea, Badge, Avatar, Dialog) are already installed

Technical Constraints

- All custom CSS (animations, utilities, color tokens) must go in `src/app/globals.css`
- Component files should only use Tailwind utility classes and shadcn components — no `style={{}} props`
- Must maintain both light and dark mode support via the existing OKLch CSS variable system
- Must not break existing functionality (auth flows, chat, diagnostics)

Task 01: globals.css Foundation

Status

complete

Wave

1

Description

Update `globals.css` to establish the visual foundation for the entire UI improvement. This includes importing the unused `tw-animate-css` package, refreshing the color palette with a subtle blue accent, adding custom keyframe animations, expanding the base layer with better defaults, and defining reusable utility classes. All subsequent tasks depend on the utilities and color tokens defined here.

Dependencies

Depends on: None (Wave 1) **Blocks:** task-05-home-page.md, task-06-dashboard.md, task-07-chat-page.md, task-08-profile-page.md, task-09-auth-pages.md, task-10-setup-checklist.md, task-11-starter-prompt-modal.md

Context from dependencies: N/A — this is the foundation task.

Files to Modify

- `src/app/globals.css` — Add `tw-animate-css` import, update color tokens, add animations, expand base layer, add utility classes

Technical Details

Implementation Steps

1. **Add `tw-animate-css` import** — The package `tw-animate-css@1.4.0` is already installed (in `package.json`) but never imported. Add the import right after the `tailwindcss` import:

```
@import "tailwindcss";
@import "tw-animate-css";
```

1. **Add animation tokens to `@theme inline`** — Add these at the end of the existing `@theme inline` block (after the `--radius-xl` line):

```
--animate-fade-in: fade-in 0.3s ease-out;
--animate-fade-up: fade-up 0.4s ease-out;
--animate-scale-in: scale-in 0.2s ease-out;
```

1. **Update light mode color tokens in `:root`** — Change these 3 values (leave all other tokens unchanged):

```
/* CHANGE these 3 lines: */
--primary: oklch(0.21 0.034 270);          /* was oklch(0.21 0.006 285.885) */
--ring: oklch(0.705 0.06 270);             /* was oklch(0.705 0.015 286.067) */
--accent: oklch(0.96 0.012 270);           /* was oklch(0.967 0.001 286.375) */
```

1. **Update dark mode color tokens in `.dark`** — Change these 3 values (leave all other tokens unchanged):

```
/* CHANGE these 3 lines: */
--primary: oklch(0.92 0.02 270);          /* was oklch(0.92 0.004 286.32) */
--ring: oklch(0.552 0.05 270);            /* was oklch(0.552 0.016 285.938) */
--accent: oklch(0.28 0.018 270);          /* was oklch(0.274 0.006 286.033) */
```

1. **Add keyframe definitions** — Add these right before the `@layer base` block:

```
@keyframes fade-in {
  from { opacity: 0; }
  to { opacity: 1; }
}

@keyframes fade-up {
  from { opacity: 0; transform: translateY(8px); }
  to { opacity: 1; transform: translateY(0); }
}

@keyframes scale-in {
  from { opacity: 0; transform: scale(0.97); }
  to { opacity: 1; transform: scale(1); }
}
```

1. **Expand the `@layer base` block** — Replace the existing `@layer base` block with:

```
@layer base {
  * {
    @apply border-border;
  }
  html {
    scroll-behavior: smooth;
  }
  body {
    @apply bg-background text-foreground;
```

```

    font-feature-settings: "rlig" 1, "calt" 1;
  }
  :focus-visible {
    @apply outline-2 outline-offset-2 outline-ring;
  }
}

```

1. **Add utility classes** — Add a new `@layer utilities` block after the `@layer base` block:

```

@layer utilities {
  .card-interactive {
    @apply transition-all duration-200 ease-out;
  }
  .card-interactive:hover {
    @apply shadow-md -translate-y-0.5;
  }
  .auth-bg {
    background-image: radial-gradient(
      circle at 50% 0%,
      var(--accent) 0%,
      transparent 50%
    );
  }
}

```

Final file structure (order of sections)

1. `@import "tailwindcss";`
2. `@import "tw-animate-css";`
3. `@theme inline { ... }` (with animation tokens added at end)
4. `:root { ... }` (with 3 updated color values)
5. `.dark { ... }` (with 3 updated color values)
6. `@keyframes` definitions (3 keyframes)
7. `@layer base { ... }` (expanded)
8. `@layer utilities { ... }` (new)

Acceptance Criteria

- `[] tw-animate-css` is imported and animation utilities like `animate-fade-up` are available
- `[] :root` has `--primary: oklch(0.21 0.034 270), --ring: oklch(0.705 0.06 270), --accent: oklch(0.96 0.012 270)`
- `[] .dark` has `--primary: oklch(0.92 0.02 270), --ring: oklch(0.552 0.05 270), --accent: oklch(0.28 0.018 270)`
- `[]` Three keyframe animations (`fade-in`, `fade-up`, `scale-in`) are defined
- `[] @theme inline` includes `--animate-fade-in`, `--animate-fade-up`, `--animate-scale-in`
- `[] .card-interactive` utility class is defined with hover shadow and translate
- `[] .auth-bg` utility class is defined with radial gradient

- [] `html` has `scroll-behavior: smooth`
- [] `:focus-visible` has global outline styling
- [] All other existing color tokens remain unchanged

Task 02: Layout Sticky Footer

Status

complete

Wave

1

Description

Update the root layout to use a flex column structure that pushes the footer to the bottom of the viewport on short-content pages. Currently, pages like login/register have minimal content and the footer floats up, leaving blank space below. Adding `min-h-screen flex flex-col` to the body and `flex-1` to the main element creates a proper sticky footer layout.

Dependencies

Depends on: None (Wave 1) **Blocks:** task-09-auth-pages.md (auth pages rely on the flex-1 main for proper vertical centering)

Context from dependencies: N/A — this is a foundation task.

Files to Modify

- `src/app/layout.tsx` — Add flex classes to body and main elements

Technical Details

Implementation Steps

1. **Add flex layout classes to the `<body>` element** (line 92). The current className is:

```
className={` ${geistSans.variable} ${geistMono.variable} antialiased`}
```

Change it to:

```
className={` ${geistSans.variable} ${geistMono.variable} antialiased min-h-screen flex flex-col`}
```

1. Add `flex-1` to the `<main>` element (line 101). The current element is:

```
<main id="main-content">{children}</main>
```

Change it to:

```
<main id="main-content" className="flex-1">{children}</main>
```

Why this works

The `ThemeProvider` component (`src/components/theme-provider.tsx`) is a pass-through wrapper around `next-themes` `ThemeProvider` — it renders children directly without adding a wrapping `<div>`. So the flex column chain from `<body>` flows directly through to `<SiteHeader>`, `<main>`, and `<SiteFooter>`.

The `min-h-screen` ensures the body fills at least the full viewport. `flex flex-col` creates a vertical flex container. `flex-1` on `<main>` makes it expand to fill all available space, pushing `<SiteFooter>` to the bottom.

Acceptance Criteria

- [] `<body>` has classes `min-h-screen flex flex-col` in addition to existing font variable classes
- [] `<main>` has `className="flex-1"`
- [] On short-content pages (e.g. `/login`), the footer sits at the bottom of the viewport
- [] On long-content pages (e.g. `/profile`), the footer sits below the content as usual
- [] No visual regressions on any page

Task 03: Site Header Responsive

Status

complete

Wave

1

Description

Update the site header component with responsive padding and sizing so it scales better on mobile devices. Currently the header uses fixed `px-4 py-4` padding and `text-2xl` logo text that could be tighter on small phones. The action buttons also use a fixed `gap-4` that is generous on narrow screens.

Dependencies

Depends on: None (Wave 1) **Blocks:** None

Context from dependencies: N/A

Files to Modify

- `src/components/site-header.tsx` — Update Tailwind classes for responsive sizing

Technical Details

Implementation Steps

1. **Update nav padding** (line 19). Change:

```
container mx-auto px-4 py-4 flex justify-between items-center
```

To:

```
container mx-auto px-3 sm:px-4 py-3 sm:py-4 flex justify-between items-center
```

1. **Update logo text size** (line 21). Change the `<h1>` className from:

```
text-2xl font-bold
```

To:

```
text-xl sm:text-2xl font-bold
```

1. **Update actions group gap** (line 38). Change:

```
flex items-center gap-4
```

To:

```
flex items-center gap-2 sm:gap-4
```

Current file reference

The full current file is at `src/components/site-header.tsx` (46 lines). The structure is:

- Skip-to-main-content link (accessibility)
- `<header className="border-b"> wrapper`
- `<nav>` with logo on left, UserProfile + ModeToggle on right

Acceptance Criteria

- [] Nav padding is `px-3 sm:px-4 py-3 sm:py-4`
- [] Logo text is `text-xl sm:text-2xl`
- [] Actions gap is `gap-2 sm:gap-4`
- [] Header looks good at 320px viewport width (no overflow, no cramping)
- [] Header looks unchanged at desktop widths (1024px+)

Task 04: Site Footer Responsive

Status

complete

Wave

1

Description

Update the site footer with responsive padding to match the header's responsive scaling and maintain consistent container padding across the app. Currently the footer uses fixed `py-6` and `px-4` that could be tighter on mobile and more generous on large screens.

Dependencies

Depends on: None (Wave 1) **Blocks:** None

Context from dependencies: N/A

Files to Modify

- `src/components/site-footer.tsx` — Update Tailwind classes for responsive padding

Technical Details

Implementation Steps

1. **Update footer padding** (line 5). Change:

```
border-t py-6 text-center text-sm text-muted-foreground
```

To:

```
border-t py-4 sm:py-6 text-center text-sm text-muted-foreground
```

1. **Update container padding** (line 6). Change:

```
container mx-auto px-4
```

To:

```
container mx-auto px-4 sm:px-6 lg:px-8
```

Current file reference

The file is at `src/components/site-footer.tsx` (24 lines). It renders a `<footer>` with a GitHubStars component and an attribution link.

Acceptance Criteria

- [] Footer padding is `py-4 sm:py-6`
- [] Container padding is `px-4 sm:px-6 lg:px-8`
- [] Footer is slightly more compact on mobile (`py-4` vs `py-6`)
- [] Footer has more generous side padding on large screens (`lg:px-8`)

Task 05: Home Page Overhaul

Status

pending

Wave

2

Description

Overhaul the home page with responsive typography, shadcn Card components replacing bare divs, hover effects on interactive cards, and better grid breakpoints. This is the highest-visibility page and sets the visual tone for the entire app. The feature cards and next steps cards are currently plain `<div>` elements with `border rounded-lg` — they need to be replaced with proper shadcn Card components with the `card-interactive` hover utility.

Dependencies

Depends on: task-01-globals-css.md **Blocks:** None

Context from dependencies: Task 01 defines the `card-interactive` utility class in `globals.css` which provides `transition-all duration-200 ease-out` with hover effects (`shadow-md -translate-y-0.5`). This task uses that class on all interactive cards. Task 01 also updates the color palette so `text-primary` on icons will show the new blue-tinted primary color.

Files to Modify

- `src/app/page.tsx` — Replace bare divs with shadcn Cards, update typography responsive classes, fix grid breakpoints

Technical Details

Implementation Steps

1. **Add Card imports** at the top of the file. Add this import:

```
import {  
  Card,  
  CardContent,
```

```
CardHeader,  
CardTitle,  
} from "@components/ui/card";
```

1. Update container spacing (line 13). Change:

```
className="flex-1 container mx-auto px-4 py-12"
```

To:

```
className="flex-1 container mx-auto px-4 sm:px-6 lg:px-8 py-8 sm:py-12"
```

1. Update hero h1 typography (line 20). Change:

```
className="text-5xl font-bold tracking-tight bg-gradient-to-r from-primary via-  
primary/90 to-primary/70 bg-clip-text text-transparent"
```

To:

```
className="text-3xl sm:text-4xl md:text-5xl font-bold tracking-tight bg-  
gradient-to-r from-primary via-primary/90 to-primary/70 bg-clip-text text-  
transparent"
```

1. Update hero h2 subtitle (line 24). Change:

```
className="text-2xl font-semibold text-muted-foreground"
```

To:

```
className="text-xl sm:text-2xl font-semibold text-muted-foreground"
```

1. Update hero description paragraph (line 27). Change:

```
className="text-xl text-muted-foreground"
```

To:

```
className="text-base sm:text-lg md:text-xl text-muted-foreground"
```

1. Update "Video Tutorial" heading (line 35). Change:

```
className="text-2xl font-semibold flex items-center justify-center gap-2"
```

To:

```
className="text-xl sm:text-2xl font-semibold flex items-center justify-center
```

```
gap-2"
```

1. **Add shadow to video embed container** (line 43). Change:

```
className="relative pb-[56.25%] h-0 overflow-hidden rounded-lg border"
```

To:

```
className="relative pb-[56.25%] h-0 overflow-hidden rounded-lg border shadow-md"
```

1. **Update feature grid** (line 55). Change:

```
className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-6 mt-12"
```

To:

```
className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-4 sm:gap-6 mt-12"
```

1. **Replace all 4 feature card divs** (lines 56-92) with shadcn Card components. Each card currently looks like:

```
<div className="p-6 border rounded-lg">
  <h3 className="font-semibold mb-2 flex items-center gap-2">
    <Shield className="h-4 w-4" />
    Authentication
  </h3>
  <p className="text-sm text-muted-foreground">
    Better Auth with Google OAuth integration
  </p>
</div>
```

Replace each with:

```
<Card className="card-interactive">
  <CardHeader className="pb-3">
    <CardTitle className="text-base flex items-center gap-2">
      <Shield className="h-4 w-4 text-primary" />
      Authentication
    </CardTitle>
  </CardHeader>
  <CardContent>
    <p className="text-sm text-muted-foreground">
      Better Auth with Google OAuth integration
    </p>
  </CardContent>
</Card>
```

Do this for all 4 cards (Authentication/Shield, Database/Database, AI Ready/Bot, UI Components/

Palette). Add `text-primary` to each icon.

1. **Update "Next Steps" heading** (line 97). Change:

```
className="text-2xl font-semibold"
```

To:

```
className="text-xl sm:text-2xl font-semibold"
```

1. **Update next steps grid** (line 98). Change:

```
className="grid grid-cols-1 md:grid-cols-2 gap-4 text-left"
```

To:

```
className="grid grid-cols-1 sm:grid-cols-2 gap-4 text-left"
```

1. **Replace all 4 next steps card divs** (lines 99-169) with shadcn Card components. Each currently looks like:

```
<div className="p-4 border rounded-lg">
  <h4 className="font-medium mb-2">1. Set up environment variables</h4>
  ...content...
</div>
```

Replace each wrapper with:

```
<Card className="card-interactive">
  <CardContent className="pt-6">
    <h4 className="font-medium mb-2">1. Set up environment variables</h4>
    ...content stays the same...
  </CardContent>
</Card>
```

Use `pt-6` on `CardContent` since there's no `CardHeader` for these simpler cards.

Acceptance Criteria

- [] Hero h1 scales: `text-3xl` on mobile, `text-4xl` on sm, `text-5xl` on md+
- [] Hero h2 and description also scale responsively
- [] Feature grid uses `sm:grid-cols-2` breakpoint (2 columns at 640px)
- [] All 4 feature cards use shadcn `Card/CardHeader/CardContent` with `card-interactive`
- [] All feature card icons have `text-primary` class
- [] All 4 next steps cards use shadcn `Card/CardContent` with `card-interactive`

- ☐ Video embed has `shadow-md`
- ☐ Container has responsive padding: `px-4 sm:px-6 lg:px-8`
- ☐ No TypeScript errors (Card imports are correct)

Task 06: Dashboard Page

Status

pending

Wave

2

Description

Update the dashboard page to use shadcn Card components instead of bare divs, add responsive breakpoints, improve the loading state, and add visual polish. The dashboard currently has two plain `<div>` cards with `border border-border rounded-lg` and a simple "Loading..." text.

Dependencies

Depends on: task-01-globals-css.md **Blocks:** None

Context from dependencies: Task 01 defines the `card-interactive` utility class in `globals.css` and updates the primary color to a blue-tinted hue. This task uses `card-interactive` on cards and `text-primary` on new icon accents.

Files to Modify

- `src/app/dashboard/page.tsx` — Replace divs with Cards, add responsive classes, improve loading

Technical Details

Implementation Steps

1. **Add new imports.** The file currently imports: `Link`, `Lock`, `UserProfile`, `Button`, `useDiagnostics`, `useSession`. Add these:

```
import { Lock, Bot, User } from "lucide-react";
import {
  Card,
  CardContent,
  CardDescription,
```

```

    CardHeader,
    CardTitle,
  } from "@components/ui/card";
import { Spinner } from "@components/ui/spinner";

```

Remove the existing `Lock` from the `lucide-react` import (it's being re-imported in the combined import).

1. **Update loading state** (lines 15-19). Change:

```

return (
  <div className="flex justify-center items-center h-screen">
    Loading...
  </div>
);

```

To:

```

return (
  <div className="flex justify-center items-center h-screen">
    <Spinner size="lg" />
  </div>
);

```

1. **Update unauthenticated state container** (line 24). Change:

```

className="container mx-auto px-4 py-12"

```

To:

```

className="container mx-auto px-4 sm:px-6 lg:px-8 py-8 sm:py-12"

```

1. **Wrap unauthenticated content in Card** (lines 26-34). Change:

```

<div className="mb-8">
  <Lock className="w-16 h-16 mx-auto mb-4 text-muted-foreground" />
  <h1 className="text-2xl font-bold mb-2">Protected Page</h1>
  <p className="text-muted-foreground mb-6">
    You need to sign in to access the dashboard
  </p>
</div>

```

To:

```

<Card className="mb-8 text-center">
  <CardContent className="pt-6">
    <Lock className="w-16 h-16 mx-auto mb-4 text-muted-foreground" />
    <h1 className="text-2xl font-bold mb-2">Protected Page</h1>
    <p className="text-muted-foreground mb-6">

```

```
    You need to sign in to access the dashboard
  </p>
</CardContent>
</Card>
```

1. Update authenticated container (line 40). Change:

```
className="container mx-auto p-6"
```

To:

```
className="container mx-auto p-4 sm:p-6 max-w-5xl"
```

1. Update title section (lines 41-43). Change:

```
<div className="flex justify-between items-center mb-8">
  <h1 className="text-3xl font-bold">Dashboard</h1>
</div>
```

To:

```
<div className="flex flex-wrap justify-between items-center gap-4 mb-6 sm:mb-8">
  <h1 className="text-2xl sm:text-3xl font-bold">Dashboard</h1>
</div>
```

1. Update card grid (line 45). Change:

```
className="grid grid-cols-1 md:grid-cols-2 gap-6"
```

To:

```
className="grid grid-cols-1 sm:grid-cols-2 gap-6"
```

1. Replace AI Chat card div (lines 46-59). Change:

```
<div className="p-6 border border-border rounded-lg">
  <h2 className="text-xl font-semibold mb-2">AI Chat</h2>
  <p className="text-muted-foreground mb-4">
    Start a conversation with AI using the Vercel AI SDK
  </p>
  ...button...
</div>
```

To:

```
<Card className="card-interactive">
  <CardHeader>
    <CardTitle className="flex items-center gap-2">
```

```

    <Bot className="h-5 w-5 text-primary" />
    AI Chat
  </CardTitle>
  <CardDescription>
    Start a conversation with AI using the Vercel AI SDK
  </CardDescription>
</CardHeader>
<CardContent>
  ...button stays the same...
</CardContent>
</Card>

```

1. Replace Profile card div (lines 62-76). Change:

```

<div className="p-6 border border-border rounded-lg">
  <h2 className="text-xl font-semibold mb-2">Profile</h2>
  <p className="text-muted-foreground mb-4">
    Manage your account settings and preferences
  </p>
  <div className="space-y-2">
    <p><strong>Name:</strong> {session.user.name}</p>
    <p><strong>Email:</strong> {session.user.email}</p>
  </div>
</div>

```

To:

```

<Card className="card-interactive">
  <CardHeader>
    <CardTitle className="flex items-center gap-2">
      <User className="h-5 w-5 text-primary" />
      Profile
    </CardTitle>
    <CardDescription>
      Manage your account settings and preferences
    </CardDescription>
  </CardHeader>
  <CardContent>
    <div className="space-y-2">
      <p><strong>Name:</strong> {session.user.name}</p>
      <p><strong>Email:</strong> {session.user.email}</p>
    </div>
  </CardContent>
</Card>

```

Acceptance Criteria

- [] Both dashboard cards use shadow `Card/CardHeader/CardTitle/CardDescription/CardContent`
- [] Both cards have `card-interactive` class
- [] AI Chat card has Bot icon with `text-primary`, Profile card has User icon with `text-primary`

- [] Loading state shows `Spinner` component instead of "Loading..." text
- [] Container has `max-w-5xl` to prevent overly wide content
- [] Grid uses `sm:grid-cols-2` breakpoint
- [] Title scales: `text-2xl` `sm:text-3xl`
- [] Unauthenticated state wraps in Card
- [] No TypeScript errors

Task 07: Chat Page

Status

pending

Wave

2

Description

Update the chat page to use shadcn Input/Button components instead of native HTML elements, add responsive breakpoints to the header and message bubbles, make the input form sticky at the bottom, and improve the empty state. This page has the most interactive elements and benefits significantly from consistent component usage.

Dependencies

Depends on: task-01-globals-css.md **Blocks:** None

Context from dependencies: Task 01 updates the color palette (so `text-primary`, `bg-primary` references use the new blue-tinted primary). No utility classes from Task 01 are used on this page (chat doesn't use `card-interactive`).

Files to Modify

- `src/app/chat/page.tsx` — Replace native elements with shadcn components, add responsive classes, sticky input

Technical Details

Implementation Steps

1. **Add Input import.** Add to the existing imports:

```
import { Input } from "@/components/ui/input";
```

The file already imports `Button` from `@/components/ui/button`.

1. **Also import Bot icon** for the improved empty state. Add `Bot` to the existing lucide-react

import:

```
import { Copy, Check, Loader2, Bot } from "lucide-react";
```

1. **Replace native `<button>` in `CopyButton`** (lines 165-177). Change the function body from:

```
return (
  <button
    onClick={handleCopy}
    className="p-1 hover:bg-muted rounded transition-colors"
    title="Copy to clipboard"
  >
    {copied ? (
      <Check className="h-3.5 w-3.5 text-green-500" />
    ) : (
      <Copy className="h-3.5 w-3.5 text-muted-foreground" />
    )}
  </button>
);
```

To:

```
return (
  <Button
    variant="ghost"
    size="icon"
    className="h-7 w-7"
    onClick={handleCopy}
    title="Copy to clipboard"
  >
    {copied ? (
      <Check className="h-3.5 w-3.5 text-green-500" />
    ) : (
      <Copy className="h-3.5 w-3.5 text-muted-foreground" />
    )}
  </Button>
);
```

1. **Update container** (line 247). Change:

```
className="container mx-auto px-4 py-8"
```

To:

```
className="container mx-auto px-4 sm:px-6 lg:px-8 py-6 sm:py-8"
```

1. **Update chat header** (lines 249-261). Change:

```
<div className="flex justify-between items-center mb-6 pb-4 border-b">
  <h1 className="text-2xl font-bold">AI Chat</h1>
```

```
<div className="flex items-center gap-4">
  <span className="text-sm text-muted-foreground">
    Welcome, {session.user.name}!
  </span>
```

To:

```
<div className="flex flex-col sm:flex-row sm:justify-between sm:items-center
gap-3 sm:gap-4 mb-4 sm:mb-6 pb-4 border-b">
  <h1 className="text-xl sm:text-2xl font-bold">AI Chat</h1>
  <div className="flex items-center gap-2 sm:gap-4">
    <span className="text-sm text-muted-foreground hidden sm:inline">
      Welcome, {session.user.name}!
    </span>
```

Key changes: flex stacks on mobile, welcome text hidden on mobile (`hidden sm:inline`), smaller gaps.

1. **Update message bubble max-widths** (line ~289-290). For user messages, change:

```
bg-primary text-primary-foreground ml-auto max-w-[80%]
```

To:

```
bg-primary text-primary-foreground ml-auto max-w-[85%] sm:max-w-[80%] md:max-
w-2xl shadow-sm
```

For AI messages, change:

```
bg-muted max-w-[80%]
```

To:

```
bg-muted max-w-[85%] sm:max-w-[80%] md:max-w-2xl shadow-sm
```

1. **Update message area** (line 271). Change:

```
className="min-h-[50vh] overflow-y-auto space-y-4 mb-4"
```

To:

```
className="min-h-[50vh] overflow-y-auto space-y-4 mb-4 pb-4"
```

1. **Improve empty state** (lines 272-275). Change:

```
<div className="text-center text-muted-foreground py-12">
  Start a conversation with AI
```

```
</div>
```

To:

```
<div className="text-center text-muted-foreground py-16 space-y-3">
  <Bot className="h-12 w-12 mx-auto text-muted-foreground/50" />
  <p className="text-lg font-medium">Start a conversation with AI</p>
  <p className="text-sm">Type a message below to begin chatting</p>
</div>
```

1. **Make input form sticky and replace native input** (lines 317-344). Change the form from:

```
<form
  onSubmit={...}
  className="flex gap-2"
>
  <input
    value={input}
    onChange={(e) => setInput(e.target.value)}
    placeholder="Type your message..."
    className="flex-1 p-2 border border-border rounded-md focus:outline-none
focus:ring-2 focus:ring-ring"
    disabled={isStreaming}
  />
  <Button type="submit" disabled={!input.trim() || isStreaming}>
    ...
  </Button>
</form>
```

To:

```
<form
  onSubmit={...}
  className="sticky bottom-0 bg-background py-4 border-t z-10 flex gap-2
sm:gap-3"
>
  <Input
    value={input}
    onChange={(e) => setInput(e.target.value)}
    placeholder="Type your message..."
    className="flex-1 min-w-0"
    disabled={isStreaming}
  />
  <Button type="submit" disabled={!input.trim() || isStreaming}>
    ...
  </Button>
</form>
```

Key changes: `sticky bottom-0 bg-background py-4 border-t z-10` makes the form stick. `Input` replaces native `<input>` for consistent styling. `min-w-0` prevents flex overflow.

Acceptance Criteria

- [] Chat input uses shadcn `Input` component, not a native `<input>`
- [] CopyButton uses shadcn `Button` `variant="ghost"` `size="icon"`, not a native `<button>`
- [] Chat header stacks vertically on mobile (`flex-col`) and is horizontal on sm+
- [] Welcome text is hidden on mobile (`hidden sm:inline`)
- [] Message bubbles have `shadow-sm` for depth
- [] Message widths scale: `max-w-[85%]` on mobile, `max-w-[80%]` on sm, `max-w-2xl` on md+
- [] Input form is sticky at bottom with `bg-background` and `border-t`
- [] Empty state shows Bot icon, title text, and subtitle
- [] Container has responsive padding: `px-4 sm:px-6 lg:px-8`
- [] No TypeScript errors (Input import is correct)

Task 08: Profile Page

Status

pending

Wave

2

Description

Update the profile page with responsive stacking, improved grid breakpoints, and flex-wrap on tight rows. The avatar section needs to stack vertically on mobile, the quick actions grid needs an intermediate `sm:` breakpoint, and all the `flex items-center justify-between` rows in cards and dialogs need `flex-wrap` to prevent overflow on narrow screens.

Dependencies

Depends on: task-01-globals-css.md **Blocks:** None

Context from dependencies: Task 01 updates the color palette. No utility classes from Task 01 are specifically used on this page (profile cards are informational, not interactive).

Files to Modify

- `src/app/profile/page.tsx` — Update responsive classes throughout

Technical Details

Implementation Steps

1. **Update container** (line 67). Change:

```
className="container max-w-4xl mx-auto py-8 px-4"
```

To:

```
className="container max-w-4xl mx-auto py-6 sm:py-8 px-4 sm:px-6 lg:px-8"
```

1. Update title row (line 68). Change:

```
className="flex items-center gap-4 mb-8"
```

To:

```
className="flex items-center gap-2 sm:gap-4 mb-6 sm:mb-8"
```

1. Update title text (line 78). Change:

```
className="text-3xl font-bold"
```

To:

```
className="text-2xl sm:text-3xl font-bold"
```

1. Update avatar section (line 85). Change:

```
className="flex items-center space-x-4"
```

To:

```
className="flex flex-col items-center sm:flex-row sm:items-center gap-4"
```

Important: remove `space-x-4` entirely — it conflicts with `flex-col`. The `gap-4` handles spacing in both directions.

1. Update avatar size (line 86). Change:

```
className="h-20 w-20"
```

To:

```
className="h-16 w-16 sm:h-20 sm:w-20"
```

1. Update text container next to avatar (line 96). Change:

```
className="space-y-2"
```

To:

```
className="space-y-2 text-center sm:text-left"
```

1. Update name heading (line 97). Change:

```
className="text-2xl font-semibold"
```

To:

```
className="text-xl sm:text-2xl font-semibold"
```

1. **Update email row** (line 98). Change:

```
className="flex items-center gap-2 text-muted-foreground"
```

To:

```
className="flex flex-wrap items-center justify-center sm:justify-start gap-2 text-muted-foreground"
```

1. **Update Account Status grid** (line 160). Change:

```
className="grid grid-cols-1 md:grid-cols-2 gap-4"
```

To:

```
className="grid grid-cols-1 sm:grid-cols-2 gap-4"
```

1. **Update all status/activity flex rows** — add `flex-wrap gap-2` and responsive padding. These appear at multiple locations. For each row that has `flex items-center justify-between p-4 border rounded-lg`, change to:

```
flex flex-wrap items-center justify-between gap-2 p-3 sm:p-4 border rounded-lg
```

Apply this to the following locations:

- Line 161 (Email Verification status)
- Line 172 (Account Type status)
- Line 196 (Current Session activity)

1. **Update Quick Actions grid** (line 224). Change:

```
className="grid grid-cols-1 md:grid-cols-3 gap-4"
```

To:

```
className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-4"
```

1. **Update dialog inner rows** — For the Security Settings dialog (lines 325, 342, 357), each row with `flex items-center justify-between p-4 border rounded-lg`, change to:

```
flex flex-wrap items-center justify-between gap-2 p-3 sm:p-4 border rounded-lg
```

1. **Update Email Preferences dialog rows** (lines 388, 396) — same pattern, change:

```
flex items-center justify-between p-4 border rounded-lg
```

To:

```
flex flex-wrap items-center justify-between gap-2 p-3 sm:p-4 border rounded-lg
```

Acceptance Criteria

- [] Avatar section stacks vertically on mobile (flex-col) and is horizontal on sm+
- [] Avatar is smaller on mobile (h-16 w-16) and full size on sm+ (h-20 w-20)
- [] Text next to avatar is centered on mobile, left-aligned on sm+
- [] Email row wraps on narrow screens (flex-wrap)
- [] Quick actions grid has intermediate breakpoint: 1 col → 2 at sm → 3 at lg
- [] Account status grid uses sm:grid-cols-2 instead of md:grid-cols-2
- [] All flex justify-between rows in cards and dialogs have flex-wrap and gap-2
- [] Container and title have responsive padding and sizing
- [] No overflow or horizontal scroll at 320px viewport
- [] Profile page title scales: `text-2xl` sm:`text-3xl`

Task 09: Auth Pages

Status

pending

Wave

2

Description

Polish all 4 auth pages (login, register, forgot-password, reset-password) with entrance animations, card shadows, a subtle background gradient, and improved vertical centering that works with the new flex layout. These pages are the first thing new users see, so visual polish matters.

Dependencies

Depends on: task-01-globals-css.md, task-02-layout.md **Blocks:** None

Context from dependencies: Task 01 defines the `auth-bg` utility class (radial gradient from accent color) and the `animate-fade-up` animation in `globals.css`. Task 02 adds `flex-1` to the `<main>` element, so auth pages can use `flex-1` to fill available vertical space properly.

Files to Modify

- `src/app/(auth)/login/page.tsx` — Add animation, shadow, background, update min-height
- `src/app/(auth)/register/page.tsx` — Same changes
- `src/app/(auth)/forgot-password/page.tsx` — Same changes
- `src/app/(auth)/reset-password/page.tsx` — Same changes

Technical Details

Implementation Steps

All 4 files follow the exact same pattern. Each has an outer wrapper div with a Card inside.

For each file, make these 2 changes:

Change 1: Update the wrapper div. Each file has:

```
<div className="flex min-h-[calc(100vh-4rem)] items-center justify-center p-4">
```

Change to:

```
<div className="auth-bg flex flex-1 min-h-[calc(100vh-8rem)] items-center justify-center p-4">
```

Changes:

- `auth-bg` — adds the subtle radial gradient background from `globals.css`
- `flex-1` — fills available space in the flex column layout (from Task 02)
- `min-h-[calc(100vh-8rem)]` — accounts for both header and footer height (was just 4rem for header)

Change 2: Add shadow and animation to the Card. Each file has:

```
<Card className="w-full max-w-md">
```

Change to:

```
<Card className="w-full max-w-md shadow-lg animate-fade-up">
```

Changes:

- `shadow-lg` — adds elevation/depth to the card
- `animate-fade-up` — entrance animation (defined in `globals.css` keyframes, registered in `@theme inline`)

File-specific locations:

login/page.tsx:

- Wrapper div: line 27
- Card: line 28

register/page.tsx:

- Wrapper div: line 20
- Card: line 21

forgot-password/page.tsx:

- Wrapper div: line 21
- Card: line 22

reset-password/page.tsx:

- Wrapper div: line 21
- Card: line 22

Acceptance Criteria

- [] All 4 auth pages have `auth-bg` class on the wrapper div
- [] All 4 auth pages have `flex-1 min-h-[calc(100vh-8rem)]` on the wrapper div
- [] All 4 auth cards have `shadow-lg` for depth
- [] All 4 auth cards have `animate-fade-up` for entrance animation
- [] Cards animate upward on page load (smooth fade + translate)
- [] Subtle radial gradient visible in the background (especially visible in dark mode)
- [] Footer stays at the bottom of the viewport on all auth pages
- [] No visual regressions to auth form functionality

Task 10: Setup Checklist Card Adoption

Status

pending

Wave

2

Description

Replace the bare `<div>` wrapper in the `SetupChecklist` component with a shadcn `Card` component for visual consistency with the rest of the app. The checklist is displayed on the home page and currently uses `p-6 border rounded-lg` which is the same pattern as the feature cards that are being upgraded to shadcn Cards.

Dependencies

Depends on: task-01-globals-css.md **Blocks:** None

Context from dependencies: Task 01 updates the color palette. The setup checklist does not use `card-interactive` since it's an informational display, not a clickable card.

Files to Modify

- `src/components/setup-checklist.tsx` — Replace wrapper div with shadcn Card

Technical Details

Implementation Steps

1. **Add Card imports.** Add at the top:

```
import {  
  Card,  
  CardContent,  
  CardHeader,  
  CardTitle,  
} from "@/components/ui/card";
```

1. Replace the outer wrapper (line 124). The current structure is:

```
<div className="p-6 border rounded-lg text-left">
  <div className="flex items-center justify-between mb-4">
    <div>
      <h3 className="font-semibold">Setup checklist</h3>
      <p className="text-sm text-muted-foreground">
        {completed}/{steps.length} completed
      </p>
    </div>
    <Button size="sm" onClick={load} disabled={loading}>
      {loading ? "Checking..." : "Re-check"}
    </Button>
  </div>

  {error ? <div className="text-sm text-destructive">{error}</div> : null}

  <ul className="space-y-2">
    ...checklist items...
  </ul>

  {data ? (
    <div className="mt-4 text-xs text-muted-foreground">
      Last checked: {new Date(data.timestamp).toLocaleString()}
    </div>
  ) : null}
</div>
```

Replace with:

```
<Card className="text-left">
  <CardHeader className="pb-4">
    <div className="flex items-center justify-between">
      <div>
        <CardTitle className="text-base">Setup checklist</CardTitle>
        <p className="text-sm text-muted-foreground">
          {completed}/{steps.length} completed
        </p>
      </div>
      <Button size="sm" onClick={load} disabled={loading}>
        {loading ? "Checking..." : "Re-check"}
      </Button>
    </div>
  </CardHeader>
  <CardContent>
    {error ? <div className="text-sm text-destructive mb-4">{error}</div> :
    null}

    <ul className="space-y-2">
      ...checklist items stay the same...
    </ul>

    {data ? (
      <div className="mt-4 text-xs text-muted-foreground">
```

```
        Last checked: {new Date(data.timestamp).toLocaleString()}
      </div>
    ) : null}
  </CardContent>
</Card>
```

Key changes:

- Outer `<div>` → `<Card>`
- Title area wrapped in `<CardHeader>` with the flex row inside
- `h3` → `<CardTitle className="text-base">` (text-base to match current visual size)
- Content wrapped in `<CardContent>`
- Error div gets `mb-4` since `CardContent` doesn't have the same padding structure
- All checklist items and timestamp stay exactly the same

Acceptance Criteria

- [] Setup checklist uses shadcn `Card/CardHeader/CardTitle/CardContent`
- [] Visual appearance is consistent — no jarring size or spacing changes
- [] Title uses `CardTitle` with `text-base` to maintain current size
- [] Re-check button still works and is positioned to the right of the title
- [] Error state still displays correctly
- [] Checklist items render identically to before
- [] No TypeScript errors

Task 11: Starter Prompt Modal

Status

pending

Wave

2

Description

Replace the native `<textarea>` in the StarterPromptModal with the shadcn Textarea component for visual consistency, and make the button row responsive so buttons stack on very small screens.

Dependencies

Depends on: task-01-globals-css.md **Blocks:** None

Context from dependencies: Task 01 updates the color palette. No utility classes from Task 01 are used in this component.

Files to Modify

- `src/components/starter-prompt-modal.tsx` — Replace native textarea, update button layout

Technical Details

Implementation Steps

1. **Add Textarea import.** Add at the top:

```
import { Textarea } from "@/components/ui/textarea";
```

1. **Replace native `<textarea>`** (lines 161-167). Change:

```
<textarea
  id="project-description"
  placeholder="e.g., A task management app for teams with real-time
collaboration, project timelines, and AI-powered task prioritization..."
```

```
value={projectDescription}
onChange={(e) => setProjectDescription(e.target.value)}
className="w-full h-24 px-3 py-2 border rounded-md resize-none text-sm"
/>
```

To:

```
<Textarea
  id="project-description"
  placeholder="e.g., A task management app for teams with real-time
collaboration, project timelines, and AI-powered task prioritization..."
  value={projectDescription}
  onChange={(e) => setProjectDescription(e.target.value)}
  className="h-24 resize-none"
/>
```

Key changes:

- `<textarea>` → `<Textarea>` (shadcn component)
- Remove explicit `w-full px-3 py-2 border rounded-md text-sm` — the shadcn `Textarea` handles these styles internally
- Keep `h-24 resize-none` as overrides

1. **Update button row** (line 174). Change:

```
className="flex gap-2"
```

To:

```
className="flex flex-col-reverse sm:flex-row gap-2"
```

This makes the buttons stack on mobile (with Cancel on top via `flex-col-reverse`) and sit side-by-side on sm+.

Acceptance Criteria

- [] Textarea uses shadcn `Textarea` component, not a native `<textarea>`
- [] Textarea has consistent focus ring styling matching other form elements
- [] Button row stacks on mobile (`flex-col-reverse`) and is horizontal on sm+ (`sm:flex-row`)
- [] "Copy Starter Prompt" button appears before "Cancel" button in both layouts
- [] Textarea placeholder text renders correctly
- [] Copy functionality still works (the `projectDescription` state binding is unchanged)
- [] No TypeScript errors

UI Redesign — Phased Plan

Status: active · **Owner decision:** Option 2 (upgrade the whole app), by phase **Progress:** Phases 0–5 ✓ — full-app deck-quality uplift complete across all 55 routes (token layer + primitives; app shell with 8K palette + mobile tab bar; every screen rebuilt to its mockup / the design language; currency-safe money screens; consistency token sweep). Verified by typecheck + lint (0/0) + full production build + all-routes no-500 check. **Pending:** authenticated visual/functional QA (blocked on a stale dev-DB password in `.env`; see memory `dev-db-credentials-stale`). **Decision record:** docs/decisions/0005-app-ui-redesign.md

Delivery policy (Phases 1–5): redesign, not rewrite. No schema changes per phase. Any mockup field without a backing column is **derived from existing data** or **omitted** — never fabricated. Every omission/derivation is recorded (see the Phase 1 notes in docs/roadmap.md).

Bring the **live application** (`agencetool.vercel.app`) up to the marketing-grade standard set by the sales deck and mockups under `marketing/`. Each phase is a visual/UX upgrade of existing screens that **preserves all current functionality, routes, and server actions** — this is a redesign, not a rewrite.

Targets & sources of truth

- **Visual target:** `marketing/index.html` + `marketing/mockups/*.html`
- **Design language:** `marketing/DESIGN-RECOMMENDATIONS.md`
- **In-code design system (source of truth, evolves here):** `DESIGN.md` + `globals.css`
- **Data shape for populated screens:** `marketing/demo-data.json`
- **Coding/UX conventions (unchanged):** `AGENTS.md`, `DESIGN.md`

The mockups are standalone HTML — a **visual target to match**, not code to copy. The app is Next.js (App Router) + React + shadcn/ui; rebuild the look with real components and real data.

Guardrails (every phase)

1. **No functional regressions** — routes, server actions, auth, tenancy, i18n (EN/FR/AR + RTL), and permissions behave exactly as before.
2. **DESIGN.md stays in sync** — when tokens/components change, update `DESIGN.md` in the same phase so it remains the source of truth.
3. **Follow the never-rules** — no free text where a dropdown exists, never expose another tenant, never sum different currencies, etc. (`AGENTS.md`).
4. **Quality gate before commit** — `lint`, `typecheck`, and `next build` pass; verify the screen in

the preview server; deploy to Vercel.

5. **Accessibility & responsive** — keyboard focus, contrast, and mobile layout checked per screen, not deferred entirely to Phase 5.

Phases

Phase 0 — Design foundation & demo data

The groundwork every later phase depends on.

- Elevate `globals.css` tokens to the marketing palette/scale (spacing, elevation/ shadows, radii, chart colors, premium accents) and update `DESIGN.md` to match.
- Build the reusable primitives the mockups imply but the app lacks: KPI/stat card with delta, KPI grid, chart components (area · donut · horizontal bars · funnel), populated empty states, skeleton loaders, refined table, refined sidebar/topbar.
- Seed the **full demo dataset** (`marketing/demo-data.json` shape) into the dev/demo agency so every screen looks populated like the mockups.
- **Done when:** tokens + primitives exist, `DESIGN.md` updated, demo agency populated, zero regressions on existing pages.

Phase 1 — Command center (hero screens) ✓

The screens users and prospects see first.

- **Dashboard** → target `mockups/dashboard.html` — ✓ hero KPIs with deltas, revenue trend, bookings-by-status, pipeline funnel, upcoming departures, derived needs-attention list; manager insights kept + restyled.
- **CRM** — client list + client profile → target `mockups/crm.html` — ✓ premium list; profile with breadcrumb, avatar header, derived stat strip, tabbed sections, spend-by-year chart, restyled timeline.
- **Pipeline** — Kanban board → target `mockups/opportunities.html` — ✓ breadcrumb, 5 KPIs, client-side filter bar, rich cards (probability, close date, purpose, owner); stage-move + funnel preserved.

Added shared primitives: `src/components/ui/{tabs,breadcrumb,tooltip}.tsx`.

Phase 2 — Sell & source

The revenue-making workflow.

- **Proposal builder** → target `mockups/proposal-builder.html`
- **Flight search** → target `mockups/flight-search.html`
- **Hotel search** → target `mockups/hotel-search.html`

Phase 3 — Operate

Fulfilment and money.

- **Bookings** — list + **Booking details** → target `mockups/booking-details.html`
- **Finance / Payments / Commissions** — apply the same card/table/KPI system

Phase 4 — Client experience & insight

- **Customer portal** → target `mockups/customer-portal.html`
- **Reports & analytics** → target `mockups/reports.html`
- **AI assistant** → target `mockups/ai-assistant.html`

Phase 5 — Polish & responsive

- Mobile/responsive pass → target `mockups/mobile.html`
- Empty / loading (skeleton) / error states across all pages
- Micro-interactions and transitions per `DESIGN-RECOMMENDATIONS.md`
- Dark-mode parity, accessibility sweep, i18n/RTL verification

Per-screen acceptance criteria

A screen is "done" when:

- Layout, hierarchy, spacing, and components visibly match the target mockup (not pixel-identical, but unmistakably the same product).
- It is populated with realistic demo data (no lorem, no empty hero states).
- It keeps every existing action/route working.
- It is responsive and passes the quality gate.
- `DESIGN.md` reflects any new shared component or token it introduced.

Sequencing & delivery

Phases run in order; Phase 0 must land first. Each screen within a phase can be built in parallel once Phase 0 primitives exist. Every phase ends with a production deploy so the live app visibly improves phase by phase.